

Trabajo fin de grado

Análisis de Nuevos Algoritmos Post-Cuánticos de Cifrado



Santiago de Prada Lorenzo

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C/ Francisco Tomás y Valiente nº 11

**UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR**



Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

**Análisis de Nuevos Algoritmos
Post-Cuánticos de Cifrado**

**Autor: Santiago de Prada Lorenzo
Tutor: Jorge Enrique López de Vergara Méndez**

febrero 2026

Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la Ley, cualquier forma de reproducción, distribución comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de la propiedad intelectual.

La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. del Código Penal*).

DERECHOS RESERVADOS

© 2026 por UNIVERSIDAD AUTÓNOMA DE MADRID

Francisco Tomás y Valiente, n.º 1

Madrid, 28049

Spain

Santiago de Prada Lorenzo

Análisis de Nuevos Algoritmos Post-Cuánticos de Cifrado

Santiago de Prada Lorenzo

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

AGRADECIMIENTOS

En primer lugar, quiero expresar mi más sincero agradecimiento a mis padres, cuyo apoyo incondicional y confianza han sido imprescindibles a lo largo de toda mi trayectoria académica. Su ayuda constante, tanto en el ámbito personal como en el académico, ha sido fundamental, especialmente en lo referente a mi proceso con la sordera bilateral y la adaptación con los implantes cocleares. Gracias a su esfuerzo, paciencia y dedicación, he podido superar dificultades y avanzar con determinación en este camino.

Asimismo, deseo agradecer a mi hermana por su cercanía y comprensión en los momentos más exigentes, y a mis amigos, por estar a mi lado y aportar ánimo, compañía y equilibrio en todas las etapas de este proceso.

Finalmente, quiero manifestar mi especial gratitud a mi tutor, el profesor Jorge Enrique López de Vergara Méndez, por su orientación, disponibilidad y valiosas sugerencias durante la elaboración de este Trabajo de Fin de Grado, que han resultado decisivas para alcanzar los objetivos propuestos.

RESUMEN

La protección de la información frente a accesos no autorizados constituye un reto central en la sociedad digital actual. La criptografía clásica ha ofrecido durante décadas mecanismos fiables de seguridad, pero el previsible impacto de la computación cuántica amenaza con dejar obsoletos algoritmos ampliamente utilizados, como RSA o las curvas elípticas, comprometiendo la confidencialidad y la integridad de los datos.

En este contexto, la criptografía post-cuántica emerge como la alternativa más prometedora, desarrollando algoritmos resistentes a ataques cuánticos. Este Trabajo de Fin de Grado aborda un estudio comparativo de diferentes propuestas dentro del proceso de estandarización del NIST. Por un lado, se analizan los esquemas finalistas de la tercera ronda (Kyber para intercambio de claves, y Dilithium, FALCON y SPHINCS+ para firma digital). Por otro, se amplía el alcance a candidatos de la cuarta ronda, como BIKE, HQC y Classic McEliece, que representan familias criptográficas de gran relevancia por su diversidad de fundamentos matemáticos.

El análisis se lleva a cabo implementando y evaluando estos algoritmos en varios lenguajes de programación (C, C#, Java, Python y Kotlin), así como en plataformas heterogéneas que incluyen ordenadores personales, dispositivos embebidos y teléfonos móviles. El propósito es ofrecer una visión completa de su rendimiento y extraer conclusiones útiles sobre su posible adopción en sistemas reales de instituciones, empresas y usuarios, contribuyendo a la preparación de un futuro escenario tecnológico dominado por la presencia de la computación cuántica.

PALABRAS CLAVE

Criptografía post-cuántica, Algoritmos de intercambio de claves, Kyber, Algoritmos de firma digital, Dilithium, FALCON, SPHINCS+, BIKE, HQC, Classic McEliece, Rendimiento, Evaluación multidispositivo, Lenguajes de programación.

ABSTRACT

Protecting information against unauthorized access is one of the major challenges in today's digital society. Classical cryptography has long provided reliable mechanisms to ensure confidentiality, integrity, and authenticity of data, but the expected impact of quantum computing threatens to render widely deployed algorithms obsolete, such as RSA or elliptic curves, jeopardizing the security of communications worldwide.

Post-quantum cryptography has emerged as the most promising response to this scenario, offering algorithms designed to resist quantum attacks. This Final Degree Project presents a comparative study of several proposals within the NIST standardization process. On the one hand, it analyzes third-round finalists, including Kyber for key encapsulation and Dilithium, FALCON, and SPHINCS+ for digital signatures. On the other hand, it extends the scope to fourth-round candidates such as BIKE, HQC, and Classic McEliece, which belong to distinct cryptographic families and broaden the evaluation spectrum.

The analysis is conducted through implementations in multiple programming languages (C, C#, Java, Python, and Kotlin) and across heterogeneous platforms, including personal computers, embedded devices, and mobile phones. The ultimate goal is to provide a comprehensive performance assessment of these algorithms and to offer insights into their viability for integration into real-world systems used by institutions, companies, and individuals, ensuring robust security in a technological landscape shaped by quantum advances.

KEYWORDS

Post-quantum cryptography, Key exchange algorithms, Kyber, Digital signature algorithms, Dilithium, FALCON, SPHINCS+, BIKE, HQC, Classic McEliece, Performance evaluation, Multi-platform analysis, Programming languages.

ÍNDICE

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	1
1.3	Fases de realización	1
1.4	Estructura de la memoria	2
2	Estado del Arte	3
2.1	Introducción	3
2.2	La necesidad de la criptografía post-cuántica y su estandarización	3
2.3	Algoritmos finalistas de la tercera ronda	4
2.4	Algoritmos candidatos de la cuarta ronda	6
2.5	Seguridad de los algoritmos post-cuánticos	7
2.5.1	Niveles de seguridad del NIST	7
2.5.2	Fundamentos de seguridad en los estándares FIPS 203, 204 y 205, y su diversificación con HQC	8
2.5.3	Ataques clásicos y cuánticos	9
2.6	Trabajos relacionados	9
2.7	Conclusión	10
3	Metodología y entorno experimental	11
3.1	Introducción	11
3.1.1	Descripción de los sistemas de prueba	12
3.1.2	Lenguajes de programación y compiladores	12
3.2	Implementaciones y librerías criptográficas	12
3.3	Adaptación del código y medición de rendimiento	13
3.4	Medición de recursos	14
3.5	Conclusión	14
4	Resultados y Análisis	15
4.1	Introducción	15
4.2	Resultados en ordenador personal	15
4.2.1	Comparación 3ª vs 4ª ronda	15
4.2.2	Estudio por operación	20
4.2.3	Consumo de recursos	23

4.3 Resultados en Raspberry Pi 5	24
4.3.1 Comparación 3ª vs 4ª ronda	25
4.3.2 Estudio por operación	28
4.3.3 Consumo de recursos	29
4.4 Resultados en teléfono móvil	30
4.4.1 Comparación 3ª vs 4ª ronda	30
4.4.2 Estudio por operación	33
4.4.3 Consumo de recursos	34
4.5 Conclusiones	35
5 Conclusiones y Trabajo Futuro	39
5.1 Conclusiones finales	39
5.2 Trabajo futuro	39
Bibliografía	45
Apéndices	47
A Criptografía post-cuántica en la Web	49
A.1 Criptografía post-cuántica en la Web: estado actual y transición	49
A.1.1 HTTPS/TLS 1.3	49
A.1.2 Acuerdo de claves híbrido (X25519 + Kyber/ML-KEM) y firmas PQC	50
A.1.3 Estado en producción, políticas de PKI y piezas adyacentes de la Web	51
A.1.4 Certificados y políticas (PKI pública)	51
A.1.5 Guías de migración en la capa de aplicación	51
A.1.6 Autenticación Web y nombres de dominio	52
B Estudio: Rendimiento de algoritmos post-cuánticos en dispositivo ordenador	53
C Estudio: Rendimiento de algoritmos de 3ª y 4ª ronda en dispositivo Raspberry Pi 5	103
D Estudio: Rendimiento de algoritmos post-cuánticos en dispositivo Android	149

LISTAS

Lista de figuras

1.1	Diagrama de Gantt de la realización del TFG	2
2.1	Línea temporal del proceso NIST PQC	4
2.2	Clasificación de esquemas PQC por familias	5
2.3	BIKE: estimaciones de la tasa de fallo de descifrado (DFR)	6
3.1	Diagrama de desarrollo	11
4.1	BIKE ordenador tiempos totales C	16
4.2	HQC ordenador tiempos por operación Java	18
4.3	MCELIECE ordenador tiempos totales Python	19
4.4	BIKE ordenador tiempos por operación C	20
4.5	MCELIECE ordenador tiempos por operación C	23
4.6	BIKE Raspberry tiempos totales Python	25
4.7	BIKE Raspberry tiempos por operación Python	28
4.8	MCELIECE Raspberry tiempos por operación C	29
4.9	BIKE resultados totales en dispositivo móvil	30
4.10	HQC resultados totales en dispositivo móvil	31
4.11	HQC en dispositivo móvil tiempos por operación	33
4.12	Resultados de generación de claves de Classic McEliece en dispositivo móvil	34
4.13	Mejores tiempos totales en PC	36
4.14	Mejores tiempos totales en Raspberry Pi	36
4.15	Mejores tiempos totales en dispositivo móvil	37
4.16	Mejores tiempos totales en dispositivo móvil	38
A.1	Esquema de encapsulación con ML-KEM (Kyber)	50

Lista de tablas

2.1	Comparación cualitativa de familias de algoritmos post-cuánticos	6
2.2	Niveles de seguridad en esquemas criptográficos	7
4.4	Resultados en PC para los algoritmos de 3ª y 4ª ronda	35

4.5 Resultados en Raspberry Pi para los algoritmos de 3ª y 4ª ronda.....	37
Tabla B.1. Resultados del algoritmo BIKE (C) en ordenador	54
Tabla B.2. Resultados del algoritmo Dilithium (C) en ordenador	55
Tabla B.3. Resultados del algoritmo Falcon (C) en ordenador	56
Tabla B.4. Resultados del algoritmo HQC (C) en ordenador	57
Tabla B.5. Resultados del algoritmo Kyber (C) en ordenador	58
Tabla B.6. Resultados del algoritmo SPHINCS+ (C) en ordenador - Parte 1	59
Tabla B.7. Resultados del algoritmo SPHINCS+ (C) en ordenador - Parte 2	60
Tabla B.8. Resultados del algoritmo SPHINCS+ (C) en ordenador - Parte 3	61
Tabla B.9. Resultados del algoritmo Classic McEliece (C) en ordenador - Parte 1	62
Tabla B.10. Resultados del algoritmo Classic McEliece (C) en ordenador - Parte 2	63
Tabla B.11. Resultados del algoritmo Classic McEliece (C) en ordenador - Parte 3	64
Tabla B.12. Resultados del algoritmo BIKE (C#) en ordenador	65
Tabla B.13. Resultados del algoritmo Dilithium (C#) en ordenador	66
Tabla B.14. Resultados del algoritmo Falcon (C#) en ordenador	67
Tabla B.15. Resultados del algoritmo HQC (C#) en ordenador	68
Tabla B.16. Resultados del algoritmo Kyber (C#) en ordenador.....	69
Tabla B.17. Resultados del algoritmo Classic McEliece (C#) en ordenador - Parte 1	70
Tabla B.18. Resultados del algoritmo Classic McEliece (C#) en ordenador - Parte 2	71
Tabla B.19. Resultados del algoritmo SPHINCS+ (C#) en ordenador - Parte 1	72
Tabla B.20. Resultados del algoritmo SPHINCS+ (C#) en ordenador - Parte 2.....	73
Tabla B.21. Resultados del algoritmo BIKE (Java) en ordenador.....	74
Tabla B.22. Resultados del algoritmo Dilithium (Java) en ordenador.....	75
Tabla B.23. Resultados del algoritmo Falcon (Java) en ordenador	76
Tabla B.24. Resultados del algoritmo HQC (Java) en ordenador	77
Tabla B.25. Resultados del algoritmo Kyber (Java) en ordenador	78
Tabla B.26. Resultados del algoritmo Classic McEliece (Java) en ordenador - Parte 1	79
Tabla B.27. Resultados del algoritmo Classic McEliece (Java) en ordenador - Parte 2	80
Tabla B.28. Resultados del algoritmo SPHINCS+ (Java) en ordenador - Parte 1	81
Tabla B.29. Resultados del algoritmo SPHINCS+ (Java) en ordenador - Parte 2	82
Tabla B.30. Resultados del algoritmo SPHINCS+ (Java) en ordenador - Parte 3	83
Tabla B.31. Resultados del algoritmo BIKE (Python) en ordenador.....	84
Tabla B.32. Resultados del algoritmo Falcon (Python) en ordenador	85
Tabla B.33. Resultados del algoritmo HQC (Python) en ordenador.....	86
Tabla B.34. Resultados del algoritmo Kyber (Python) en ordenador	87
Tabla B.35. Resultados del algoritmo Classic McEliece (Python) en ordenador - Parte 1	88
Tabla B.36. Resultados del algoritmo Classic McEliece (Python) en ordenador - Parte 2	89
Tabla B.37. Resultados del algoritmo SPHINCS+ (Python) en ordenador - Parte 1	90

Tabla B.38. Resultados del algoritmo SPHINCS+ (Python) en ordenador - Parte 2	91
Tabla B.39. Resultados del algoritmo Dilithium (Python) en ordenador	92
Tabla B.40. Recursos de memoria y CPU - Parte 1.	93
Tabla B.41. Recursos de memoria y CPU - Parte 2.	94
Tabla B.42. Recursos de memoria y CPU - Parte 3.	95
Tabla B.43. Recursos de memoria y CPU - Parte 4.	96
Tabla B.44. Recursos de memoria y CPU - Parte 5.	97
Tabla B.45. Recursos de memoria y CPU - Parte 6.	98
Tabla B.46. Recursos de memoria y CPU - Parte 7.	99
Tabla B.47. Recursos de memoria y CPU - Parte 8.	100
Tabla B.48. Recursos de memoria y CPU - Parte 9.	101
Tabla C.1. Resultados del algoritmo BIKE (C#) en Raspberry Pi 5	104
Tabla C.2. Resultados del algoritmo Dilithium (C#) en Raspberry Pi 5	105
Tabla C.3. Resultados del algoritmo FALCON (C#) en Raspberry Pi 5	106
Tabla C.4. Resultados del algoritmo HQC (C#) en Raspberry Pi 5	107
Tabla C.5. Resultados del algoritmo Kyber (C#) en Raspberry Pi 5	108
Tabla C.6. Resultados del algoritmo Classic McEliece (C#) - Parte 1 en Raspberry Pi 5	109
Tabla C.7. Resultados del algoritmo Classic McEliece (C#) - Parte 2 en Raspberry Pi 5	110
Tabla C.8. Resultados del algoritmo SPHINCS+ (C#) - Parte 1 en Raspberry Pi 5	111
Tabla C.9. Resultados del algoritmo SPHINCS+ (C#) - Parte 2 en Raspberry Pi 5	112
Tabla C.10. Resultados del algoritmo Dilithium (C) en Raspberry Pi 5	113
Tabla C.11. Resultados del algoritmo Falcon (C) en Raspberry Pi 5	114
Tabla C.12. Resultados del algoritmo Kyber (C) en Raspberry Pi 5	115
Tabla C.13. Resultados del algoritmo SPHINCS+ (C) - Parte 1 en Raspberry Pi 5	116
Tabla C.14. Resultados del algoritmo SPHINCS+ (C) - Parte 2 en Raspberry Pi 5	117
Tabla C.15. Resultados del algoritmo BIKE (C) en Raspberry Pi 5	118
Tabla C.16. Resultados del algoritmo HQC (C) en Raspberry Pi 5	119
Tabla C.17. Resultados del algoritmo Classic McEliece (C) - Parte 1 en Raspberry Pi 5	120
Tabla C.18. Resultados del algoritmo Classic McEliece (C) - Parte 2 en Raspberry Pi 5	121
Tabla C.19. Resultados del algoritmo BIKE (Java) en Raspberry Pi 5	122
Tabla C.20. Resultados del algoritmo Dilithium (Java) en Raspberry Pi 5	123
Tabla C.21. Resultados del algoritmo Falcon (Java) en Raspberry Pi 5	124
Tabla C.22. Resultados del algoritmo Kyber (Java) en Raspberry Pi 5	125
Tabla C.23. Resultados del algoritmo Classic McEliece (Java) - Parte 1 en Raspberry Pi 5 ..	126
Tabla C.24. Resultados del algoritmo Classic McEliece (Java) - Parte 2 en Raspberry Pi 5 ..	127
Tabla C.25. Resultados del algoritmo SPHINCS+ (Java) - Parte 1 en Raspberry Pi 5	128
Tabla C.26. Resultados del algoritmo SPHINCS+ (Java) - Parte 2 en Raspberry Pi 5	129
Tabla C.27. Resultados del algoritmo HQC (Java) en Raspberry Pi 5	130

Tabla C.28. Resultados del algoritmo BIKE (Python) en Raspberry Pi 5	131
Tabla C.29. Resultados del algoritmo Dilithium (Python) - Parte 1 en Raspberry Pi 5	132
Tabla C.30. Resultados del algoritmo Dilithium (Python) - Parte 2 en Raspberry Pi 5	133
Tabla C.31. Resultados del algoritmo Falcon (Python) en Raspberry Pi 5	134
Tabla C.32. Resultados del algoritmo HQC (Python) en Raspberry Pi 5	135
Tabla C.33. Resultados del algoritmo Kyber (Python) en Raspberry Pi 5	136
Tabla C.34. Resultados del algoritmo Classic McEliece (Python) - Parte 1 en Raspberry Pi 5	137
Tabla C.35. Resultados del algoritmo Classic McEliece (Python) - Parte 2 en Raspberry Pi 5	138
Tabla C.36. Resultados del algoritmo Classic McEliece (Python) - Parte 3 en Raspberry Pi 5	139
Tabla C.37. Resultados del algoritmo SPHINCS+ (Python) - Parte 1 en Raspberry Pi 5	140
Tabla C.38. Resultados del algoritmo SPHINCS+ (Python) - Parte 2 en Raspberry Pi 5	141
Tabla C.39. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 - Parte 1	142
Tabla C.40. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 - Parte 2	143
Tabla C.41. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 - Parte 3	144
Tabla C.42. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 - Parte 4	145
Tabla C.43. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 - Parte 5	146
Tabla C.44. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 - Parte 6	147
Tabla C.45. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 - Parte 7	148
Tabla D.1. Resultados del algoritmo BIKE (Kotlin) en dispositivo Android	150
Tabla D.2. Resultados del algoritmo Dilithium (Kotlin) en dispositivo Android	151
Tabla D.3. Resultados del algoritmo Falcon (Kotlin) en dispositivo Android	152
Tabla D.4. Resultados del algoritmo HQC (Kotlin) en dispositivo Android	153
Tabla D.5. Resultados del algoritmo Kyber (Kotlin) en dispositivo Android	154
Tabla D.6. Resultados del algoritmo Classic McEliece (Kotlin) en dispositivo Android	155
Tabla D.7. Resultados del algoritmo SPHINCS+ (Kotlin) en dispositivo Android	156
Tabla D.8. Consumo de recursos (CPU y memoria) en Kotlin en dispositivo Android	157

INTRODUCCIÓN

1.1. Motivación

La criptografía post-cuántica surge como respuesta a la amenaza que los ordenadores cuánticos representan para los sistemas clásicos, ya que algoritmos como RSA (Rivest, Shamir, Adleman) o ECDSA (*Elliptic Curve Digital Signature Algorithm*, Algoritmo de Firma Digital con Curvas Elípticas) serían vulnerables ante técnicas como las de Shor o Grover. Este trabajo analiza los algoritmos estandarizados en la tercera ronda del NIST (Instituto Nacional de Estándares y Tecnología de Estados Unidos), (Kyber, Dilithium, FALCON y SPHINCS+) junto con los candidatos de la cuarta (BIKE, HQC y Classic McEliece), permitiendo comparar su rendimiento y nivel de madurez.

1.2. Objetivos

El proyecto compara diversas propuestas de criptografía post-cuántica del proceso de estandarización del NIST, abarcando tanto los finalistas de la tercera ronda como los candidatos de la cuarta. Se evalúan en detalle las operaciones de cada esquema, generación de claves, encapsulación, desencapsulación, firma y verificación, midiendo tiempos y consumo de recursos para identificar las fases más costosas y las más eficientes. La evaluación se realiza en distintos tipos de hardware, así como implementaciones con distintos lenguajes de programación. En conjunto, se ofrece una perspectiva práctica que facilite valorar su viabilidad para integrarse de forma efectiva en sistemas reales cara a su futura adopción en sistemas de comunicación y seguridad.

1.3. Fases de realización

En la figura 1.1 se muestran las fases en las que se ha estructurado el desarrollo de este Trabajo de Fin de Grado (TFG), reflejando temporalmente las tareas realizadas y su planificación.

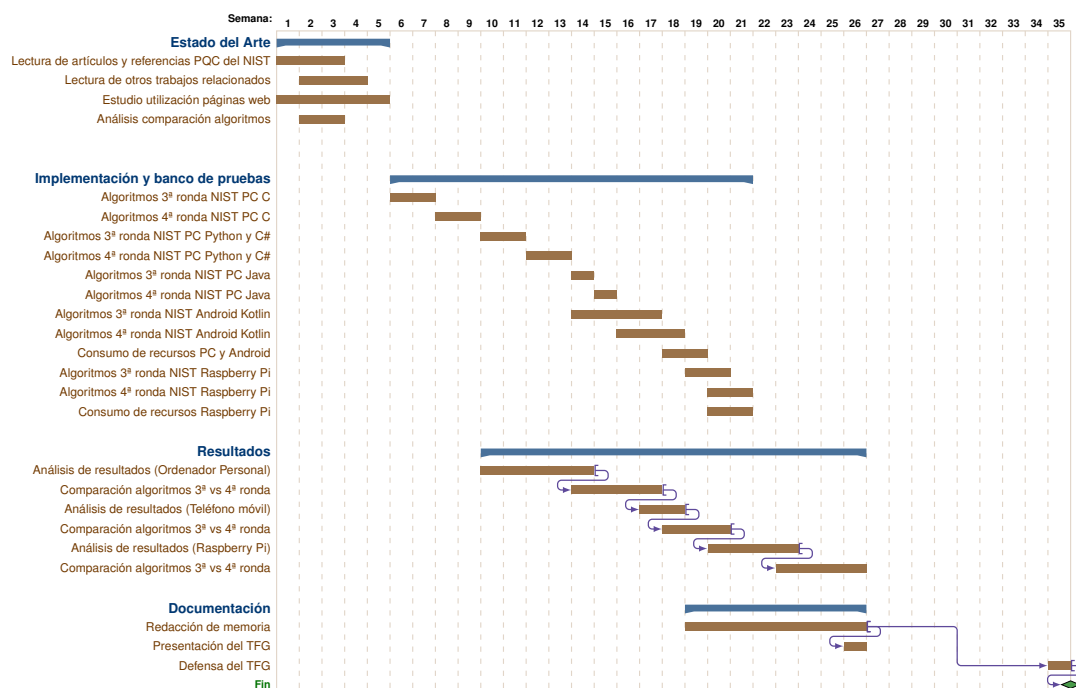


Figura 1.1. Diagrama de Gantt de la realización del TFG

1.4. Estructura de la memoria

El resto de la memoria se organiza de la siguiente manera:

En el capítulo 2 se presenta el estado del arte, revisando los fundamentos de la criptografía post-cuántica, la necesidad de su adopción, el proceso de estandarización del NIST, los algoritmos seleccionados en la tercera ronda y los candidatos de la cuarta ronda, así como los aspectos de seguridad asociados y los trabajos relacionados existentes.

El capítulo 3 describe la metodología y el entorno experimental, detallando los sistemas utilizados, los lenguajes y compiladores, las bibliotecas criptográficas empleadas, la adaptación del código para las mediciones y el procedimiento seguido para evaluar tanto rendimiento como consumo de recursos.

Tras ello, el capítulo 4 expone los resultados obtenidos y su análisis, diferenciando los escenarios de ordenador personal, Raspberry Pi 5 y teléfono móvil, incluyendo la comparación entre rondas del NIST, el estudio por operación y la evaluación del uso de recursos, junto con una síntesis comparativa general.

Finalmente, el capítulo 5 recoge las conclusiones finales del trabajo y las posibles líneas de trabajo futuro, destacando las aportaciones del estudio y las direcciones que podrían explorarse para continuar este estudio.

ESTADO DEL ARTE

2.1. Introducción

Este capítulo introduce la criptografía post-cuántica, explica por qué RSA y ECC (*Elliptic Curve Cryptography*, Criptografía de Curvas Elípticas) dejan de ser seguros frente a ordenadores cuánticos y resume las principales familias resistentes. También presenta el proceso de estandarización del NIST, su cronología y criterios, y contextualiza los finalistas de la tercera ronda y los candidatos de la cuarta. Finalmente, expone sus implicaciones prácticas en tamaños y rendimiento, estableciendo la base para los análisis experimentales que se desarrollan más adelante.

2.2. La necesidad de la criptografía post-cuántica y su estandarización

La criptografía post-cuántica (*Post-Quantum Cryptography*, PQC) es la rama de la criptografía moderna diseñada para resistir adversarios con capacidad cuántica. Igual que la clásica, emplea algoritmos y claves para proteger confidencialidad, integridad y autenticidad en múltiples entornos, pero se apoya en supuestos de dureza diferentes: problemas matemáticos que se consideran intratables tanto para ordenadores convencionales como para los cuánticos (por ejemplo, estructuras de retículas o códigos). Surge como respuesta directa a las amenazas introducidas por la computación cuántica y busca garantizar que los sistemas actuales y futuros sigan siendo seguros sin depender de RSA o ECC.

La convocatoria inicial del NIST se lanzó para preparar con antelación estándares públicos que resistan a futuros ordenadores cuánticos y sustituyan a RSA o ECC antes de que el riesgo sea operativo: ya en IR 8105 (2016) NIST fijó como objetivo disponer de criptografía segura frente a computadores clásicos y cuánticos, interoperable con protocolos existentes, y delineó su plan inicial para avanzar en este campo [1]. Además, NIST venía advirtiendo de la urgencia por el escenario “*harvest now, decrypt later*”, adversarios que capturan y almacenan hoy para descifrar mañana, y del tiempo largo de mi-

gración que implica cambiar infraestructuras criptográficas a escala gubernamental e industrial [2]. Por eso, en diciembre de 2016 formalizó una llamada de propuestas con requisitos y criterios de evaluación (seguridad, rendimiento, implementabilidad, diversidad), y un proceso por rondas, de 3–5 años, abierto a la revisión pública [3]. En la cronología oficial se recoge este arranque: anuncio del plan en 2016, publicación de IR 8105, Call for Proposals el 20 de diciembre de 2016 y recepción de candidaturas hasta el 30 de noviembre de 2017, pasos que encaminaron el proceso de estandarización que culminaría en los FIPS de 2024 [4].

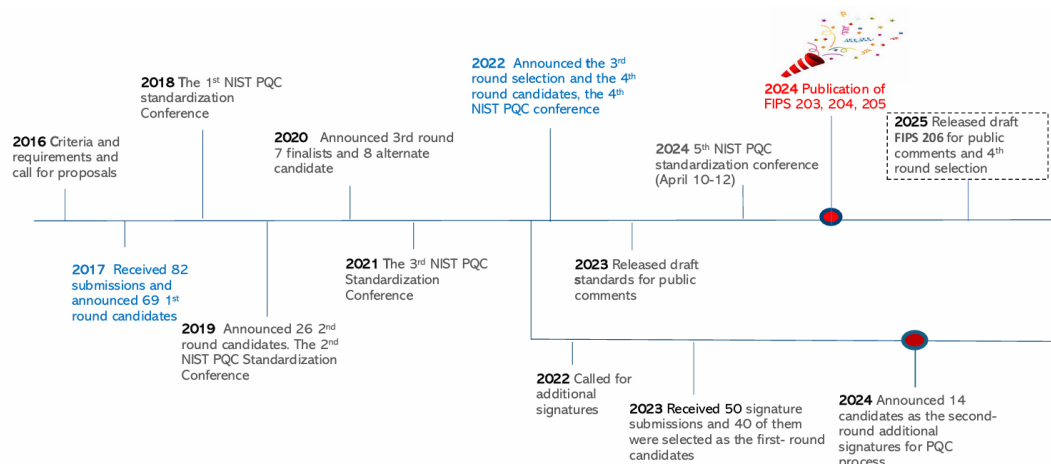


Figura 2.1. Línea temporal del proceso de estandarización PQC del NIST (2016-2025) [5].

En la figura 2.1 se puede ver como, desde 2016, NIST abrió la llamada y fijó criterios (IR 8105), recibió 82 propuestas en 2017 y redujo a 69 candidatas de 1^a ronda. En 2019 pasó a 26 candidatas (2^a ronda) y en 2020 anunció 7 finalistas y 8 alternativos (3^a ronda). En 2022 seleccionó a los primeros estándares (Kyber, Dilithium, SPHINCS+ y el camino para FALCON) y arrancó la 4^a ronda de KEM no reticulares. En 2023 publicó borradores para comentarios y, en 2024, los FIPS 203–205; para 2025 queda el cierre de la 4^a ronda (HQC y FIPS 206) y la continuación del proceso adicional de firmas.

2.3. Algoritmos finalistas de la tercera ronda

Durante la tercera ronda del NIST (2019–2022) se evaluaron algoritmos finalistas y alternativos, agrupados por la familia matemática en la que basan su seguridad. En la figura 2.2 se visualiza esta clasificación, donde se observa el peso principal de los esquemas basados en retículas y la presencia de representantes de otras familias.

Basados en retículas. Estos esquemas se apoyan en problemas como el Aprendizaje con Errores (*Learning With Errors*, LWE) o el Problema del Vector más Corto (*Shortest Vector Problem*, SVP), considerados difíciles tanto para ordenadores clásicos como para cuánticos. En esta familia destacaron

CRYSTALS-Kyber (KEM) y las firmas CRYSTALS-Dilithium y FALCON, todos ellos finalistas de la ronda tres y más tarde seleccionados por NIST como candidatos a estandarización [6].

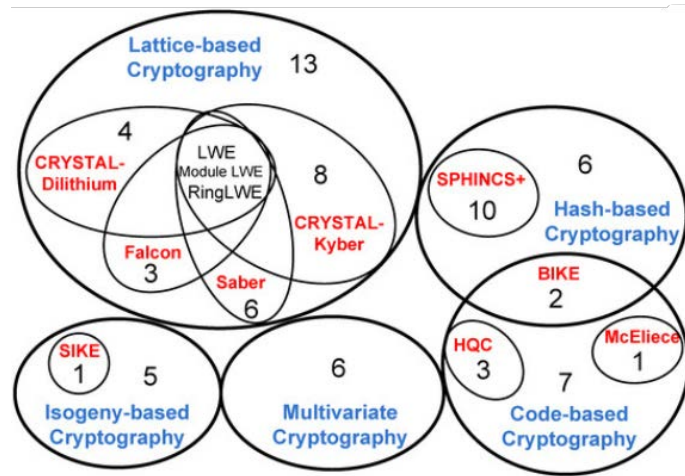


Figura 2.2. Clasificación de los esquemas de criptografía post-cuántica (PQC) por familias y candidatos representativos en el proceso NIST [7].

Basados en funciones *hash*. En este enfoque, la seguridad se fundamenta en las propiedades criptográficas de las funciones *hash*, en particular en su resistencia a colisiones y a preimágenes. La resistencia a preimágenes implica que, dado un valor *hash*, resulta computacionalmente inviable encontrar un mensaje que lo produzca, mientras que la resistencia a colisiones impide hallar dos mensajes distintos con el mismo *hash*. Estas propiedades permiten construir esquemas de firma digital seguros frente a ataques cuánticos. Un ejemplo representativo es SPHINCS+, considerado una opción conservadora para diversificar los estándares, a pesar del gran tamaño de sus firma [6].

Basados en códigos. Estos algoritmos aprovechan la dificultad de decodificar códigos lineales aleatorios sin la clave adecuada, un problema resistente desde hace décadas. El finalista fue Classic McEliece, mientras que BIKE y HQC quedaron como alternativos por su interés como esquemas no reticulares, a pesar del inconveniente de claves públicas muy grandes [6].

Basados en isogenias de curvas elípticas. Se apoyan en la complejidad de encontrar isogenias entre curvas elípticas supersingulares, lo que permitía claves públicas compactas [8]. El candidato fue SIKE, pero en agosto de 2022 un ataque práctico lo rompió, descartándolo como opción segura a largo plazo [9].

Basados en sistemas multivariantes. Su seguridad proviene de la dificultad de resolver sistemas de polinomios multivariantes sobre cuerpos finitos, un problema NP-*hard* para entradas grandes [10]. El representante fue Rainbow, que llegó a ser finalista en firmas digitales, pero fue roto en 2022 mediante un ataque que recuperaba las claves propuestas [11].

En la Tabla 2.1 se sintetizan las características principales de cada familia de algoritmos post-cuánticos, lo que permite comparar de forma rápida sus compromisos de seguridad y costes prácticos.

Característica	Hash	Isogenias	Redes	Códigos	Multivariantes
Nivel de seguridad	Alto	Moderado	Alto	Alto	Moderado
Rendimiento	Moderado	Lento	Moderado	Rápido	Lento
Tamaño de clave	Pequeño	Pequeño	Moderado	Moderado	Grande
Complejidad de implementación	Moderada	Alta	Moderada	Moderada	Alta
Adecuación a redes móviles	Moderada	Baja	Moderada	Alta	Baja

Tabla 2.1. Comparación cualitativa de familias de algoritmos post-cuánticos según seguridad, rendimiento, tamaño de claves, complejidad de implementación y adecuación a redes móviles, basado en [12].

2.4. Algoritmos candidatos de la cuarta ronda

La Ronda 4 se centró en KEM de códigos: Classic McEliece [13], BIKE y HQC. Classic McEliece aporta un historial de seguridad excelente, pero sus claves públicas muy grandes y una generación de claves lenta limitan su uso en muchos entornos, aunque la encapsulación/desencapsulación sea rápida cuando se reutiliza la clave.

BIKE utiliza códigos cuasi-cíclicos de comprobación de paridad con densidad moderada (QC-MDPC). En estos códigos, la matriz de comprobación de paridad es una tabla de 0 y 1 que sirve para detectar y corregir errores en los bits. En el caso QC-MDPC, esta matriz tiene una cantidad intermedia de unos y está formada por bloques repetidos con desplazamientos circulares, lo que permite usar claves más compactas y aplicar una decodificación iterativa por inversión de bits para corregir los errores introducidos durante el cifrado [14]. Es competitivo en tamaños y velocidad, pero su tasa de fallos de descifrado (DFR) debe mantenerse estrictamente acotada para evitar vectores de ataque, motivo de evaluación intensa.

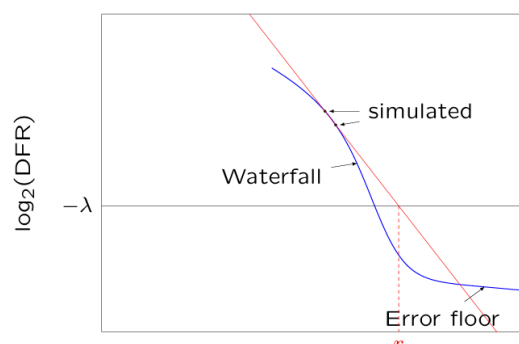


Figura 2.3. BIKE: estimaciones de la tasa de fallo de descifrado (DFR) frente a la longitud de bloque. La curva simulada (azul) muestra la región *waterfall* y el *error floor*; la línea guía (roja) extrapola el comportamiento, y se indica la cota $2^{-\lambda}$ y el punto r a partir del cual $\text{DFR} \leq 2^{-\lambda}$, basado en [15]

La figura 2.3 muestra que en BIKE puede haber fallos de descifrado (*Decrypt Failure Rate*, DFR), y que esa probabilidad depende del tamaño del bloque del código. Cuando el bloque es pequeño, los fallos son frecuentes. A medida que aumentas el tamaño del bloque, la curva baja muy rápido (zona

“waterfall”) y los fallos se vuelven muy poco probables. A partir de cierto punto r , los fallos son tan raros que quedan por debajo del umbral de seguridad exigido $2^{-\lambda}$. Aun así, siempre queda un “suelo” de error (*error floor*), es decir, una probabilidad mínima que no baja más aunque el bloque siga creciendo. La línea roja de la figura es una aproximación teórica: sirve para extrapolar el comportamiento de la curva azul (la simulada) y estimar dónde caerá la tasa de fallos cuando ya no se pueden hacer más simulaciones prácticas (porque fallan tan poco que sería imposible medirlo en un tiempo razonable).

HQC ofrece un equilibrio entre tamaños, rendimiento y análisis de seguridad (incluida la gestión de fallos), y por ello fue seleccionado por el NIST como KEM adicional de reserva; a nivel teórico, sus keygen/encaps/decaps son equilibrados y los parámetros propuestos facilitan el despliegue práctico. En resumen: McEliece maximiza confianza histórica a costa de tamaño; BIKE optimiza eficiencia condicionada a controlar la DFR; HQC equilibra seguridad y despliegue, justificando su elección en la Ronda 4 [16].

2.5. Seguridad de los algoritmos post-cuánticos

2.5.1. Niveles de seguridad del NIST

El NIST definió cinco niveles de seguridad para su proceso de estandarización de criptografía post-cuántica. En la práctica, los más usados son Nivel 1, Nivel 3 y Nivel 5, y se comparan con la fuerza bruta contra AES (*Advanced Encryption Standard* (Tabla 2.2):

Categoría NIST	Equivalencia	Descripción
Nivel 1	AES-128	Seguridad básica. Equivalente a la dificultad de romper AES con una clave de 128 bits.
Nivel 3	AES-192	Seguridad intermedia. Equivalente a la resistencia de AES-192 frente a ataques. Recomendado para aplicaciones con requisitos elevados.
Nivel 5	AES-256	Seguridad alta y robusta frente a adversarios avanzados. Equivalente a la seguridad proporcionada por AES-256.

Tabla 2.2. Niveles de seguridad en esquemas criptográficos, basado en [17]

Nivel 1 ($\approx$ AES-128): Se considera la seguridad básica. Un adversario necesitaría un esfuerzo equivalente al de romper AES con clave de 128 bits. Es suficiente para la mayoría de aplicaciones generales en Internet (p.ej., TLS, *Transport Layer Security*).

Nivel 3 ($\approx$ AES-192): Seguridad intermedia, adecuada para entornos con requisitos más altos (p.ej., infraestructuras críticas, defensa). La equivalencia con AES-192 implica que un ataque requiere un

esfuerzo exponencial aún mayor.

Nivel 5 ($\approx$ AES-256): Máxima categoría definida por NIST. Es la seguridad “para adversarios estatales avanzados” o para datos que deban mantenerse confidenciales durante décadas. Equivalente al coste de romper AES-256.

ML-KEM (FIPS 203) ofrece parámetros para Nivel 1, 3 y 5. Por Ejemplo: Kyber-512 $\approx$ Nivel 1, Kyber-768 $\approx$ Nivel 3, Kyber-1024 $\approx$ Nivel 5. ML-DSA (FIPS 204), también con parámetros para niveles 1, 3 y 5. SLH-DSA (FIPS 205), sus parámetros equivalen a Niveles 1, 3 y 5, aunque con tamaños de firma más grandes. Además, HQC también tiene opciones de Nivel 1, 3 y 5.

2.5.2. Fundamentos de seguridad en los estándares FIPS 203, 204 y 205, y su diversificación con HQC

Desde 2024, el NIST ha elevado a estándares FIPS tres pilares de la PQC: ML-KEM (KEM derivado de Kyber), ML-DSA (firmas derivadas de Dilithium) y SLH-DSA (firmas hash-basadas, derivadas de SPHINCS+) [18]. El estándar FIPS 203 (que formaliza a Kyber como ML-KEM) afirma que la seguridad del esquema se reduce a la dificultad del problema Module-LWE. Eso significa que, si alguien pudiera romper ML-KEM (recuperar la clave secreta o descifrar sin permiso), entonces también podría resolver instancias del problema Module-LWE. Además, como no existe ni ataque clásico eficiente ni algoritmo cuántico tipo Shor/Grover que lo resuelva, hoy se considera resistente incluso frente a un atacante con ordenador cuántico [19].

En paralelo, FIPS 204 recoge que ML-DSA está pensado para resistir también a adversarios con capacidad cuántica, manteniendo la funcionalidad de firma y verificación en entornos reales [20]. Por su parte, FIPS 205 fija la estandarización de SLH-DSA, apoyada únicamente en la resistencia de las funciones *hash*, lo que facilita razonar su seguridad ante mejoras cuánticas tipo Grover mediante el aumento de tamaños de salida [21].

El NIST, en su informe de la ronda 3 (NISTIR 8413), decidió convertir en estándar a Kyber y a las firmas Dilithium, FALCON y SPHINCS+ porque superaron las pruebas; el resto de candidatos pasó a una ronda 4 para seguir evaluándolos [22].

En ronda 4, NIST analizó BIKE, Classic McEliece, HQC y SIKE como KEM alternativos; HQC fue finalmente seleccionado para estandarización adicional, consolidando una diversificación más allá de las retículas [23].

2.5.3. Ataques clásicos y cuánticos

En el plano clásico, la seguridad de los esquemas de retículas (Kyber, Dilithium, Falcon) se apoya en los problemas LWE y SIS; su resolución eficiente es desconocida y se considera intratable con los métodos actuales [24].

Para romper LWE/SIS necesitas usar BKZ (*Block Korkine–Zolotarev*), algoritmo que, su objetivo es encontrar patrones “cortos” o “simples” dentro de un problema matemático muy grande. Si los encuentra, el sistema se rompe. Pero BKZ se vuelve exponencialmente más caro cuanto más lo potencias. Los parámetros de Kyber/Dilithium se eligen para que necesites un BKZ tan grande que el esfuerzo supere $\approx 2^{128}$ operaciones (nivel AES-128) [25].

En los sistemas basados en códigos (Classic McEliece, HQC, BIKE), la seguridad depende de lo difícil que es descifrar un código sin su estructura. En HQC y BIKE existe una pequeña posibilidad de que el descifrado falle (DFR), y es crucial controlarla. HQC tiene un análisis más sólido y preciso de estos fallos que BIKE, y por eso NIST lo prefirió en la Ronda 4 [26].

Además de ser matemáticamente fuertes, deben estar protegidos contra ataques de canal lateral, como los que miden el tiempo de ejecución, consumo de potencia o emisiones electromagnéticas, y contra ataques por fallos. Estos ataques no rompen el problema matemático, pero sí pueden revelar la clave si la implementación filtra información o se comporta de forma diferente bajo condiciones adversas [27].

Un ordenador cuántico con un número suficiente de cúbits podría romper RSA y ECC rápidamente gracias al algoritmo de Shor, mientras que el algoritmo de Grover hace que los ataques de fuerza bruta sean mucho más rápidos, reduciendo el esfuerzo de 2^n a $\approx 2^{n/2}$ [28].

2.6. Trabajos relacionados

Diversos trabajos previos han analizado el rendimiento y la viabilidad de la criptografía post-cuántica en diferentes plataformas y contextos. A continuación se describen brevemente y se señalan sus principales limitaciones, lo que motiva y justifica el alcance de este TFG.

El Trabajo de Fin de Grado de José Alberto Benito Corral, junto con su posterior publicación en JITEL [29], constituye uno de los estudios más completos sobre el rendimiento de los algoritmos estandarizados por el NIST en la tercera ronda. Evalúa Kyber/ML-KEM, Dilithium/ML-DSA, Falcon/FN-DSA y SPHINCS+/SLH-DSA en diferentes lenguajes y dispositivos PC, Raspberry Pi 5 y teléfono Android, midiendo tiempos de operación y tamaños de claves/firma. Sin embargo, su principal limitación es que no incluye los algoritmos candidatos de la cuarta ronda del NIST.

El artículo publicado en Internet of Things, realizado por Luis Cruz-Piris, Andrés Marín-López, Ma-

nuel Álvarez-Campana, Mario Sanz, José Ignacio Moreno y David Arroyo [30], realiza también un análisis de rendimiento de algoritmos post-cuánticos, centrado exclusivamente en dispositivos Raspberry Pi 2, 3 y 4. Su estudio es valioso para evaluar entornos embebidos y dispositivos con recursos limitados. Como limitación, los autores no incluyen la Raspberry Pi 5, ya que su consumo energético, superior a 5 W, hace que no pueda considerarse un dispositivo de baja potencia en el contexto analizado.

Finalmente, el trabajo redactado por José A. Montenegro, Rubén Ríos y Javier López-Cerezo, del NICS Lab de la Universidad de Málaga [31], analiza la integración de criptografía post-cuántica en TLS y evalúa el impacto de usar algoritmos clásicos, híbridos y post-cuánticos durante el *handshake*. El rendimiento se mide mediante pruebas de OpenSSL que contabilizan el número de operaciones ejecutadas en intervalos de 10 segundos, además de los *handshakes* completados y los datos transmitidos. Como limitación, el estudio no especifica la plataforma hardware utilizada en las mediciones, ni realiza una evaluación comparativa entre dispositivos heterogéneos (PC, ARM, móviles), lo que dificulta extrapolar sus resultados a distintos entornos.

En conjunto, estos trabajos proporcionan una base sólida, pero dejan abierta la necesidad de un estudio actualizado que compare algoritmos de cuarta ronda frente a los de la tercera en plataformas modernas y heterogéneas. Este TFG aborda precisamente esa carencia, ampliando y continuando el trabajo previo de J.A. Benito Corral y complementando la perspectiva centrada en TLS del trabajo de la U. Málaga, con el fin de ofrecer una evaluación más completa y actualizada del estado de la criptografía post-cuántica.

2.7. Conclusión

La criptografía post-cuántica se consolida como respuesta directa a la amenaza que la computación cuántica plantea sobre RSA y ECC, especialmente por el algoritmo de Shor. El proceso del NIST, desde IR 8105 hasta los FIPS 203–205 (figura 2.1), ha fijado criterios de seguridad y rendimiento y ha seleccionado un núcleo estable: ML-KEM (Kyber), ML-DSA (Dilithium) y SLH-DSA (SPHINCS+), manteniendo, además, una vía de diversificación con HQC en Ronda 4.

El panorama por familias (resumido en la tabla 2.1) muestra perfiles de riesgo y coste complementarios: retículas con la mejor relación eficiencia-seguridad; firmas *hash*-basadas con máxima robustez a cambio de tamaños; y códigos (Classic McEliece, BIKE, HQC) que aportan independencia estructural, con atención especial a la DFR y su acotación (figura 2.3). Estos diseños cubren los niveles de seguridad NIST 1, 3 y 5 (tabla 2.2) y ofrecen parámetros prácticos para distintos entornos.

METODOLOGÍA Y ENTORNO

EXPERIMENTAL

3.1. Introducción

Este capítulo describe la metodología y el entorno experimental empleados para comparar, por operación, distintos algoritmos post-cuánticos en múltiples plataformas y lenguajes de programación, atendiendo a métricas de tiempo de ejecución, consumo de memoria y viabilidad práctica. El conjunto evaluado incluye CRYSTALS-Kyber/ML-KEM, CRYSTALS-Dilithium/ML-DSA, FALCON/FN-DSA y SPHINCS+/SLH-DSA, así como HQC, BIKE y Classic McEliece. No se consideran algoritmos clásicos como RSA o ECDSA, ya analizados en [29].

El flujo metodológico seguido se resume en la figura 3.1 e incluye: la selección de algoritmos (tercera y cuarta ronda del proceso NIST), su implementación en distintos lenguajes (C, C#, Java, Python y Kotlin) y plataformas (PC, Raspberry Pi y dispositivo móvil), la ejecución de pruebas repetidas - aproximadamente 1000 iteraciones por versión, con excepciones en esquemas más costosos como McEliece o SPHINCS+- y la extracción de métricas estadísticas (media, desviación estándar, tiempo por operación y consumo de CPU y memoria).

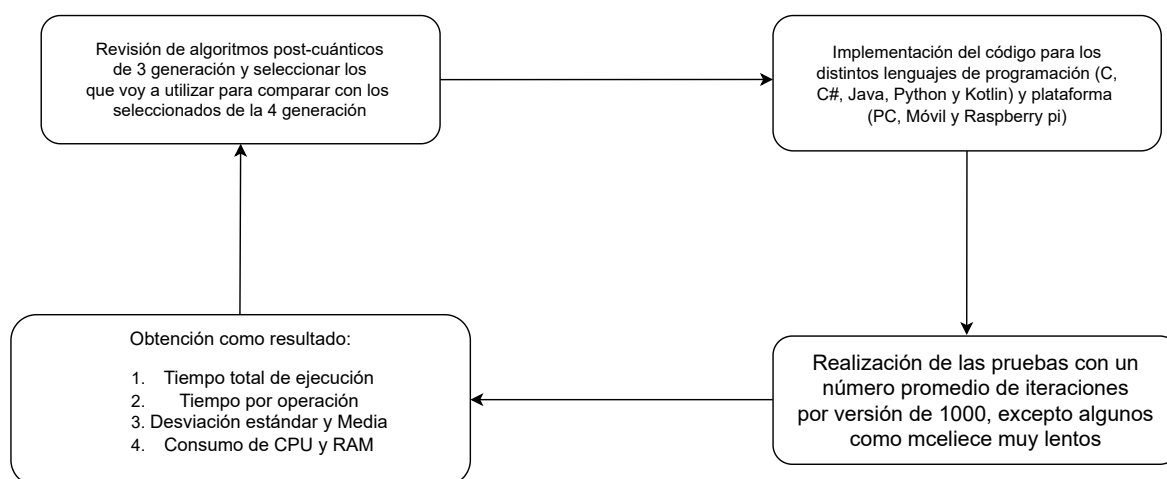


Figura 3.1. Diagrama de desarrollo

3.1.1. Descripción de los sistemas de prueba

Con el objetivo de evaluar la portabilidad, escalabilidad y eficiencia de los algoritmos post-cuánticos en distintos entornos, se emplearon tres plataformas hardware representativas:

Ordenador personal. Equipo portátil con procesador Intel® Core™ i7-10510U (4 núcleos / 8 hilos, hasta 4.9 GHz) y 16 GB de memoria RAM DDR4 a 2400 MT/s, ejecutando Ubuntu 20.04 LTS.

Raspberry Pi 5. Dispositivo basado en un procesador ARM Cortex-A76 de 4 núcleos a 2.4 GHz y 8 GB de memoria LPDDR4X-4267, con sistema operativo Debian GNU/Linux 12.

Teléfono móvil. Las pruebas en entorno móvil se realizaron sobre un Samsung Galaxy A36 5G, equipado con un procesador Qualcomm Snapdragon 6 Gen 3 (arquitectura ARM de 64 bits) y 8 GB de memoria RAM, ejecutando Android 15 bajo la capa One UI 7. El desarrollo se realizó mediante Android Studio Electric Eel, empleando Kotlin (versión 1.9.24).

Estos tres perfiles permiten analizar la portabilidad del código entre arquitecturas heterogéneas, así como la escalabilidad del rendimiento desde sistemas de escritorio hasta dispositivos con restricciones energéticas y térmicas.

3.1.2. Lenguajes de programación y compiladores

Para garantizar la comparabilidad de los resultados, se fijaron las versiones de compiladores e intérpretes empleadas en cada plataforma:

- **PC (Ubuntu 20.04 LTS):** GCC 9.4.0, .NET SDK 7.0.410, Java 21.0.2 LTS y Python 3.11.4.
- **Raspberry Pi 5 (Debian 12):** GCC 12.2.0, .NET SDK 8.0.405, Java 17.0.13 y Python 3.11.2.
- **Teléfono móvil (Android 15):** Kotlin 1.9.24.

En todos los casos se utilizaron los mismos parámetros criptográficos por algoritmo y nivel de seguridad, registrando los tiempos por operación y las estadísticas agregadas para su posterior análisis comparativo.

3.2. Implementaciones y librerías criptográficas

Para la ejecución de los algoritmos evaluados se emplearon tanto implementaciones de referencia oficiales, publicadas en el marco del proceso de estandarización de criptografía post-cuántica del NIST, como bibliotecas ampliamente utilizadas en la práctica y verificadas por la comunidad criptográfica. En determinados casos, se recurrió además a implementaciones prácticas procedentes de fuentes técnicas reconocidas, como AsecuritySite [32], con el objetivo de facilitar la validación experimental y la reproducibilidad de los resultados.

En el caso de los esquemas de la tercera ronda -Kyber, Dilithium, Falcon y SPHINCS+- se utilizaron las versiones oficiales mantenidas por sus respectivos equipos de desarrollo: Kyber [33], Dilithium [34], Falcon [35] y SPHINCS+ [36]. Asimismo, se tomó como referencia el trabajo previo de José Alberto Benito [37], cuyo repositorio sirvió como guía para la organización del código y la integración multilenguaje.

Java. Las implementaciones en Java se desarrollaron utilizando la biblioteca Bouncy Castle, en sus módulos `bouncycastle.pqc.crypto` y `bouncycastle.pqc.jcajce`, que proporcionan soporte nativo para los algoritmos post-cuánticos estandarizados por el NIST.

- **BIKE, Kyber y McEliece:** implementados mediante las interfaces `KEMGenerateSpec`, `KEMExtractSpec` y la clase `SecretKeyWithEncapsulation`.
- **Dilithium:** mediante el proveedor `net.thiim.dilithium`, compatible con el estándar final FIPS 204.
- **Falcon, HQC y SPHINCS+:** utilizando los generadores de claves y parámetros disponibles en el paquete `org.bouncycastle.pqc.crypto`.

Python. Las implementaciones en Python se basaron principalmente en la interfaz de `Open Quantum Safe`, a través del módulo `oqs`, que permite invocar de forma directa los KEM y esquemas de firma definidos por el NIST.

C#. En C# se empleó la biblioteca Bouncy Castle (`BouncyCastle.Cryptography.dll`), manteniendo una estructura de clases coherente con la API de Java y soporte completo para los algoritmos post-cuánticos evaluados.

Kotlin/Android. Las implementaciones en Kotlin se ejecutaron en entorno Android utilizando Bouncy Castle como proveedor criptográfico, integrando las clases JCA/JCE del módulo `bouncycastle.pqc.jcajce`.

3.3. Adaptación del código y medición de rendimiento

Con el fin de evaluar el rendimiento de los algoritmos en cada entorno, las implementaciones descritas fueron adaptadas para incorporar instrumentación específica que permitiera medir los tiempos de ejecución de cada operación criptográfica: generación de claves, encapsulación y desencapsulación en los KEM, y firma y verificación en los esquemas de firma digital.

En C se partió de las implementaciones de referencia oficiales, incorporando tanto variantes *portable* como optimizadas cuando estaban disponibles. En particular, para BIKE se utilizó la implementación mantenida por *AWS Labs* [38], que permite seleccionar dinámicamente la ruta de ejecución en función de las capacidades del procesador. Para HQC se emplearon las versiones de referencia y op-

timizadas en AVX2, mientras que Classic McEliece se integró a partir del repositorio PQClean [39], permitiendo comparar directamente variantes *reference* y optimizadas.

En Python se utilizaron *bindings* y bibliotecas de terceros ampliamente validadas, principalmente Open Quantum Safe, así como implementaciones específicas para HQC y Classic McEliece [40, 41].

En Java y C# se adaptaron ejemplos oficiales de *Bouncy Castle*, utilizando mecanismos de temporización de alta resolución (`System.nanoTime()` y `Stopwatch`, respectivamente). En Kotlin/Android, las pruebas se integraron como *instrumented tests*, empleando `measureNanoTime` y controlando fases de *warm-up* para reducir la variabilidad debida a la compilación JIT y la recolección de basura.

Los resultados se almacenaron automáticamente en archivos `.csv`, que contienen los tiempos individuales, medias y desviaciones estándar, y constituyen la base para los diagramas de caja utilizados en el análisis comparativo. El código completo se encuentra disponible en el repositorio del proyecto: <https://github.com/saantiidp/TFG>.

3.4. Medición de recursos

La medición del consumo de recursos se realizó mediante herramientas específicas en cada entorno. En PC, las pruebas se automatizaron mediante scripts en `bash`, utilizando la utilidad `/usr/bin/time` para registrar el tiempo total de ejecución, el porcentaje medio de uso de CPU y la memoria máxima residente. En Android, las mediciones se llevaron a cabo mediante *Android Studio Profiler*, monitorizando el uso de CPU, memoria y eventos de recolección de basura durante la ejecución de las pruebas.

3.5. Conclusión

En este capítulo se ha descrito la metodología experimental y el entorno empleado para la evaluación comparativa de algoritmos post-cuánticos en múltiples plataformas y lenguajes. La definición precisa de los dispositivos, herramientas, bibliotecas y procedimientos de medición garantiza la reproducibilidad de los experimentos y establece una base sólida para el análisis de resultados presentado en el capítulo siguiente.

RESULTADOS Y ANÁLISIS

4.1. Introducción

En este capítulo presentaremos y analizaremos los resultados experimentales de los algoritmos post-cuánticos ejecutados en distintas plataformas, evaluando la evolución entre la tercera y la cuarta ronda del proceso de estandarización del NIST. El estudio compara el impacto de las optimizaciones recientes, como el uso de instrucciones `AVX2` y la compilación con `-march=native`.

Los resultados se organizan por plataforma (ordenador personal, Raspberry Pi 5 y teléfono móvil), y en cada caso se ofrece una comparación entre rondas y un análisis por operación (generación de claves, encapsulación, desencapsulación, firma y verificación). El capítulo concluye con una síntesis global que recoge los patrones comunes y las diferencias más relevantes entre algoritmos y entornos.

4.2. Resultados en ordenador personal

4.2.1. Comparación 3ª vs 4ª ronda

BIKE

Los resultados obtenidos para BIKE en ordenador personal muestran un comportamiento estable y un rendimiento intermedio en comparación con los algoritmos de la tercera ronda del NIST. En la implementación en C, tanto la versión Portable como la optimizada con `AVX2` presentan tiempos de ejecución muy próximos, con diferencias inferiores al 10 % en la mayoría de las operaciones. El tiempo total oscila desde 1.45 ms en el nivel de seguridad L1 hasta aproximadamente 12.5 ms en L5, como se muestra en la figura 4.1.

Este resultado muestra que las optimizaciones con instrucciones vectoriales `AVX2` no aportan mejoras importantes en el rendimiento de BIKE. Esto se puede deber a que el propio diseño del algoritmo, basado en operaciones binarias muy simples y repetitivas, ya aprovecha bien los recursos del procesador incluso sin esas extensiones.

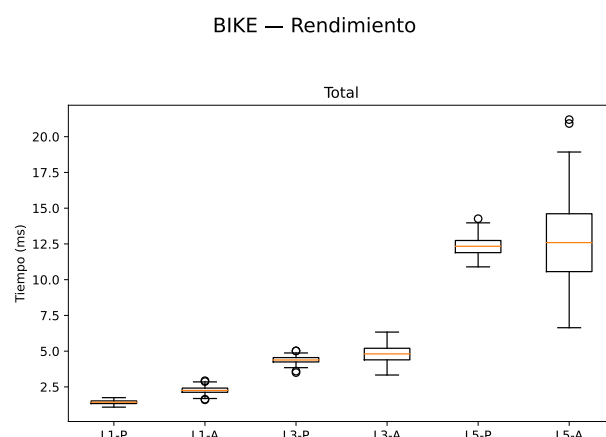


Figura 4.1. BIKE: tiempos totales en ordenador personal (C). L1-L5 indican el nivel de seguridad; P y A corresponden a las implementaciones Portable y AVX2.

En comparación con los algoritmos de la tercera ronda del NIST, BIKE se sitúa en una posición intermedia en términos de rendimiento. Frente a Kyber y Dilithium, ambos basados en retículas y con tiempos totales por debajo del milisegundo en sus versiones AVX2, BIKE resulta entre 5 y 15 veces más lento. No obstante, continúa siendo considerablemente más rápido que Falcon y SPHINCS+, cuyos tiempos pueden superar fácilmente los 25 ms y varios segundos respectivamente, especialmente en los niveles de seguridad más altos.

En los lenguajes de alto nivel, el comportamiento mantiene la misma tendencia observada en C. En C#, los tiempos aumentan de forma casi lineal con el nivel de seguridad, desde 2.19 ms en BIKE-128 hasta 15.37 ms en BIKE-256. Estos resultados lo sitúan por debajo de los obtenidos para Falcon y SPHINCS+, y ligeramente por encima de los de Kyber y Dilithium.

En Java, los tiempos son mayores, con valores que oscilan entre 36.7 ms y casi 196 ms según el nivel de seguridad. El crecimiento del tiempo de ejecución con el tamaño de clave se mantiene prácticamente lineal, aunque se observa cierta dispersión. Aun así, BIKE en Java, si bien es superado por Falcon en tiempos totales, continúa siendo más eficiente que SPHINCS+, cuyas ejecuciones superan los 3 s en los niveles de seguridad más bajos y alcanzan entre 6 y 7 s en las configuraciones de mayor seguridad.

En Python, los resultados son particularmente destacables: los tiempos totales se sitúan entre 1.5 ms y 10.6 ms, lo que convierte a BIKE en el algoritmo post-cuántico más rápido dentro de las implementaciones en este lenguaje. Estos tiempos reducidos pueden deberse a que su implementación en Python ejecuta las partes críticas en código nativo, minimizando la sobrecarga del intérprete.

En comparación, Kyber, el más rápido en todos los lenguajes, requiere entre 6 y 15 ms, mientras que Falcon y SPHINCS+ superan ampliamente el segundo de ejecución.

En conjunto, los resultados confirman que BIKE mantiene un perfil de rendimiento muy estable

entre lenguajes y niveles de seguridad. A diferencia de otros esquemas que han mejorado de forma sustancial con las instrucciones AVX2, BIKE muestra una dependencia mínima de las extensiones vectoriales.

HQC

Los resultados obtenidos para HQC en ordenador personal muestran que, a diferencia de BIKE, sí se beneficia significativamente de las optimizaciones introducidas mediante instrucciones AVX2, reflejando una mejora clara de rendimiento gracias al uso de instrucciones vectoriales.

En la implementación en C, la diferencia entre las versiones de referencia y las optimizadas es muy significativa: el tiempo total de ejecución se reduce de 1.52 ms a 0.197 ms en HQC-128, y de 8.12 ms a 0.837 ms en HQC-256, lo que supone una mejora de hasta un orden de magnitud. El crecimiento con el tamaño de clave es prácticamente lineal y las desviaciones son reducidas, lo que confirma la estabilidad del esquema y su buena escalabilidad.

En comparación con los algoritmos de tercera ronda, HQC presenta un comportamiento especialmente interesante. En su versión de referencia, HQC es más lento que Dilithium, situándose en un rango intermedio entre éste y BIKE. Sin embargo, con la optimización AVX2, el rendimiento mejora de forma drástica: HQC-128 alcanza tiempos similares a los de Kyber-512 sin vectorización y logra superar incluso a Dilithium con AVX2, manteniendo una ventaja de varios factores sobre los esquemas de firma Falcon y, especialmente, SPHINCS+, cuyos tiempos en C llegan a varios segundos. Esta diferencia evidencia que HQC se beneficia mucho más que BIKE de las optimizaciones vectoriales.

En las implementaciones en C#, HQC presenta un rendimiento modesto, con tiempos que aumentan de forma lineal desde 26.4 ms en HQC-128 hasta 127.5 ms en HQC-256. En comparación con los algoritmos de tercera ronda, HQC se sitúa claramente por detrás de los principales esquemas de encapsulación: Kyber y Dilithium completan sus operaciones en menos de 2 ms, mientras que BIKE alcanza entre 2.2 ms y 15.4 ms según el nivel de seguridad. Incluso el esquema de firma Falcon, que presenta una estructura más compleja, logra tiempos inferiores, entre 16 ms y 48 ms, lo que evidencia el elevado coste computacional de HQC en este lenguaje. Solo los esquemas de firma basados en *hash*, como SPHINCS+, presentan tiempos claramente superiores, del orden de varios segundos. Por tanto, HQC en C# se posiciona como uno de los algoritmos más lentos del conjunto.

En Java, los resultados muestran un comportamiento consistente: los tiempos totales oscilan entre 12.7 ms (HQC-128) y 67.7 ms (HQC-256). El crecimiento con el tamaño de clave se mantiene lineal y moderado como se puede observar en la figura 4.2.

En este caso, HQC es superado por Falcon, especialmente en las variantes de menor seguridad, pero se sitúa claramente por delante de BIKE, cuyos tiempos alcanzan entre 36.7 ms y 195 ms según el nivel de seguridad. Además, HQC continúa siendo mucho más rápido que SPHINCS+, cuyas

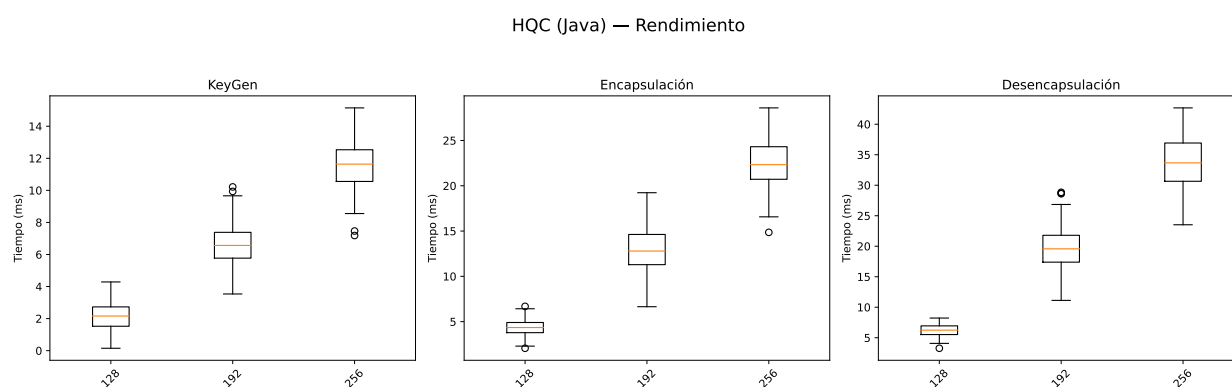


Figura 4.2. HQC tiempos por operación en PC (Java) para los niveles de seguridad 128, 192 y 256.

ejecuciones en Java se sitúan entre 3 y 7 s.

Por último, en Python, presenta tiempos más elevados, con valores totales de 46.3 ms para HQC-128 y 254.3 ms para HQC-256. El crecimiento con el nivel de seguridad es lineal y estable, aunque el incremento en HQC-256 es notable. En las configuraciones de menor seguridad, HQC-128 logra un rendimiento destacable: es claramente más rápido que Dilithium2 (62 ms) y supera con gran margen a Falcon, cuyas ejecuciones superan varios segundos. Sin embargo, al aumentar el tamaño de clave, el tiempo total se incrementa de forma significativa, y HQC-256 pasa a situarse por detrás de Dilithium5, aunque sigue siendo mucho más eficiente que los esquemas de firma basados en hash, especialmente SPHINCS+, que en Python alcanza entre 2 y 4 s de ejecución.

En conjunto, los resultados de cuarta ronda confirman que HQC se beneficia de forma clara de las optimizaciones en el lenguaje C, mostrando una ganancia de rendimiento muy superior a la de BIKE.

Classic McEliece

El algoritmo Classic McEliece, presenta un comportamiento diferenciado dentro del conjunto de KEM de cuarta ronda. Su principal característica es el elevado coste de la fase de generación de claves, que domina el tiempo total de ejecución en todas las configuraciones y lenguajes analizados.

En la implementación en C, incluso con el uso de instrucciones AVX2, los tiempos siguen siendo notablemente superiores a los de HQC y BIKE. En la variante de menor seguridad, McEliece-348864f, el tiempo total ronda los 60 ms, mientras que en los niveles más altos (McEliece-8192128f) supera los 295 ms, observándose incluso un ligero empeoramiento con el uso de optimizaciones AVX2. La comparación entre las versiones “f” (*fast*) y las versiones estándar pone de manifiesto una mejora notable: las variantes “f” reducen el tiempo total entre un 30 % y un 50 % respecto a sus equivalentes normales, gracias al uso de parámetros más ligeros y a una construcción más eficiente del código de Goppa, que simplifica el proceso de generación de claves. Las optimizaciones AVX2 apenas afectan a la fase de generación de claves. No obstante, mantiene una ventaja significativa en las fases de

encapsulación y desencapsulación, con tiempos muy reducidos, del orden de microsegundos, lo que refuerza su idoneidad en escenarios donde las claves se generan de forma esporádica y se reutilizan durante largos periodos.

En las implementaciones en C#, los tiempos aumentan considerablemente, con valores entre 146 ms (McEliece-348864f) y 683 ms (McEliece-8192128f), manteniendo un crecimiento casi lineal con el tamaño de clave. Estos resultados lo sitúan por detrás de todos los KEM de tercera y cuarta ronda analizados: Kyber, Dilithium, BIKE y HQC son sustancialmente más rápidos en este entorno, mientras que SPHINCS+ continúa siendo el único esquema con tiempos muy superiores (del orden de segundos).

En Java, el comportamiento es similar, pero con un incremento aún mayor en los tiempos. Las variantes más pequeñas, como McEliece-348864f, alcanzan unos 105 ms, mientras que las configuraciones de mayor seguridad (McEliece-8192128f) superan los 4.9 s. El crecimiento temporal es casi exponencial con el tamaño de clave, en contraste con HQC y BIKE, que presentan un patrón lineal mucho más controlado. A pesar de ello, McEliece sigue siendo significativamente más rápido que los esquemas de firma SPHINCS+, pero mucho más lento que Kyber, Dilithium y Falcon, lo que refuerza su perfil de esquema seguro pero pesado computacionalmente.

Por último, en Python mantiene un comportamiento coherente, con tiempos totales que van desde 41 ms (McEliece-348864f) hasta 160 ms (McEliece-8192128f) como se puede observar en la figura ??.

Estas cifras lo sitúan ligeramente por encima de BIKE y HQC-128, y algo por debajo de HQC-256, reflejando una implementación estable; esta diferencia se puede deber a que HQC-256 depende de operaciones binarias densas y convoluciones que resultan especialmente costosas en un lenguaje interpretado como Python, mientras que en McEliece el coste dominante se concentra en la generación de claves.

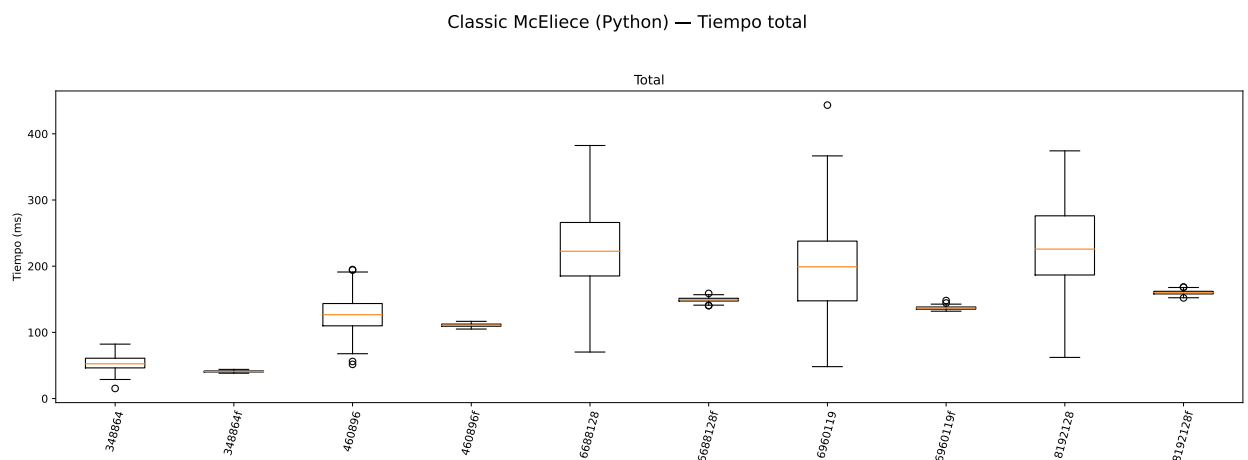


Figura 4.3. Tiempos totales de Classic McEliece en PC (Python) para distintos niveles de seguridad. Las variantes “f” (*fast*) emplean parámetros optimizados que reducen el coste de generación de claves.

En resumen, Classic McEliece destaca por su excelente rendimiento en las fases de encapsulación y desencapsulación, pero su generación de claves continúa siendo el factor limitante. A diferencia de HQC o BIKE, su escalabilidad se ve comprometida en niveles de seguridad altos, donde el tamaño de las claves y la complejidad de las operaciones sobre campos finitos incrementan de forma significativa el tiempo total de ejecución.

4.2.2. Estudio por operación

BIKE

El análisis detallado por operación de BIKE revela un comportamiento equilibrado entre sus tres fases, aunque con una clara dominancia de la desencapsulación en el tiempo total.

En la implementación en C, los tiempos medios varían según el nivel de seguridad, pero mantienen una distribución similar. En el nivel L1, la generación de claves requiere aproximadamente 0.28 ms, la encapsulación 0.06 ms y la desencapsulación alrededor de 1.11 ms, sumando un total cercano a 1.45 ms. En el nivel más alto (L5), los tiempos aumentan de forma casi lineal hasta 2.44 ms, 0.33 ms y 9.59 ms respectivamente, con un total de 12.36 ms. La desencapsulación concentra entre el 75 % y el 80 % del tiempo total de ejecución, debido a que este proceso corrige errores mediante múltiples iteraciones sobre grandes vectores binarios, analizando y modificando bits de forma repetitiva hasta reconstruir el mensaje original, aumentando el coste computacional.

Las versiones optimizadas con AVX2 muestran resultados muy similares a las portables, como se puede observar en la figura 4.4, lo que confirma la limitación en el impacto de las optimizaciones vectoriales.

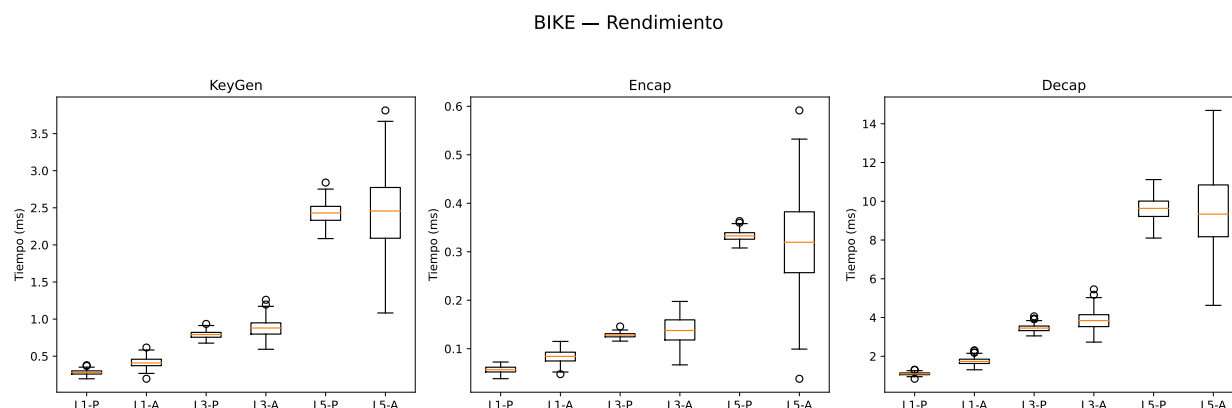


Figura 4.4. BIKE tiempos por operación en PC lenguaje C. L1, L3 y L5 indican el nivel de seguridad; P y A corresponden a las versiones portable y AVX2.

En comparación con los algoritmos de tercera ronda, la distribución temporal de BIKE difiere notablemente. En Kyber, tanto la encapsulación como la desencapsulación presentan pesos similares y

tiempos inferiores al milisegundo, con una generación de claves aún más rápida (≈ 0.04 ms). En Dilithium y Falcon, el patrón se invierte: la generación de claves y la verificación son marginales frente al proceso de firma, que domina el tiempo total. En BIKE, por el contrario, la mayor carga recae en la fase de desencapsulación, reflejando su dependencia del número de errores corregidos.

En C#, las versiones de menor seguridad (BIKE-128) presentan tiempos similares en la generación de claves y la desencapsulación, ambos en torno a 1 ms, mientras que la encapsulación permanece muy por debajo del milisegundo. Sin embargo, a medida que aumenta el nivel de seguridad, la generación de claves se vuelve proporcionalmente más costosa: en BIKE-256 alcanza alrededor de 10 ms, casi el doble del tiempo empleado en la desencapsulación (4.5 ms).

En Python, el comportamiento interno se mantiene proporcional: la desencapsulación es hasta 15 veces más lenta que la encapsulación, pero la generación de claves y el encapsulado continúan siendo operaciones muy ligeras.

En conjunto, el estudio por operación confirma que el rendimiento global de BIKE está determinado casi exclusivamente por la desencapsulación, mientras que la generación de claves sería la segunda operación más costosa, y la encapsulación presenta una latencia muy baja y estable. Su perfil de tiempos es coherente en todos los lenguajes y niveles de seguridad.

HQC

El estudio por operación de HQC muestra una distribución de tiempos más heterogénea que la de BIKE, con una clara dominancia de las fases de encapsulación y, especialmente, de desencapsulación. Estas fases concentran entre el 65 % y el 75 % del tiempo total, mientras que la generación de claves tiene un peso menor, aunque crece de forma más pronunciada al aumentar el nivel de seguridad.

En la implementación en C, la versión de referencia presenta tiempos de 0.23 ms, 0.49 ms y 0.80 ms para las fases de generación, encapsulación y desencapsulación en HQC-128, respectivamente. Tras la optimización con AVX2, estas cifras se reducen a 0.027 ms, 0.063 ms y 0.107 ms, lo que supone una mejora de casi un 90 % del tiempo total. El crecimiento del tiempo con el tamaño de clave es prácticamente lineal y estable, confirmando la escalabilidad del esquema.

Comparado con los algoritmos de la tercera ronda, HQC en su versión optimizada alcanza un rendimiento muy competitivo. En HQC-128, el tiempo total de ejecución es del orden de 0.2 ms, comparable al de Kyber-512 sin optimización vectorial e incluso ligeramente inferior al de Dilithium2 con AVX2. Aunque la fase de desencapsulación sigue siendo la más costosa, el esquema completo se ejecuta en menos de un milisegundo. Frente a BIKE, muestra una mejora muy significativa, especialmente en la fase de descifrado, donde reduce el tiempo de ejecución aproximadamente diez veces.

En las implementaciones en C#, los tiempos se amplían en todas las fases por la ausencia de optimización a bajo nivel, pasando de 4.38 ms a 21.17 ms en la generación de claves y de 8.74 ms a

42.38 ms en la encapsulación, según el nivel de seguridad. La desencapsulación mantiene su papel dominante, con valores de hasta 64 ms en HQC-256, lo que supone cerca del 50 % del total.

En Java, la distribución de tiempos por operación sigue la misma tendencia. La generación de claves crece moderadamente (de 2.1 ms a 11.3 ms), mientras que la encapsulación y la desencapsulación alcanzan 22 ms y 34 ms en HQC-256, respectivamente.

En Python, como se puede observar en la tabla 4.1, los tiempos por operación son mayores, con 7.38 ms en la generación de claves, 14.95 ms en la encapsulación y 23.96 ms en la desencapsulación para HQC-128. En HQC-256, estas cifras aumentan a 41.36 ms, 83.27 ms y 129.58 ms, respectivamente, manteniendo una relación casi lineal entre niveles de seguridad.

Lenguaje	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
Python	HQC-128	7.3813 ms (± 0.1170)	14.9464 ms (± 0.2851)	23.9656 ms (± 0.3498)	46.3443 ms (± 0.7314)
Python	HQC-256	41.3651 ms (± 0.5118)	83.2793 ms (± 0.9721)	129.5886 ms (± 1.0228)	254.2775 ms (± 1.6345)

Tabla 4.1: HQC tiempos por operación en PC lenguaje Python.

En conjunto, el análisis por operación confirma que el rendimiento global de HQC está dominado por las fases de encapsulación y desencapsulación, aunque la optimización mediante AVX2 permite reducirlas de forma drástica en C.

Classic McEliece

El análisis por operación de Classic McEliece confirma su carácter atípico dentro de los esquemas de encapsulación post-cuánticos. La fase de generación de claves domina de forma absoluta el tiempo total de ejecución, superando en varios órdenes de magnitud a la encapsulación y la desencapsulación. Esto se debe a la complejidad inherente del proceso de construcción de los códigos de Goppa y de las matrices de control, que implica operaciones intensivas sobre grandes estructuras algebraicas.

En la implementación en C, los tiempos de generación de claves varían desde 95 ms en McEliece-348864 hasta más de 820 ms en McEliece-8192128, mientras que las fases de encapsulación y desencapsulación se mantienen prácticamente constantes en todos los niveles de seguridad, con valores del orden de microsegundos a pocas décimas de milisegundo. Este fuerte desequilibrio refleja que el cuello de botella principal reside en la inicialización del esquema y no en su uso operativo. El uso de instrucciones AVX2 no introduce mejoras significativas e incluso produce un leve empeoramiento en algunos casos como se puede observar en la figura 4.5.

Las versiones marcadas con f (por ejemplo, McEliece-348864f o 8192128f) muestran una reducción notable del tiempo de generación de claves, con valores entre un 30 % y un 50 % inferiores respecto a las variantes estándar. No obstante, en las fases de encapsulación y desencapsulación ambas versiones ofrecen rendimientos prácticamente idénticos, manteniendo tiempos por debajo de un milisegundo en todos los niveles de seguridad para la encapsulación.

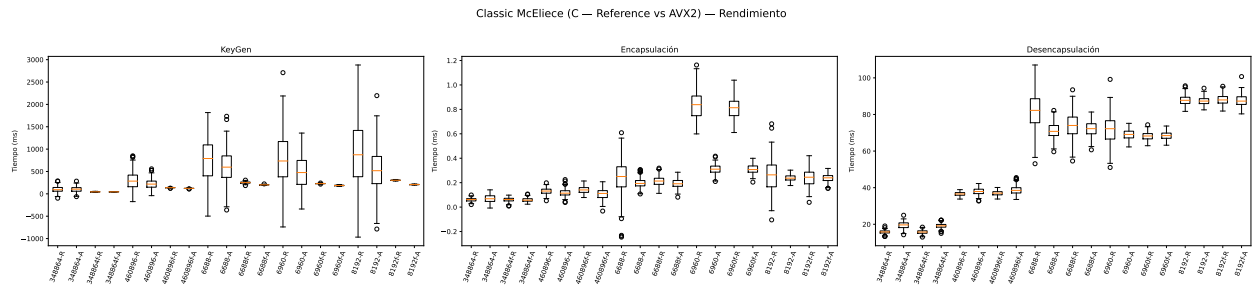


Figura 4.5. Classic McEliece tiempos por operación en PC lenguaje C; R y A corresponden a las versiones reference y AVX2.

En comparación con los algoritmos de tercera ronda, McEliece se sitúa en una posición singular: la fase de generación de claves es entre dos y tres órdenes de magnitud más lenta que la de Kyber o Dilithium, pero sus operaciones de encapsulación y desencapsulación son incluso más rápidas en algunos casos, situándose por debajo de 0.2 ms en encapsulación. Frente a HQC y BIKE, McEliece presenta tiempos totales muy superiores, aunque con un coste operativo mucho menor una vez generadas las claves. Su perfil resulta, por tanto, más adecuado para entornos en los que las claves se generan de forma esporádica y se reutilizan en numerosas sesiones, como servidores persistentes o infraestructuras de certificación.

En los lenguajes de alto nivel, el comportamiento conserva la misma tendencia. En C#, la generación de claves pasa de 131 ms en McEliece-348864f a 1276 ms en McEliece-6960119, mientras que la encapsulación y la desencapsulación permanecen estables (por debajo de 6 ms y 90 ms, respectivamente). La relación entre fases sigue siendo idéntica: la generación domina completamente el tiempo total, con una escalabilidad aproximadamente lineal con el tamaño de clave. En Java, los tiempos aumentan considerablemente, con valores que alcanzan 17.5 s en McEliece-8192128. A pesar de ello, las fases de encapsulación y desencapsulación continúan siendo muy rápidas.

Por último, en Python, los resultados se mantienen dentro del mismo patrón: la generación de claves oscila entre 41 ms y 226 ms en las variantes más representativas, mientras que la encapsulación y desencapsulación apenas contribuyen al tiempo total. En contraste, los esquemas de firma como SPHINCS+ siguen siendo entre uno y dos órdenes de magnitud más lentos en todas las operaciones.

Las versiones “f” ofrecen una mejora tangible en ese aspecto, reduciendo el tiempo de inicialización sin afectar al rendimiento operativo. Aunque su complejidad lo sitúa como el KEM más pesado en la generación, su extraordinaria eficiencia en encapsulación y desencapsulación lo convierte en una opción idónea para aplicaciones donde la generación de claves no es un proceso frecuente.

4.2.3. Consumo de recursos

BIKE

En C, el consumo de memoria es extremadamente reducido, en torno a 2–2.7 MB tanto en las versiones AVX2 como Portable, y el uso de CPU se mantiene estable entre el 98 % y el 100 %. En C#, la memoria crece de forma significativa hasta unos 47 MB, con una carga de CPU constante del 100 %. En Java, el consumo se sitúa en torno a 44 MB (aproximadamente 18 veces más que en C) y con una utilización de CPU de entre el 148 % y el 156 %, lo que indica un uso eficaz del paralelismo. Por el contrario, la versión en Python es mucho más ligera en memoria (entre 19 MB y 20 MB), aunque la CPU se mantiene por debajo del 100 %, entre el 98 % y el 99 %.

McEliece

El algoritmo McEliece destaca por su elevado tamaño de clave y, en consecuencia, por un consumo de memoria más alto que otros KEMs. En C, los valores se sitúan entre 3.4 MB y 5.8 MB, con un uso de CPU muy estable en torno al 98–99 %. En C#, el uso de memoria aumenta hasta unos 63 MB (más de diez veces más que en C), aunque la carga de CPU se mantiene en el 99 % en todas las variantes. En Java, la diferencia es aún más notable, con un uso de memoria entre 276 MB y 292 MB, y una CPU ligeramente superior al 101–102 %, reflejando una carga constante pero sin paralelismo intensivo. En Python, el comportamiento es más contenido: la memoria oscila entre 20 y 23 MB, con una CPU en torno al 96–99 %, lo que indica una ejecución más eficiente en memoria, aunque con menor aprovechamiento del hardware multinúcleo.

HQC.

En C, las versiones reference de HQC son las más exigentes en memoria dentro de las implementaciones nativas, con entre 5 y 5.2 MB y un uso de CPU del 99 %. Las variantes optimizadas con AVX2, en cambio, reducen notablemente la memoria requerida (1.8–2.0 MB) y mantienen la CPU cerca del 100 %. En C#, la memoria máxima alcanza unos 46–47 MB con un uso constante del 99 %. En Java, la memoria empleada asciende a unos 226 MB con una CPU de 101–102 %, mientras que en Python, la demanda de memoria crece hasta cifras muy superiores, entre 164 MB y 173 MB, con una CPU entre el 84 % y el 92 %, situándose entre las implementaciones más pesadas en este lenguaje.

4.3. Resultados en Raspberry Pi 5

La Raspberry Pi 5 incorpora una arquitectura ARM Cortex-A76 a 2.4 GHz con soporte para NEON, pero sin extensiones AVX2. Este entorno permite analizar cómo se comportan los algoritmos post-cuánticos en hardware de bajo consumo, mostrando diferencias significativas respecto a la arquitectura x86_64 utilizada en la sección anterior.

4.3.1. Comparación 3ª vs 4ª ronda

BIKE

En Raspberry Pi, BIKE muestra un comportamiento significativamente distinto al observado en ordenador personal. Aunque la desencapsulación continúa siendo la operación dominante, la arquitectura ARM penaliza de forma desigual a cada entorno de ejecución.

En C, tanto la versión de referencia como la optimizada con NEON/FPU presentan tiempos prácticamente idénticos, con una variación inferior al 1 %, lo que confirma que no se beneficia del paralelismo SIMD (*Single Instruction, Multiple Data*) disponible en ARM como pasaba en el ordenador personal.

En C#, hay también una penalización considerable respecto al PC. BIKE-128 pasa de 2.19 ms en PC a más de 19 ms en Raspberry Pi, mientras que BIKE-256 alcanza más de 214 ms, multiplicando por un factor de entre 8 y 14 los tiempos obtenidos en ordenador personal.

En Java, el impacto es desigual. Para BIKE-128, la penalización frente a PC es mínima: pasa de 36.7 ms a unos 37.4 ms. Sin embargo, para BIKE-256 la penalización crece de forma considerable, alcanzando 288 ms en Raspberry Pi frente a los 196 ms en PC.

En Python, BIKE presenta una pérdida notable de eficiencia respecto a sus resultados en PC. En ordenador personal, BIKE era sorprendentemente uno de los esquemas más rápidos en este lenguaje, situándose entre 1.5 ms y 10.6 ms. En Raspberry Pi, sin embargo, como se puede observar en la figura 4.6, los tiempos aumentan significativamente: BIKE-128 alcanza aproximadamente 38.7 ms y BIKE-256 supera los 400 ms. Estos tiempos provocan que Python deje de ser un entorno particularmente favorable para BIKE, a diferencia de lo que ocurría en x86.

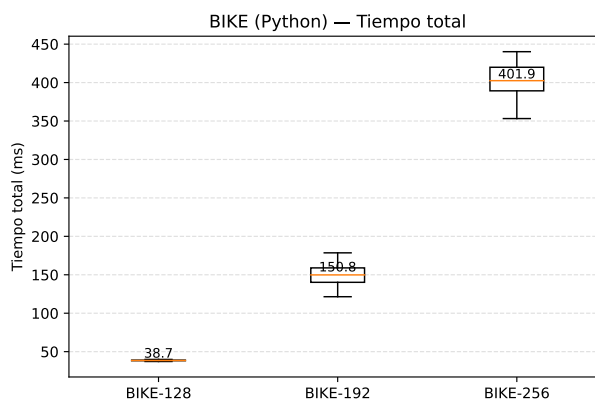


Figura 4.6. BIKE tiempos totales en Raspberry Pi lenguaje Python (niveles 128, 192 y 256).

En comparación con los algoritmos de la tercera ronda, la brecha de rendimiento de BIKE en Raspberry Pi es mucho mayor que en PC. Kyber-512, por ejemplo, requiere tan solo 0.21 ms en C y menos de 0.6 ms en C#, mientras que Dilithium presenta tiempos muy reducidos en C y C# (entre

0.1 y 3 ms), pero mucho más elevados en Python (entre 75 ms y 180 ms). Aun así, en todos los lenguajes los esquemas reticulares continúan siendo muy superiores a BIKE: BIKE-128 requiere cerca de 38 ms en C y más de 19 ms en C#, lo que sitúa a BIKE entre 30 y 170 veces por encima de Kyber y entre 10 y 60 veces por encima de Dilithium según el lenguaje. Esta penalización es mucho mayor que la observada en PC, donde BIKE era entre 5 y 15 veces más lento que los KEM reticulares. En Raspberry Pi, se amplifica esta diferencia, especialmente en lenguajes interpretados como Python.

En contraste, la comparación con Falcon se invierte parcialmente respecto a PC. En ordenador personal, Falcon era más lento que BIKE en todos los lenguajes excepto en Java; sin embargo, en Raspberry Pi Falcon se mantiene más estable, con tiempos de 25 ms en C# y entre 20 y 55 ms en Java. Esto sitúa a Falcon-512 por encima de BIKE-128 en rendimiento, un cambio significativo respecto al entorno x86. Por tanto, Falcon resulta comparativamente menos castigado que BIKE en ARM.

En conjunto, los resultados muestran que BIKE pierde competitividad en los cuatro lenguajes evaluados, ampliando la brecha frente a Kyber y Dilithium, pero manteniendo una ventaja estable frente a SPHINCS+ y un rendimiento comparable al de Falcon en ciertos casos.

HQC

En Raspberry Pi, en C, HQC mantiene una posición competitiva frente a los algoritmos de la tercera ronda. HQC-128 (14.2 ms) es significativamente más lento que Kyber-512 (0.21 ms) y que Dilithium-2 (0.85 ms), pero resulta comparable e incluso ligeramente más rápido que Falcon-512 y Falcon-1024, que alcanzan aproximadamente 29 ms y 81 ms, respectivamente. Así, HQC se sigue situando en un punto intermedio: claramente por detrás de los esquemas reticulares, pero competitivo frente a Falcon y muy por delante de SPHINCS+.

En los lenguajes de alto nivel, el panorama cambia. En C#, como se ve en la tabla 4.2, los tiempos totales de HQC en Raspberry Pi son de 35.5 ms, 107.5 ms y 197.6 ms para HQC-128, HQC-192 y HQC-256, respectivamente. Estas cifras lo sitúan claramente por detrás de Falcon, cuyas variantes 512 y 1024 se ejecutan entre 24 ms y 63 ms, y también muy por detrás de Kyber y Dilithium, que se mantienen por debajo de los 3 ms. En este entorno, HQC pasa a ser uno de los KEM más lentos del conjunto, solo por delante de SPHINCS+, que sigue teniendo tiempos del orden de cientos de milisegundos o segundos.

Lenguaje	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
C#	HQC-128	5.900 ms (± 1.428)	11.806 ms (± 2.342)	17.820 ms (± 2.660)	35.53 ms (± 6.43)
C#	HQC-192	17.939 ms (± 0.332)	35.792 ms (± 0.234)	53.801 ms (± 0.303)	107.53 ms (± 0.87)
C#	HQC-256	32.878 ms (± 0.220)	65.561 ms (± 0.267)	99.112 ms (± 0.579)	197.55 ms (± 1.07)

Tabla 4.2. HQC tiempos por operación en Raspberry Pi lenguaje C#.

En Java se observa un comportamiento similar: HQC-128, HQC-192 y HQC-256 obtienen tiempos totales de 25.9 ms, 75.9 ms y 137.3 ms, respectivamente, mientras que Falcon-512 y Falcon-1024 se

sitúan entre 19 ms y 55 ms. De nuevo, HQC es más lento que Falcon en todos los niveles de seguridad, y muy superior en tiempo a Kyber y Dilithium, que operan por debajo del milisegundo. No obstante, HQC mantiene una ventaja clara frente a SPHINCS+, cuyos tiempos se cuentan por segundos incluso en las variantes rápidas.

En Python, la diferencia se acentúa todavía más: HQC-128 ronda los 118 ms, HQC-192 alcanza unos 349 ms y HQC-256 supera los 615 ms, mientras que Falcon-512 y Falcon-1024 se ejecutan en aproximadamente 14 ms y 40 ms, respectivamente. Los esquemas reticulares (Kyber y Dilithium) también obtienen mejores resultados en este lenguaje, con Kyber por debajo del milisegundo y Dilithium entre 75 ms y 180 ms, según el nivel de seguridad. De nuevo, HQC queda claramente por detrás de Falcon, Kyber y Dilithium en Python, pero mantiene una ventaja muy notable frente a SPHINCS+, cuyos tiempos siguen siendo de varios segundos.

En resumen, en Raspberry Pi HQC mantiene un buen comportamiento en C, donde compite razonablemente bien con Falcon, pero pierde competitividad en los lenguajes de alto nivel, donde Falcon, Kyber y Dilithium le superan con claridad.

Classic McEliece

El algoritmo Classic McEliece muestra en Raspberry Pi un comportamiento muy similar al observado en PC: la generación de claves domina el tiempo total de ejecución de forma abrumadora. Sin embargo, la penalización respecto a PC es aún mayor en ARM, llegando a superar 1.7 s en las variantes de mayor seguridad.

Las versiones “f” vuelven a reducir este tiempo, como se observa en la tabla 4.3, aunque sin modificar el rendimiento de las fases operativas, que permanecen extremadamente rápidas, igual que en PC.

La comparación con los algoritmos de la tercera ronda refleja un comportamiento similar al de la arquitectura x86, pero con penalizaciones más acentuadas: McEliece es extremadamente lento en la generación de claves en Raspberry Pi, muy por detrás de Kyber, Dilithium o Falcon, cuyos tiempos de inicialización son casi insignificantes. No obstante, sus fases de encapsulación y desencapsulación siguen siendo de las más rápidas del conjunto, superando ampliamente a Falcon y con un coste operativo menor que el de SPHINCS+.

Lenguaje	Versión	T Gen. Claves (ms)	T Encapsulación (ms)	T Desencapsulación (ms)	T Total (ms)
Java	McEliece-348864	253.98 (± 186.52)	1.61 (± 2.98)	17.30 (± 4.50)	273.02 (± 191.01)
Java	McEliece-348864f	122.65 (± 30.44)	1.85 (± 3.06)	17.30 (± 4.31)	141.91 (± 38.04)
Java	McEliece-460896	731.02 (± 453.21)	2.82 (± 4.26)	46.85 (± 4.75)	780.81 (± 459.31)
Java	McEliece-460896f	337.14 (± 68.61)	2.52 (± 3.85)	46.97 (± 4.79)	386.76 (± 77.63)
Java	McEliece-6688128	1596.76 (± 1180.10)	4.16 (± 5.82)	84.24 (± 7.84)	1685.28 (± 1188.22)
Java	McEliece-6688128f	546.73 (± 71.07)	4.14 (± 5.92)	91.00 (± 12.29)	641.99 (± 89.71)
Java	McEliece-6960119	1708.14 (± 1243.34)	6.01 (± 7.80)	88.70 (± 6.36)	1802.99 (± 1244.21)
Java	McEliece-6960119f	548.43 (± 88.46)	7.50 (± 11.04)	87.66 (± 9.14)	643.73 (± 107.55)
Java	McEliece-8192128	1882.28 (± 1074.37)	5.16 (± 7.58)	115.78 (± 10.87)	2003.36 (± 1075.24)
Java	McEliece-8192128f	681.23 (± 132.35)	5.28 (± 7.38)	118.07 (± 17.24)	804.71 (± 150.95)

Tabla 4.3. Resultados del algoritmo Classic McEliece (implementación en Java) en dispositivo Raspberry Pi.

En C#, Java y Python aparece la misma tendencia: la generación de claves es claramente el factor limitante, pero el uso operativo sigue siendo muy ligero, manteniendo un perfil adecuado para entornos donde las claves se generan de forma esporádica.

4.3.2. Estudio por operación

BIKE

Un resultado especialmente relevante aparece al analizar C# y Python. A diferencia de C y Java, donde la desencapsulación sigue siendo con claridad la operación dominante, en estos dos lenguajes interpretados/gestionados se observa un cambio estructural en la distribución del coste computacional: la generación de claves pasa a ser la operación más costosa del esquema en Raspberry Pi.

En C#, los tiempos de *keygen* crecen de forma desproporcionada en comparación con PC, alcanzando 14.8 ms para BIKE-128, 63 ms para BIKE-192 y más de 181 ms en BIKE-256, superando holgadamente a la desencapsulación en todas las variantes. Esto supone un comportamiento anómalo respecto al modelo teórico de BIKE, donde la fase de decodificación debería dominar el coste total. El caso de Python es aún más llamativo: aunque la desencapsulación sigue siendo costosa,

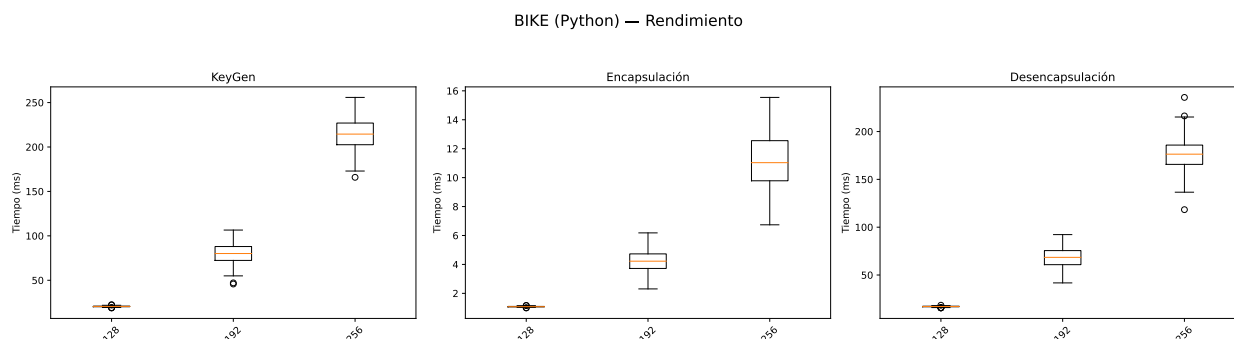


Figura 4.7. BIKE tiempos por operación en Raspberry Pi lenguaje Python (niveles 128, 192 y 256).

la generación de claves se convierte de nuevo en la operación predominante en BIKE-128, como se

puede observar en la figura 4.7, y mantiene un peso muy elevado en las versiones superiores. Para BIKE-128, el *keygen* alcanza 20.5 ms, superando ligeramente a la desencapsulación (17 ms), algo totalmente opuesto a lo que ocurre tanto en C como en PC. En BIKE-256, la penalización es extrema: más de 212 ms sólo para generar la clave.

HQC

El comportamiento interno de HQC en Raspberry Pi es más equilibrado. Las fases de encapsulación y desencapsulación concentran entre el 70 % y el 75 % del tiempo total, igual que en PC, mientras que la generación de claves sigue siendo muy ligera. La principal diferencia respecto a la arquitectura x86 es que la ausencia de AVX2 impide las aceleraciones drásticas vistas en PC optimizado.

Classic McEliece

Classic McEliece reproduce exactamente la misma estructura temporal observada en PC: la generación de claves domina absolutamente el tiempo total, mientras que encapsulación y desencapsulación se ejecutan en tiempos inferiores al milisegundo. Las variantes “f” vuelven a reducir el tiempo de forma notable sobre todo en la fase de generación de claves como vemos en la figura 4.8, aunque sin afectar a las fases operativas del algoritmo.

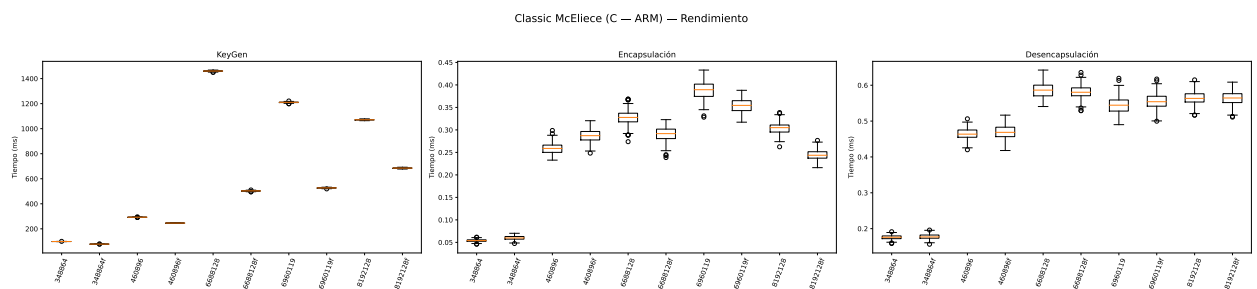


Figura 4.8. Classic McEliece tiempos por operación en Raspberry Pi lenguaje C (ARM), comparando variantes estándar y *fast*.

4.3.3. Consumo de recursos

El análisis del consumo de recursos en Raspberry Pi muestra patrones diferentes a los observados en PC, especialmente en términos de memoria y uso efectivo de CPU.

CPU

En C, todos los algoritmos mantienen un uso de CPU cercano al 100 %, igual que en ordenador personal. En Java, el uso llega a superar el 200 %, e incluso más que en PC. En Python, por el contrario, la utilización se reduce en torno al 25 %.

Memoria

El consumo de memoria en Raspberry Pi es sustancialmente más bajo que en PC:

- En C, los algoritmos se ejecutan entre 1 y 10 MB, siendo más eficientes que sus equivalentes en x86.
- En C#, el uso ronda los 42–50 MB, ligeramente inferior al observado en PC.
- En Java, la memoria se sitúa entre 150 y 210 MB, bastante menor que los valores típicos de PC (200–300 MB).
- En Python, el consumo es muy estable, en torno a 66 MB, mucho más eficiente que en PC, especialmente para HQC y SPHINCS+.

4.4. Resultados en teléfono móvil

4.4.1. Comparación 3ª vs 4ª ronda

BIKE

En dispositivo móvil, BIKE presenta tiempos totales, como se puede ver en la figura 4.9 de 51.98 ms, 146.06 ms y 343.94 ms para BIKE-128, BIKE-192 y BIKE-256, manteniendo una escalabilidad predecible con el nivel de seguridad. Aunque su comportamiento es estable, su rendimiento queda por detrás de los algoritmos más eficientes en Android. Frente a los esquemas de la tercera ronda, BIKE es claramente más lento: Kyber alcanza entre 9 y 12 ms, mientras que Dilithium se sitúa entre 16 y 20 ms. Falcon, aunque más costoso que los anteriores, resulta más eficiente que BIKE en los niveles altos, con tiempos entre 79 ms y 203 ms. Comparado con los algoritmos de cuarta ronda, HQC supera a BIKE en todos los niveles (37–171 ms), mientras que Classic McEliece y SPHINCS+ quedan muy por detrás, con tiempos totales muy superiores a los de BIKE.

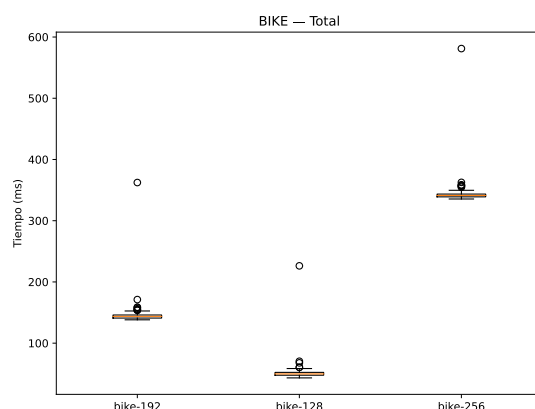


Figura 4.9. BIKE resultados totales en dispositivo móvil para los niveles de seguridad 128, 192 y 256.

En relación con otras plataformas, el rendimiento de BIKE en móvil es sustancialmente inferior al observado en ordenador personal y también, en la mayoría de casos, más lento que en Raspberry Pi, aunque con diferencias que dependen claramente del lenguaje utilizado. En PC, BIKE ofrece tiempos

totales muy reducidos en C y Python (entre 1.4 ms y 12.5 ms), con un crecimiento lineal y estable según el nivel de seguridad. En Raspberry Pi, sus tiempos aumentan de forma notable, situándose entre 37 ms y 214 ms, especialmente en lenguajes interpretados o con menor optimización. En Android, los tiempos en Kotlin son todavía mayores, alcanzando entre 52 ms y 344 ms. Aun así, en las tres plataformas el algoritmo mantiene un patrón coherente y predecible de escalado con el tamaño de las claves.

En conjunto, BIKE ofrece en móvil un rendimiento intermedio: superado con claridad por Kyber, Dilithium y HQC; cercano a Falcon en los niveles altos; y por delante únicamente de Classic McEliece y SPHINCS+.

HQC

En el entorno móvil Android, HQC presenta un rendimiento claramente más favorable que BIKE y se sitúa en una posición intermedia dentro de los algoritmos analizados. Los tiempos totales obtenidos en Kotlin son de 37 ms, 97 ms y 171 ms para HQC-128, HQC-192 y HQC-256, respectivamente, que se puede observar en la figura 4.10, evidencia una escalabilidad lineal y estable con el nivel de seguridad. La diferencia entre configuraciones es proporcional y no se observan incrementos abruptos, lo que confirma un comportamiento predecible del algoritmo en esta plataforma.

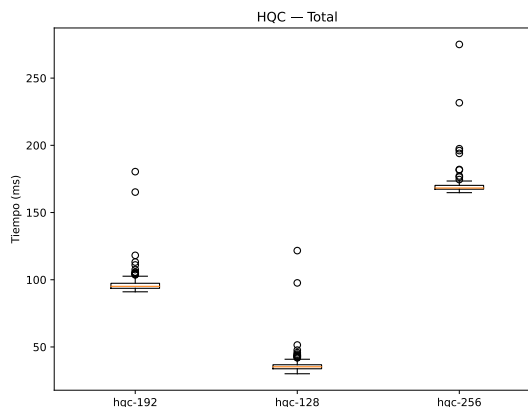


Figura 4.10. HQC resultados totales en dispositivo móvil para los niveles de seguridad 128, 192 y 256.

En comparación directa con los algoritmos de tercera ronda, HQC queda por detrás de Kyber, que continúa siendo el KEM más eficiente en móvil con tiempos entre 9 ms y 12 ms. Del mismo modo, HQC es más lento que Dilithium, cuyas firmas y verificaciones se mantienen por debajo de los 21 ms incluso en los niveles más altos de seguridad. Sin embargo, HQC ofrece un desempeño claramente superior al de Falcon, que alcanza entre 79 ms y más de 200 ms según el nivel de seguridad, y resulta también mucho más ligero que Classic McEliece, cuyos tiempos superan con facilidad los 780 ms e incluso rebasan los 6 s en las configuraciones más exigentes. Frente a SPHINCS+ la diferencia es todavía mayor: sus tiempos totales, entre 231 ms y más de 750 ms, lo sitúan muy por detrás de HQC en todos los niveles.

Así, HQC se posiciona como un esquema de rendimiento medio en el entorno móvil: más pesado que los algoritmos reticulares de la tercera ronda, pero significativamente más rápido y estable que Falcon, SPHINCS+ y Classic McEliece.

Respecto a otras plataformas, el rendimiento de HQC en móvil es notablemente inferior al de ordenador personal, donde sus tiempos totales en C oscilan entre 0.19 ms y 0.83 ms para las versiones optimizadas con AVX2. En Raspberry Pi, sus resultados se sitúan entre 14 ms y 77 ms en C y entre 35 ms y 197 ms en C#, lo que lo coloca en un punto intermedio entre PC y móvil.

Classic McEliece

En el entorno móvil Android, Classic McEliece es el esquema más pesado entre todos los algoritmos evaluados. Sus tiempos totales en Kotlin oscilan entre 781 ms para la variante más ligera (McEliece-348864f) y más de 6.2 s en las configuraciones de seguridad más elevadas, reflejando un coste extremadamente alto en generación de claves y una penalización creciente a medida que aumentan los parámetros. Aunque las operaciones de encapsulación y desencapsulación son muy rápidas (por debajo de 40 ms incluso en los niveles más altos), la generación de claves domina por completo el tiempo total, lo que condiciona su viabilidad práctica en dispositivos móviles.

En comparación con los algoritmos de tercera ronda, McEliece queda muy por detrás en todos los casos. Kyber completa sus operaciones en menos de 12 ms, y Dilithium mantiene tiempos entre 16 ms y 20 ms. Incluso Falcon, aunque más costoso que los anteriores, se sitúa entre 79 ms y 203 ms, valores muy inferiores a cualquier instancia de McEliece. En relación con los algoritmos de cuarta ronda, McEliece también se posiciona como el algoritmo más lento. HQC, por ejemplo, presenta tiempos totales entre 37 ms y 171 ms, mientras que BIKE, aun siendo más pesado que los esquemas reticulares, permanece siempre por debajo de 344 ms. SPHINCS+, pese a ser un esquema de firma basado en árboles hash, solo supera a McEliece en los niveles más altos, con tiempos entre 231 ms y 753 ms, situándose aún así por debajo de varias variantes de McEliece. En conjunto, Classic McEliece es con diferencia el esquema de mayor coste computacional en el entorno móvil.

En cuanto a su comportamiento en otras plataformas, Classic McEliece también muestra tiempos elevados tanto en ordenador personal como en Raspberry Pi, aunque en ningún caso alcanza los valores observados en móvil. En PC, sus tiempos totales oscilan entre 60 ms y 900 ms según la variante, y en Raspberry Pi se sitúan aproximadamente entre 70 ms y 420 ms en C# y entre 78 ms y 686 ms en C, antes de superar el segundo de ejecución en los niveles de seguridad más altos. Aun así, en ambas plataformas mantiene el mismo patrón observado en móvil: tiempos de encapsulación y desencapsulación muy reducidos, acompañados de una generación de claves extremadamente costosa que define por completo su rendimiento global.

4.4.2. Estudio por operación

BIKE

En Android, el reparto de tiempos entre operaciones en BIKE, la desencapsulación es claramente la fase dominante, concentrando entre el 60 % y el 70 % del tiempo total (32 ms en BIKE-128 y más de 190 ms en BIKE-256). La generación de claves gana peso relativo respecto a PC, pasando de 18 ms a más de 140 ms según el nivel de seguridad, mientras que la encapsulación se mantiene como la operación más ligera, entre 1 ms y 8 ms.

En conjunto, Android conserva la jerarquía típica del esquema—desencapsulación > generación de claves > encapsulación—aunque con una penalización proporcionalmente mayor en las fases más costosas.

HQC

En Android, HQC mantiene la misma distribución de costes observada en las demás plataformas. La desencapsulación continúa siendo la operación más costosa, situándose entre 17 ms en HQC-128 y cerca de 84 ms en HQC-256. La encapsulación también presenta un peso relevante, especialmente en los niveles altos, donde supera los 50 ms, como se puede observar en la figura 4.11. La generación de claves, aunque más ligera que en BIKE, aumenta de forma progresiva con el nivel de seguridad, desde 7.5 ms hasta más de 30 ms.

El patrón entre fases se conserva en todo el esquema—desencapsulación > encapsulación > generación de claves—con un crecimiento regular y sin cambios estructurales respecto a su comportamiento en PC o Raspberry Pi.

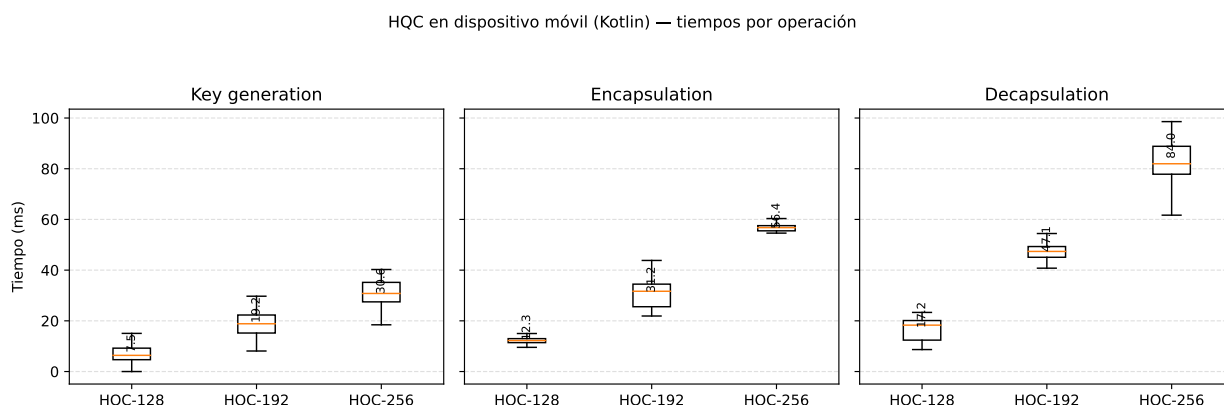


Figura 4.11. HQC tiempos por operación en en dispositivo móvil para los distintos niveles de seguridad.

Classic McEliece

En Android, Classic McEliece la generación de claves domina claramente el coste total, incluso en las variantes más ligeras. En MCEliece-348864f, se puede observar en la figura 4.12, ya supera los 700 ms, y en los niveles superiores llega a más de 6 s, muy por encima del resto de operaciones. Por contraste, la encapsulación y la desencapsulación se mantienen extremadamente ligeras en todas las configuraciones, con valores inferiores a 10 ms incluso en las variantes de mayor seguridad. Esto hace que el tiempo total esté prácticamente determinado por la fase de generación de claves.

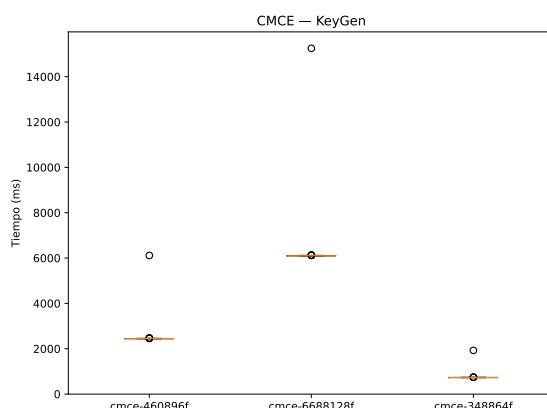


Figura 4.12. Resultados de generación de claves de Classic McEliece en dispositivo móvil de las versiones *fast*.

4.4.3. Consumo de recursos

El análisis del consumo de recursos en dispositivo móvil Android muestra un comportamiento distinto respecto al observado en ordenador personal Raspberry Pi, tanto en el uso de CPU como en el consumo de memoria.

CPU. La utilización de CPU presenta una variabilidad notable entre familias de algoritmos:

- Los KEM reticulares (Kyber) mantienen los valores más bajos, entre el 51 % y el 55 %, lo que confirma su alta eficiencia también en entorno móvil.
- Dilithium se sitúa en un rango moderado (65–75 %), consistente con el mayor coste de la fase de firma.
- BIKE y Classic McEliece alcanzan niveles próximos al 100 %, reflejando su mayor carga computacional, especialmente en la generación de claves.
- Falcon y HQC son los algoritmos con mayor uso de CPU, alcanzando entre el 105 % y el 128 % en HQC y hasta el 127 % en Falcon, similar a lo observado en Raspberry Pi para Java.

En conjunto, los algoritmos más ligeros en CPU son Kyber y Dilithium, mientras que Falcon y HQC muestran el comportamiento más intensivo, seguido de BIKE y McEliece.

Memoria. El consumo de memoria en Android se mantiene dentro de rangos relativamente estables, aunque con diferencias apreciables entre familias:

- Los algoritmos más eficientes son Kyber y McEliece, con consumos entre 130 MB y 150 MB.
- BIKE y Dilithium se sitúan en torno a los 170–180 MB.
- SPHINCS+ presenta valores similares (181–184 MB), manteniéndose sorprendentemente contenido pese a su elevado coste computacional.
- HQC es el algoritmo con mayor consumo de memoria, alcanzando los 376–377 MB en todas sus variantes, duplicando a la mayoría de esquemas evaluados.

En resumen, el entorno Android penaliza especialmente a Falcon y HQC en uso de CPU y a HQC en memoria, mientras que Kyber se mantiene como el esquema más eficiente del conjunto.

4.5. Conclusiones

Los resultados experimentales ponen de manifiesto que los algoritmos estandarizados en la tercera ronda del NIST presentan, en conjunto, un rendimiento superior al de los candidatos de cuarta ronda, tanto en tiempo absoluto como en estabilidad al cambiar de plataforma. Esta tendencia se observa de forma consistente al comparar ordenador personal, Raspberry Pi y dispositivo móvil.

En el entorno de ordenador personal, el lenguaje C con optimizaciones AVX2 es claramente la opción más eficiente. La Tabla 4.4 resume los mejores resultados obtenidos en esta plataforma y permite observar que Kyber y Dilithium alcanzan los menores tiempos totales de todo el estudio, situándose en el orden de las centésimas y décimas de milisegundo respectivamente. Estos valores los consolidan como las opciones más eficientes y regulares en sistemas de propósito general.

Lenguaje	Versión	T Gen. Claves	T Operación 1	T Operación 2	T Total	Memoria	CPU
C	Kyber-768 AVX2	0.0170 ms	0.0168 ms	0.0180 ms	0.0517 ms	1715 KB	99%
C	Dilithium-3 AVX2	0.0525 ms	0.1485 ms	0.0580 ms	0.2591 ms	2806 KB	99%
C	Falcon-512 AVX2	8.3794 ms	0.2644 ms	0.0456 ms	8.6895 ms	2078 KB	99%
C	SPHINCS+ SHAKE256-128F AVX2	0.71 ms	15.20 ms	1.07 ms	16.98 ms	3012 KB	100%
C	HQC-192 AVX2	0.0647 ms	0.1465 ms	0.2253 ms	0.4365 ms	1928 KB	99%
C	BIKE-L3 AVX2	0.868 ms	0.140 ms	3.757 ms	4.765 ms	2372 KB	99%
C	McEliece460896f AVX2	123.04 ms	0.109 ms	38.55 ms	161.70 ms	4013 KB	98%

Tabla 4.4. Resultados en PC para los algoritmos de 3ª y 4ª ronda.

Falcon presenta tiempos sensiblemente superiores, dominados principalmente por el coste de la generación de claves, aunque todavía se mantiene en un orden de magnitud claramente inferior al de los esquemas *hash-based*. En el caso de SPHINCS+, incluso considerando sus variantes más optimizadas con AVX2 —que representan el mejor escenario posible para este algoritmo—, los tiempos totales se sitúan en el rango de varias decenas de milisegundos o más. Estos valores resultan comparables, e incluso superiores en algunos casos, a los observados en Classic McEliece, y claramente por encima de los obtenidos por HQC y BIKE. Este comportamiento confirma que, pese a las optimizaciones vectoriales, el coste intrínseco de las operaciones de firma en SPHINCS+ sigue siendo elevado y limita su competitividad en escenarios sensibles a latencia.

Entre los candidatos de cuarta ronda, HQC es el algoritmo que más se beneficia de AVX2, alcan-

zando tiempos inferiores al milisegundo y situándose como el esquema más próximo a los algoritmos de tercera ronda en PC. BIKE permanece en el rango de los milisegundos incluso con optimizaciones, mientras que Classic McEliece queda claramente penalizado por el elevado coste de la generación de claves, que domina su tiempo total a pesar de tener operaciones de encapsulación y desencapsulación relativamente rápidas.

La Figura 4.13 complementa esta información mostrando visualmente los mejores tiempos totales en PC. En ella se aprecia con claridad la ventaja de Kyber y Dilithium frente al resto de algoritmos, así como la distancia que separa a los esquemas de tercera ronda de los candidatos de cuarta ronda cuando se dispone de optimizaciones vectoriales.

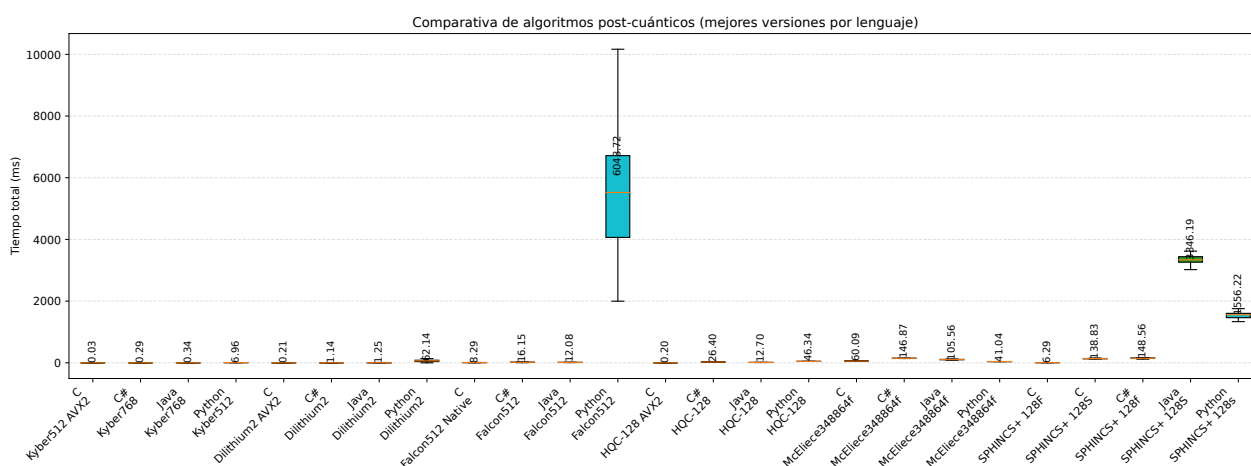


Figura 4.13. Mejores tiempos totales en PC. Las variantes “f” (*fast*) emplean parámetros optimizados que reducen el coste de generación de claves.

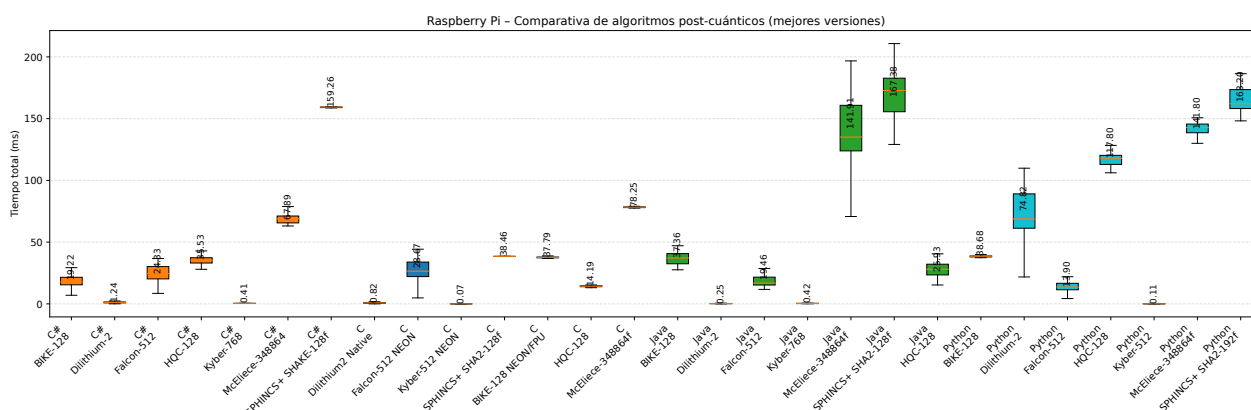


Figura 4.14. Mejores tiempos totales en Raspberry Pi. Las variantes “f” (*fast*) emplean parámetros optimizados que reducen el coste de generación de claves.

Al trasladar la ejecución a Raspberry Pi, la Figura 4.14 y la Tabla 4.5 muestran un incremento generalizado de los tiempos para todos los algoritmos, debido a la ausencia de AVX2 y a las características propias de la arquitectura ARM. En esta plataforma, el lenguaje C sigue ofreciendo los mejores resultados absolutos, especialmente para Kyber y Dilithium, que aprovechan las optimizaciones NEON y

mantienen tiempos relativamente contenidos. Sin embargo, entre los candidatos de cuarta ronda no se observa un dominador claro: HQC presenta buenos resultados en C, pero en lenguajes de más alto nivel como C# y Python BIKE obtiene tiempos totales inferiores, lo que indica que el rendimiento relativo entre ambos depende en gran medida del lenguaje y de la calidad de las implementaciones disponibles. Classic McEliece vuelve a mostrar tiempos elevados, especialmente en la generación de claves, lo que limita su viabilidad práctica en sistemas ARM con recursos ajustados.

Lenguaje	Versión	T Gen. Claves	T Operación 1	T Operación 2	T Total	Memoria	CPU
C	Kyber-512 NEON	0.0204 ms	0.0267 ms	0.0252 ms	0.0724 ms	1616 KB	99 %
C	Dilithium-2 NEON	0.1183 ms	0.5900 ms	0.1398 ms	0.8481 ms	1088 KB	100 %
C	Falcon-512 NEON	22.1882 ms	6.4347 ms	0.0426 ms	28.6655 ms	1120 KB	99 %
C	HQC-128 Reference	2.2307 ms	4.5384 ms	7.4254 ms	14.1944 ms	1120 KB	99 %
C	BIKE-128 NEON/FPU	2.8017 ms	0.2336 ms	34.7562 ms	37.7915 ms	9472 KB	99 %
C	McEliece-348864f ARM	78.01 ms	0.060 ms	0.178 ms	78.25 ms	6291 KB	99 %

Tabla 4.5. Resultados en Raspberry Pi para los algoritmos de 3ª y 4ª ronda.

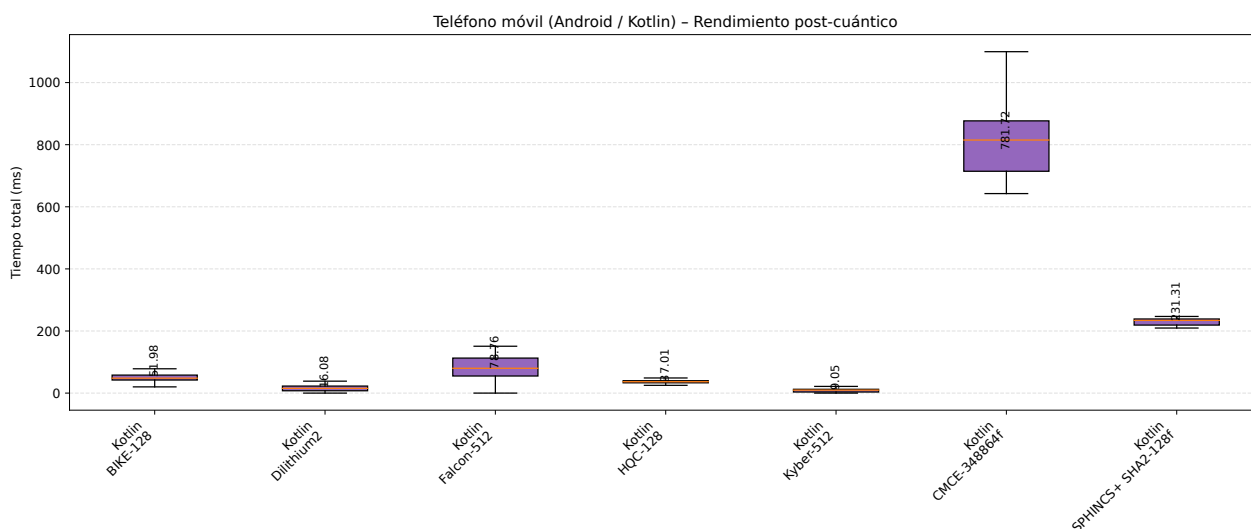


Figura 4.15. Mejores tiempos totales en dispositivo móvil. Las variantes “f” (fast) emplean parámetros optimizados que reducen el coste de generación de claves.

En el entorno móvil Android, representado en la Figura 4.16, el análisis se centra en Kotlin como lenguaje único. En este escenario, las limitaciones hardware y del entorno de ejecución se traducen en tiempos notablemente superiores a los observados en PC y Raspberry Pi. Aun así, Kyber y Dilithium continúan siendo los algoritmos más eficientes, manteniendo una ventaja clara frente al resto. HQC vuelve a destacar como el algoritmo de cuarta ronda con mejor comportamiento transversal, mientras que BIKE presenta un coste un poco más elevado y Classic McEliece resulta poco viable debido al alto coste de la generación de claves.

Si se comparan las plataformas entre sí, el ordenador personal es el entorno donde todos los algoritmos alcanzan su mejor rendimiento, seguido de Raspberry Pi, que ofrece resultados aceptables para esquemas bien optimizados en arquitecturas ARM. El dispositivo móvil es la plataforma más restrictiva, aunque los algoritmos más eficientes siguen siendo utilizables para determinados *use cases*.

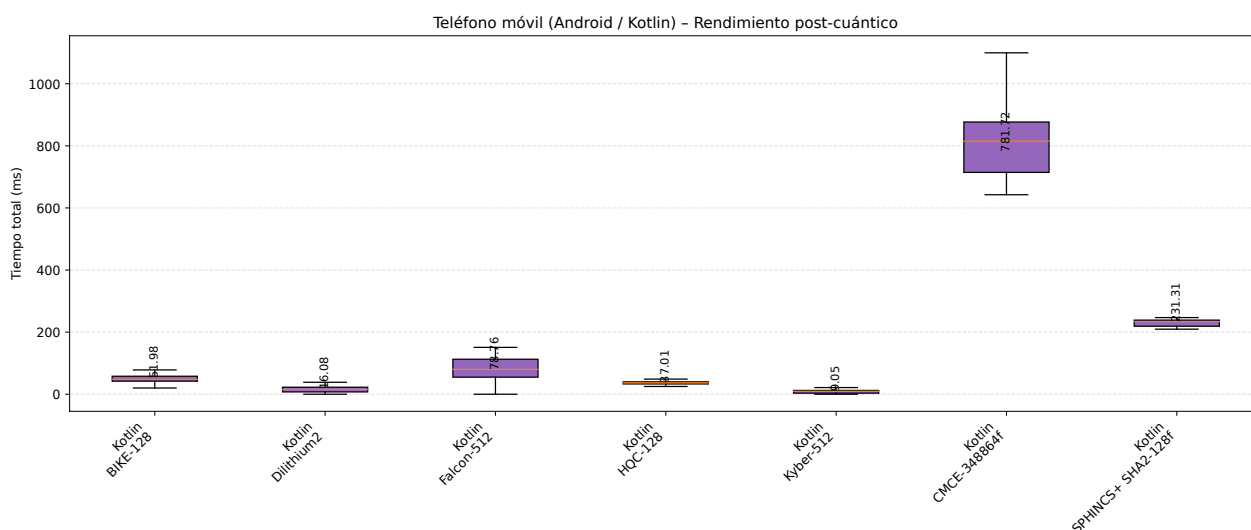


Figura 4.16. Mejores tiempos totales en dispositivo móvil. Las variantes “f” (*fast*) emplean parámetros optimizados que reducen el coste de generación de claves.

En términos generales, Kyber es el algoritmo que presenta el mejor rendimiento absoluto y el mejor escalado entre plataformas, seguido de Dilithium. HQC se consolida como la alternativa más equilibrada entre los candidatos de cuarta ronda, mientras que BIKE y, especialmente, Classic McEliece muestran las mayores penalizaciones a medida que se reducen los recursos disponibles. Estos resultados confirman que los algoritmos de tercera ronda no solo son los más rápidos en una plataforma concreta, sino también los que mejor se adaptan a arquitecturas heterogéneas.

CONCLUSIONES Y TRABAJO FUTURO

5.1. Conclusiones finales

Realizar este trabajo ha permitido comprender en profundidad los mecanismos necesarios para garantizar la seguridad de la información y la importancia de seguir avanzando frente a amenazas emergentes, especialmente en un contexto post-cuántico. Estas ideas, reforzadas en asignaturas como Redes de Comunicación 2, Ciberseguridad, Introducción a la Computación Cuántica y Complejidad y Computación, han puesto de manifiesto la necesidad de adoptar nuevos esquemas criptográficos capaces de sustituir de forma segura a los métodos clásicos.

Tras el análisis realizado, puede concluirse que, entre los algoritmos de cuarta ronda evaluados, HQC es el que ofrece el rendimiento más equilibrado y competitivo. Su comportamiento es estable en todos los dispositivos, mantiene tiempos reducidos en encapsulación y desencapsulación y escala de forma muy regular con el nivel de seguridad. BIKE muestra también un rendimiento sólido y predecible, mientras que Classic McEliece destaca únicamente por su rapidez operativa, aunque queda claramente limitado por el elevado coste de generación de claves.

No obstante, los resultados muestran que ninguno de los algoritmos de cuarta ronda supera a los mejores de la tercera, ya que Kyber y Dilithium continúan siendo las propuestas más eficientes y con mejores tiempos de ejecución en el conjunto de pruebas realizadas.

El código desarrollado para este TFG se encuentra disponible en el repositorio: <https://github.com/saantiidp/TFG>.

5.2. Trabajo futuro

Como principal línea de trabajo futura, se propone profundizar en el análisis de la seguridad práctica de los algoritmos post-cuánticos, centrándose especialmente en su resistencia frente a ataques de canal lateral. Aunque este trabajo se ha orientado principalmente al rendimiento, resulta fundamental estudiar el impacto de técnicas como *timing attacks*, *cache-timing*, análisis de potencia o ataques

derivados del comportamiento interno de las microarquitecturas modernas. Estos ataques no intentan romper el algoritmo matemático subyacente, sino que explotan pequeñas variaciones en el tiempo de ejecución, en los accesos a la memoria caché o en el consumo eléctrico del dispositivo para inferir información secreta. Dado que algoritmos como Kyber, Dilithium, SPHINCS+ o FALCON realizan operaciones complejas y con numerosos accesos a memoria, su estudio detallado permitiría identificar vulnerabilidades que surgen no del diseño criptográfico, sino de la forma en que estos esquemas se implementan y ejecutan en hardware real.

Otro aspecto relevante de investigación es el análisis de esquemas híbridos que combinan algoritmos clásicos y post-cuánticos en protocolos existentes. Por ejemplo, el grupo de trabajo de TLS del IETF (*Internet Engineering Task Force*) ha propuesto recientemente grupos híbridos de intercambio de claves para TLS 1.3 que combinan un algoritmo clásico (ECDHE, *Elliptic Curve Diffie-Hellman Ephemeral*) con un mecanismo post-cuántico (ML-KEM), con el objetivo de ofrecer protección frente a adversarios tanto clásicos como cuánticos [42]. De forma más general, la IETF ha definido la terminología y el marco conceptual para esquemas y protocolos criptográficos híbridos PQ/T (*Post-Quantum / Traditional*), que abarcan tanto el intercambio de claves como las firmas digitales y los certificados [43]. Además, implementaciones tempranas como el despliegue experimental del intercambio híbrido X25519+Kyber en navegadores y servidores muestran que estas técnicas están siendo evaluadas en entornos reales [44].

Asimismo, resulta necesario evaluar el coste real de la transición hacia infraestructuras post-cuánticas en entornos productivos. Este estudio debería contemplar aspectos como el aumento del tamaño de claves, certificados y firmas; el impacto en el almacenamiento y el tráfico de red; la compatibilidad con sistemas heredados; y el coste computacional asociado al despliegue masivo de PQC en servicios empresariales, dispositivos IoT (*Internet of Things*) o redes de alta carga. Medir estos factores permitiría estimar el esfuerzo requerido para una adopción completa y anticipar posibles cuellos de botella operativos.

Estas líneas de investigación complementarían los resultados obtenidos en este trabajo y ofrecerían una visión más completa sobre los desafíos reales, tanto de seguridad como de infraestructura, que acompañan a la transición hacia un ecosistema criptográfico plenamente post-cuántico.

BIBLIOGRAFÍA

- [1] L. Chen, S. Jordan, Y.-K. Liu, D. Moody, R. Peralta, R. Perlner, and D. Smith-Tone, “Report on post-quantum cryptography,” NIST Interagency/Internal Report (NISTIR) 8105, National Institute of Standards and Technology, 2016. Disponible en: <https://doi.org/10.6028/NIST.IR.8105>.
- [2] Information Security and Privacy Advisory Board, “Information security and privacy advisory board (ispab) minutes, october 25–26, 2023,” tech. rep., National Institute of Standards and Technology (NIST), Washington, D.C., USA, Oct. 2023. Disponible en <https://csrc.nist.gov/csrc/media/Events/2023/ispab-october-meeting/documents/ISPAB-Oct2023-Minutes.pdf>.
- [3] National Institute of Standards and Technology (NIST), “Post-quantum cryptography: Call for proposals,” tech. rep., U.S. Department of Commerce, dec 2016. Disponible en: <https://csrc.nist.gov/csrc/media/projects/post-quantum-cryptography/documents/call-for-proposals-final-dec-2016.pdf>.
- [4] National Institute of Standards and Technology (NIST), “Post-quantum cryptography: Workshops and timeline.” <https://csrc.nist.gov/projects/post-quantum-cryptography/workshops-and-timeline>, 2025. Accedido en octubre de 2025.
- [5] D. Moody, “Nist pqc: The road ahead.” Presentation at the Real World Crypto Symposium, March 2025, 2025.
- [6] National Institute of Standards and Technology (NIST), “Status report on the third round of the NIST post-quantum cryptography standardization process,” Tech. Rep. NIST IR 8413, NIST, 2022.
- [7] D.-T. Dam, T.-H. Tran, V.-P. Hoang, C.-K. Pham, and T.-T. Hoang, “A survey of post-quantum cryptography: Start of a new race,” *Cryptography*, vol. 7, p. 40, Aug. 2023.
- [8] D. Jao, C. Costello, L. De Feo, P. Longa, M. Naehrig, and J. Renes, “Supersingular isogeny key encapsulation (SIKE): Submission to the NIST post-quantum cryptography project (round 4).” NIST Post-Quantum Cryptography Project, 2022. Version: September 15, 2022.
- [9] W. Castryck and T. Decru, “An efficient key recovery attack on SIDH,” in *Advances in Cryptology – EUROCRYPT 2023* (C. Hazay and M. Stam, eds.), vol. 13507 of *Lecture Notes in Computer Science*, (Cham), pp. 423–447, Springer Nature Switzerland, 2023.
- [10] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, and D. Smith-Tone, “Status report on the third round of the NIST post-quantum cryptography standardization process (update 1),” Tech. Rep. NIST IR 8413-upd1, National Institute of Standards and Technology, 2022.
- [11] W. Beullens, “Breaking rainbow takes a weekend on a laptop.” IBM Research Blog, 2022.
- [12] L. Zhang and X. Wang, “On a parabolic equation in MEMS with an external pressure,” 2023.

- [13] Classic McEliece Team, "Classic mceliece: Conservative code-based cryptography: Guide for implementors." <https://classic.mceliece.org/mceliece-impl-20221023.pdf>, oct 2022. Version 2022-10-23.
- [14] J. Cai and X. Zhang, "Efficient layered new bit-flipping QC-MDPC decoder for BIKE post-quantum cryptography," *arXiv preprint arXiv:2412.11997*, dec 2024. Visitado: 2025-09-28.
- [15] N. Aragon, P. L. Barreto, S. Bettaleb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, S. Ghosh, S. Gueron, T. Güneysu, C. A. Melchor, R. Misoczki, E. Persichetti, J. Richter-Brockmann, N. Sendrier, J.-P. Tillich, V. Vasseur, and G. Zémor, "BIKE: Status update and recent developments." NIST PQC Seminar Presentation, Sept. 2024. Disponible en: <https://csrc.nist.gov/csrc/media/Projects/post-quantum-cryptography/documents/pqc-seminars/presentations/21-bike-09272024.pdf>.
- [16] National Institute of Standards and Technology, "Status report on the fourth round of the NIST post-quantum cryptography standardization process," NIST Interagency/Internal Report NIST IR 8545, NIST, mar 2025. Visitado: 2025-09-28.
- [17] D. Moody, "Let's get ready to rumble: The NIST PQC competition." Presentation at PQCrypto 2018, Apr. 2018. Disponible en: https://csrc.nist.gov/CSRC/media/Presentations/Let-s-Get-Ready-to-Rumble-The-NIST-PQC-Competiti/images-media/PQCrypto-April2018_Moody.pdf.
- [18] National Institute of Standards and Technology (NIST), "NIST releases first 3 finalized post-quantum encryption standards." NIST News Release, aug 2024. Disponible en: <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>.
- [19] National Institute of Standards and Technology (NIST), "FIPS 203: Module-lattice-based key-encapsulation mechanism standard (ML-KEM)," Federal Information Processing Standards Publication 203, U.S. Department of Commerce, aug 2024. Disponible en: <https://csrc.nist.gov/pubs/fips/203/final>.
- [20] National Institute of Standards and Technology (NIST), "FIPS 204: Module-lattice-based digital signature standard (ML-DSA)," Federal Information Processing Standards Publication 204, U.S. Department of Commerce, aug 2024. Disponible en: <https://csrc.nist.gov/pubs/fips/204/final>.
- [21] National Institute of Standards and Technology (NIST), "FIPS 205: Stateless hash-based digital signature standard (SLH-DSA)," Federal Information Processing Standards Publication 205, U.S. Department of Commerce, aug 2024. Disponible en: <https://csrc.nist.gov/pubs/fips/205/final>.
- [22] National Institute of Standards and Technology (NIST), "Status report on the third round of the NIST post-quantum cryptography standardization process," NIST Interagency/Internal Report (NISTIR) 8413, U.S. Department of Commerce, jul 2022. Disponible en: <https://csrc.nist.gov/pubs/ir/8413/final>.

- [23] National Institute of Standards and Technology (NIST), “Status report on the fourth round of the NIST post-quantum cryptography standardization process,” NIST Interagency/Internal Report (NISTIR) 8545, U.S. Department of Commerce, Mar. 2025. Disponible en: <https://nvlpubs.nist.gov/nistpubs/ir/2025/NIST.IR.8545.pdf>.
- [24] J. Ribeiro, “Codes and lattices in cryptography: Mini-course notes.” Lecture Notes, FCT-UNL, 2023. https://jmlribeiro.github.io/crypto_mini_course_notes34.pdf. Accedido en septiembre de 2025.
- [25] J. Li and P. Q. Nguyen, “A complete analysis of the BKZ lattice reduction algorithm.” Cryptology ePrint Archive, Paper 2020/1237, 2020.
- [26] J. Ernst and A. Mitrokovtsa, “A framework for UC secure privacy preserving biometric authentication using efficient functional encryption.” Cryptology ePrint Archive, Paper 2023/481, 2023.
- [27] A. Shaller, L. Zamir, and M. Nojournian, “Roadmap of post-quantum cryptography standardization: Side-channel attacks and countermeasures,” *Information and Computation*, pp. 1–26, 2023.
- [28] Fortinet, “Understanding shor’s and grover’s algorithms and their impact on cybersecurity.” Fortinet CyberGlossary, 2025. Disponible en: <https://www.fortinet.com/resources/cyberglossary/shors-grovers-algorithms>. Accedido.
- [29] J. A. Benito-Corral, J. E. López de Vergara, L. de Pedro, I. González, F. Almenares, L. Cruz-Piris, and A. Marín, “Análisis de algoritmos post-cuánticos de cifrado en plataformas heterogéneas,” in *Actas de las XVII Jornadas de Ingeniería Telemática (JITEL 2025)*, (Cáceres, España), pp. 1–4, Nov. 2025.
- [30] L. Cruz-Piris, A. Marín-López, M. Álvarez Campana, M. Sanz, J. I. Moreno, and D. Arroyo, “Measuring the impact of post quantum cryptography in industrial IoT scenarios,” *Internet of Things*, vol. 34, p. 101793, 2025.
- [31] J. A. Montenegro, R. Ríos, and J. López-Cerezo, “A performance evaluation framework for post-quantum TLS,” *Future Generation Computer Systems*, vol. 175, p. 108062, 2026.
- [32] W. J. Buchanan, “Asecuritysite.com.” <https://asecuritysite.com/>. Accedido en octubre de 2025.
- [33] PQCrystals Team, “Kyber: Post-quantum key encapsulation mechanism.” <https://pq-crystals.org/kyber/>, 2024. Accedido en septiembre de 2025.
- [34] PQCrystals Team, “Dilithium: Post-quantum digital signature algorithm.” <https://pq-crystals.org/dilithium/>, 2024. Accedido en septiembre de 2025.
- [35] Falcon Project Team, “Falcon: Digital signature algorithm (reference implementation).” <https://github.com/tprest/falcon>, 2024. Accedido en septiembre de 2025.
- [36] SPHINCS+ Team, “SPHINCS+: Stateless hash-based signature scheme.” <https://sphincs.org/>, 2024. Accedido en septiembre de 2025.
- [37] J. A. Benito Corral, “Repositorio TFG - implementaciones PQC en c, c#, java y python.” https://github.com/Alberto1-2/TFG_CODE, 2024. Accedido en septiembre de 2025.
- [38] AWS Labs, “BIKE-KEM: AVX2-optimized implementation (decoding module).” <https://>

- github.com/aws-labs/bike-kem/tree/master/src/decode, 2024. Accedido en septiembre de 2025.
- [39] PQClean Contributors, "PQClean: Clean and reference implementations of PQC algorithms." <https://github.com/PQClean/PQClean>, 2024. Accedido en septiembre de 2025.
- [40] S. Bowne, "HQC example in python (project c810)." <https://samsclass.info/141/proj/C810.htm>, 2025. Accedido en septiembre de 2025.
- [41] J. Mojzis, "python-mceliece: McEliece implementation in python." <https://github.com/janmojzis/python-mceliece>, 2024. Accedido en septiembre de 2025.
- [42] K. Kwiatkowski, P. Kampanakis, B. E. Westerbaan, and D. Stebila, "Post-quantum hybrid ECDHE-MLKEM key agreement for TLS 1.3," Internet-Draft draft-ietf-tls-ecdhe-mlkem-03, Internet Engineering Task Force (IETF), Nov. 2025. Work in Progress.
- [43] F. Driscoll, M. Parsons, and B. Hale, "Terminology for Post-Quantum Traditional Hybrid Schemes." RFC 9794, June 2025. Category: Informational.
- [44] D. O'Brien, "Protecting chrome traffic with hybrid kyber KEM." Chromium Blog, Aug. 2023. Accedido en septiembre de 2025.
- [45] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3." RFC 8446, Aug. 2018.
- [46] N. Sullivan, "A detailed look at RFC 8446 (a.k.a. TLS 1.3)." Cloudflare Blog, Aug. 2018. Accedido en noviembre de 2025.
- [47] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid key exchange in TLS 1.3," Internet-Draft draft-ietf-tls-hybrid-design-16, Internet Engineering Task Force, Sept. 2025. Work in Progress.
- [48] National Institute of Standards and Technology, "FIPS 203: Module-lattice-based key-encapsulation mechanism standard (ML-KEM)," Federal Information Processing Standards Publication FIPS 203, U.S. Department of Commerce, 2024. Accedido en septiembre de 2025.
- [49] National Institute of Standards and Technology, "FIPS 204: Module-lattice-based digital signature standard (ML-DSA)," Federal Information Processing Standards Publication FIPS 204, U.S. Department of Commerce, 2024. Accedido en septiembre de 2025.
- [50] National Institute of Standards and Technology, "FIPS 205: Stateless hash-based digital signature standard (SLH-DSA)," Federal Information Processing Standards Publication FIPS 205, U.S. Department of Commerce, 2024. Accedido en septiembre de 2025.
- [51] W. J. Buchanan, "Hybrid encryption with kyber (ML-KEM) and AES." https://asecuritysite.com/hybrid/go_hybrid3, 2025. Accedido en septiembre de 2025.
- [52] CA/Browser Forum, "Baseline requirements for the issuance and management of publicly-trusted TLS server certificates, version 2.1.2." CA/Browser Forum, dec 2024.
- [53] A. Banerjee, T. Reddy, K. D. Schoiniakakis, T. Hollebeek, and M. Ounsworth, "Post-Quantum Cryptography for Engineers," Internet-Draft draft-ietf-pquip-pqc-engineers-14, Internet Engineering Task Force, Aug. 2025. Work in Progress.
- [54] FIDO Alliance, "Addressing FIDO alliance's technologies in a post-quantum world." White Paper, jan 2024. Accedido en septiembre de 2025.

- [55] C. Schutijser, R. Koning, E. Lastdrager, and C. Hesselman, “Evaluating post-quantum cryptography in DNSSEC signing for top-level domain operators,” in *Proceedings of the 2025 Network Traffic Measurement and Analysis Conference (TMA 2025)*, (Bordeaux, France), IFIP, jun 2025. Accedido en septiembre de 2025.

APÉNDICES

CRIPTOGRAFÍA POST-CUÁNTICA EN LA WEB

A.1. Criptografía post-cuántica en la Web: estado actual y transición

Aquí presentamos un panorama del uso actual y la transición hacia criptografía post-cuántica en la Web. Partimos de cómo HTTPS/TLS 1.3 y la PKI pública proporcionan hoy autenticación y cifrado, y explicamos por qué la llegada de la computación cuántica obliga a sustituir RSA/ECDSA y ECDHE por alternativas resistentes. Resumimos los estándares NIST (ML-KEM, ML-DSA, SLH-DSA) y el enfoque híbrido en TLS como vía práctica de migración, revisando su despliegue real en navegadores/CDNs y el papel de las políticas del CA/Browser Forum. Por último, señalamos líneas de adaptación en WebAuthn/FIDO y DNSSEC, para enmarcar una transición gradual y compatible con el ecosistema actual.

A.1.1. HTTPS/TLS 1.3

En la Web actual, HTTPS es HTTP sobre TLS 1.3, el protocolo que autentica a las partes y cifra el canal para impedir escuchas, manipulación y suplantación. Su especificación describe con detalle el *handshake* y el proceso para derivar las claves de sesión [45]. Durante ese *handshake* el servidor presenta un certificado y ejecuta una firma digital que el navegador verifica con la clave pública del certificado, demostrando que posee la privada y que el material presentado no ha sido alterado [45].

Aquí encaja la criptografía post-cuántica sin alterar la lógica de la Web: se mantienen las mismas funciones (autenticar y cifrar en el *handshake*), sustituyendo RSA/ECDSA y X25519/ECDHE por ML-DSA/SLH-DSA y ML-KEM, de modo que la conexión siga funcionando igual, pero con algoritmos preparados frente a ataques cuánticos [46].

A.1.2. Acuerdo de claves híbrido (X25519 + Kyber/ML-KEM) y firmas PQC

La transición práctica ha llegado vía intercambios híbridos en TLS 1.3 que combinan un algoritmo clásico (p. ej., X25519) con uno post-cuántico en un único *handshake*, de forma que la conexión siga segura incluso si en el futuro uno de los dos se debilitara [47]. Este patrón ya se probó a escala: Chrome añadió soporte para X25519Kyber768 a partir de la versión 116 para establecer secretos de sesión, con impacto controlado en latencia y tamaños [44]. La pieza PQC de ese híbrido es ML-KEM (Kyber), publicada por NIST como FIPS 203 con parámetros y formatos interoperables listos para productos y protocolos [48].

En paralelo, la autenticación también tiene alternativas: NIST estandarizó ML-DSA (derivada de Dilithium) para firmas digitales con conjuntos de parámetros oficiales, pensada para sustituir progresivamente a RSA/ECDSA en certificados y software [49]. Para casos donde se prioriza la robustez a muy largo plazo, NIST también estandarizó SLH-DSA (basada en funciones *hash* y derivada de SPHINCS+), una alternativa muy conservadora con el estándar final ya publicado y parámetros definidos [50].

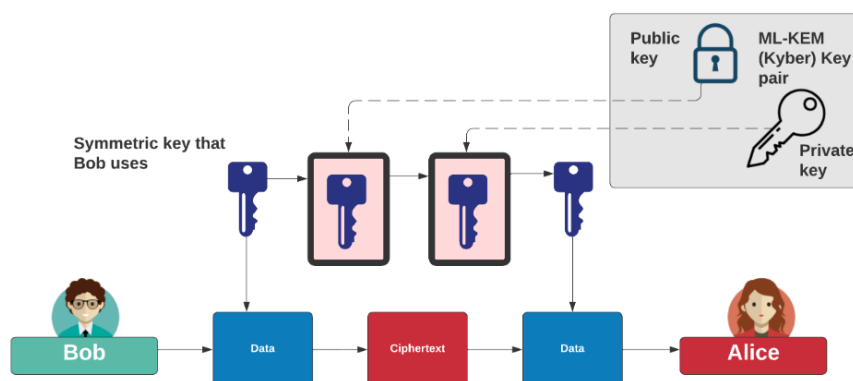


Figura A.1. Esquema simplificado de encapsulación con ML-KEM (Kyber). Bob utiliza la clave pública de Alice para encapsular una clave simétrica, con la que cifra los datos. Alice, mediante su clave privada, puede recuperar esa clave simétrica y descifrar el mensaje [51].

En la figura A.1 se muestra de forma esquemática el papel de ML-KEM (Kyber) en el acuerdo de claves: Bob encapsula una clave simétrica con la clave pública de Alice y le envía el resultado; Alice, con su clave privada, recupera la misma clave. Así ambos comparten un secreto de sesión con el que cifran la comunicación posterior de manera eficiente.

A.1.3. Estado en producción, políticas de PKI y piezas adyacentes de la Web

En producción, algunos distribuidores de contenidos (CDNs) y operadores ya han habilitado TLS 1.3 híbrido (p. ej., X25519+ML-KEM/ Kyber) tanto en las conexiones desde el borde hacia los servidores de origen como en el tráfico interno entre microservicios. Han publicado informes técnicos con cómo lo desplegaron, qué métricas observaron (tamaños del *handshake*, latencia, tasa de éxito) y qué buenas prácticas siguieron para introducirlo gradualmente (*canary*, porcentajes, *fallback*) sin afectar la experiencia de usuario [46].

Qué algoritmos puede usar un sitio público y qué aceptan los navegadores lo marcan las *Baseline Requirements* del CA/Browser Forum (algoritmos/tamaños mínimos, emisión, vigencia, revocación), que rigen los certificados de servidor confiables [52]. Esas políticas se aplican a través de los programas de raíces de los navegadores: por ejemplo, la *Mozilla Root Store Policy* explica qué autoridades certificadoras (CA) pueden incluirse en el almacén de confianza del navegador, bajo qué requisitos y cómo se auditan y supervisan una vez incluidas.

Más allá de TLS, el ecosistema web ya está preparando otras piezas: el IETF publica guías de migración para integrar criptografía post-cuántica en distintos protocolos de Internet sin romper compatibilidad, con recomendaciones sobre perfiles y fases de adopción [53]. También progresa la adaptación de la autenticación WebAuthn/FIDO al escenario post-cuántico, con hojas de ruta para actualizar autenticadores y proveedores de identidad sin perder compatibilidad [54]. También en DNSSEC se prueban firmas PQC y combinaciones de algoritmos para equilibrar rendimiento y robustez frente a los usos actuales con RSA/ECDSA [55].

A.1.4. Certificados y políticas (PKI pública)

Qué algoritmos y tamaños puede usar un sitio público y qué aceptan los navegadores lo determinan las *Baseline Requirements* del CA/Browser Forum, que hoy exigen, por ejemplo, $\text{RSA} \geq 2048$ o EC P-256/P-384, además de reglas de emisión, vigencia y revocación para que un certificado sea públicamente confiable [52].

A.1.5. Guías de migración en la capa de aplicación

El IETF está publicando guías de migración a criptografía post-cuántica para los protocolos de Internet: describen los retos (tamaño de mensajes, *fallbacks*, compatibilidad), proponen perfiles “*quantum-ready*” y sugieren fases de despliegue para reducir el riesgo operativo. Además, hay materiales específicos para ingenieros que aterrizan los impactos prácticos (cambios de API, *buffers*, telemetría, pruebas A/B) cuando se integra PQC en pilas reales [53].

A.1.6. Autenticación Web y nombres de dominio

En autenticación Web, la FIDO Alliance ha publicado un *white paper* con líneas para adaptar WebAuthn/FIDO2 al mundo post-cuántico, abordando interoperabilidad, formatos de credenciales y rutas de transición para proveedores de identidad y navegadores [54]. En DNSSEC, estudios recientes evalúan firmas PQC frente a RSA/ECDSA en operadores de DNS (tamaños de respuesta, latencias, fiabilidad UDP/TCP) y proponen mecanismos de fragmentación/optimización para mantener tiempos de resolución razonables; también se sugiere diversificar algoritmos (uno conservador + otro eficiente) para asegurar buen rendimiento y *fallback* robusto en la migración [55].

ESTUDIO: RENDIMIENTO DE ALGORITMOS POST-CUÁNTICOS EN DISPOSITIVO ORDENADOR

A continuación se presentan los resultados completos obtenidos en las pruebas de rendimiento de los algoritmos post-cuánticos BIKE, HQC, Classic McEliece, Kyber, Dilithium, Falcon y SPHINCS+, evaluados en las implementaciones desarrolladas en los lenguajes C, C#, Python y Java para el dispositivo ordenador personal.

Las tablas incluidas en este anexo recogen los tiempos medios y sus correspondientes desviaciones estándar para cada una de las operaciones fundamentales de los diferentes esquemas: generación de claves, encapsulación, desencapsulación, firma y verificación. Estos resultados permiten comparar el rendimiento tanto entre los distintos niveles de seguridad de cada algoritmo como entre sus implementaciones en diferentes lenguajes de programación.

Adicionalmente, se incluyen tablas relativas al consumo de recursos, donde se detalla el uso máximo de CPU y la memoria alcanzada durante las ejecuciones. Estos datos complementan el análisis temporal al aportar una visión más completa del coste computacional que implica cada implementación.

El conjunto de tablas B.1 a B.54 organiza toda esta información de manera exhaustiva, clasificada por algoritmo, lenguaje y tipo de operación, constituyendo una referencia detallada que sirve como base para el análisis comparativo llevado a cabo en este trabajo.

BIKE

Lenguaje	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
C	BIKE L1 Portable	0.280 ms ($\pm$ 0.034)	0.058 ms ($\pm$ 0.007)	1.108 ms ($\pm$ 0.082)	1.446 ms ($\pm$ 0.123)
C	BIKE L1 AVX2	0.412 ms ($\pm$ 0.065)	0.084 ms ($\pm$ 0.014)	1.746 ms ($\pm$ 0.182)	2.242 ms ($\pm$ 0.261)
C	BIKE L3 Portable	0.798 ms ($\pm$ 0.048)	0.128 ms ($\pm$ 0.005)	3.464 ms ($\pm$ 0.188)	4.390 ms ($\pm$ 0.241)
C	BIKE L3 AVX2	0.868 ms ($\pm$ 0.122)	0.140 ms ($\pm$ 0.027)	3.757 ms ($\pm$ 0.468)	4.765 ms ($\pm$ 0.617)
C	BIKE L5 Portable	2.435 ms ($\pm$ 0.136)	0.333 ms ($\pm$ 0.010)	9.587 ms ($\pm$ 0.582)	12.355 ms ($\pm$ 0.728)
C	BIKE L5 AVX2	2.458 ms ($\pm$ 0.548)	0.323 ms ($\pm$ 0.088)	9.726 ms ($\pm$ 2.023)	12.507 ms ($\pm$ 2.659)

BIKE — Rendimiento

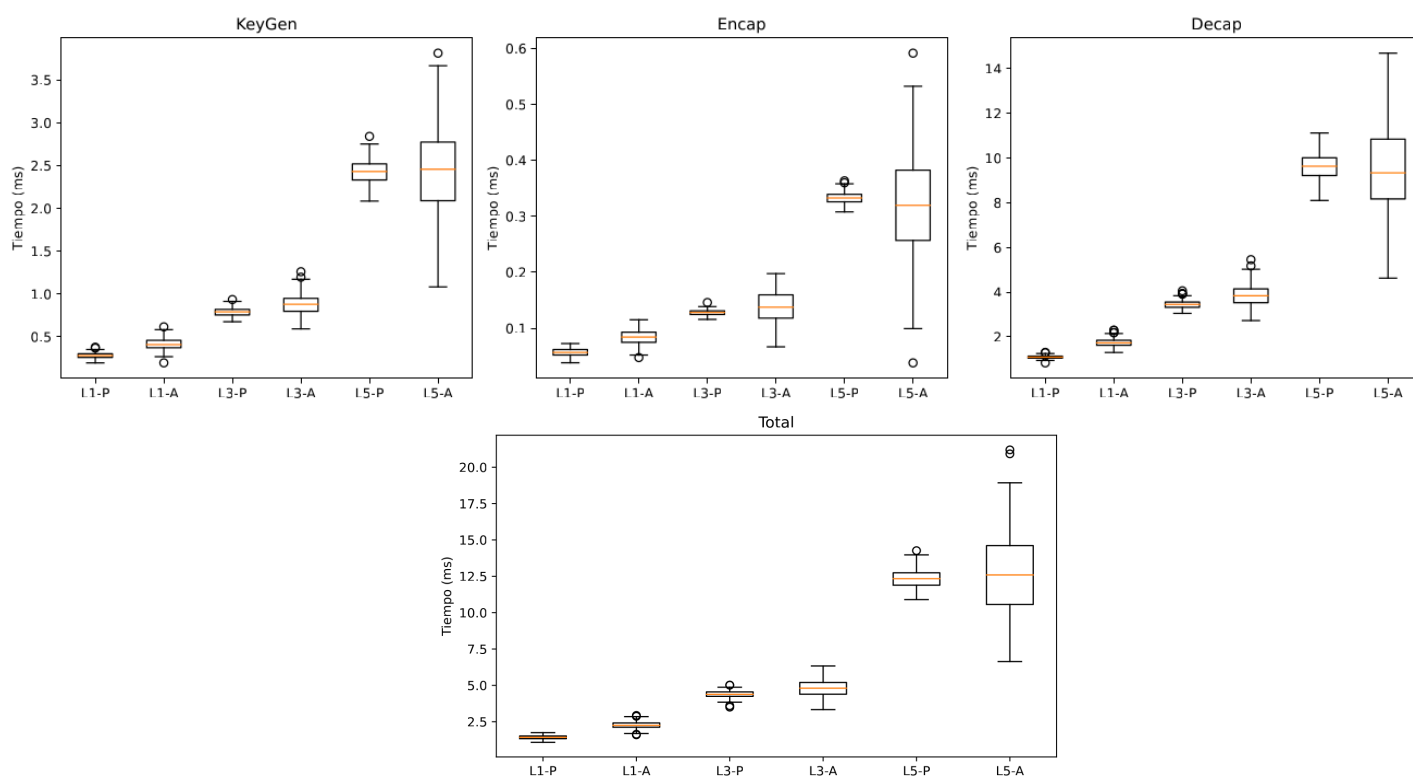
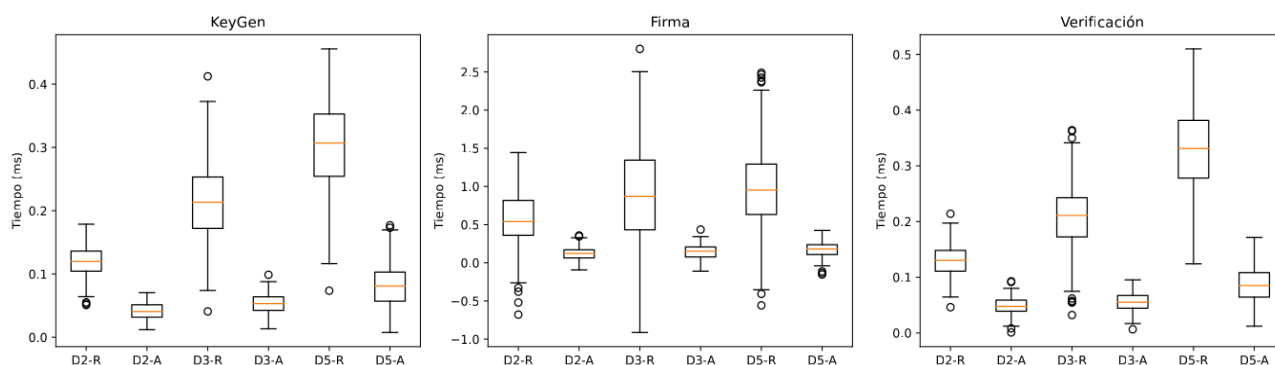


Tabla B.1. Resultados del algoritmo **BIKE** (implementación en **C**) en dispositivo ordenador.

DILITHIUM

LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
C	Dilithium2 Reference	0.1174 ms (± 0.0242)	0.5345 ms (± 0.3664)	0.1341 ms (± 0.0290)	0.7860 ms (± 0.3765)
C	Dilithium2 AVX2	0.0403 ms (± 0.0135)	0.1192 ms (± 0.0796)	0.0499 ms (± 0.0173)	0.2094 ms (± 0.0957)
C	Dilithium3 Reference	0.2113 ms (± 0.0658)	0.8608 ms (± 0.6505)	0.2109 ms (± 0.0616)	1.2830 ms (± 0.6921)
C	Dilithium3 AVX2	0.0525 ms (± 0.0150)	0.1485 ms (± 0.0961)	0.0580 ms (± 0.0172)	0.2591 ms (± 0.1093)
C	Dilithium5 Reference	0.3062 ms (± 0.0726)	0.9538 ms (± 0.6082)	0.3254 ms (± 0.0754)	1.5854 ms (± 0.6508)
C	Dilithium5 AVX2	0.0805 ms (± 0.0336)	0.1699 ms (± 0.1115)	0.0853 ms (± 0.0362)	0.3356 ms (± 0.1638)

Dilithium — Rendimiento



Dilithium — Rendimiento

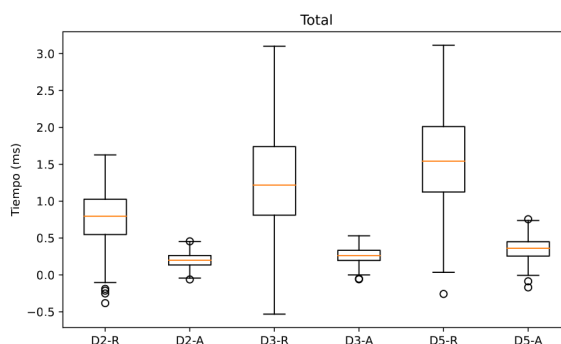


Tabla B.2. Resultados del algoritmo **Dilithium** (implementación en **C**) en dispositivo ordenador.

FALCON

LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
C	Falcon 512 Reference	18.6774 ms (± 6.4237)	5.2753 ms (± 0.2409)	0.0458ms (± 0.0070)	23.9985 ms (± 6.4419)
C	Falcon 512 AVX2	8.3794 ms (± 2.3638)	0.2644 ms (± 0.0382)	0.0456 ms (± 0.0087)	8.6895 ms (± 2.3661)
C	Falcon 512 Native	7.9890 ms (± 2.3160)	0.2532 ms (± 0.0416)	0.0446 ms (± 0.0080)	8.2868 ms (± 2.3257)
C	Falcon 1024	59.0729 ms (± 23.3993)	12.5074 ms (± 1.3203)	0.1099 ms (± 0.0281)	71.6902 ms (± 23.6523)
C	Falcon 1024 AVX2	25.6746 ms (± 7.1850)	0.5409 ms (± 0.0423)	0.0957 ms (± 0.0108)	26.3112 ms (± 7.1878)
C	Falcon 1024 Native	23.5175 ms (± 6.5236)	0.5079 ms (± 0.0707)	0.0947 ms (± 0.0175)	24.1201ms (± 6.5362)

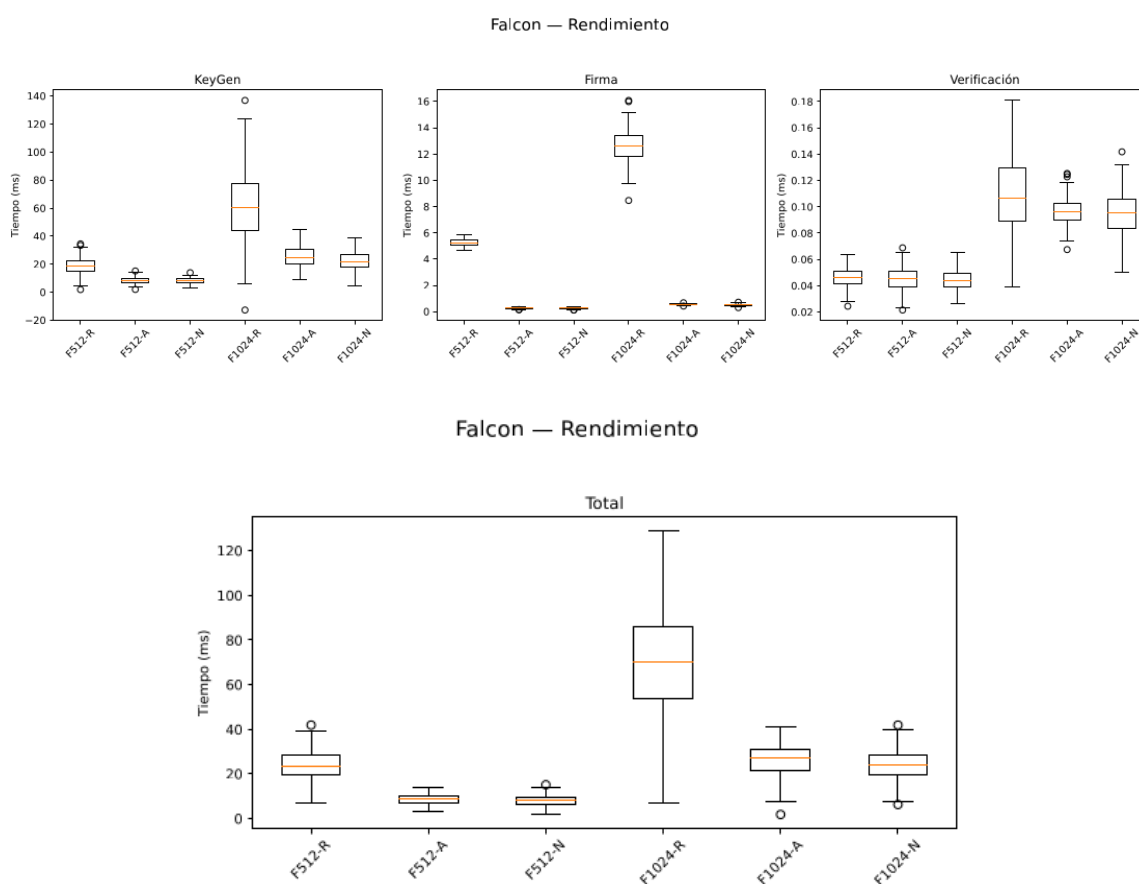


Tabla B.3. Resultados del algoritmo **Falcon** (implementación en **C**) en dispositivo ordenador.

HQC

LENGUAJE	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
C	hqc 128 Reference	0.2343 ms (± 0.0566)	0.4919 ms (± 0.1047)	0.7986 ms (± 0.1700)	1.5248 ms (± 0.3041)
C	hqc 128 AVX2	0.0274 ms (± 0.0038)	0.0629 ms (± 0.0064)	0.1070 ms (± 0.0106)	0.1972 ms (± 0.0185)
C	hqc 192 Reference	0.7709 ms (± 0.2312)	1.5790 ms (± 0.4121)	2.4487 ms (± 0.6135)	4.7986 ms (± 1.1097)
C	hqc 192 AVX2	0.0647 ms (± 0.0056)	0.1465 ms (± 0.0111)	0.2253 ms (± 0.0167)	0.4365 ms (± 0.0301)
C	hqc 256 Reference	1.2830 ms (± 0.2926)	2.6821 ms (± 0.6133)	4.1602 ms (± 0.8413)	8.1253 ms (± 1.5831)
C	hqc 256 AVX2	0.1254 ms (± 0.0139)	0.2761 ms (± 0.0274)	0.4361 ms (± 0.0430)	0.8376 ms (± 0.0757)

HQC — Rendimiento

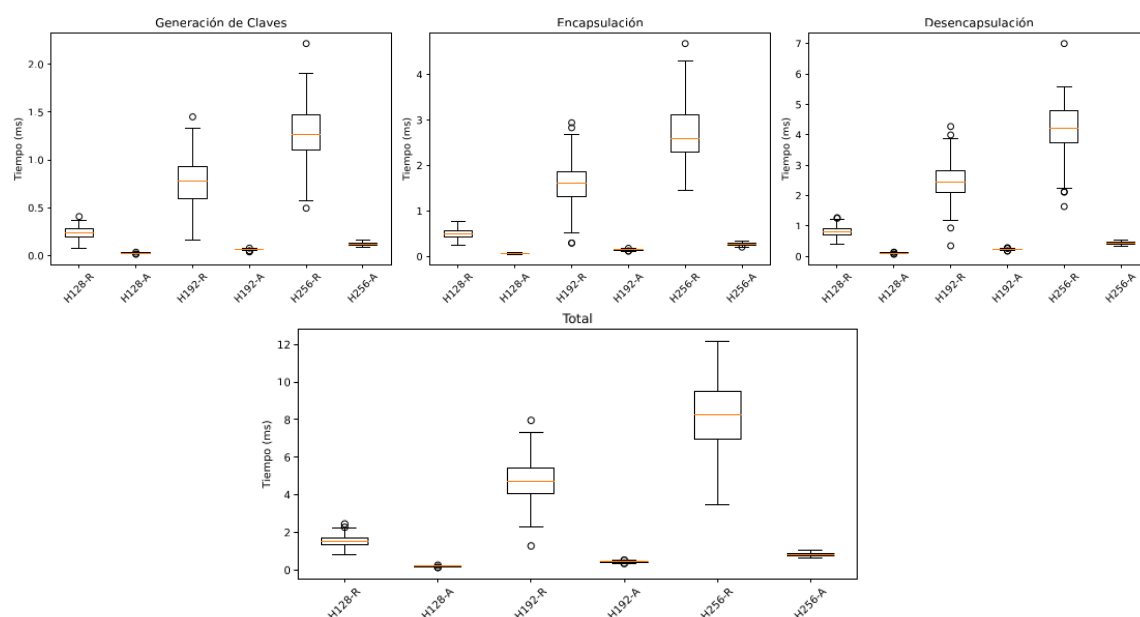


Tabla B.4. Resultados del algoritmo HQC (implementación en C) en dispositivo ordenador.

Kyber

LENGUAJE	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
C	Kyber 512 Reference	0.0439 ms (± 0.0084)	0.0528 ms (± 0.0104)	0.0663 ms (± 0.0122)	0.1630 ms (± 0.0251)
C	Kyber 512 AVX2	0.0099 ms (± 0.0021)	0.0108 ms (± 0.0029)	0.0114 ms (± 0.0026)	0.0321 ms (± 0.0060)
C	Kyber 768 Reference	0.0752 ms (± 0.1595)	0.0814 ms (± 0.0241)	0.0988 ms (± 0.0292)	0.2554 ms (± 0.1735)
C	Kyber 768 AVX2	0.0170 ms (± 0.0075)	0.0168 ms (± 0.0073)	0.0180 ms (± 0.0079)	0.0517 ms (± 0.0206)
C	Kyber 1024 Reference	0.1017 ms (± 0.0479)	0.1179 ms (± 0.0552)	0.1384 ms (± 0.0647)	0.3579 ms (± 0.1647)
C	Kyber 1024 AVX2	0.0214 ms (± 0.0063)	0.0219 ms (± 0.0057)	0.0232 ms (± 0.0056)	0.0665 ms (± 0.0145)

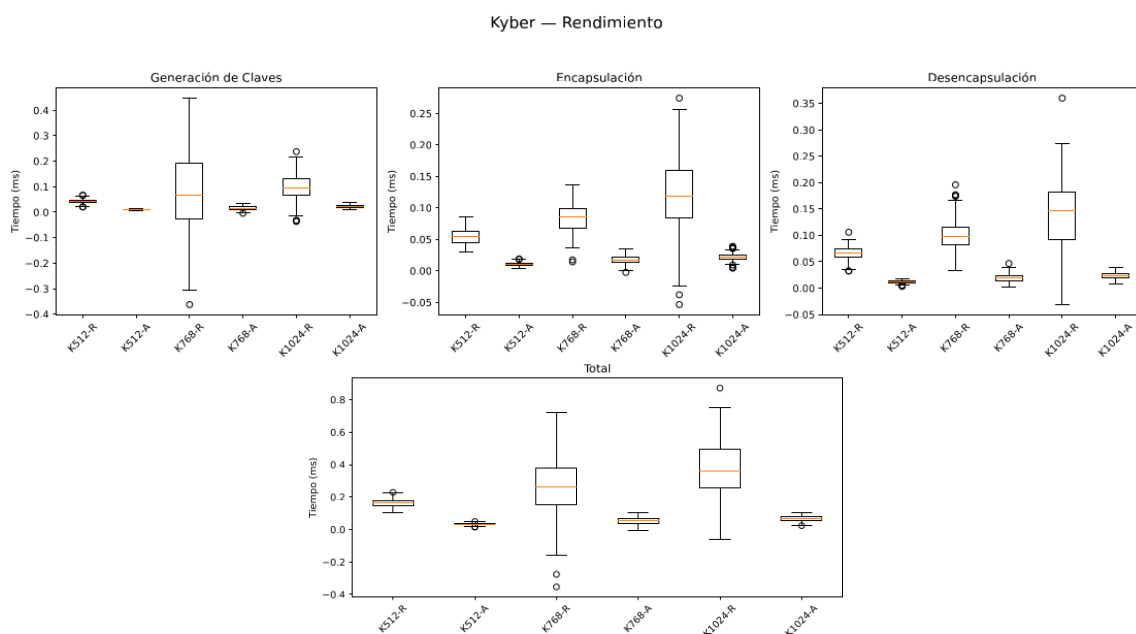


Tabla B.5. Resultados del algoritmo **Kyber** (implementación en **C**) en dispositivo ordenador.

SPHINCS+

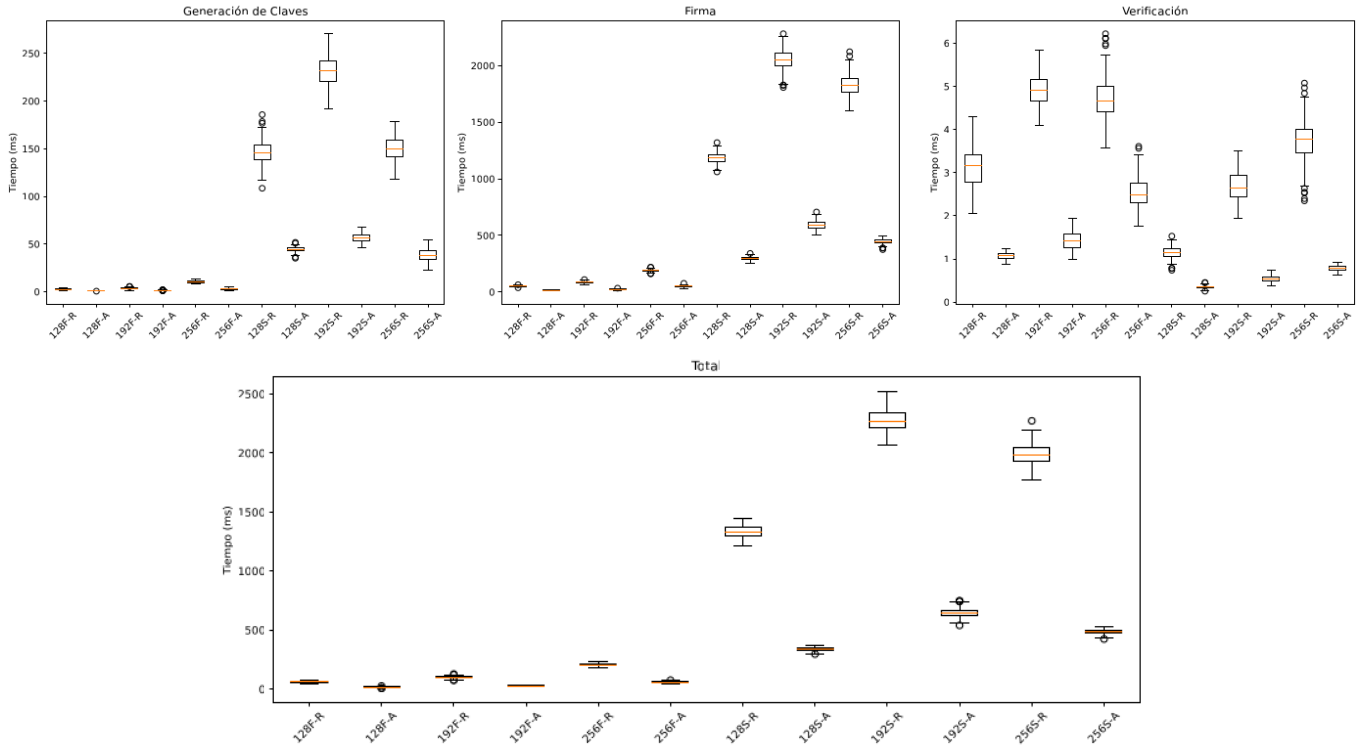
LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
C	SHA256-128F Simple Reference	1.21 ms ($\pm$ 0.27)	28.85 ms ($\pm$ 0.83)	1.90 ms ($\pm$ 0.13)	31.96 ms ($\pm$ 0.88)
C	SHA256-192F Simple Reference	2.46 ms ($\pm$ 0.19)	51.05 ms ($\pm$ 0.90)	2.69 ms ($\pm$ 0.62)	56.20 ms ($\pm$ 1.11)
C	SHA256-256F Simple Reference	4.93 ms ($\pm$ 0.38)	102.34 ms ($\pm$ 16.58)	2.87 ms ($\pm$ 0.11)	110.14 ms ($\pm$ 16.58)
C	SHA256-128S Simple Reference	85.11 ms ($\pm$ 10.07)	651.08 ms ($\pm$ 5.30)	0.61 ms ($\pm$ 0.04)	736.80 ms ($\pm$ 11.38)
C	SHA256-192S Simple Reference	124.65 ms ($\pm$ 8.75)	1133.94 ms ($\pm$ 29.93)	0.98 ms ($\pm$ 0.15)	1259.58 ms ($\pm$ 31.18)
C	SHA256-256S Simple Reference	82.98 ms ($\pm$ 9.64)	989.36 ms ($\pm$ 83.65)	1.53 ms ($\pm$ 0.18)	1073.87 ms ($\pm$ 84.20)
C	SHAKE256-128F Simple Reference	2.45 ms ($\pm$ 0.50)	54.11 ms ($\pm$ 4.15)	3.08 ms ($\pm$ 0.42)	59.65 ms ($\pm$ 4.20)
C	SHAKE256-192F Simple Reference	3.27 ms ($\pm$ 0.81)	89.88 ms ($\pm$ 8.68)	4.89 ms ($\pm$ 0.37)	98.04 ms ($\pm$ 8.72)
C	SHAKE256-256F Simple Reference	10.48 ms ($\pm$ 1.20)	190.37 ms ($\pm$ 9.76)	4.80 ms ($\pm$ 0.47)	205.65 ms ($\pm$ 9.84)
C	SHAKE256-128S Simple Reference	147.50 ms ($\pm$ 12.30)	1184.03 ms ($\pm$ 45.91)	1.14 ms ($\pm$ 0.14)	1332.66 ms ($\pm$ 47.53)
C	SHAKE256-192S Simple Reference	230.28 ms ($\pm$ 15.13)	2044.56 ms ($\pm$ 87.22)	2.65 ms ($\pm$ 0.33)	2277.49 ms ($\pm$ 88.52)
C	SHAKE256-256S Simple Reference	151.01 ms ($\pm$ 12.49)	1834.82 ms ($\pm$ 92.23)	3.72 ms ($\pm$ 0.46)	1989.55 ms ($\pm$ 93.07)
C	SHAKE256-128F Simple AVX2	0.71 ms ($\pm$ 0.04)	15.20 ms ($\pm$ 3.13)	1.07 ms ($\pm$ 0.08)	16.98 ms ($\pm$ 3.13)
C	SHAKE256-192F Simple AVX2	1.48 ms ($\pm$ 0.13)	24.05 ms ($\pm$ 3.23)	1.43 ms ($\pm$ 0.22)	26.95 ms ($\pm$ 3.24)
C	SHAKE256-256F Simple AVX2	2.98 ms ($\pm$ 0.75)	50.71 ms ($\pm$ 6.74)	2.61 ms ($\pm$ 0.34)	56.29 ms ($\pm$ 6.79)

Tabla B.6. SPHINCS+ (C) — Parte 1.

C	SHAKE256-128S Simple AVX2	43.94 ms (± 2.62)	295.44 ms (± 15.61)	0.35 ms (± 0.03)	339.73 ms (± 15.83)
C	SHAKE256-192S Simple AVX2	56.53 ms (± 4.27)	589.23 ms (± 37.57)	0.53 ms (± 0.07)	646.29 ms (± 37.81)
C	SHAKE256-256S Simple AVX2	38.67 ms (± 6.76)	443.64 ms (± 19.29)	0.78 ms (± 0.06)	483.09 ms (± 20.44)
C	SHA256-128F Simple AVX2	0.23 ms (± 0.05)	5.60 ms (± 0.65)	0.46 ms (± 0.10)	6.29 ms (± 0.66)
C	SHA256-192F Simple AVX2	0.42 ms (± 0.06)	9.67 ms (± 0.72)	0.68 ms (± 0.18)	10.77 ms (± 0.74)
C	SHA256-256F Simple AVX2	1.62 ms (± 0.22)	20.31 ms (± 1.28)	0.73 ms (± 0.04)	22.66 ms (± 1.30)
C	SHA256-128S Simple AVX2	16.99 ms (± 1.14)	121.66 ms (± 9.33)	0.19 ms (± 0.01)	138.83 ms (± 9.40)
C	SHA256-192S Simple AVX2	21.64 ms (± 1.51)	212.57 ms (± 2.23)	0.30 ms (± 0.02)	234.51 ms (± 2.69)
C	SHA256-256S Simple AVX2	15.26 ms (± 2.45)	192.24 ms (± 4.80)	0.38 ms (± 0.03)	207.88 ms (± 5.39)

Tabla B.7. SPHINCS+ (C) — Parte 2.

SPHINCS+ (SHAKE-256) — Rendimiento



SPHINCS+ (SHA-256) — Rendimiento

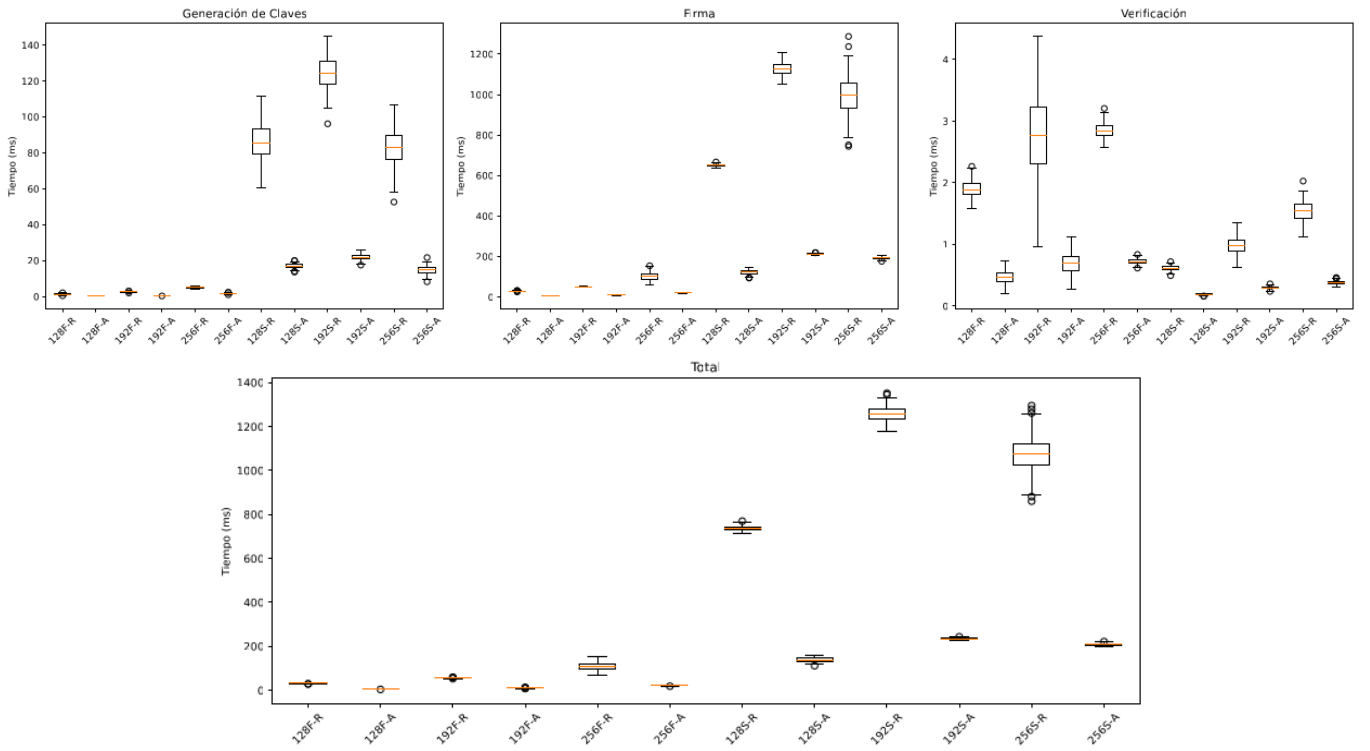


Tabla B.8. SPHINCS+ (C) — Parte 3.

McEliece					
Lenguaje	Versión	T Gen. Claves (ms)	T Enc (ms)	T Dec (ms)	T Total (ms)
C	McEliece348864 Reference	95.07 (±65.51)	0.0619 (±0.0133)	15.78 (±0.85)	110.91 (±65.31)
C	McEliece348864 AVX2	98.40 (±65.68)	0.0667 (±0.0300)	19.50 (±1.84)	117.97 (±65.62)
C	McEliece348864f Reference	44.24 (±4.68)	0.0605 (±0.0160)	15.79 (±0.96)	60.09 (±5.17)
C	McEliece348864f AVX2	44.66 (±1.90)	0.0591 (±0.0145)	19.08 (±1.21)	63.80 (±2.82)
C	McEliece460896 Reference	288.33 (±184.58)	0.130 (±0.0241)	36.56 (±1.03)	325.01 (±184.75)
C	McEliece460896 AVX2	222.58 (±115.66)	0.117 (±0.0299)	38.02 (±1.85)	260.72 (±115.40)
C	McEliece460896f Reference	137.76 (±4.60)	0.139 (±0.0280)	36.89 (±1.27)	174.79 (±5.00)
C	McEliece460896f AVX2	123.04 (±4.20)	0.109 (±0.0417)	38.55 (±2.31)	161.70 (±5.26)
C	McEliece6688128 Reference	767.56 (±514.26)	0.254 (±0.141)	81.28 (±10.17)	849.09 (±515.22)
C	McEliece6688128 AVX2	531.36 (±351.85)	0.195 (±0.0342)	71.30 (±4.15)	602.85 (±351.83)

Tabla B.9. Classic McEliece (C) — Parte 1.

C	McEliece6688128f Reference	250.90 (±20.45)	0.211 (±0.0394)	73.41 (±6.65)	324.52 (±26.50)
C	McEliece6688128f AVX2	203.52 (±6.19)	0.195 (±0.0347)	72.25 (±3.78)	275.96 (±8.09)
C	McEliece6960119 Reference	704.12 (±563.27)	0.832 (±0.113)	72.32 (±7.37)	777.28 (±564.29)
C	McEliece6960119 AVX2	475.29 (±360.41)	0.308 (±0.039)	69.09 (±2.41)	544.69 (±360.42)
C	McEliece6960119f Reference	229.60 (±4.26)	0.808 (±0.091)	68.40 (±2.16)	298.80 (±4.82)
C	McEliece6960119f AVX2	188.21 (±8.94)	0.310 (±0.038)	68.66 (±2.21)	257.18 (±10.56)
C	McEliece8192128 Reference	820.04 (±725.96)	0.263 (±0.122)	87.79 (±2.77)	908.09 (±725.47)
C	McEliece8192128 AVX2	583.82 (±493.36)	0.236 (±0.024)	87.57 (±2.20)	671.63 (±493.36)
C	McEliece8192128f Reference	303.59 (±8.59)	0.242 (±0.067)	88.17 (±2.50)	392.00 (±9.52)
C	McEliece8192128f AVX2	207.89 (±5.50)	0.238 (±0.035)	87.34 (±3.04)	295.46 (±7.23)

Tabla B.10. Classic McEliece (C) — Parte 2.

Classic McEliece — Rendimiento (Reference vs AVX2)

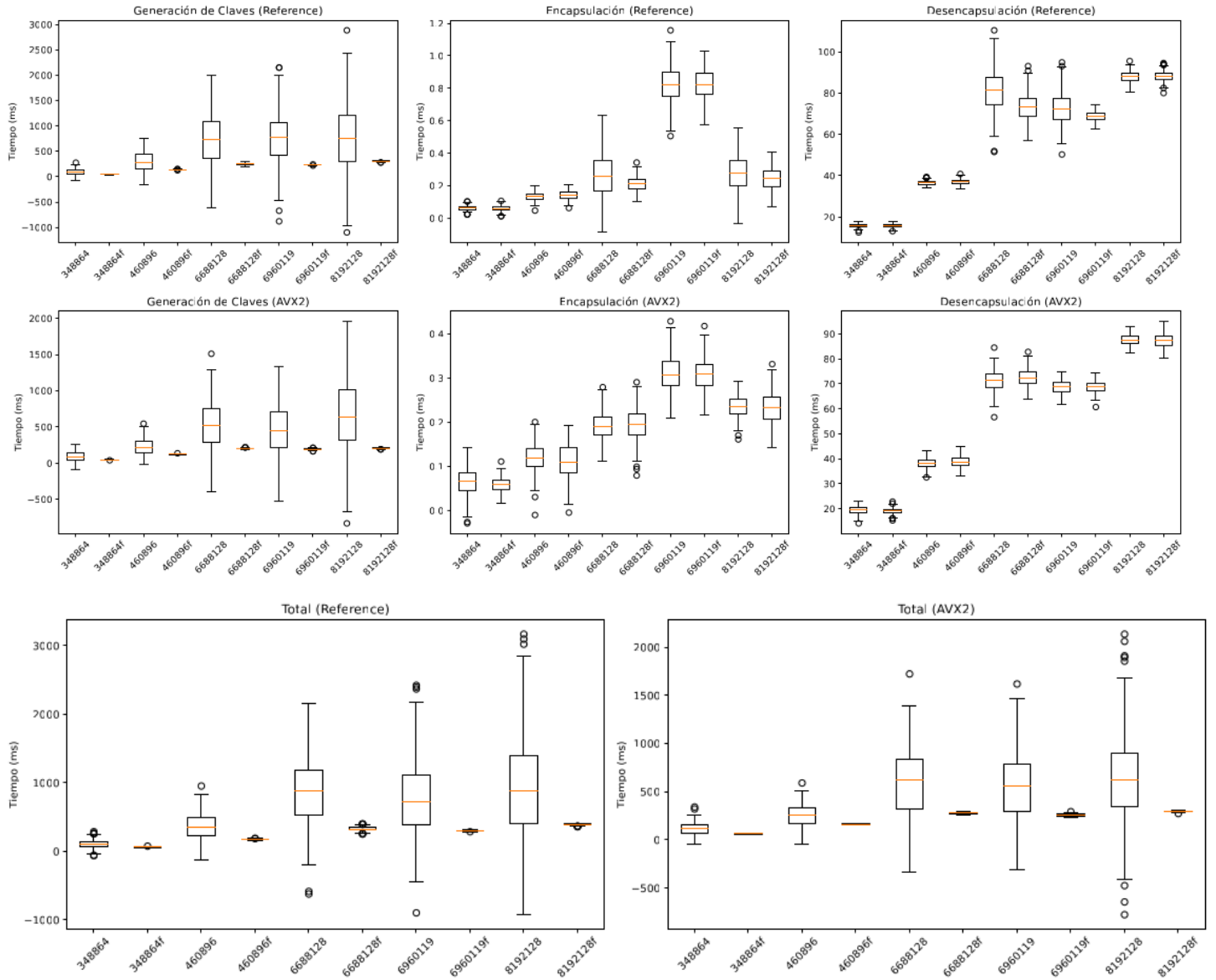


Tabla B.11. Classic McEliece (C) — Parte 3.

BIKE

LENGUAJE	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
C#	BIKE-128	1.0323 ms ($\pm$ 0.3348)	0.1332 ms ($\pm$ 0.1324)	1.0266 ms ($\pm$ 0.2570)	2.1921 ms ($\pm$ 0.7025)
C#	BIKE-192	3.7797 ms ($\pm$ 0.1692)	0.3165 ms ($\pm$ 0.0338)	2.3398 ms ($\pm$ 0.1336)	6.4359 ms ($\pm$ 0.2465)
C#	BIKE-256	10.1351 ms ($\pm$ 0.3645)	0.6960 ms ($\pm$ 0.0576)	4.5440 ms ($\pm$ 0.2384)	15.3751 ms ($\pm$ 0.5259)

BIKE (C#) — Rendimiento

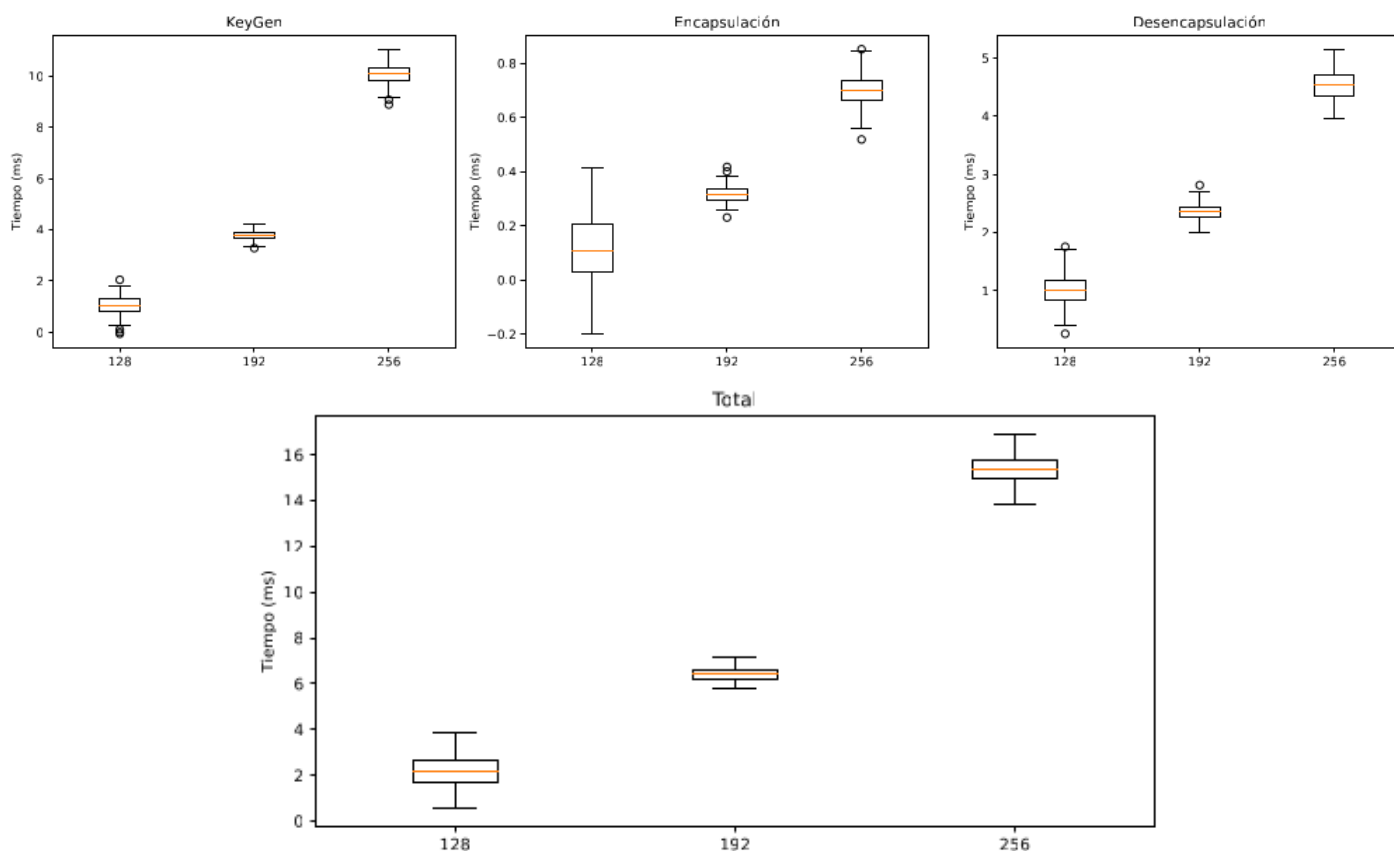


Tabla B.12. Resultados del algoritmo **BIKE** (C#) en dispositivo ordenador.

Dilithium

LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
C#	Dilithium 2	0.1854 ms ($\pm$ 0.0670)	0.7540 ms ($\pm$ 0.5644)	0.1962 ms ($\pm$ 0.0620)	1.1357 ms ($\pm$ 0.6047)
C#	Dilithium 3	0.2859 ms ($\pm$ 0.0644)	1.0899 ms ($\pm$ 0.7749)	0.2770 ms ($\pm$ 0.0627)	1.6528 ms ($\pm$ 0.7992)
C#	Dilithium 5	0.3892 ms ($\pm$ 0.0344)	1.1612 ms ($\pm$ 0.6884)	0.4033 ms ($\pm$ 0.0365)	1.9536 ms ($\pm$ 0.6903)

Dilithium (C#) — Rendimiento

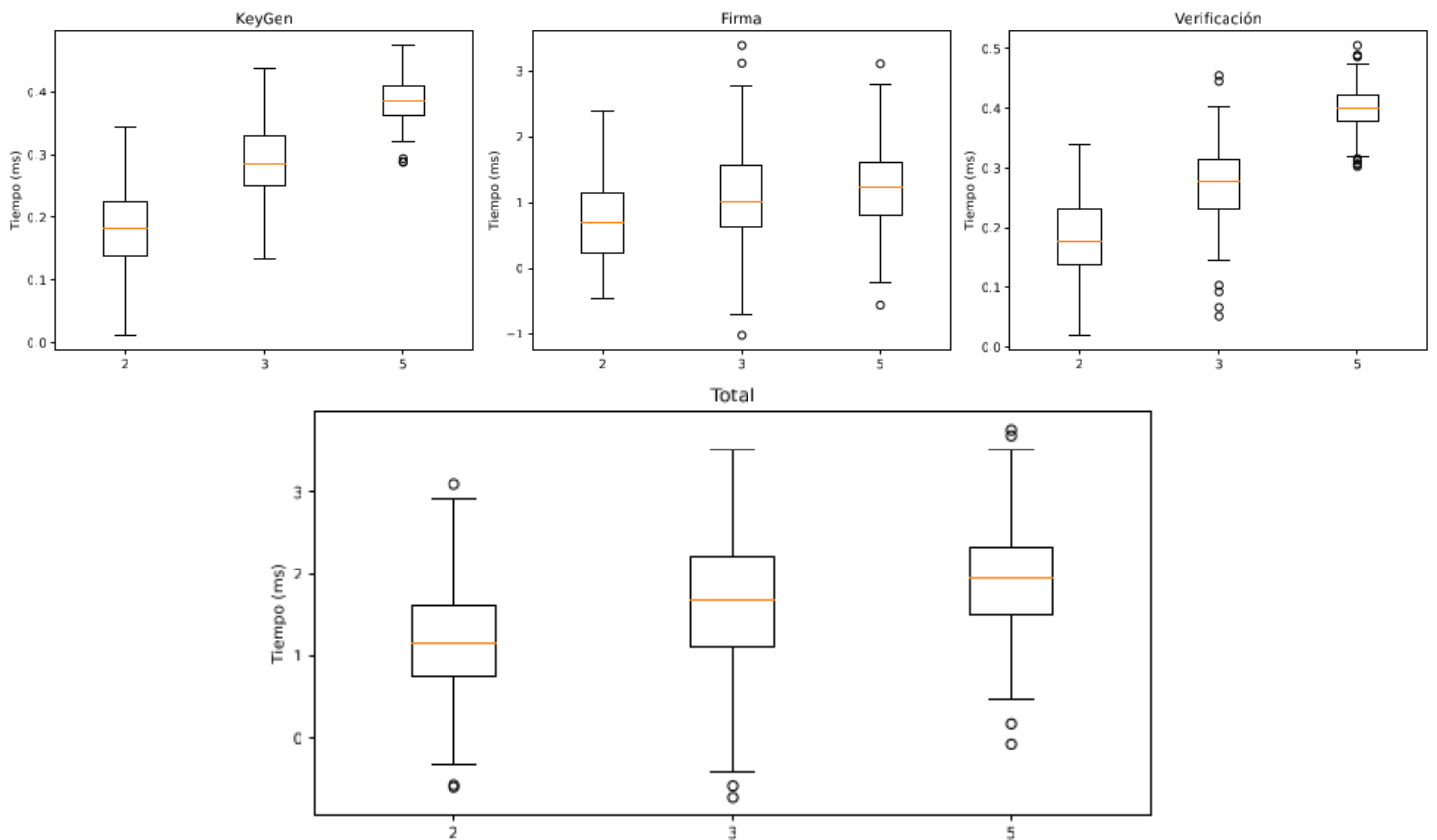


Tabla B.13. Resultados del algoritmo **Dilithium** (C#) en dispositivo ordenador.

Falcon

LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
C#	Falcon512 (grande)	14.2816 ms (± 5.6703)	1.7049 ms (± 0.3969)	0.1603 ms (± 0.0423)	16.1468 ms (± 5.8251)
C#	Falcon512 (pequeño)	15.9106 ms (± 7.0673)	1.8441 ms (± 0.5687)	0.0990 ms (± 0.0487)	17.8536 ms (± 7.3926)
C#	Falcon1024 (grande)	43.8473 ms (± 17.2116)	4.0270 ms (± 0.9964)	0.2863 ms (± 0.0796)	48.1606 ms (± 17.5864)
C#	Falcon1024 (pequeño)	37.4007 ms (± 12.8316)	3.3043 ms (± 0.5514)	0.1761 ms (± 0.0389)	40.8811 ms (± 12.9635)

Falcon (C#) — Rendimiento

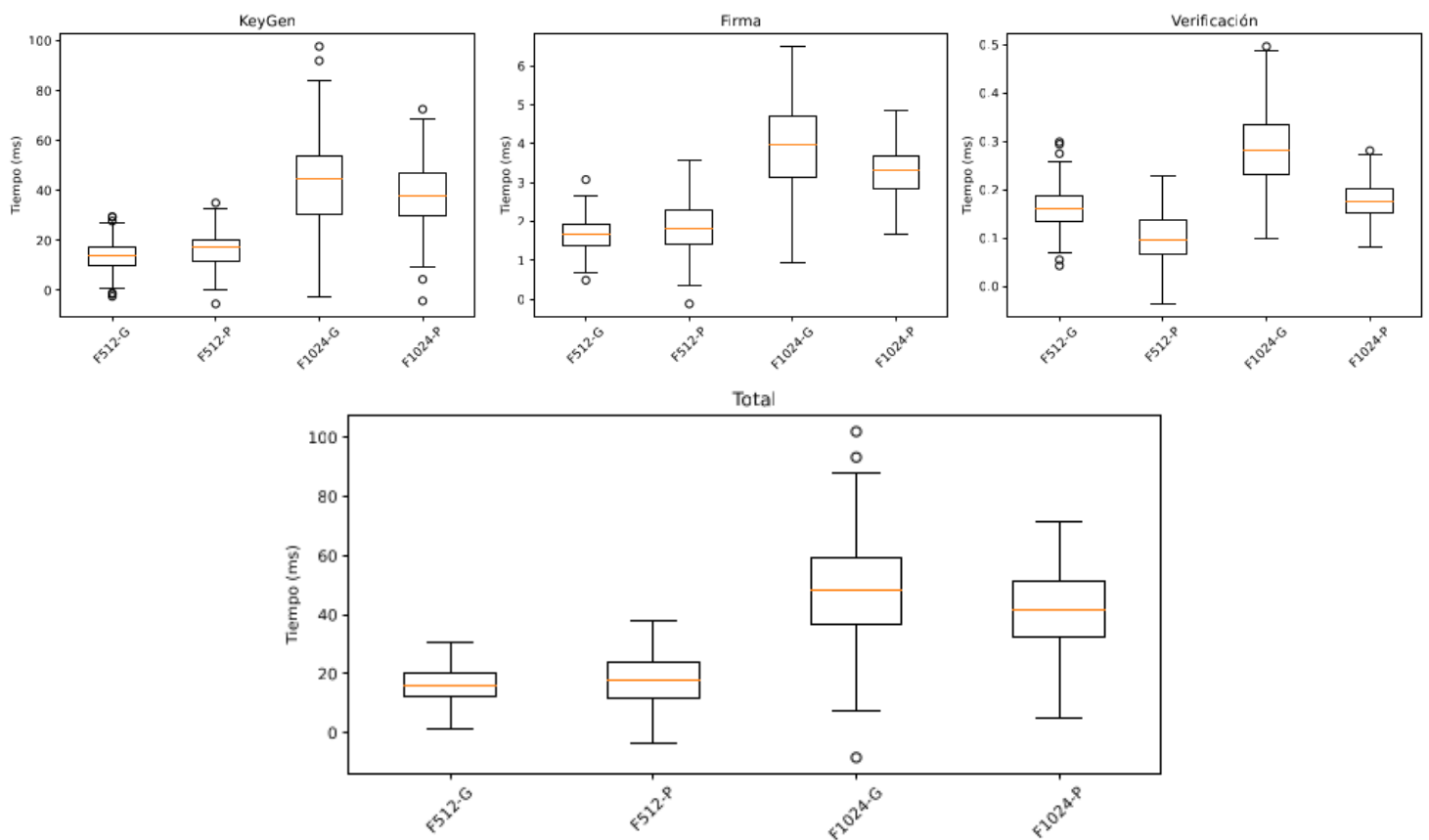


Tabla B.14. Resultados del algoritmo **Falcon** (C#) en dispositivo ordenador.

HQC

LENGUAJE	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
C#	HQC-128	4.3790 ms ($\pm$ 0.9817)	8.7427 ms ($\pm$ 1.7449)	13.2814 ms ($\pm$ 2.6616)	26.4031 ms ($\pm$ 5.1799)
C#	HQC-192	11.6284 ms ($\pm$ 0.5589)	23.2481 ms ($\pm$ 1.1592)	35.1153 ms ($\pm$ 1.7031)	69.9919 ms ($\pm$ 2.9876)
C#	HQC-256	21.1682 ms ($\pm$ 1.2557)	42.3825 ms ($\pm$ 2.5611)	63.9672 ms ($\pm$ 4.5167)	127.5178 ms ($\pm$ 7.3870)

HQC (C#) — Rendimiento

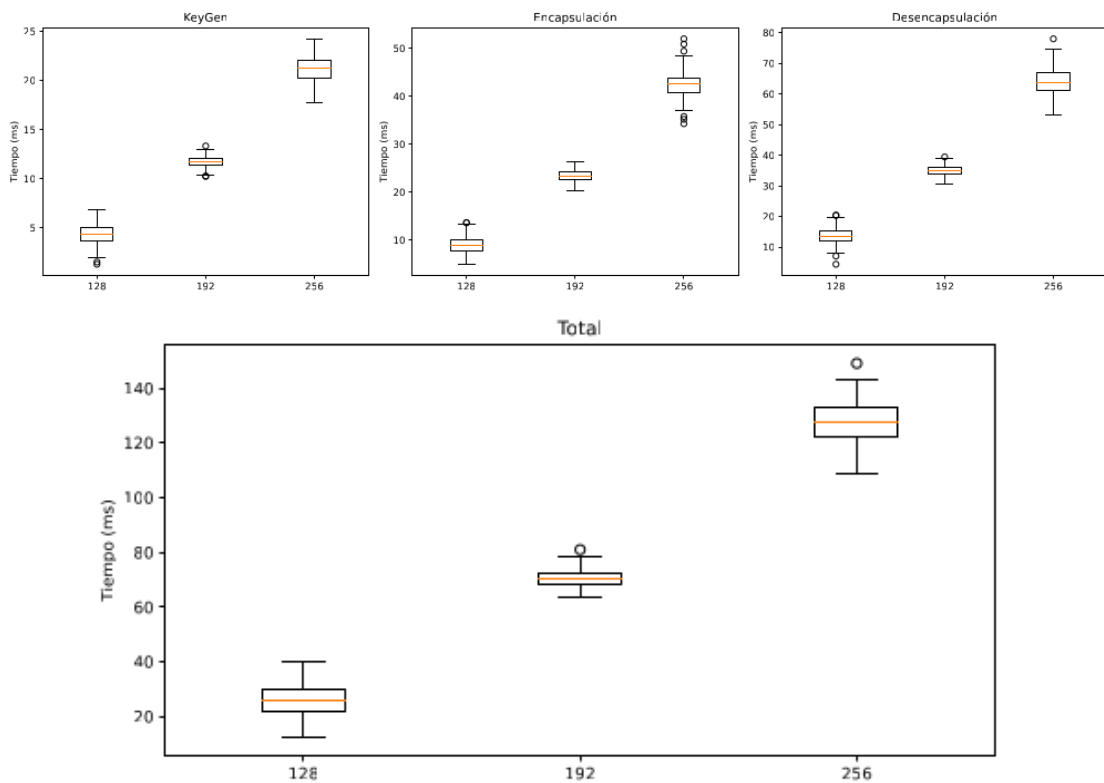


Tabla B.15. Resultados del algoritmo **HQC** (C#) en dispositivo ordenador.

Kyber

LENGUAJE	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
C#	Kyber-512	0.1143 ms ($\pm$ 0.0594)	0.1380 ms ($\pm$ 0.0696)	0.1727 ms ($\pm$ 0.0809)	0.4250 ms ($\pm$ 0.2057)
C#	Kyber-768	0.0806 ms ($\pm$ 0.0150)	0.0964 ms ($\pm$ 0.0185)	0.1118 ms ($\pm$ 0.0192)	0.2888 ms ($\pm$ 0.0348)
C#	Kyber-1024	0.1235 ms ($\pm$ 0.0190)	0.1429 ms ($\pm$ 0.0234)	0.1639 ms ($\pm$ 0.0258)	0.4303 ms ($\pm$ 0.0463)

Kyber (C#) — Rendimiento

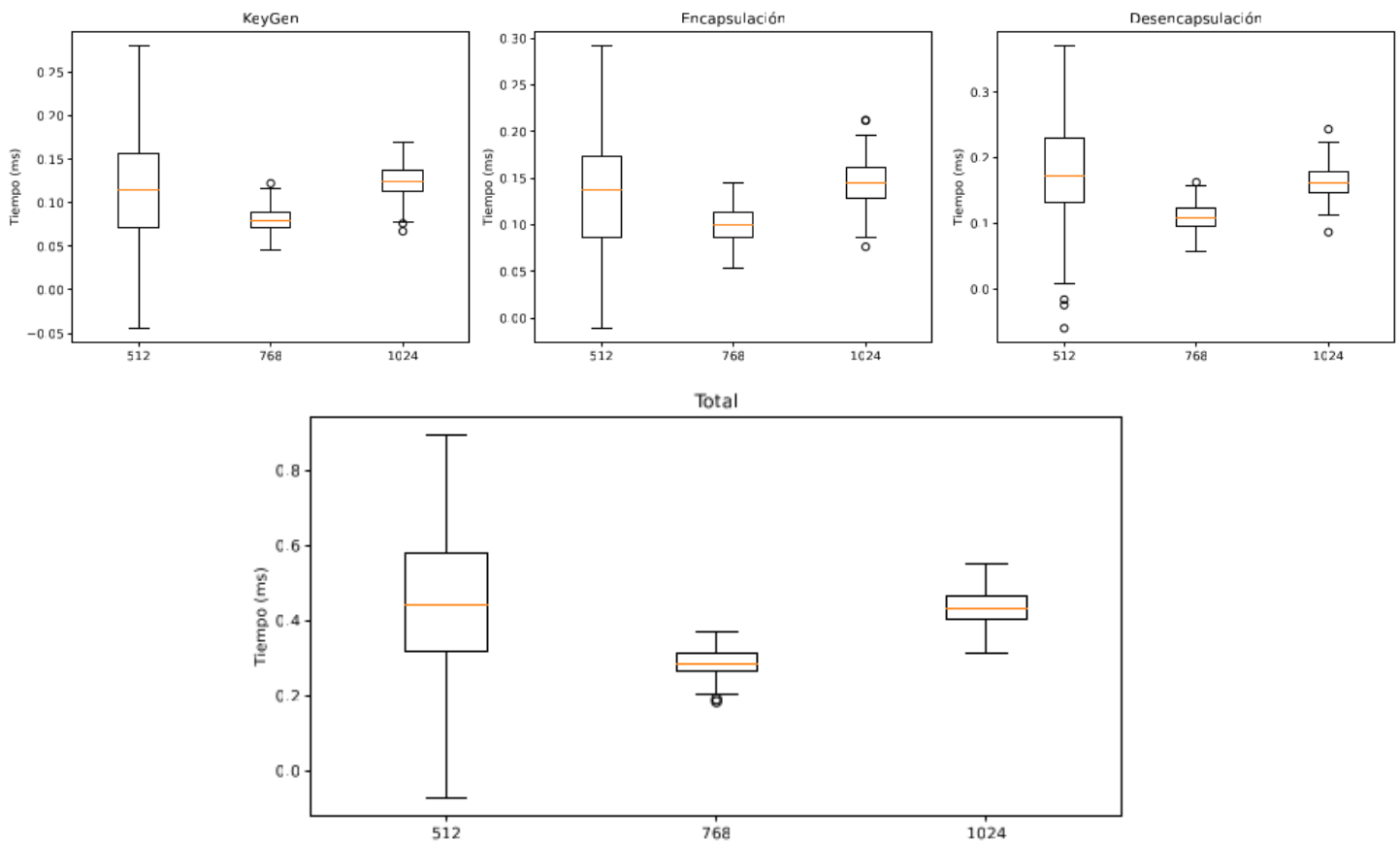


Tabla B.16. Resultados del algoritmo **Kyber** (C#) en dispositivo ordenador.

McEliece

LENGUAJE	Versión	T Gen. Claves	T Encapsulaci ón	T Desencapsulaci ón	T Total
C#	McEliece3488 64f	130.9914 ms (± 4.0021)	0.8505 ms (± 0.1237)	15.0241 ms (± 1.0982)	146.8687 ms (± 4.4664)
C#	McEliece3488 64	276.0427 ms (± 150.7761)	0.9973 ms (± 0.6379)	16.2201 ms (± 1.8193)	293.2863 ms (± 151.5192)
C#	McEliece4608 96f	380.0095 ms (± 28.2363)	1.6794 ms (± 0.1665)	36.0942 ms (± 2.8540)	417.7869 ms (± 29.1105)
C#	McEliece4608 96	551.2056 ms (± 224.6669)	2.0281 ms (± 0.4391)	36.0250 ms (± 3.9673)	589.2630 ms (± 225.2119)
C#	McEliece6688 128f	583.8130 ms (± 35.7870)	3.6458 ms (± 0.9735)	78.9996 ms (± 7.5208)	666.4632 ms (± 40.0028)
C#	McEliece6688 128	1254.412 9 ms (± 1117.365 6)	3.2171 ms (± 0.3576)	68.7999 ms (± 8.4329)	1326.433 7 ms (± 1116.307 1)
C#	McEliece8192 128	1270.172 9 ms (± 764.9907)	3.7900 ms (± 0.2731)	78.2809 ms (± 2.3796)	1352.247 1 ms (± 765.7607)
C#	McEliece8192 128f	591.0409 ms (± 79.2273)	4.0277 ms (± 0.4681)	88.3522 ms (± 15.8295)	683.4249 ms (± 90.1185)
C#	McEliece6960 119	1276.442 5 ms (± 742.7241)	6.0327 ms (± 0.9693)	75.7443 ms (± 4.7411)	1358.224 1 ms (± 743.3804)

Tabla B.17. Classic McEliece (C#) — Parte 1.

C#

McEliece6960
119f

536.9070
ms (±
48.3346)

5.2271 ms (±
0.5333)

69.4581 ms (±
8.1334)

611.5962
ms (±
56.0417)

McEliece (C#) — Rendimiento

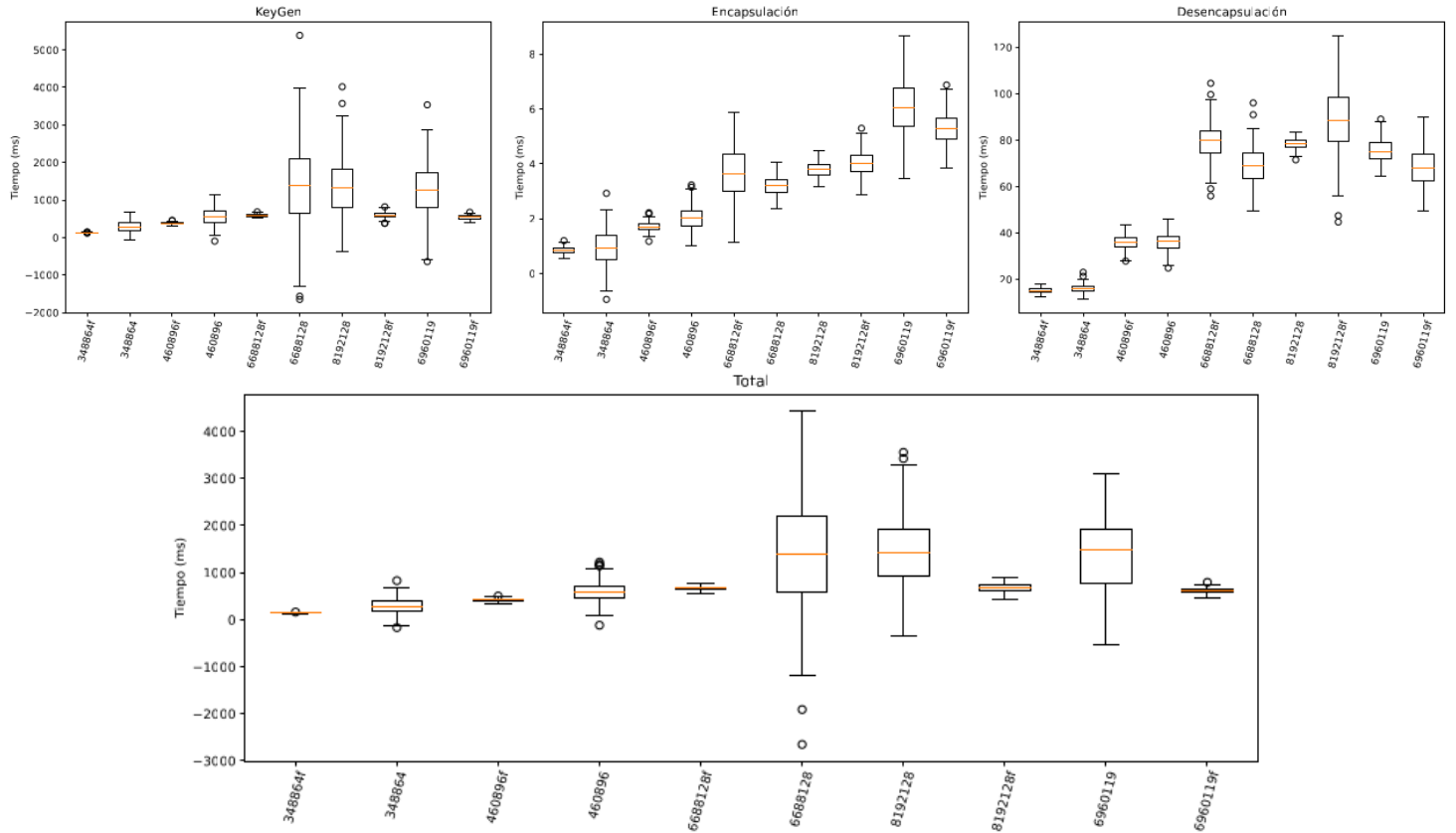


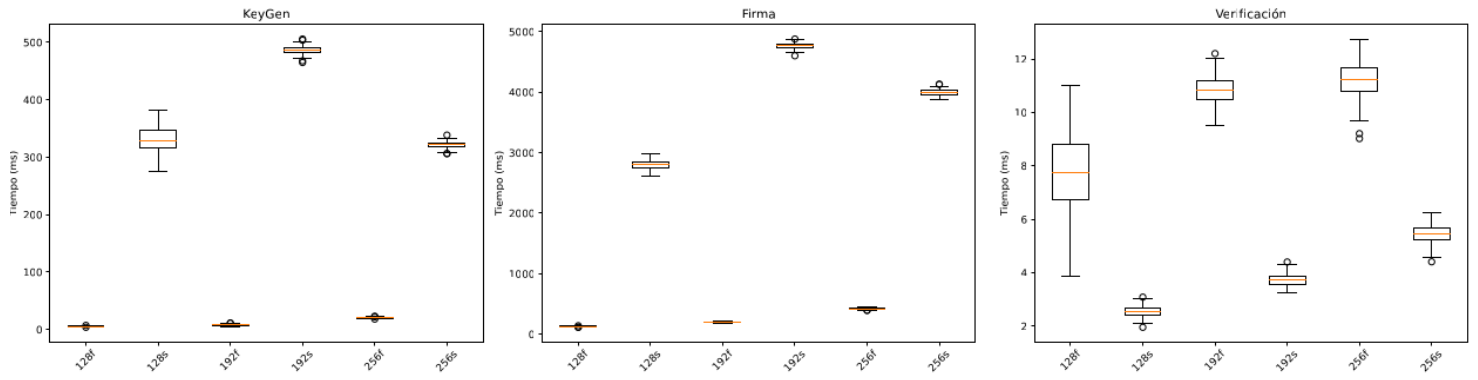
Tabla B.18. Classic McEliece (C#) — Parte 2.

SPHINCS+

LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
C#	SHA2-128f	5.5597 ms ($\pm$ 0.4875)	134.9195 ms ($\pm$ 10.4120)	8.0769 ms ($\pm$ 0.5498)	148.5575 ms ($\pm$ 11.0189)
C#	SHA2-128s	531.2919 ms ($\pm$ 168.6988)	4587.9249 ms ($\pm$ 970.7649)	4.0878 ms ($\pm$ 1.4313)	5123.3124 ms ($\pm$ 1028.5655)
C#	SHA2-192f	8.1385 ms ($\pm$ 0.3656)	216.8756 ms ($\pm$ 10.3913)	11.8456 ms ($\pm$ 0.7645)	236.8607 ms ($\pm$ 11.0775)
C#	SHA2-192s	517.7863 ms ($\pm$ 8.4143)	5090.0693 ms ($\pm$ 44.8718)	4.0434 ms ($\pm$ 0.1948)	5611.9003 ms ($\pm$ 51.4150)
C#	SHA2-256f	28.0866 ms ($\pm$ 0.8660)	574.2459 ms ($\pm$ 11.3621)	15.6364 ms ($\pm$ 0.6411)	617.9700 ms ($\pm$ 11.7229)
C#	SHA2-256s	449.0627 ms ($\pm$ 5.9306)	5425.3683 ms ($\pm$ 59.1710)	7.5869 ms ($\pm$ 0.2932)	5882.0194 ms ($\pm$ 63.0726)
C#	SHAKE-12 8f	5.3556 ms ($\pm$ 0.6654)	127.6777 ms ($\pm$ 4.4865)	7.7498 ms ($\pm$ 1.3189)	140.7841 ms ($\pm$ 5.2076)
C#	SHAKE-12 8s	330.9551 ms ($\pm$ 22.1937)	2804.7353 ms ($\pm$ 79.3863)	2.5476 ms ($\pm$ 0.2041)	3138.2402 ms ($\pm$ 88.9811)
C#	SHAKE-19 2f	7.7283 ms ($\pm$ 0.9785)	200.8252 ms ($\pm$ 8.3034)	10.8853 ms ($\pm$ 0.5184)	219.4400 ms ($\pm$ 9.1286)
C#	SHAKE-19 2s	487.3310 ms ($\pm$ 6.3786)	4755.9410 ms ($\pm$ 45.4990)	3.7488 ms ($\pm$ 0.2410)	5247.0228 ms ($\pm$ 49.8735)
C#	SHAKE-25 6f	20.2344 ms ($\pm$ 0.7795)	419.2671 ms ($\pm$ 9.1821)	11.2111 ms ($\pm$ 0.6942)	450.7141 ms ($\pm$ 10.0372)
C#	SHAKE-25 6s	321.1620 ms ($\pm$ 5.5478)	3996.4310 ms ($\pm$ 50.0486)	5.4832 ms ($\pm$ 0.3373)	4323.0781 ms ($\pm$ 52.5053)

Tabla B.19. SPHINCS+ (C#) — Parte 1.

SPHINCS+ (SHAKE-256, C#) — Rendimiento



SPHINCS+ (SHA-256, C#) — Rendimiento

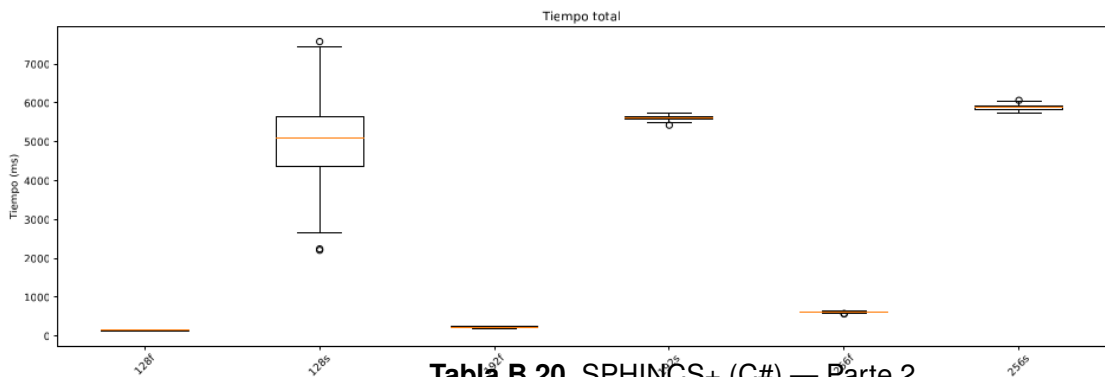
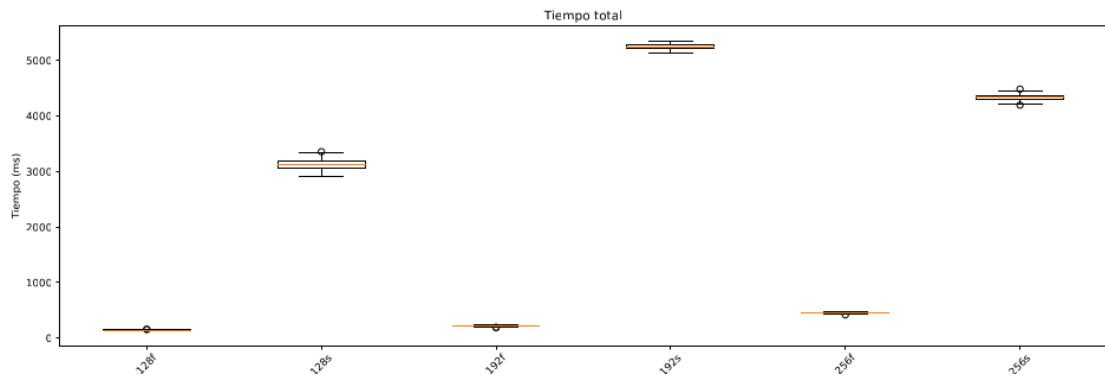
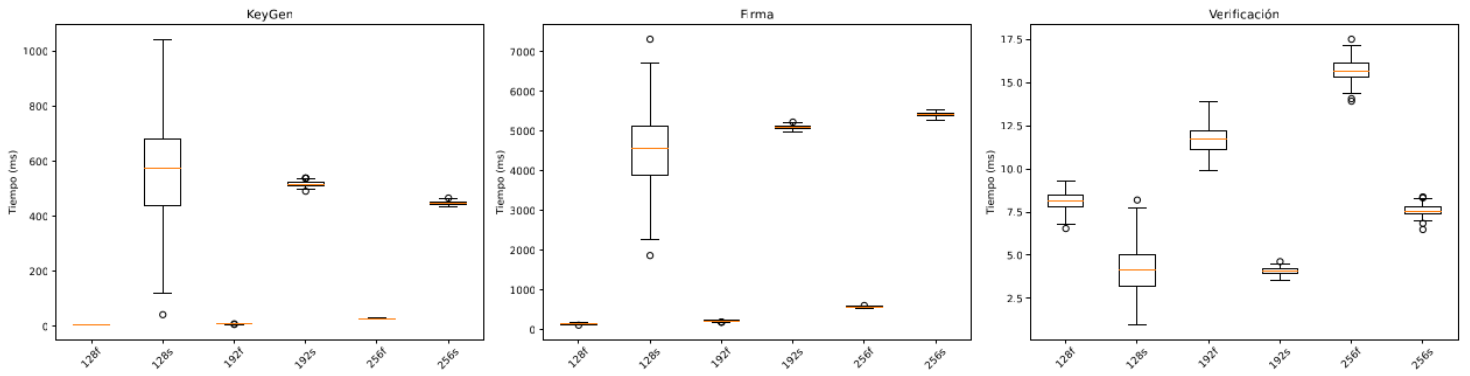


Tabla B.20. SPHINCS+ (C#) — Parte 2.

BIKE					
LENGUAJE	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
Java	BIKE-12 8	14.1298 ms ($\pm$ 200.364 3)	0.9645 ms ($\pm$ 13.3292)	21.5737 ms ($\pm$ 4.4479)	36.6680 ms ($\pm$ 202.9956)
Java	BIKE-19 2	27.0578 ms ($\pm$ 2.3293)	1.4751 ms ($\pm$ 0.3170)	51.1962 ms ($\pm$ 3.8756)	79.7291 ms ($\pm$ 5.9122)
Java	BIKE-25 6	77.4327 ms ($\pm$ 1.9930)	3.9409 ms ($\pm$ 0.2615)	114.6054 ms ($\pm$ 2.5721)	195.9790 ms ($\pm$ 4.1862)

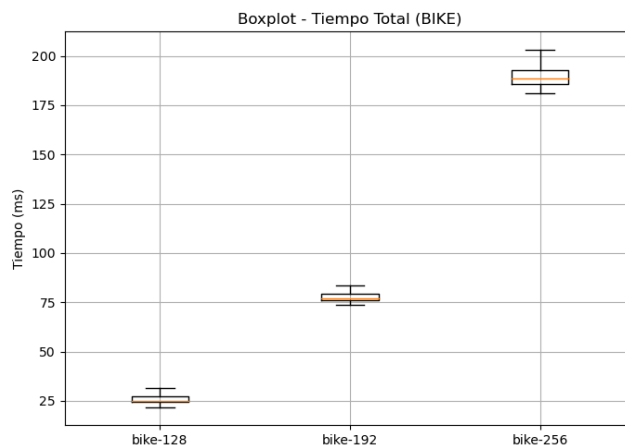
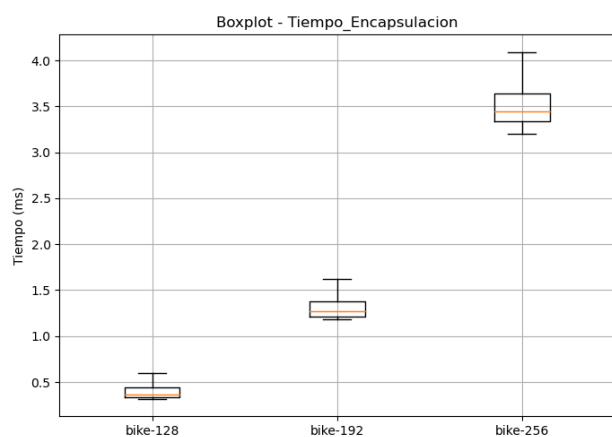
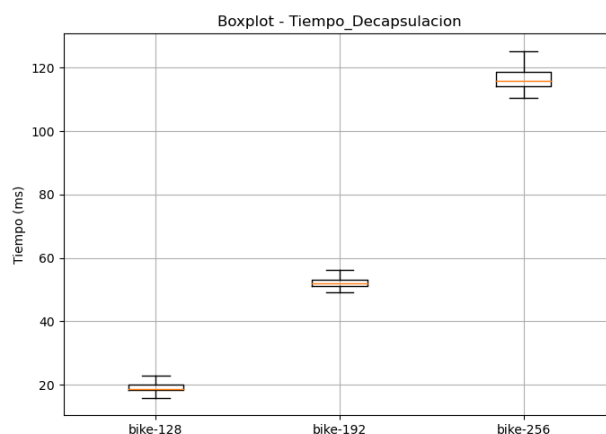
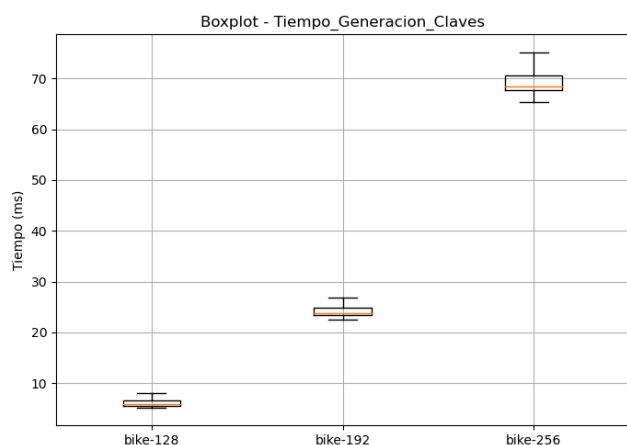


Tabla B.21. BIKE (Java) en dispositivo ordenador.

Dilithium

LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
Java	Dilithium2 (Pequeño)	0.5512 ms (± 9.3197)	0.5814 ms (± 0.5520)	0.1128 ms (± 0.1007)	1.2454 ms (± 9.3366)
Java	Dilithium2 (Grande)	0.5512 ms (± 9.3197)	0.5936 ms (± 0.5770)	0.1591 ms (± 0.1540)	1.3039 ms (± 9.3388)
Java	Dilithium3 (Pequeño)	0.3571 ms (± 0.0649)	0.8023 ms (± 0.7534)	0.1318 ms (± 0.0935)	1.2912 ms (± 0.7619)
Java	Dilithium3 (Grande)	0.3571 ms (± 0.0649)	0.8179 ms (± 0.6296)	0.1712 ms (± 0.0790)	1.3462 ms (± 0.6378)
Java	Dilithium5 (Pequeño)	0.5205 ms (± 0.1650)	0.9448 ms (± 0.8111)	0.1914 ms (± 0.0825)	1.6567 ms (± 0.8318)
Java	Dilithium5 (Grande)	0.5205 ms (± 0.1650)	0.9964 ms (± 0.8138)	0.2402 ms (± 0.1142)	1.7571 ms (± 0.8382)

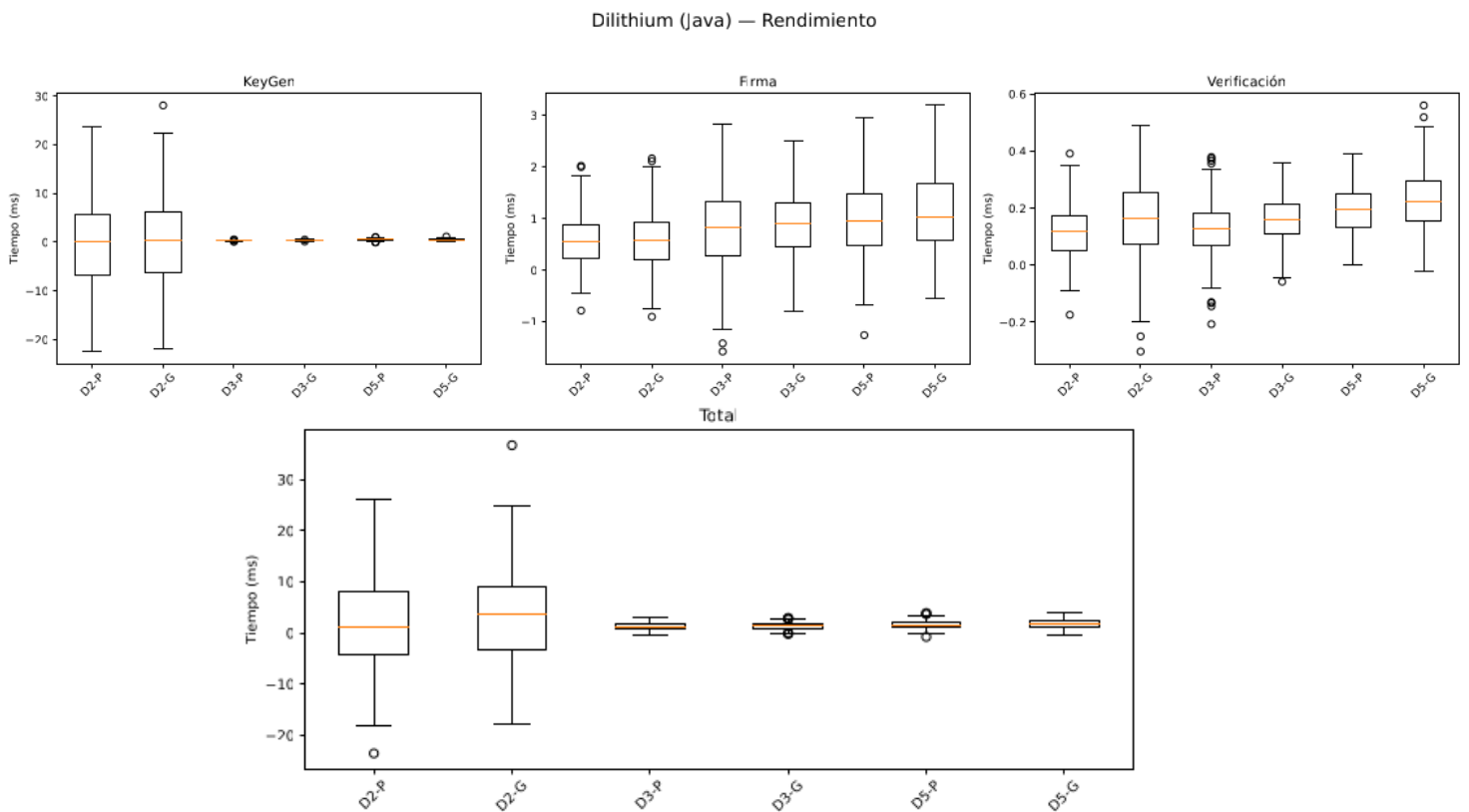


Tabla B.22. Dilithium (Java) en dispositivo ordenador.

Falcon

LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
Java	falcon-512 (corto)	11.5389 ms (± 5.1774)	1.2359 ms (± 0.8508)	0.0941 ms (± 0.1119)	12.8689 ms (± 5.2480)
Java	falcon-512 (largo)	10.8487 ms (± 3.7987)	1.1086 ms (± 0.2304)	0.1239 ms (± 0.0277)	12.0812 ms (± 3.8058)
Java	falcon-1024 (corto)	31.6011 ms (± 10.9482)	2.4456 ms (± 0.6343)	0.1376 ms (± 0.0391)	34.1843 ms (± 10.9666)
Java	falcon-1024 (largo)	35.1033 ms (± 13.0152)	2.7835 ms (± 0.8786)	0.2280 ms (± 0.0676)	38.1148 ms (± 13.0450)

Falcon (Java) — Rendimiento

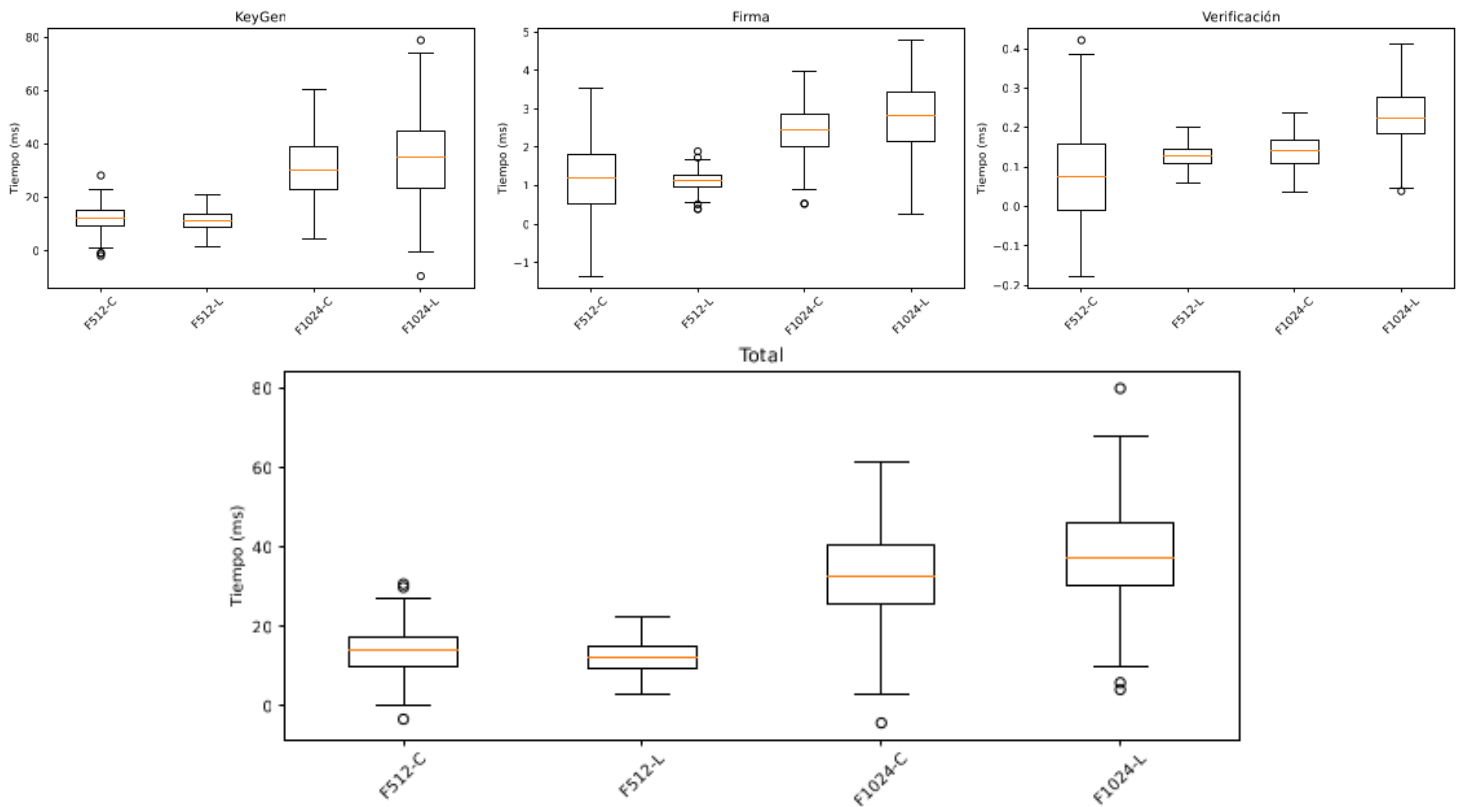


Tabla B.23. Falcon (Java) en dispositivo ordenador.

LENGUAJE	Versión	T Gen. Claves	HQC		T Total
			T Encapsulación	T Desencapsulación	
Java	HQC-1 28	2.1317 ms ($\pm$ 0.8254)	4.2096 ms ($\pm$ 0.8331)	6.3601 ms ($\pm$ 0.9857)	12.7014 ms ($\pm$ 1.5320)
Java	HQC-1 92	6.4935 ms ($\pm$ 1.2246)	13.0500 ms ($\pm$ 2.3620)	19.5737 ms ($\pm$ 3.1802)	39.1172 ms ($\pm$ 4.1464)
Java	HQC-2 56	11.3427 ms ($\pm$ 1.3565)	22.5546 ms ($\pm$ 2.3997)	33.8210 ms ($\pm$ 3.8391)	67.7183 ms ($\pm$ 4.7262)

HQC (Java) — Rendimiento

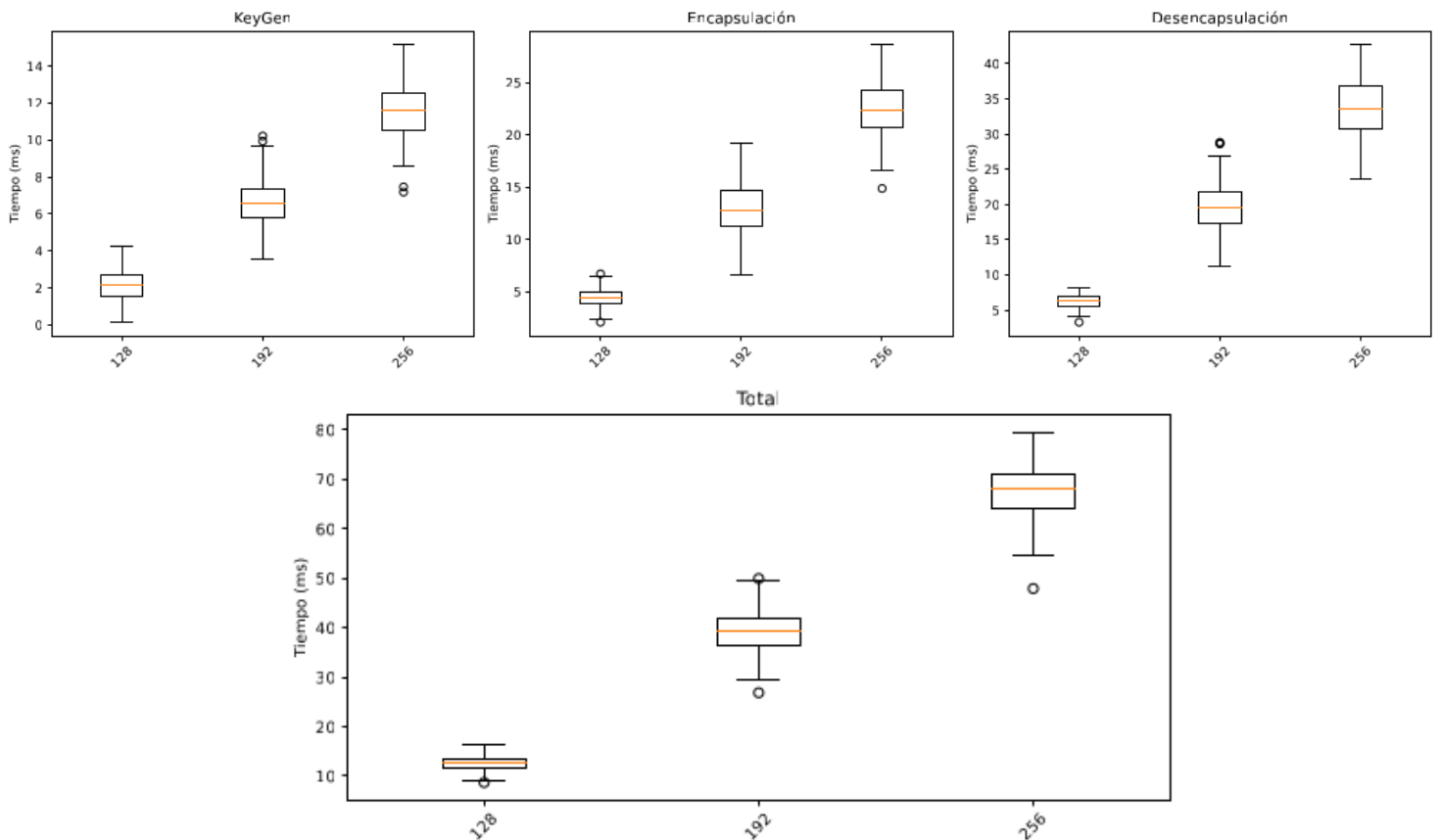


Tabla B.24. HQC (Java) en dispositivo ordenador.

Kyber

LENGUAJE	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
Java	KYBER512	0.1227 ms (± 0.0893)	0.0982 ms (± 0.0700)	0.1208 ms (± 0.1035)	0.3431 ms (± 0.2602)
Java	KYBER768	0.1115 ms (± 0.0320)	0.1075 ms (± 0.0276)	0.1189 ms (± 0.0313)	0.3387 ms (± 0.0880)
Java	KYBER1024	0.1219 ms (± 0.0299)	0.1249 ms (± 0.0266)	0.1367 ms (± 0.0308)	0.3843 ms (± 0.0762)

Kyber (Java) — Rendimiento

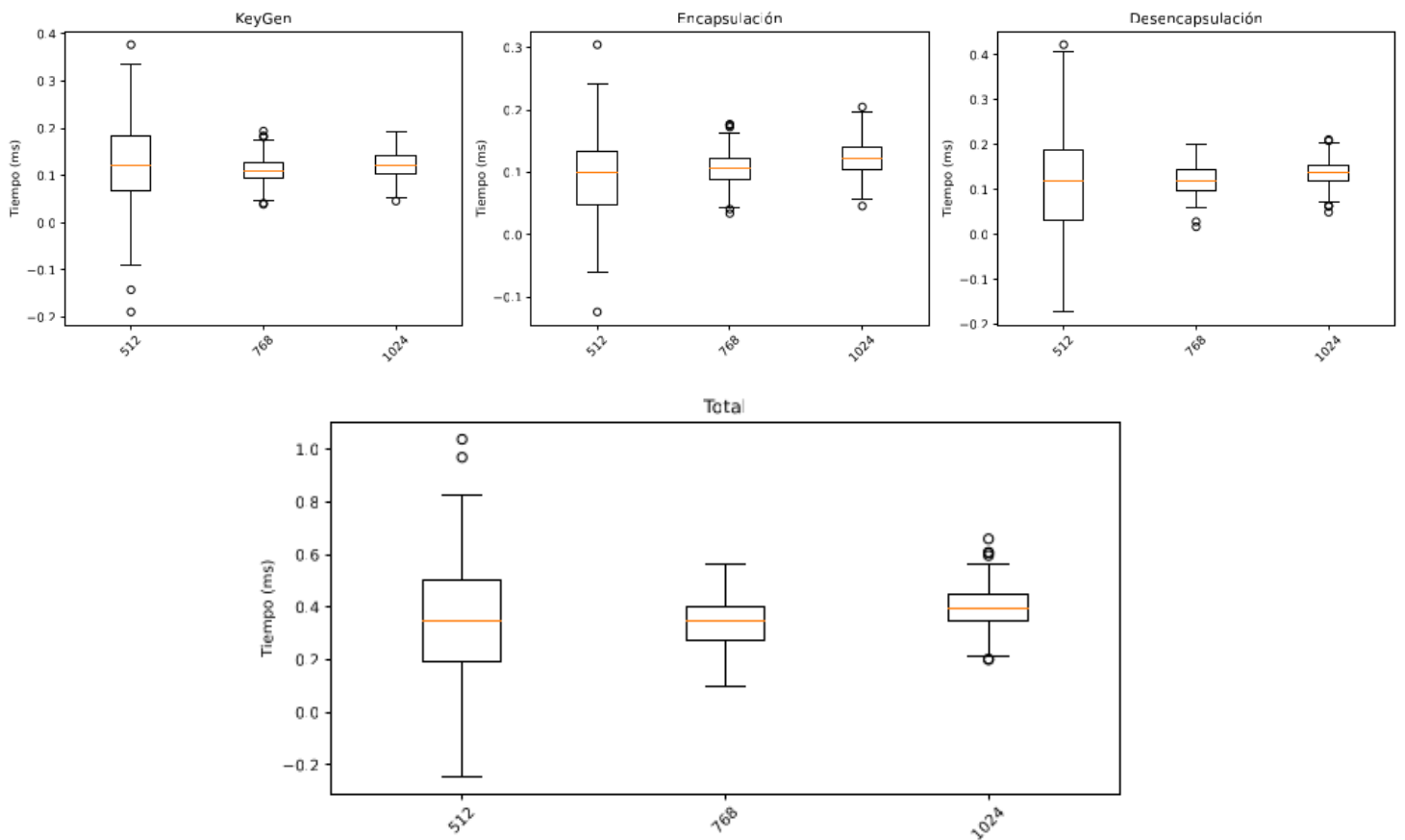


Tabla B.25. Kyber (Java) en dispositivo ordenador.

McEliece					
LENGUAJE	Versión	T Gen. Claves	T Encapsulaci ón	T Desencapsulac ión	T Total
Java	mceliece3488 64	283.2359 ms ($\pm$ 301.6281)	0.9242 ms ($\pm$ 1.5952)	15.8071 ms ($\pm$ 2.7939)	300.0130 ms ($\pm$ 303.3639)
Java	mceliece3488 64f	89.3953 ms ($\pm$ 12.0907)	0.5782 ms ($\pm$ 0.0938)	15.5796 ms ($\pm$ 1.3467)	105.5608 ms ($\pm$ 12.6885)
Java	mceliece4608 96	5549.3864 ms ($\pm$ 3631.0415)	1.1369 ms ($\pm$ 0.2968)	37.7806 ms ($\pm$ 5.1533)	5588.3130 ms ($\pm$ 3631.3529)
Java	mceliece4608 96f	1479.0081 ms ($\pm$ 90.1120)	0.9754 ms ($\pm$ 0.1504)	33.4015 ms ($\pm$ 2.2551)	1513.3941 ms ($\pm$ 91.0572)
Java	mceliece6688 128	14474.400 8 ms ($\pm$ 12404.475 0)	1.6542 ms ($\pm$ 0.1335)	62.0347 ms ($\pm$ 2.7650)	14538.096 7 ms ($\pm$ 12404.256 5)
Java	mceliece6688 128f	3506.2804 ms ($\pm$ 41.6845)	1.6734 ms ($\pm$ 0.1559)	61.9454 ms ($\pm$ 2.3959)	3569.9063 ms ($\pm$ 41.5944)
Java	mceliece6960 119	10905.545 8 ms ($\pm$ 9516.5638)	3.0088 ms ($\pm$ 4.2100)	60.8364 ms ($\pm$ 4.2466)	10969.398 3 ms ($\pm$ 9518.0156)
Java	mceliece6960 119f	3166.0684 ms ($\pm$ 43.5397)	2.3337 ms ($\pm$ 0.1728)	59.9447 ms ($\pm$ 2.0452)	3228.3553 ms ($\pm$ 44.3645)
Java	mceliece8192 128	17424.063 3 ms ($\pm$ 12257.923 4)	2.5583 ms ($\pm$ 0.5939)	82.6650 ms ($\pm$ 9.8769)	17509.296 9 ms ($\pm$ 12256.919 6)
Java	mceliece8192 128f	4838.4050 ms ($\pm$ 309.6527)	2.3941 ms ($\pm$ 0.4683)	84.1481 ms ($\pm$ 7.4034)	4924.9592 ms ($\pm$ 312.5998)

Tabla B.26. Classic McEliece (Java) — Parte 1.

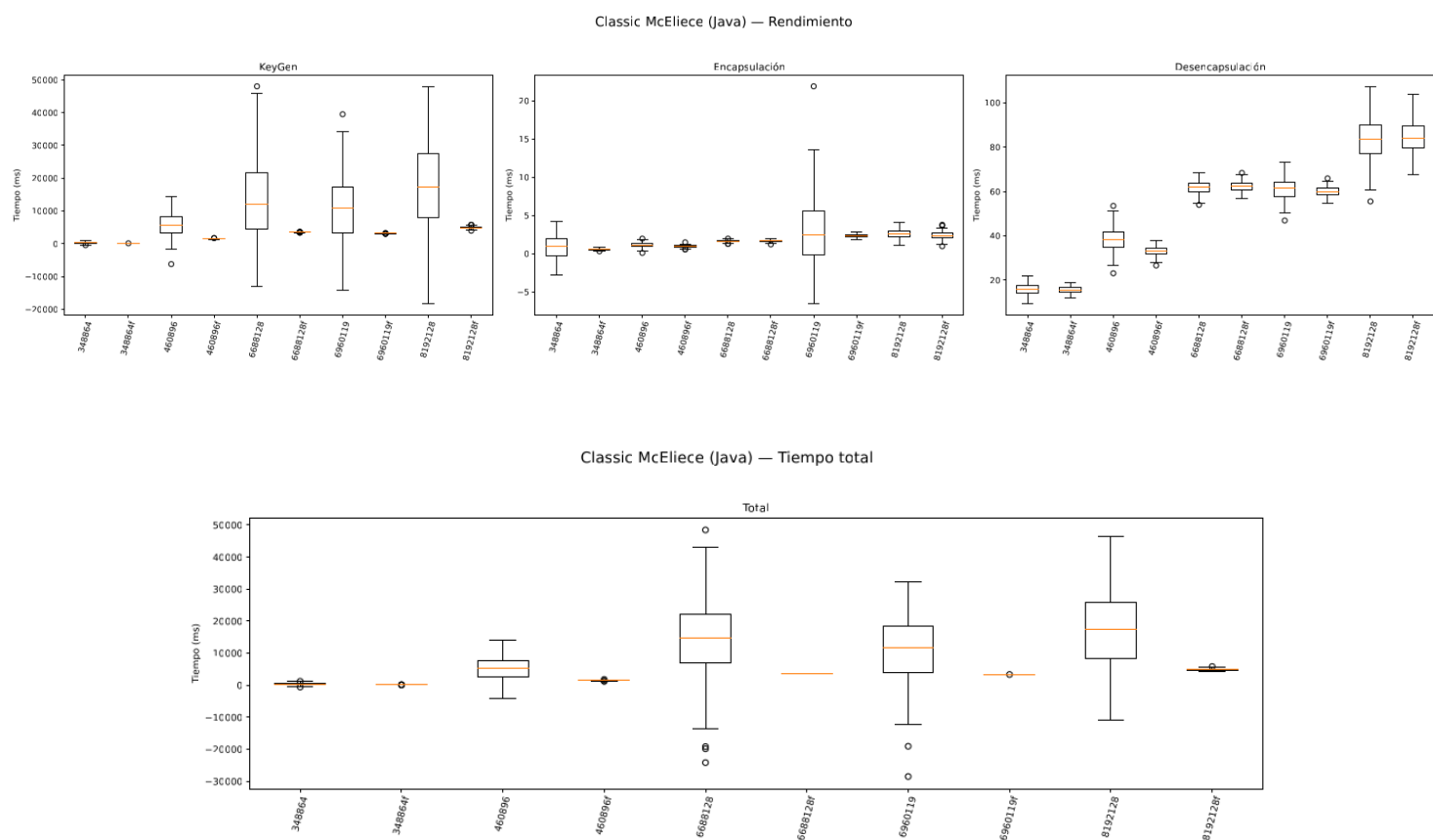


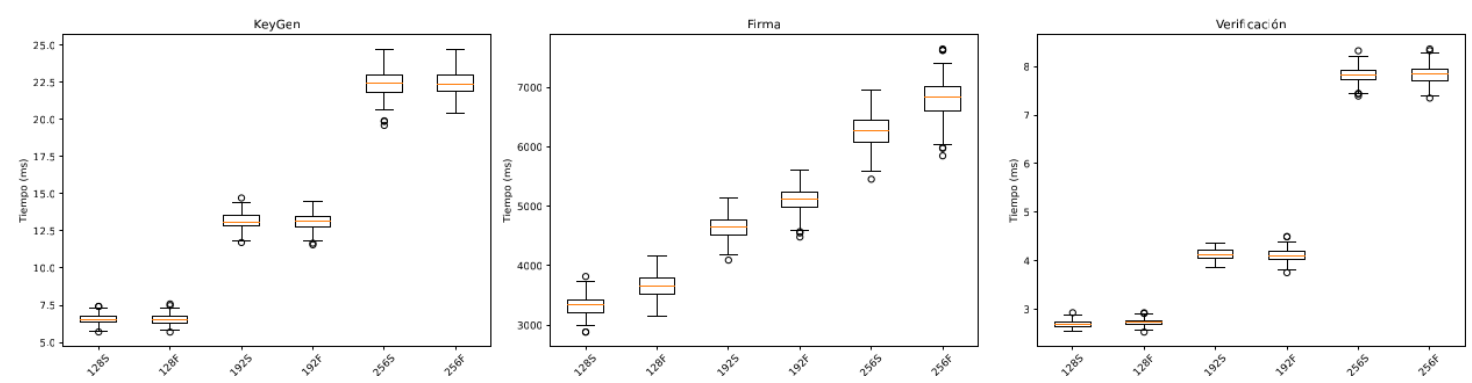
Tabla B.27. Classic McEliece (Java) — Parte 2.

SPHINCS+					
LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
Java	SPHINCS+-S HA2-128S	6.5462 ms (± 0.3261)	3336.9520 ms ($\pm$ 155.4857)	2.6937 ms ($\pm$ 0.0734)	3346.1919 ms ($\pm$ 155.8355)
Java	SPHINCS+-S HA2-128F	6.5434 ms (± 0.3314)	3660.1276 ms ($\pm$ 187.4115)	2.7254 ms ($\pm$ 0.0743)	3669.3964 ms ($\pm$ 187.8313)
Java	SPHINCS+-S HA2-192S	13.1243 ms ($\pm$ 0.5109)	4645.7928 ms ($\pm$ 197.7741)	4.1128 ms ($\pm$ 0.1118)	4663.0299 ms ($\pm$ 198.2534)
Java	SPHINCS+-S HA2-192F	13.1128 ms ($\pm$ 0.4976)	5092.7645 ms ($\pm$ 210.9864)	4.1158 ms ($\pm$ 0.1095)	5110.0471 ms ($\pm$ 211.4476)
Java	SPHINCS+-S HA2-256S	22.4579 ms ($\pm$ 0.8354)	6258.6237 ms ($\pm$ 280.5416)	7.8412 ms ($\pm$ 0.1762)	6288.9228 ms ($\pm$ 281.0954)
Java	SPHINCS+-S HA2-256F	22.4498 ms ($\pm$ 0.8398)	6839.0728 ms ($\pm$ 325.7982)	7.8485 ms ($\pm$ 0.1771)	6869.3711 ms ($\pm$ 326.3851)
Java	SPHINCS+-S HAKE-128S	6.5476 ms (± 0.3255)	3328.2642 ms ($\pm$ 152.9870)	2.6975 ms ($\pm$ 0.0714)	3337.5093 ms ($\pm$ 153.4440)
Java	SPHINCS+-S HAKE-128F	6.5501 ms (± 0.3239)	3655.5286 ms ($\pm$ 183.9923)	2.7269 ms ($\pm$ 0.0712)	3664.8056 ms ($\pm$ 184.4620)
Java	SPHINCS+-S HAKE-192S	13.1329 ms ($\pm$ 0.5053)	4641.6674 ms ($\pm$ 193.2483)	4.1186 ms ($\pm$ 0.1094)	4658.9189 ms ($\pm$ 193.7260)
Java	SPHINCS+-S HAKE-192F	13.1305 ms ($\pm$ 0.5081)	5090.5624 ms ($\pm$ 207.7163)	4.1154 ms ($\pm$ 0.1115)	5107.8083 ms ($\pm$ 208.1987)
Java	SPHINCS+-S HAKE-256S	22.4550 ms ($\pm$ 0.8301)	6251.8778 ms ($\pm$ 277.5094)	7.8419 ms ($\pm$ 0.1730)	6282.1746 ms ($\pm$ 278.0656)

Tabla B.28. SPHINCS+ (Java) — Parte 1.

Java	SPHINCS+ HAKE-256F	22.4588 ms (± 0.8347)	6829.9663 ms (± 321.7512)	7.8488 ms (± 0.1742)	6860.2740 ms (± 322.3327)
------	-----------------------	-----------------------------	---------------------------------	-------------------------	---------------------------------

SPHINCS+ (SHAKE, Java) — Rendimiento



SPHINCS+ (SHA2, Java) — Rendimiento

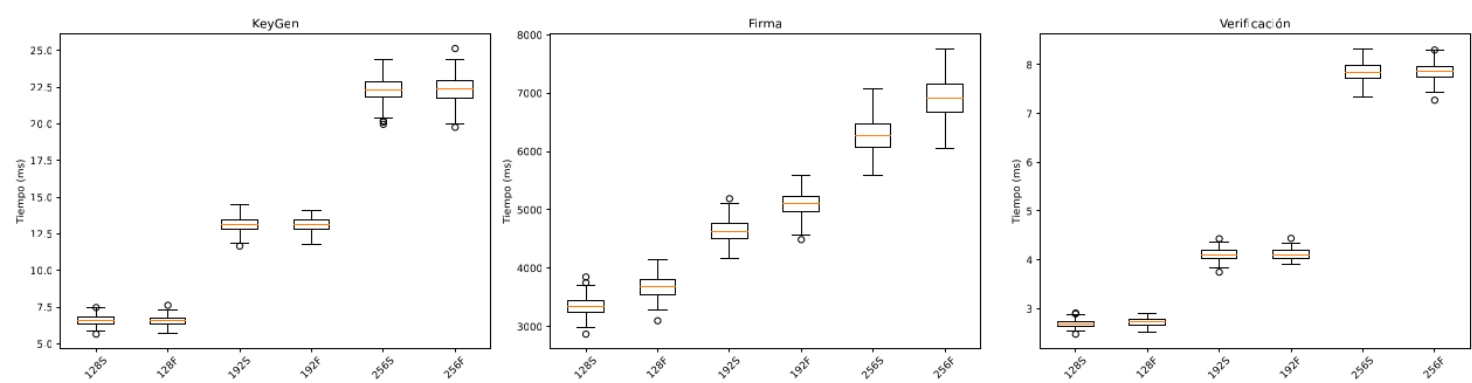
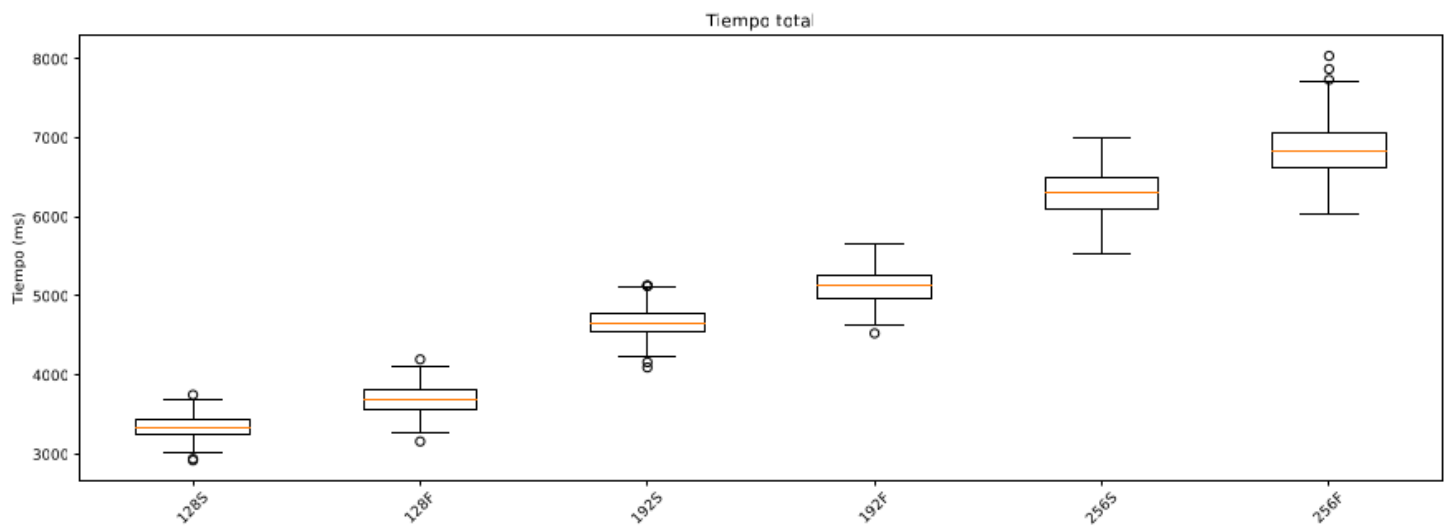


Tabla B.29. SPHINCS+ (Java) — Parte 2.

SPHINCS+ (SHAKE, Java) — Total



SPHINCS+ (SHA2, Java) — Total

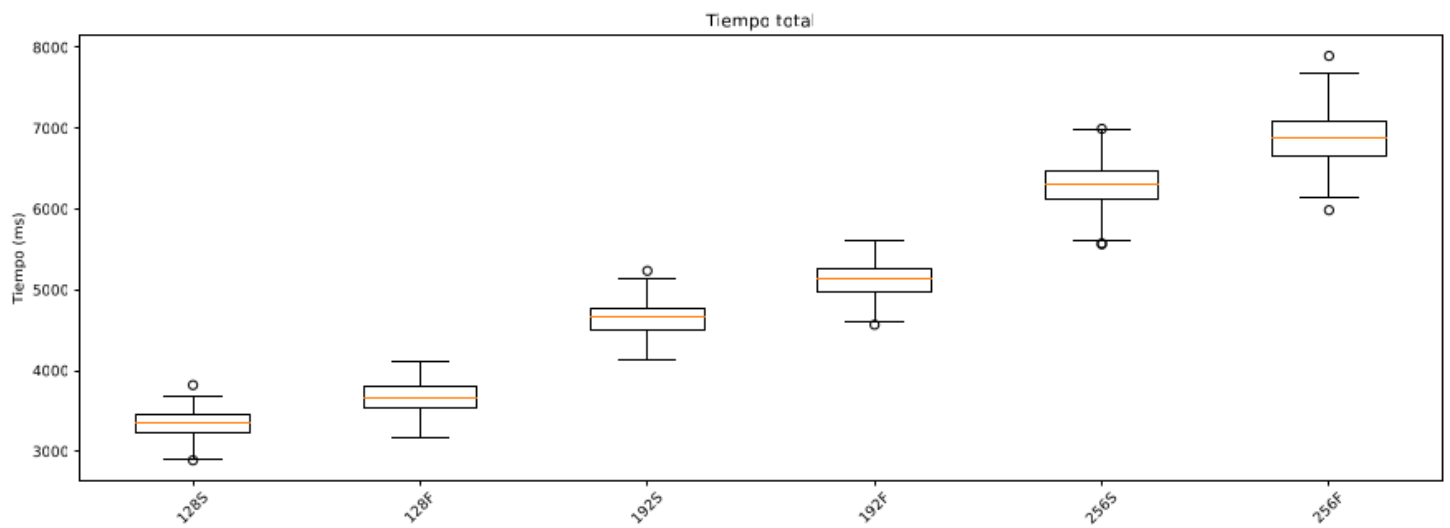


Tabla B.30. SPHINCS+ (Java) — Parte 3.

BIKE					
LENGUAJE	Versi ón	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
Python	BIKE -L1	0.2867 ms ($\pm$ 0.0297)	0.0542 ms ($\pm$ 0.0075)	1.2281 ms ($\pm$ 0.0985)	1.5699 ms ($\pm$ 0.1251)
Python	BIKE -L3	0.7895 ms ($\pm$ 0.0312)	0.1134 ms ($\pm$ 0.0060)	3.8805 ms ($\pm$ 0.1606)	4.7858 ms ($\pm$ 0.1861)
Python	BIKE -L5	1.9645 ms ($\pm$ 0.1217)	0.2355 ms ($\pm$ 0.0225)	8.4053 ms ($\pm$ 0.3399)	10.6041 ms ($\pm$ 0.4031)

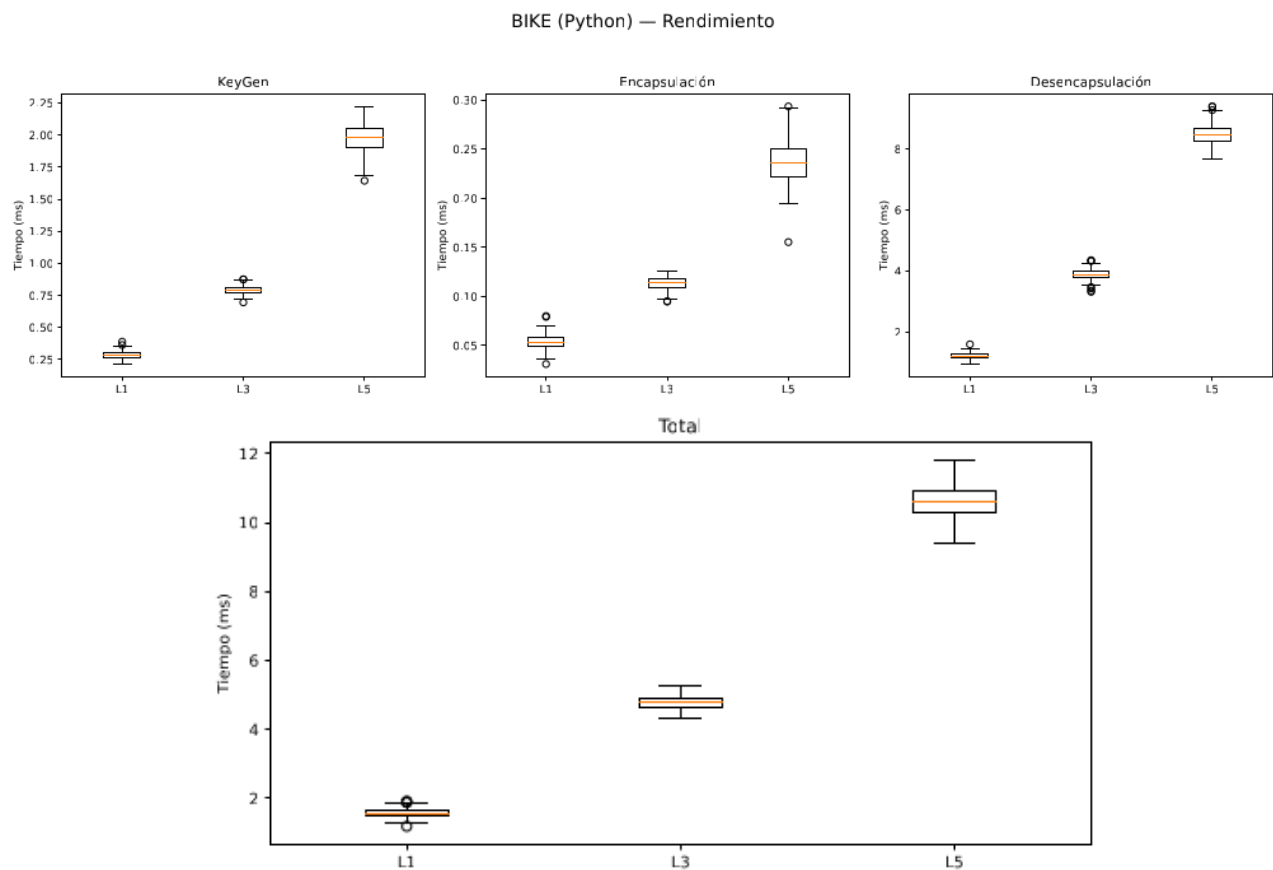


Tabla B.31. BIKE (Python) en dispositivo ordenador.

Falcon

LENGUAJE	Versión	T Gen. Claves (ms)	T Firma (ms)	T Verificación (ms)	T Total (ms)
Python	Falcon-5 12 (Corto)	6020.28 ($\pm$ 2793.73)	19.34 ($\pm$ 0.57)	3.98 ($\pm$ 0.08)	6043.72 ($\pm$ 2793.64)
Python	Falcon-5 12 (Largo)	7641.44 ($\pm$ 4445.16)	19.67 ($\pm$ 0.66)	4.02 ($\pm$ 0.10)	7665.14 ($\pm$ 4445.20)
Python	Falcon-1 024 (Corto)	23477.37 ($\pm$ 14424.31)	39.75 ($\pm$ 1.43)	8.54 ($\pm$ 0.24)	23525.95 ($\pm$ 14424.26)
Python	Falcon-1 024 (Largo)	26729.16 ($\pm$ 14432.58)	40.11 ($\pm$ 0.80)	8.62 ($\pm$ 0.12)	26778.14 ($\pm$ 14432.43)

Falcon (Python) — Rendimiento

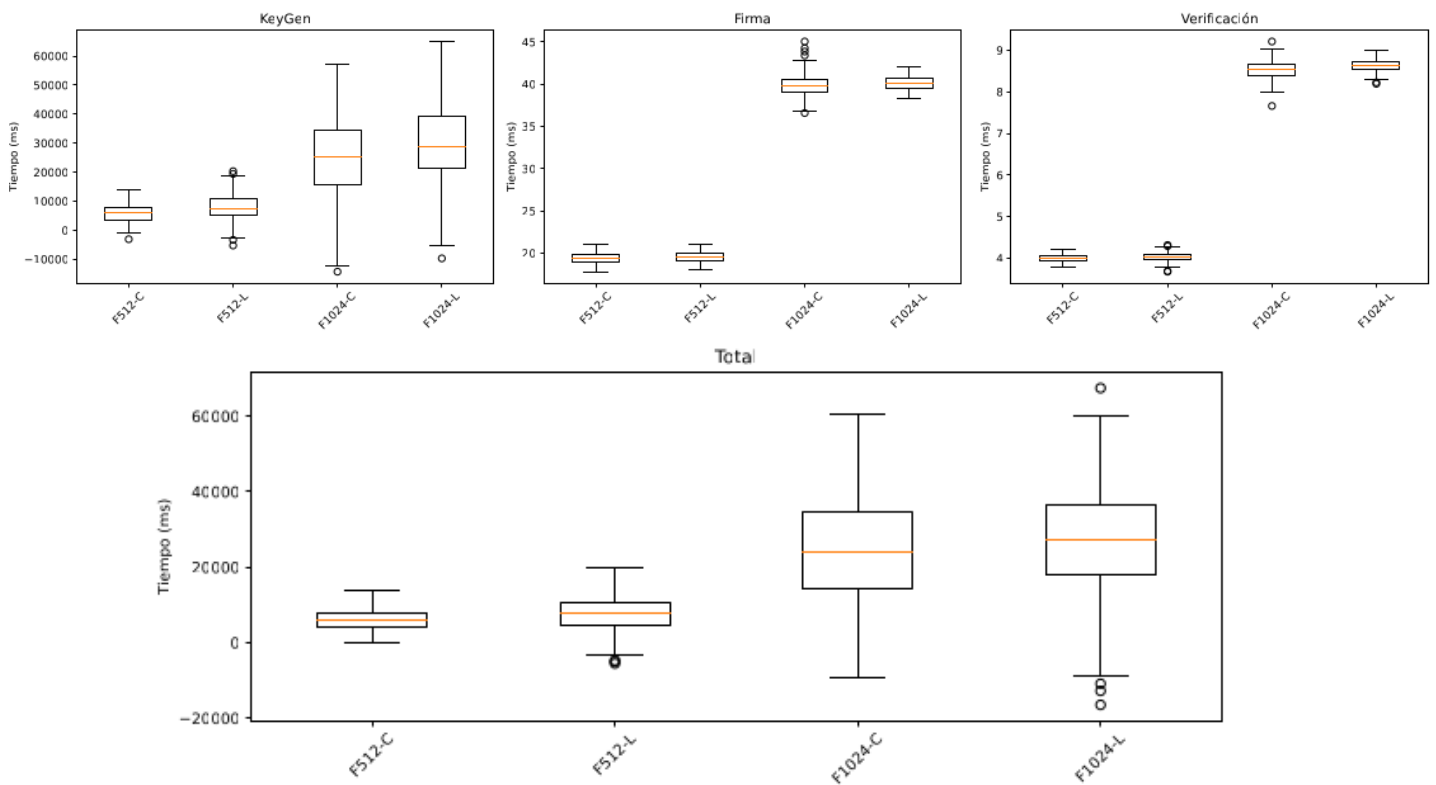


Tabla B.32. Falcon (Python) en dispositivo ordenador.

LENGUAJE	Versión	T Gen. Claves	HQC		T Total
			T Encapsulación	T Desencapsulación	
Python	HQC-12 8	7.3813 ms ($\pm$ 0.1170)	14.9464 ms ($\pm$ 0.2851)	23.9656 ms ($\pm$ 0.3498)	46.3443 ms ($\pm$ 0.7314)
Python	HQC-25 6	41.3651 ms ($\pm$ 0.5118)	83.2793 ms ($\pm$ 0.9721)	129.5886 ms ($\pm$ 1.0228)	254.2775 ms ($\pm$ 1.6345)

HQC (Python) — Rendimiento

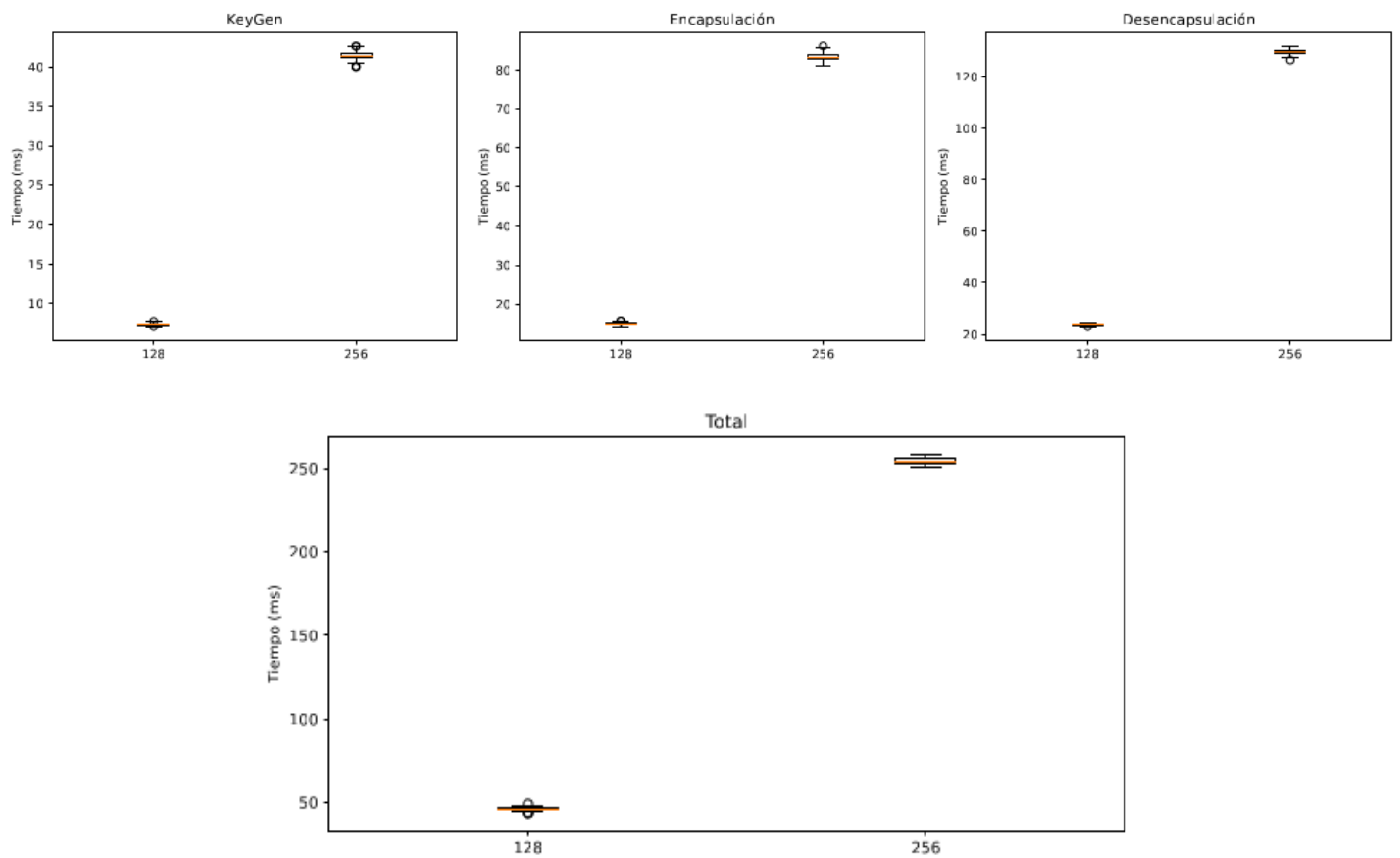


Tabla B.33. HQC (Python) en dispositivo ordenador.

LENGUAJE	Versión	Kyber			
		T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
Python	Kyber512	1.5874 ms ($\pm$ 0.1783)	2.2039 ms ($\pm$ 0.2388)	3.1713 ms ($\pm$ 0.2962)	6.9626 ms ($\pm$ 0.7133)
Python	Kyber768	2.6538 ms ($\pm$ 0.1856)	3.3806 ms ($\pm$ 0.2569)	4.6591 ms ($\pm$ 0.3030)	10.6935 ms ($\pm$ 0.7455)
Python	Kyber1024	4.0339 ms ($\pm$ 0.3476)	4.8650 ms ($\pm$ 0.4117)	6.4779 ms ($\pm$ 0.4439)	15.3768 ms ($\pm$ 1.2032)

Kyber (Python) — Rendimiento

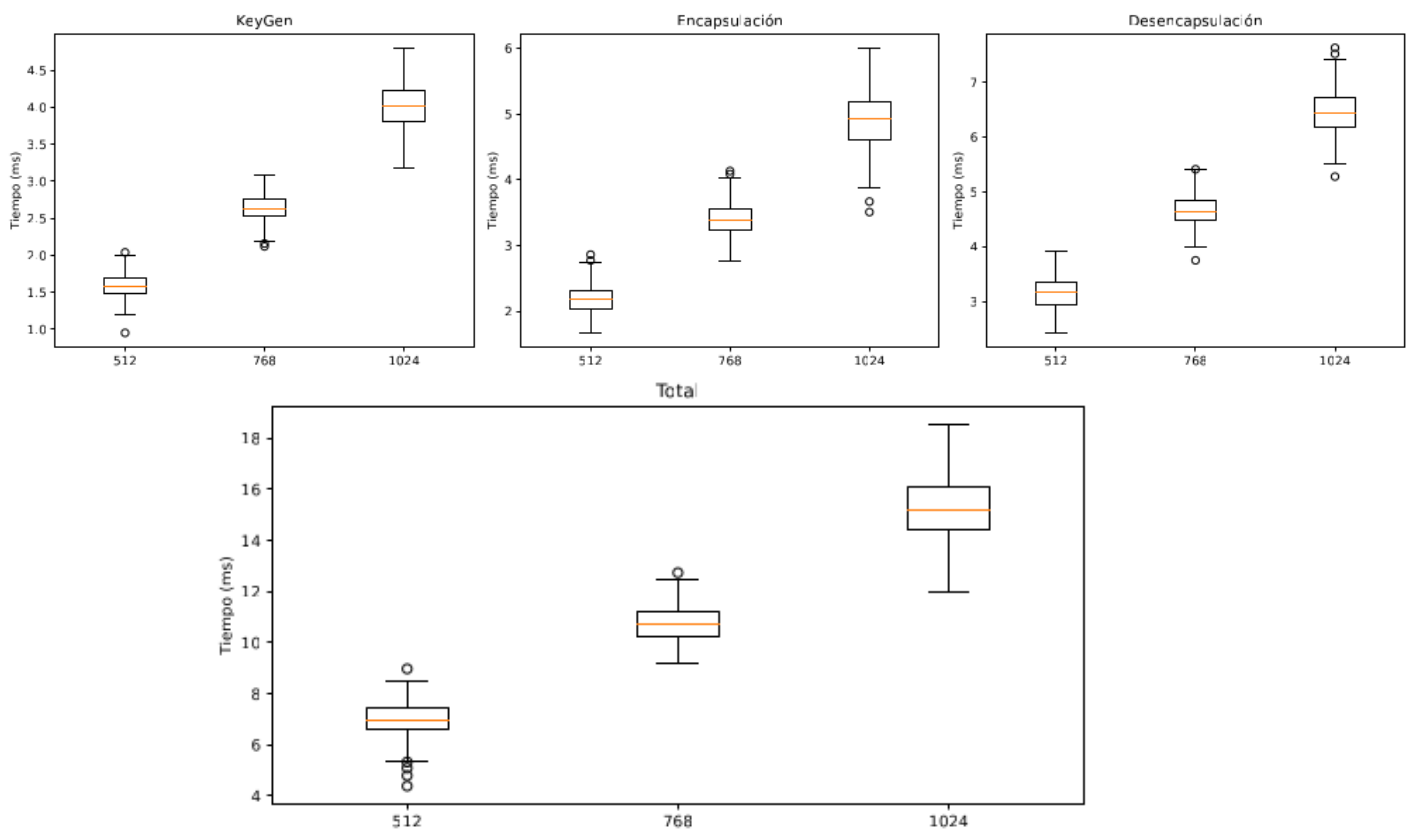


Tabla B.34. Kyber (Python) en dispositivo ordenador.

LENGUAJE	Versión	McEliece			
		T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
Python	Classic-McEliece-348864	53.6754 ms ($\pm$ 11.4710)	0.0556 ms ($\pm$ 0.0158)	0.0500 ms ($\pm$ 0.0106)	53.7828 ms ($\pm$ 11.4762)
Python	Classic-McEliece-348864f	40.9449 ms ($\pm$ 1.0530)	0.0497 ms ($\pm$ 0.0139)	0.0483 ms ($\pm$ 0.0096)	41.0432 ms ($\pm$ 1.0640)
Python	Classic-McEliece-460896	127.0735 ms ($\pm$ 23.9046)	0.1023 ms ($\pm$ 0.0211)	0.0912 ms ($\pm$ 0.0164)	127.2754 ms ($\pm$ 23.9077)
Python	Classic-McEliece-460896f	110.4798 ms ($\pm$ 2.1828)	0.1029 ms ($\pm$ 0.0234)	0.0937 ms ($\pm$ 0.0192)	110.6774 ms ($\pm$ 2.1946)
Python	Classic-McEliece-6688128	215.6868 ms ($\pm$ 64.0485)	0.2648 ms ($\pm$ 0.0288)	0.1136 ms ($\pm$ 0.0208)	216.0733 ms ($\pm$ 64.0498)
Python	Classic-McEliece-6688128f	148.8978 ms ($\pm$ 3.0133)	0.2537 ms ($\pm$ 0.0339)	0.1078 ms ($\pm$ 0.0189)	149.2618 ms ($\pm$ 3.0220)
Python	Classic-McEliece-6960119	198.0291 ms ($\pm$ 62.9405)	0.2537 ms ($\pm$ 0.0277)	0.0992 ms ($\pm$ 0.0180)	198.3851 ms ($\pm$ 62.9543)
Python	Classic-McEliece-6960119f	136.2338 ms ($\pm$ 2.3293)	0.2549 ms ($\pm$ 0.0296)	0.1020 ms ($\pm$ 0.0190)	136.5926 ms ($\pm$ 2.3356)
Python	Classic-McEliece-8192128	226.8936 ms ($\pm$ 64.4108)	0.3129 ms ($\pm$ 0.0355)	0.1084 ms ($\pm$ 0.0156)	227.3179 ms ($\pm$ 64.4087)

Tabla B.35. Classic McEliece (Python) — Parte 1.

Python	Classic-McEliece-8192128f	159.666 9 ms ($\pm$ 2.8558)	0.3198 ms ($\pm$ 0.0318)	0.1112 ms ($\pm$ 0.0193)	160.098 6 ms ($\pm$ 2.8599)
--------	---------------------------	------------------------------------	------------------------------	------------------------------	------------------------------------

Classic McEliece (Python) — Rendimiento

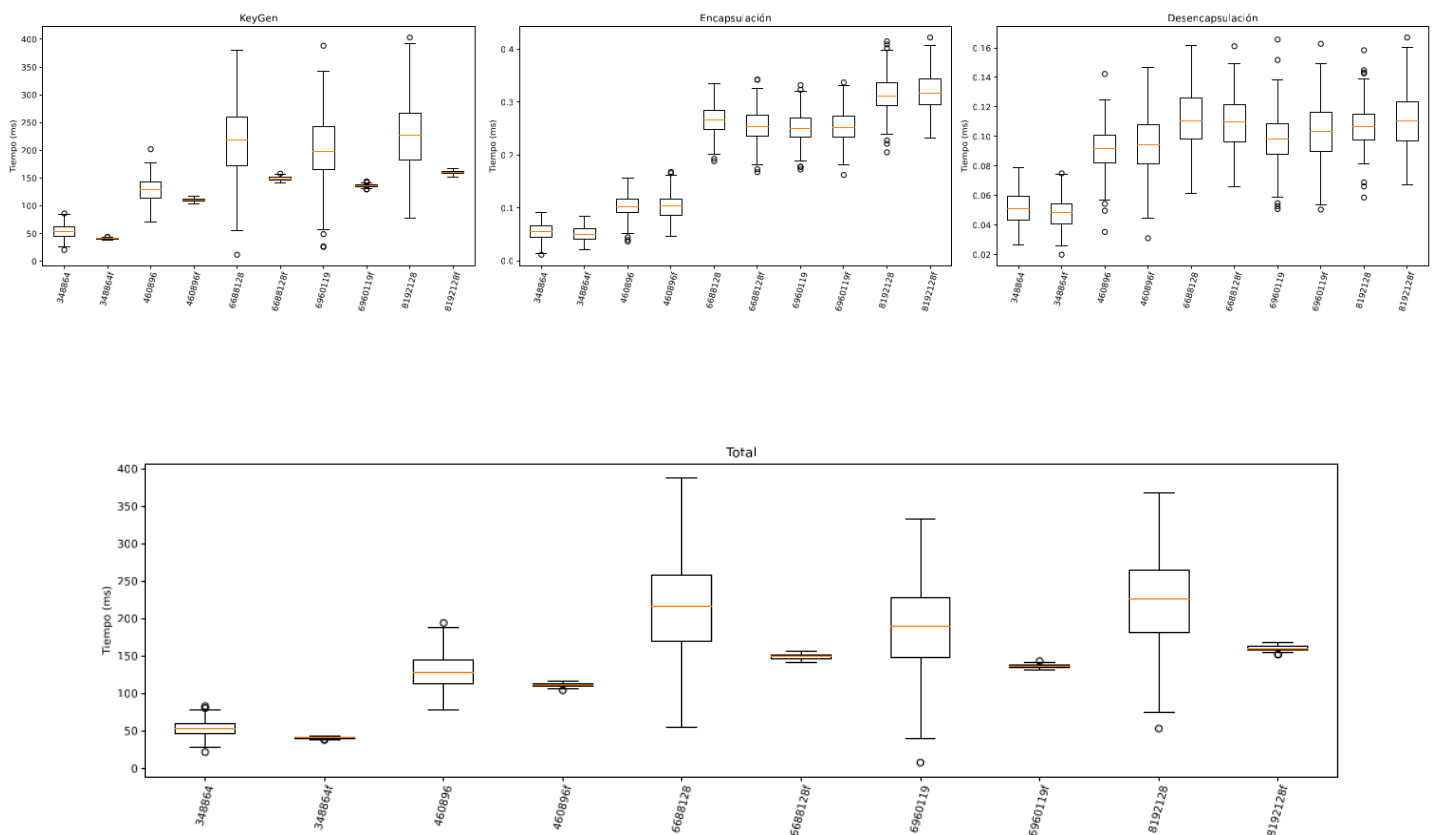


Tabla B.36. Classic McEliece (Python) — Parte 2.

SPHINCS+					
LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
Python	SPHINCS+-S HA2-128s	179.4616 ms ($\pm$ 16.2212)	1374.0530 ms ($\pm$ 111.0037)	1.4834 ms ($\pm$ 0.2317)	1556.2235 ms ($\pm$ 120.1321)
Python	SPHINCS+-S HA2-192s	245.6651 ms ($\pm$ 4.6428)	2217.0938 ms ($\pm$ 36.0795)	2.0126 ms ($\pm$ 0.0950)	2464.7381 ms ($\pm$ 39.2632)
Python	SPHINCS+-S HA2-256s	226.3345 ms ($\pm$ 10.4626)	2508.8363 ms ($\pm$ 46.4120)	3.9344 ms ($\pm$ 0.1896)	2739.5295 ms ($\pm$ 50.8069)
Python	SPHINCS+-S HAKE-128s	286.6102 ms ($\pm$ 16.0213)	2140.2179 ms ($\pm$ 87.2323)	2.2373 ms ($\pm$ 0.1636)	2429.0957 ms ($\pm$ 95.2386)
Python	SPHINCS+-S HAKE-192s	404.5870 ms ($\pm$ 14.3309)	3541.9017 ms ($\pm$ 87.2203)	3.1406 ms ($\pm$ 0.1659)	3949.4130 ms ($\pm$ 97.6649)
Python	SPHINCS+-S HAKE-256s	266.4675 ms ($\pm$ 7.9722)	3046.1070 ms ($\pm$ 77.3147)	4.6080 ms ($\pm$ 0.2022)	3317.6260 ms ($\pm$ 82.0395)

SPHINCS+ (SHAKE, Python) — Rendimiento

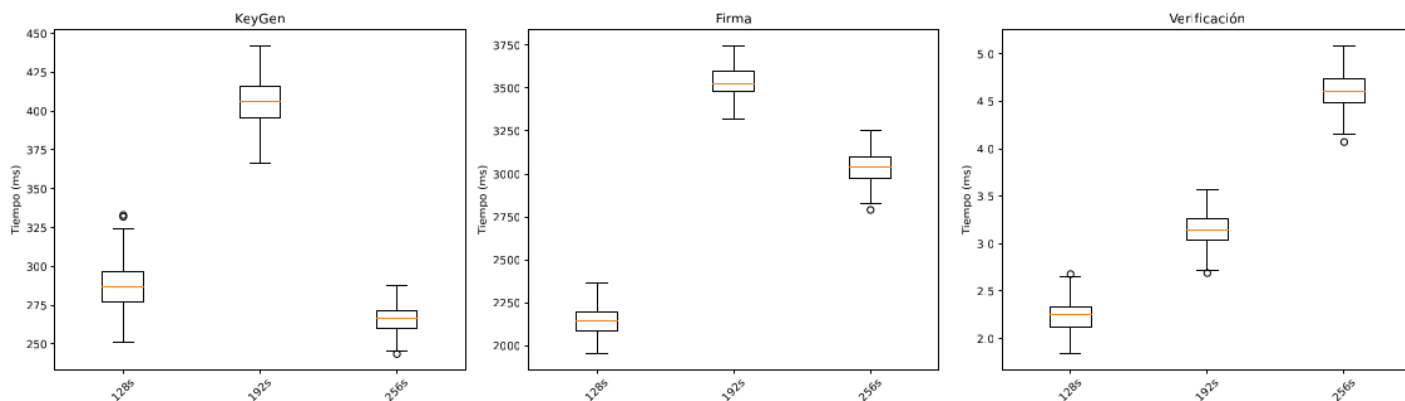
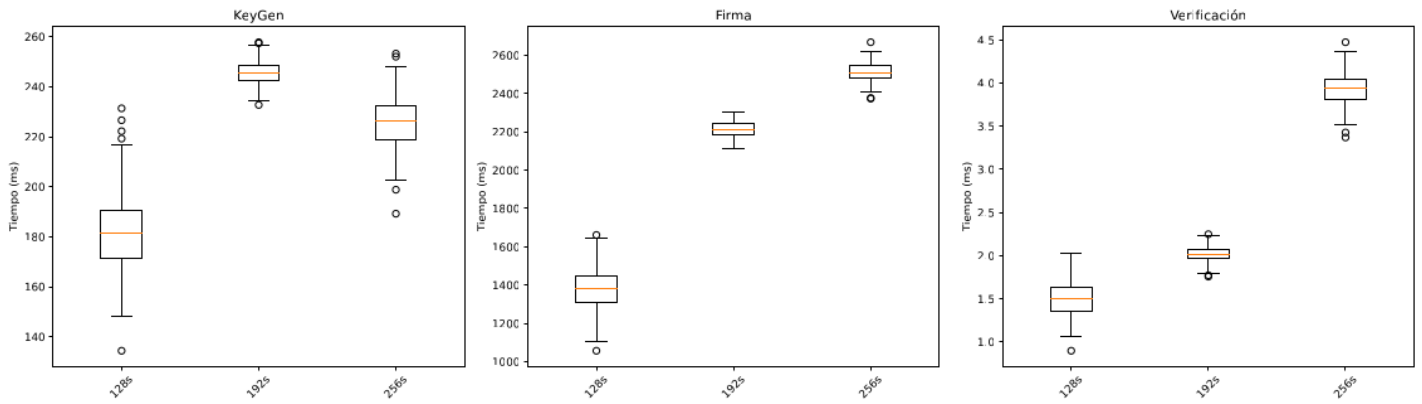
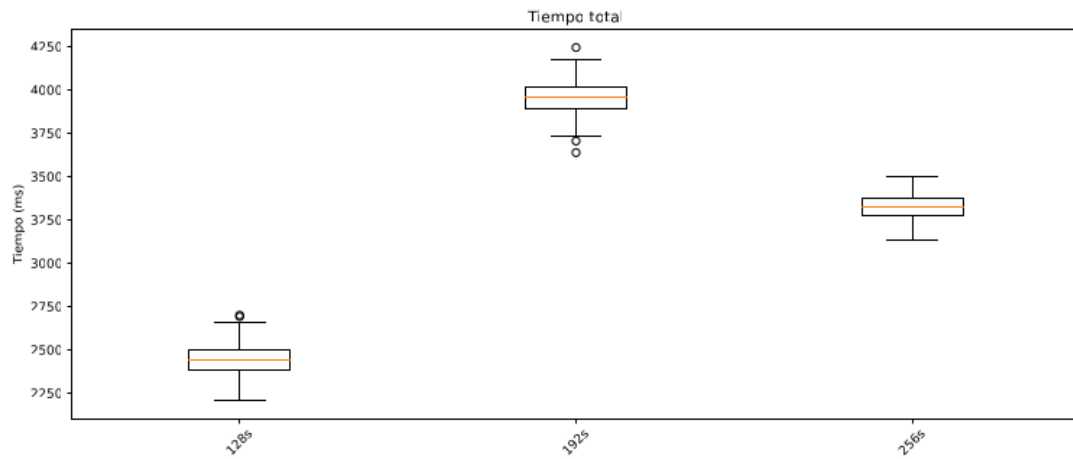


Tabla B.37. SPHINCS+ (Python) en dispositivo ordenador— Parte 1.

SPHINCS+ (SHA2, Python) — Rendimiento



SPHINCS+ (SHAKE, Python) — Total



SPHINCS+ (SHA2, Python) — Total

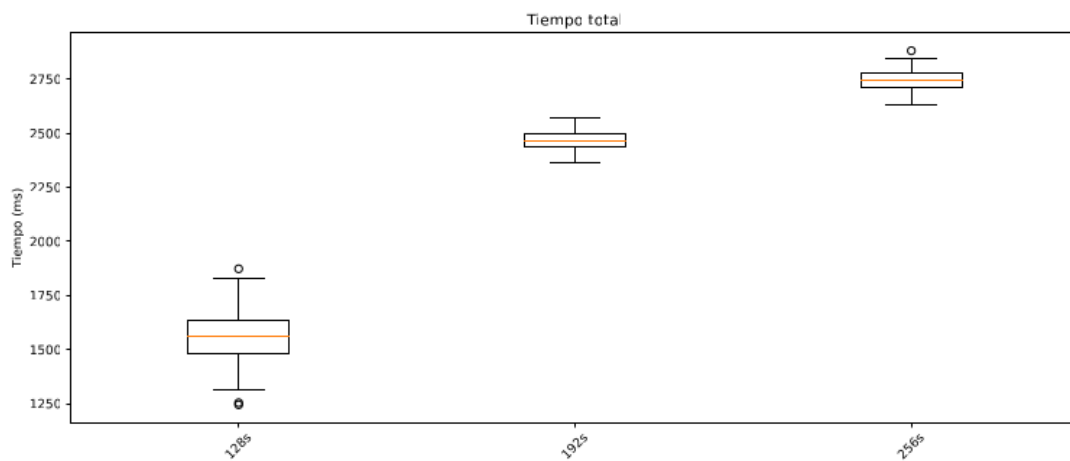


Tabla B.38. SPHINCS+ (Python) en dispositivo ordenador— Parte 2.

Dilithium

LENGUAJE	Versión	T Gen. Claves	T Firma	T Verificación	T Total
Python	Dilithiu m2	6.93 ms ($\pm$ 1.39)	46.71 ms ($\pm$ 32.60)	8.50 ms ($\pm$ 1.26)	62.14 ms ($\pm$ 32.52)
Python	Dilithiu m3	12.58 ms ($\pm$ 3.44)	59.67 ms ($\pm$ 55.04)	14.02 ms ($\pm$ 2.45)	86.27 ms ($\pm$ 57.60)
Python	Dilithiu m5	17.35 ms ($\pm$ 4.36)	70.84 ms ($\pm$ 43.96)	21.09 ms ($\pm$ 6.56)	109.29 ms ($\pm$ 44.75)

Dilithium (Python) — Rendimiento

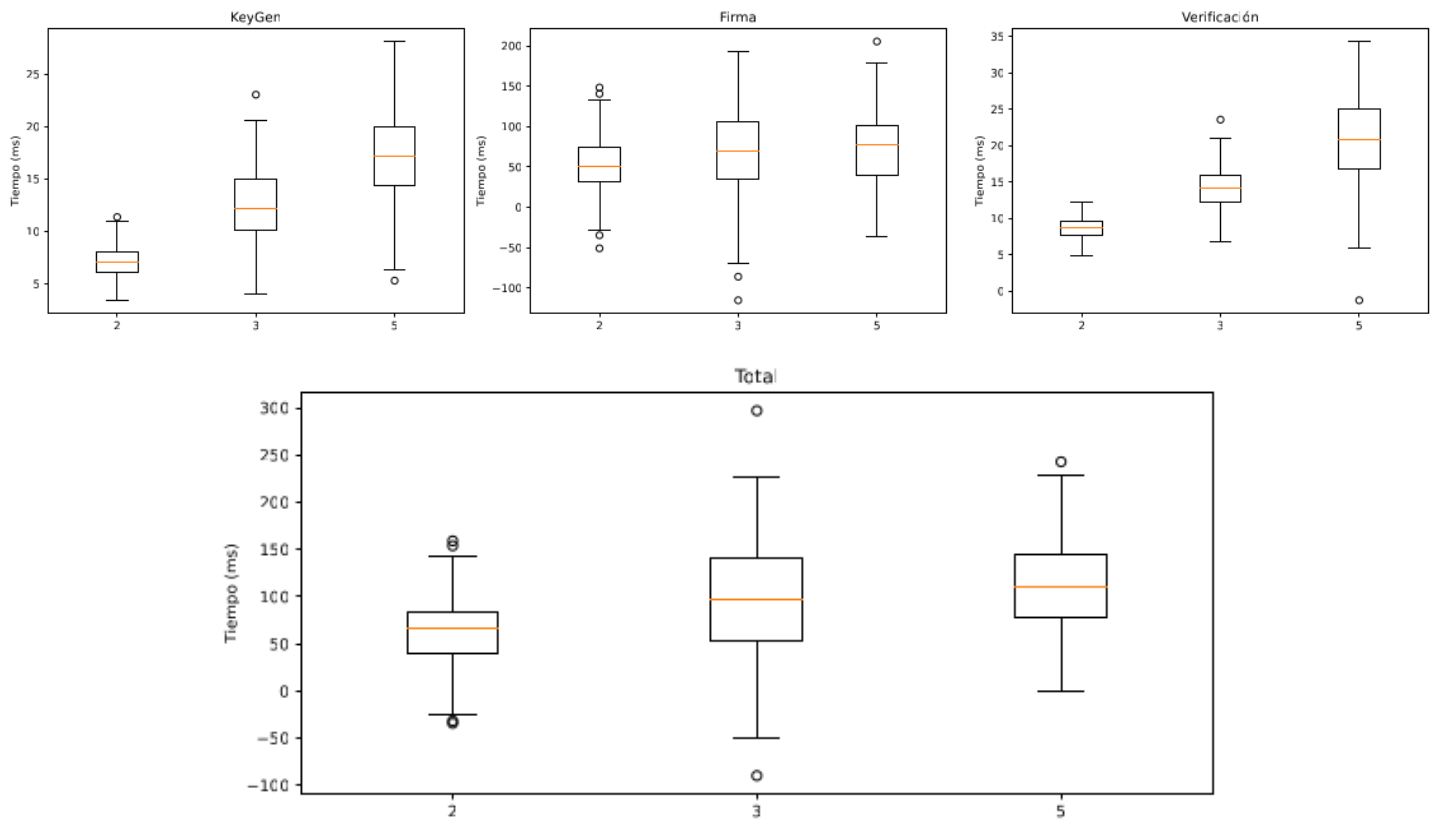


Tabla B.39. Dilithium (Python) en dispositivo ordenador.

Consumo de Recursos

Lenguaje	Versión	Memoria en Kbytes	Uso Máximo CPU
C	BIKE AVX2 L1	2244	98 %
C	BIKE AVX2 L3	2372	99 %
C	BIKE AVX2 L5	2600	100 %
C	BIKE Portable L1	2156	100 %
C	BIKE Portable L3	2384	100 %
C	BIKE Portable L5	2704	100 %
C	Dilithium-2 avx2	2890	100 %
C	Dilithium-2 ref	2858	99 %
C	Dilithium-3 avx2	2806	99 %
C	Dilithium-3 ref	2850	99 %
C	Dilithium-5 avx2	2764	98 %
C	Dilithium-5 ref	2742	99 %
C	Falcon-512	2139	99 %
C	Falcon-1024	2202	99 %
C	Falcon-512 avx2	2078	99 %
C	Falcon-1024 avx2	2066	99 %
C	HQC-128 ref	5024	99 %
C	HQC-192 ref	5164	99 %
C	HQC-256 ref	5192	99 %
C	HQC-128 avx2	1836	100 %
C	HQC-192 avx2	1928	99 %
C	HQC-256 avx2	2032	99 %
C	Kyber 512	1572	99 %
C	Kyber 768	1580	99 %
C	Kyber 1024	1586	99 %

Tabla B.40. Recursos de memoria y CPU — Parte 1.

C	Kyber 512 AVX2	1681	99 %
C	Kyber 768 AVX2	1715	99 %
C	Kyber 1024 AVX2	1693	99 %
C	McEliece 348864	3465	98 %
C	McEliece 348864f	3478	98 %
C	McEliece 460896	4090	98 %
C	McEliece 460896f	4013	98 %
C	McEliece 6688128	5353	99 %
C	McEliece 6688128f	5413	99 %
C	McEliece 6960119	5337	99 %
C	McEliece 6960119f	5391	99 %
C	McEliece 8192128	5886	99 %
C	McEliece 8192128f	5850	99 %
C	SPHINCS+ sha256-128f-simple ref	3272	99 %
C	SPHINCS+ sha256-128f-simple sha2_avx2	2931	99 %
C	SPHINCS+ sha256-128s-simple ref	3360	99 %
C	SPHINCS+ sha256-128s-simple sha2_avx2	3078	99 %
C	SPHINCS+ sha256-192f-simple ref	3271	100 %
C	SPHINCS+ sha256-192f-simple sha2_avx2	3079	99 %
C	SPHINCS+ sha256-192s-simple ref	3272	99 %
C	SPHINCS+ sha256-192s-simple sha2_avx2	2929	99 %
C	SPHINCS+ sha256-256f-simple ref	3412	100 %
C	SPHINCS+ sha256-256f-simple sha2_avx2	3012	99 %
C	SPHINCS+ sha256-256s-simple ref	3446	99 %

Tabla B.41. Recursos de memoria y CPU — Parte 2.

C	SPHINCS+ sha256-256s-simple sha2_avx2	3033	99 %
C	SPHINCS+ shake256-128f-simple shake_avx2	2982	99 %
C	SPHINCS+ shake256-128s-simple shake_avx2	2989	99 %
C	SPHINCS+ shake256-192f-simple shake_avx2	2918	99 %
C	SPHINCS+ shake256-192s-simple shake_avx2	3017	99 %
C	SPHINCS+ shake256-256f-simple shake_avx2	2955	99 %
C	SPHINCS+ shake256-256s-simple shake_avx2	2957	99 %
Java	DILITHIUM 2	263902	182,00 %
Java	DILITHIUM 3	267801	162,60 %
Java	DILITHIUM 5	259148	151,10 %
Java	KYBER 512	255152	250,00 %
Java	KYBER 768	251761	209,00 %
Java	KYBER 1024	253896	217,70 %
Java	FALCON 512 corto	279359	112,90 %
Java	FALCON 512 largo	280225	114,10 %
Java	FALCON 1024 corto	280768	114,00 %
Java	FALCON 1024 largo	280083	113,90 %

Tabla B.42. Recursos de memoria y CPU — Parte 3.

Java	HQC 128	226919	101,50 %
Java	HQC 192	226507	102,00 %
Java	HQC 256	227052	102,00 %
Java	BIKE 128	44159	155,70 %
Java	BIKE 192	44126	148,10 %
Java	BIKE 256	44151	151,90 %
Java	mceliece348864	285308	102,33 %
Java	mceliece348864f	282877	102,00 %
Java	mceliece460896	292300	102,00 %
Java	mceliece460896f	276285	101,67 %
Java	mceliece6688128	285037	101,67 %
Java	mceliece6688128f	278525	102,00 %
Java	mceliece6960119	276976	102,00 %
Java	mceliece6960119f	288079	101,67 %
Java	mceliece8192128	283393	101,67 %
Java	mceliece8192128f	278213	101,67 %
Java	SHA2-128s	240240	107,00 %
Java	SHA2-128f	235764	227,00 %

Tabla B.43. Recursos de memoria y CPU — Parte 4.

Java	SHA2-192s	253212	105,00 %
Java	SHA2-192f	234692	171,00 %
Java	SHA2-256s	255704	118,00 %
Java	SHA2-256f	222048	130,00 %
Java	SHAKE-128s	214840	104,00 %
Java	SHAKE-128f	122988	166,00 %
Java	SHAKE-192s	218464	102,00 %
Java	SHAKE-192f	200904	145,00 %
Java	SHAKE-256s	215088	102,00 %
Java	SHAKE-256f	217052	125,00 %
C#	BIKE 128	47196	100 %
C#	BIKE 192	47196	100 %
C#	BIKE 256	47196	100 %
C#	Dilithium 2.0	47831	100 %
C#	Dilithium 3.0	48725	101 %
C#	Dilithium 5.0	48360	101 %
C#	Falcon 512 (pequeño)	65724	100 %
C#	Falcon 512 (grande)	65724	100 %

Tabla B.44. Recursos de memoria y CPU — Parte 5.

C#	Falcon 1024 (pequeño)	65724	100 %
C#	Falcon 1024 (grande)	65724	100 %
C#	HQC 128	46756	99 %
C#	HQC 192	46932	99 %
C#	HQC 256	46548	99 %
C#	Kyber 512	46443	103 %
C#	Kyber 768	46443	103 %
C#	Kyber 1024	46443	103 %
C#	McEliece 348864	63449	99 %
C#	McEliece 348864f	63449	99 %
C#	McEliece 460896	63449	99 %
C#	McEliece 460896f	63449	99 %
C#	McEliece 6688128	63449	99 %
C#	McEliece 6688128f	63449	99 %
C#	McEliece 6960119	63449	99 %
C#	McEliece 6960119f	63449	99 %
C#	McEliece 8192128	63449	99 %
C#	McEliece 8192128f	63449	99 %

Tabla B.45. Recursos de memoria y CPU — Parte 6.

C#	SPHINCS+ SHA2-128s	46616	98 %
C#	SPHINCS+ SHA2-128f	46820	103 %
C#	SPHINCS+ SHA2-192s	46832	100 %
C#	SPHINCS+ SHA2-192f	46840	102 %
C#	SPHINCS+ SHA2-256s	47164	96 %
C#	SPHINCS+ SHA2-256f	47192	100 %
C#	SPHINCS+ SHAKE-128s	46324	98 %
C#	SPHINCS+ SHAKE-128f	43168	102 %
C#	SPHINCS+ SHAKE-192s	46832	96 %
C#	SPHINCS+ SHAKE-192f	45392	94 %
C#	SPHINCS+ SHAKE-256s	46328	96 %
C#	SPHINCS+ SHAKE-256f	46540	102 %
Python	BIKE L1	19301	98 %
Python	BIKE L3	19384	99 %
Python	BIKE L5	19729	99 %
Python	Dilithium 2 (mensaje corto)	8680	5 %
Python	Dilithium 2 (mensaje largo)	8732	4 %
Python	Dilithium 3 (mensaje corto)	8732	3 %

Tabla B.46. Recursos de memoria y CPU — Parte 7.

Python	Dilithium 3 (mensaje largo)	8648	3 %
Python	Dilithium 5 (mensaje corto)	8784	3 %
Python	Dilithium 5 (mensaje largo)	8680	4 %
Python	Falcon 512	12740	22 %
Python	Falcon 1024	12732	20 %
Python	HQC 128	163996	86 %
Python	HQC 192	164026	92 %
Python	HQC 256	172992	84 %
Python	Kyber 512	8744	4 %
Python	Kyber 768	8829	5 %
Python	Kyber 1024	8839	4 %
Python	McEliece 348864	20428	96 %
Python	McEliece 348864f	20560	96 %
Python	McEliece 460896	21368	97 %
Python	McEliece 460896f	21356	99 %
Python	McEliece 6688128	22692	98 %
Python	McEliece 6688128f	22560	97 %
Python	McEliece 6960119	22576	98 %

Tabla B.47. Recursos de memoria y CPU — Parte 8.

Python	McEliece 6960119f	22692	98 %
Python	McEliece 8192128	23456	98 %
Python	McEliece 8192128f	23452	98 %
Python	SPHINCS+ SHA2-128s	12934	30 %
Python	SPHINCS+ SHA2-192s	12958	30 %
Python	SPHINCS+ SHA2-256s	12954	18 %
Python	SPHINCS+ SHAKE-128s	12934	40 %
Python	SPHINCS+ SHAKE-192s	12964	40 %
Python	SPHINCS+ SHAKE-256s	13006	20 %

Tabla B.48. Recursos de memoria y CPU — Parte 9.

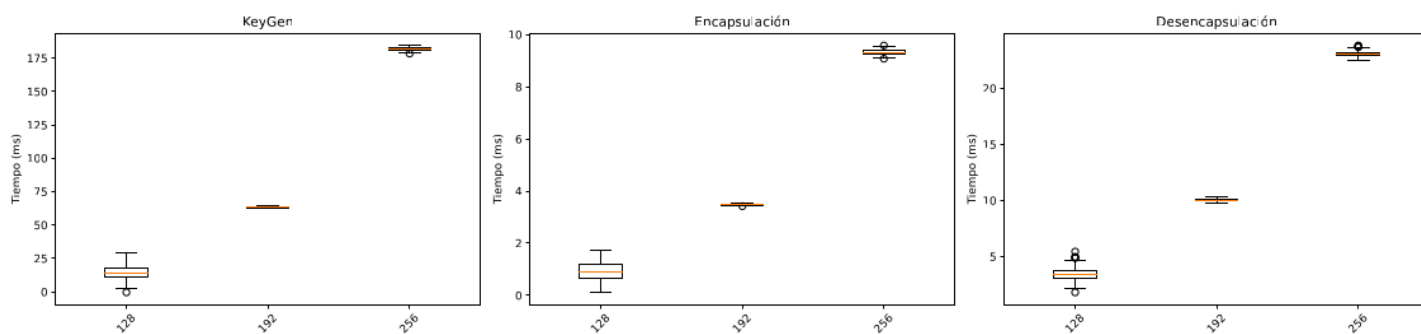
ESTUDIO: RENDIMIENTO DE ALGORITMOS DE 3^a Y 4^a RONDA EN DISPOSITIVO RASPBERRY PI 5

A continuación se presentan los resultados completos obtenidos para los algoritmos post-cuánticos ejecutados en el dispositivo Raspberry Pi 5. Se incluyen las mediciones en C, C#, Python y Java, mostrando tiempos medios (en milisegundos), desviación estándar y consumo de recursos.

BIKE

Lenguaje	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
C#	BIKE-128	14.790 ms (± 5.421)	0.951 ms (± 0.384)	3.481 ms (± 0.593)	19.22 ms (± 6.40)
C#	BIKE-192	63.136 ms (± 0.399)	3.473 ms (± 0.020)	10.031 ms (± 0.112)	76.64 ms (± 0.53)
C#	BIKE-256	181.624 ms (± 1.047)	9.319 ms (± 0.099)	23.078 ms (± 0.236)	214.02 ms (± 1.38)

BIKE (C# — Raspberry Pi) — Rendimiento



BIKE (C# — Raspberry Pi) — Tiempo Total

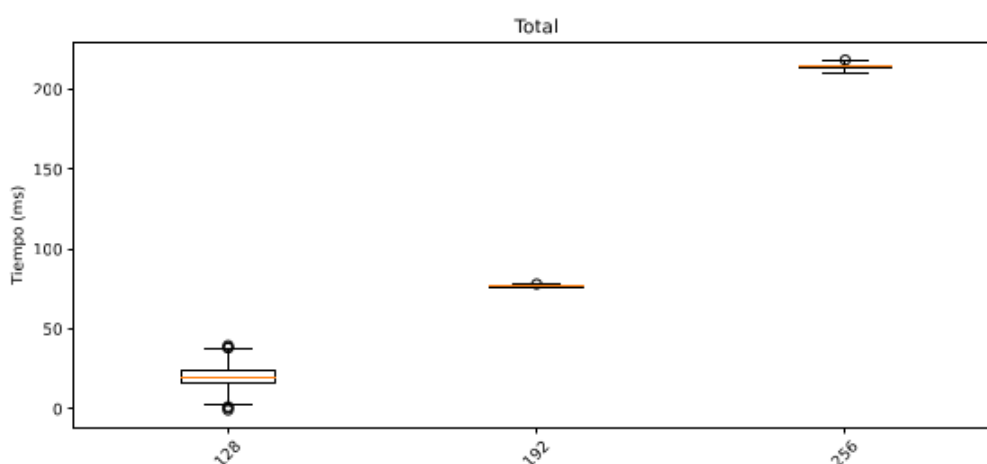
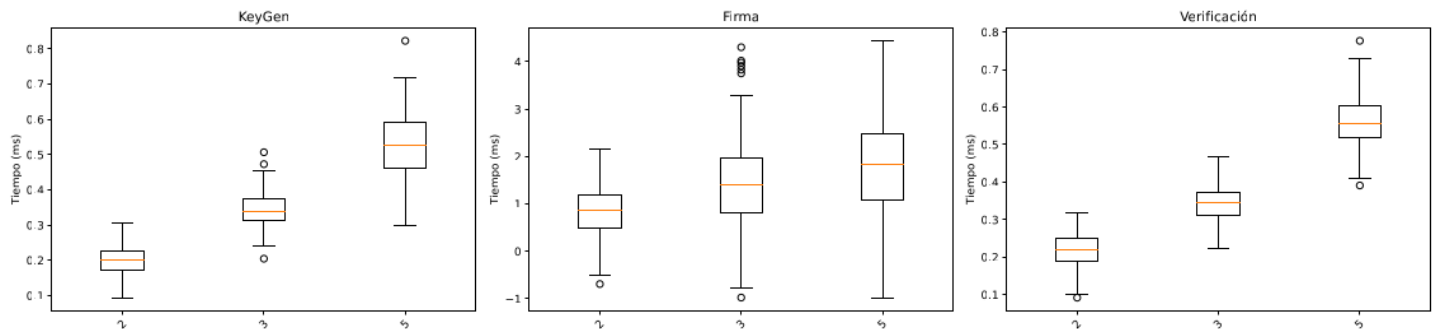


Tabla C.1. Resultados del algoritmo BIKE (implementación en C#) en Raspberry Pi 5.

Dilithium

Lenguaje	Versión	T Gen. Claves	T Firma	T Verificación	T Total
C#	Dilithium-2	0.198 ms ($\pm$ 0.040)	0.826 ms ($\pm$ 0.515)	0.218 ms ($\pm$ 0.047)	1.242 ms ($\pm$ 0.602)
C#	Dilithium-3	0.337 ms ($\pm$ 0.049)	1.422 ms ($\pm$ 1.018)	0.344 ms ($\pm$ 0.048)	2.103 ms ($\pm$ 1.115)
C#	Dilithium-5	0.527 ms ($\pm$ 0.084)	1.754 ms ($\pm$ 1.045)	0.563 ms ($\pm$ 0.075)	2.844 ms ($\pm$ 1.204)

Dilithium (C# — Raspberry Pi) — Rendimiento



Dilithium (C# — Raspberry Pi) — Tiempo Total

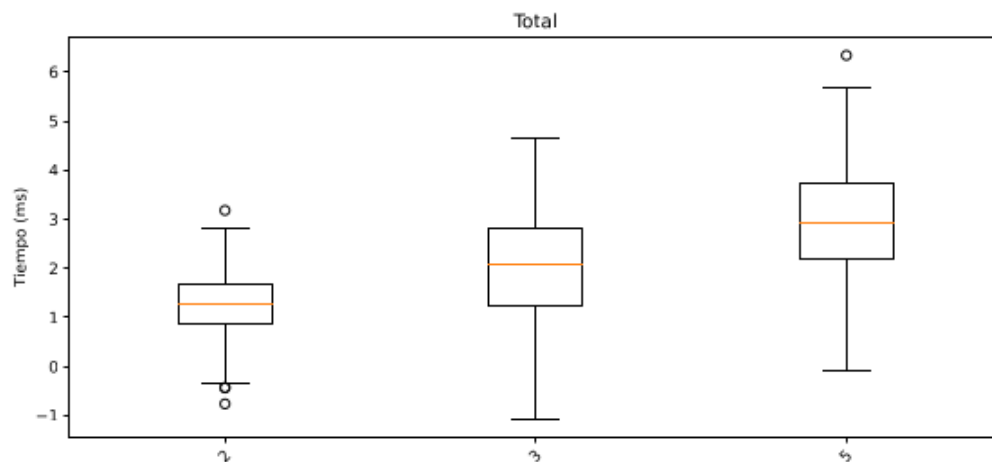
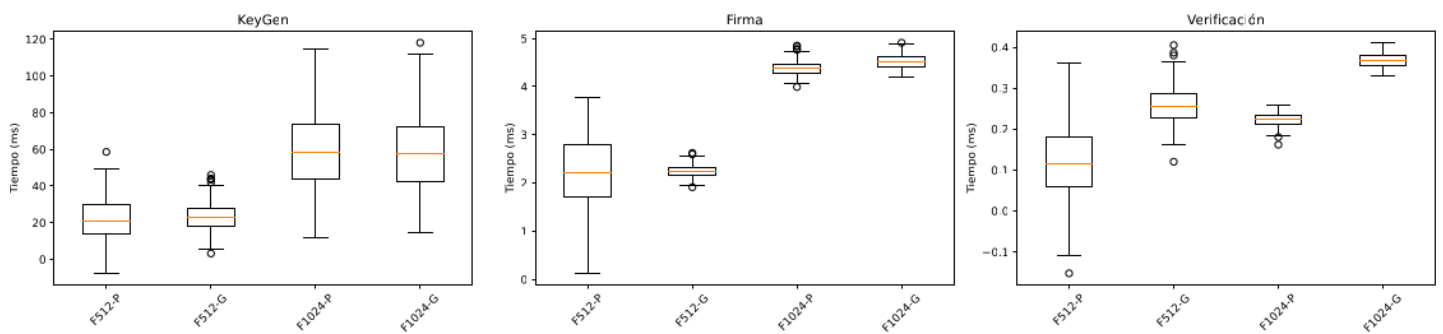


Tabla C.2. Resultados del algoritmo Dilithium (implementación en C#) en Raspberry Pi 5.

Falcon					
Lenguaje	Versión	T Gen. Claves	T Firma	T Verificación	T Total
C#	Falcon-512 (Pequeño)	22.745 ms ($\pm$ 11.883)	2.174 ms ($\pm$ 0.714)	0.123 ms ($\pm$ 0.101)	25.04 ms ($\pm$ 12.70)
C#	Falcon-512 (Grande)	21.833 ms ($\pm$ 7.781)	2.239 ms ($\pm$ 0.128)	0.258 ms ($\pm$ 0.049)	24.33 ms ($\pm$ 7.96)
C#	Falcon-1024 (Pequeño)	58.100 ms ($\pm$ 19.165)	4.372 ms ($\pm$ 0.138)	0.221 ms ($\pm$ 0.015)	62.69 ms ($\pm$ 19.32)
C#	Falcon-1024 (Grande)	58.210 ms ($\pm$ 20.259)	4.524 ms ($\pm$ 0.146)	0.365 ms ($\pm$ 0.017)	63.10 ms ($\pm$ 20.42)

Falcon (C# — Raspberry Pi) — Rendimiento



Falcon (C# — Raspberry Pi) — Tiempo Total

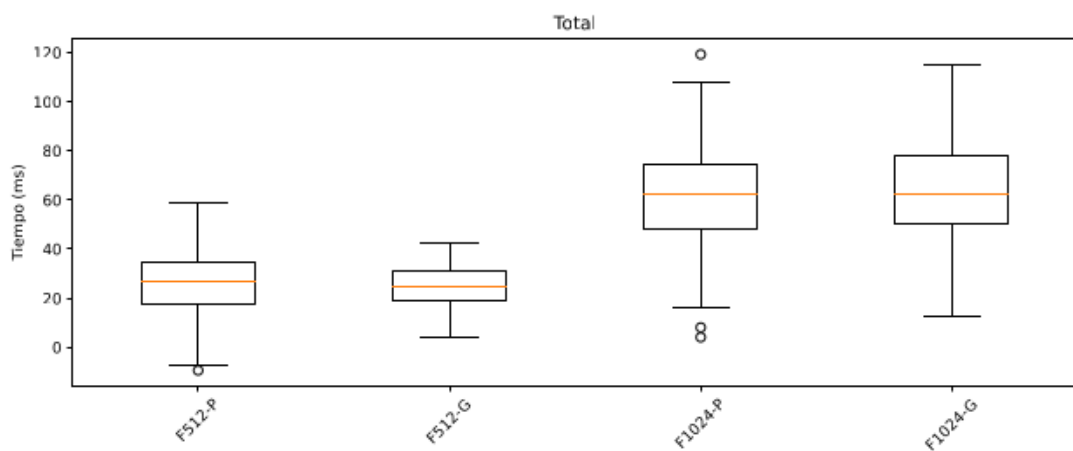
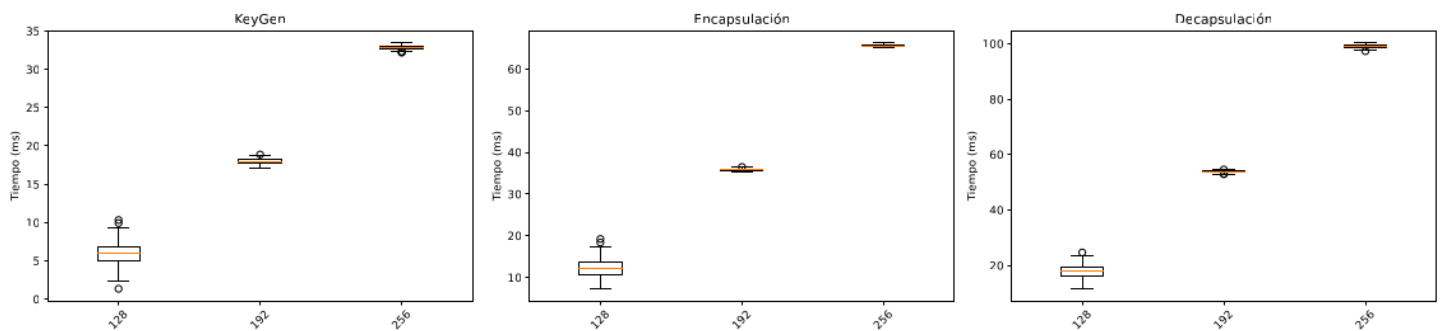


Tabla C.3. Resultados del algoritmo FALCON (implementación en C#) en Raspberry Pi 5.

HQC

Lenguaje	Versión	T Gen. Claves	T Encapsulación	T Decapsulación	T Total
C#	HQC-12 8	5.900 ms ($\pm$ 1.428)	11.806 ms ($\pm$ 2.342)	17.820 ms ($\pm$ 2.660)	35.53 ms ($\pm$ 6.43)
C#	HQC-19 2	17.939 ms ($\pm$ 0.332)	35.792 ms ($\pm$ 0.234)	53.801 ms ($\pm$ 0.303)	107.53 ms ($\pm$ 0.87)
C#	HQC-25 6	32.878 ms ($\pm$ 0.220)	65.561 ms ($\pm$ 0.267)	99.112 ms ($\pm$ 0.579)	197.55 ms ($\pm$ 1.07)

HQC (C# — Raspberry Pi) — Rendimiento



HQC (C# — Raspberry Pi) — Tiempo Total

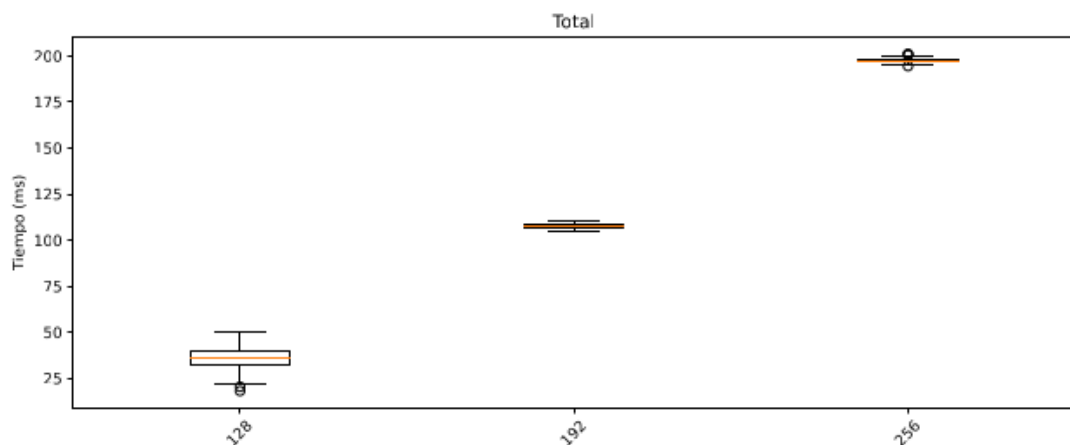
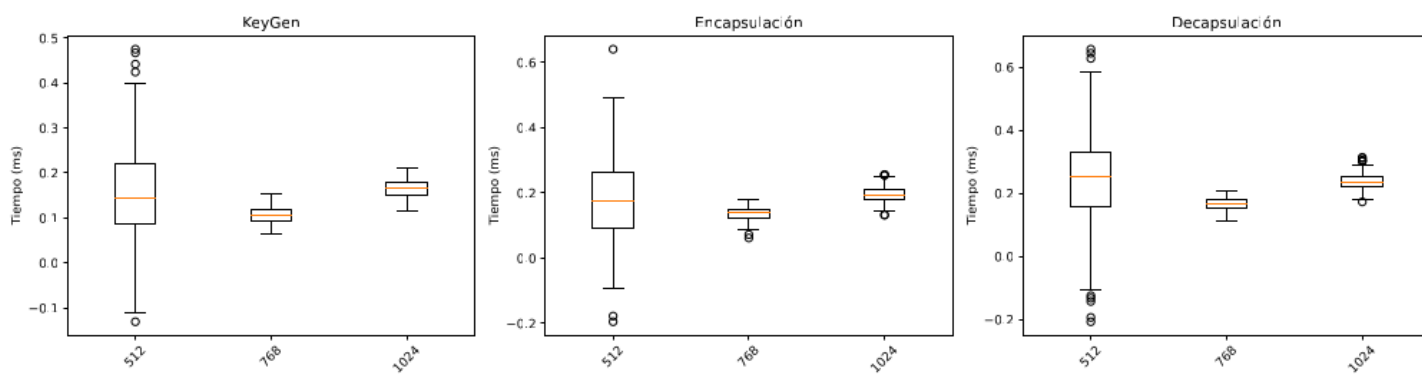


Tabla C.4. Resultados del algoritmo HQC (implementación en C#) en Raspberry Pi 5.

Kyber

Lenguaje	Versión	T Gen. Claves	T Encapsulación	T Decapsulación	T Total
C#	Kyber-512	0.139 ms ($\pm$ 0.111)	0.170 ms ($\pm$ 0.131)	0.232 ms ($\pm$ 0.165)	0.541 ms ($\pm$ 0.41)
C#	Kyber-768	0.108 ms ($\pm$ 0.018)	0.135 ms ($\pm$ 0.021)	0.166 ms ($\pm$ 0.021)	0.409 ms ($\pm$ 0.06)
C#	Kyber-1024	0.165 ms ($\pm$ 0.020)	0.196 ms ($\pm$ 0.022)	0.237 ms ($\pm$ 0.025)	0.598 ms ($\pm$ 0.07)

Kyber (C# — Raspberry Pi) — Rendimiento



Kyber (C# — Raspberry Pi) — Tiempo Total

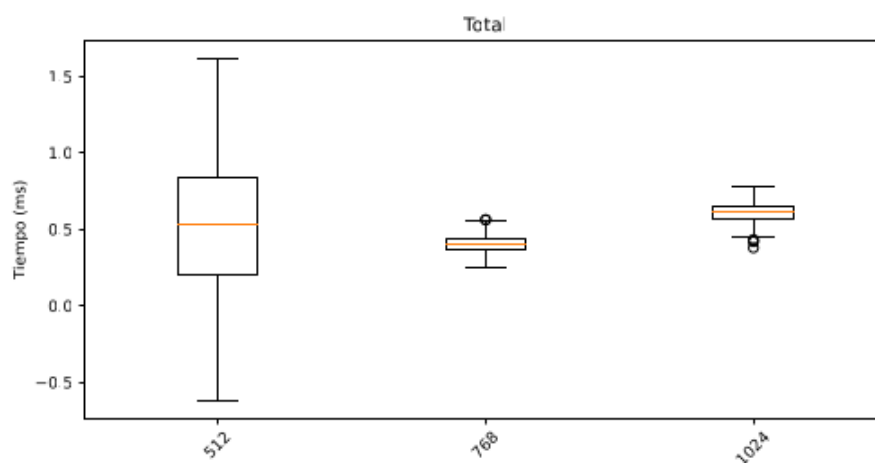


Tabla C.5. Resultados del algoritmo Kyber (implementación en C#) en Raspberry Pi 5.

McEliece					
Lenguaje	Versión	T Gen. Claves (ms)	T Encapsulación (ms)	T Decapsulación (ms)	T Total (ms)
C#	mceliece348864	66.833 (± 3.944)	0.579 (± 0.048)	0.474 (± 0.033)	67.886 (± 4.020)
C#	mceliece348864f	68.421 (± 3.671)	0.591 (± 0.045)	0.474 (± 0.041)	69.486 (± 3.751)
C#	mceliece460896	111.562 (± 4.872)	0.609 (± 0.046)	0.512 (± 0.036)	112.683 (± 4.949)
C#	mceliece460896f	114.123 (± 5.019)	0.617 (± 0.044)	0.503 (± 0.034)	115.243 (± 5.095)
C#	mceliece6688128	210.875 (± 7.533)	0.739 (± 0.057)	0.604 (± 0.047)	212.218 (± 7.640)
C#	mceliece6688128 f	213.766 (± 7.385)	0.727 (± 0.052)	0.609 (± 0.041)	215.102 (± 7.496)
C#	mceliece6960119	287.933 (± 8.251)	0.842 (± 0.059)	0.689 (± 0.052)	289.464 (± 8.362)
C#	mceliece6960119 f	289.852 (± 8.134)	0.836 (± 0.058)	0.682 (± 0.051)	291.370 (± 8.243)
C#	mceliece8192128	415.473 (± 9.907)	0.967 (± 0.067)	0.785 (± 0.060)	417.225 (± 10.042)

Tabla C.6. Resultados del algoritmo Classic McEliece (implementación en C#) — Parte 1, en Raspberry Pi 5.

C#

mceliece8192128
f

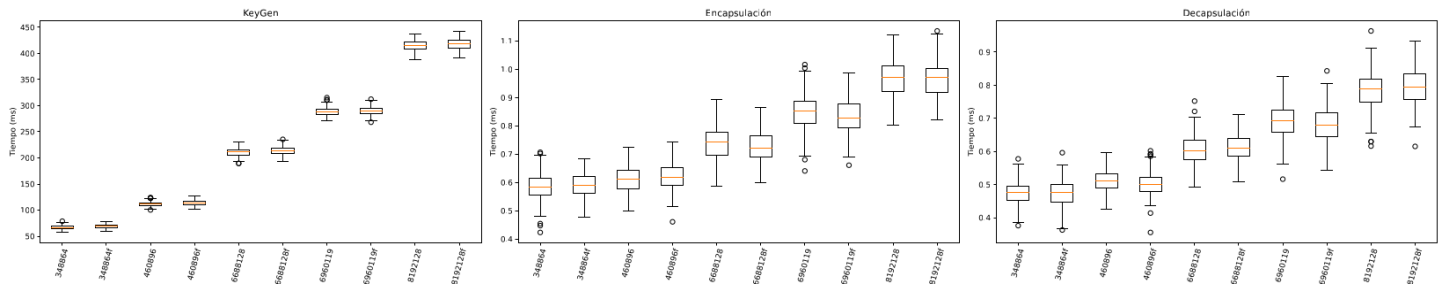
418.093
(±
9.866)

0.958 (± 0.062)

0.796 (± 0.058)

419.847
(±
9.987)

Classic McEliece (C# — Raspberry Pi) — Rendimiento



Classic McEliece (C# — Raspberry Pi) — Tiempo Total

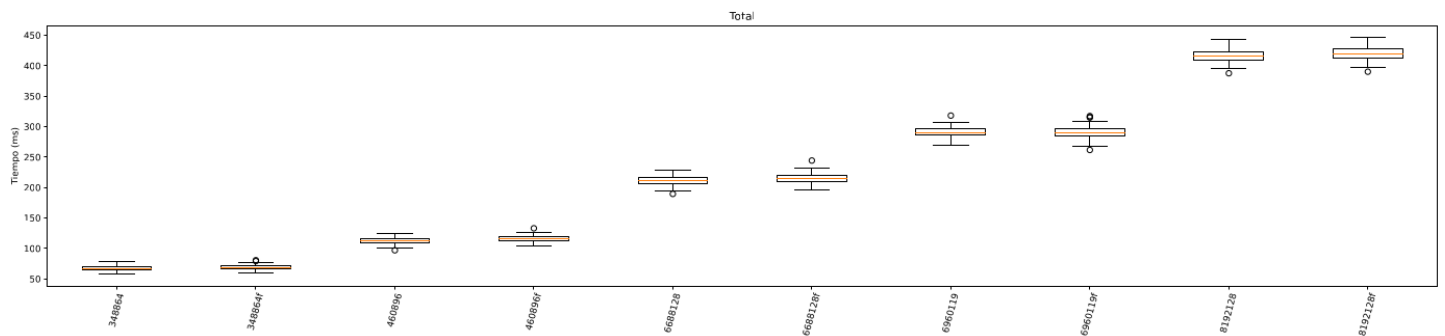
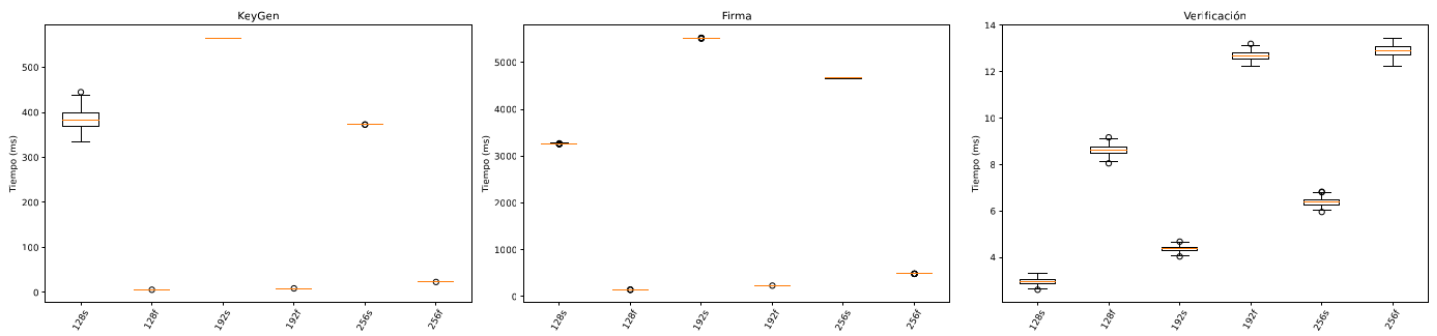


Tabla C.7. Resultados del algoritmo Classic McEliece (implementación en C#) — Parte 2, en Raspberry Pi 5.

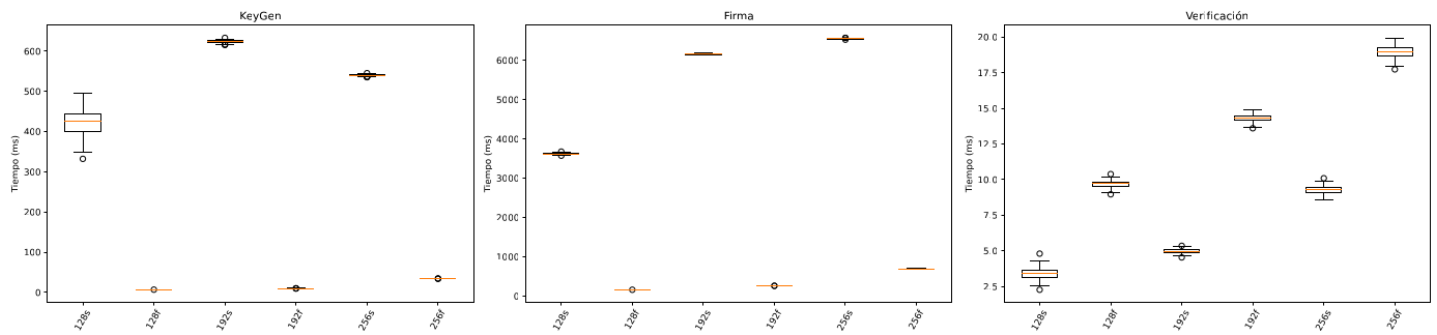
SPHINCS+					
Lenguaje	Versión	T Gen. Claves (ms)	T Firma (ms)	T Verificación (ms)	T Total (ms)
C#	SHA2-128s	425.602 ($\pm$ 27.560)	3615.902 ($\pm$ 17.227)	3.393 ($\pm$ 0.406)	4044.897 ($\pm$ 32.63)
C#	SHA2-128f	6.630 ($\pm$ 0.048)	160.113 ($\pm$ 0.289)	9.656 ($\pm$ 0.236)	176.399 ($\pm$ 0.57)
C#	SHA2-192s	623.499 ($\pm$ 2.980)	6154.876 ($\pm$ 11.097)	4.965 ($\pm$ 0.128)	6783.340 ($\pm$ 14.20)
C#	SHA2-192f	9.794 ($\pm$ 0.096)	259.231 ($\pm$ 0.587)	14.297 ($\pm$ 0.252)	283.322 ($\pm$ 0.94)
C#	SHA2-256s	540.137 ($\pm$ 1.649)	6548.963 ($\pm$ 7.243)	9.261 ($\pm$ 0.243)	7098.361 ($\pm$ 9.14)
C#	SHA2-256f	33.788 ($\pm$ 0.136)	691.440 ($\pm$ 1.607)	19.025 ($\pm$ 0.403)	744.253 ($\pm$ 2.15)
C#	SHAKE-128s	384.527 ($\pm$ 19.529)	3265.648 ($\pm$ 2.685)	2.972 ($\pm$ 0.123)	3653.147 ($\pm$ 22.34)
C#	SHAKE-128f	5.988 ($\pm$ 0.027)	144.636 ($\pm$ 0.156)	8.637 ($\pm$ 0.202)	159.261 ($\pm$ 0.38)
C#	SHAKE-192s	564.649 ($\pm$ 0.450)	5515.927 ($\pm$ 3.444)	4.360 ($\pm$ 0.118)	6084.936 ($\pm$ 4.01)
C#	SHAKE-192f	8.831 ($\pm$ 0.034)	232.943 ($\pm$ 0.276)	12.682 ($\pm$ 0.182)	254.456 ($\pm$ 0.49)
C#	SHAKE-256s	373.769 ($\pm$ 0.260)	4664.367 ($\pm$ 3.741)	6.380 ($\pm$ 0.155)	5044.516 ($\pm$ 4.16)
C#	SHAKE-256f	23.387 ($\pm$ 0.081)	485.079 ($\pm$ 0.355)	12.890 ($\pm$ 0.243)	521.356 ($\pm$ 0.68)

Tabla C.8. Resultados del algoritmo SPHINCS+ (implementación en C#) — Parte 1, en Raspberry Pi 5.

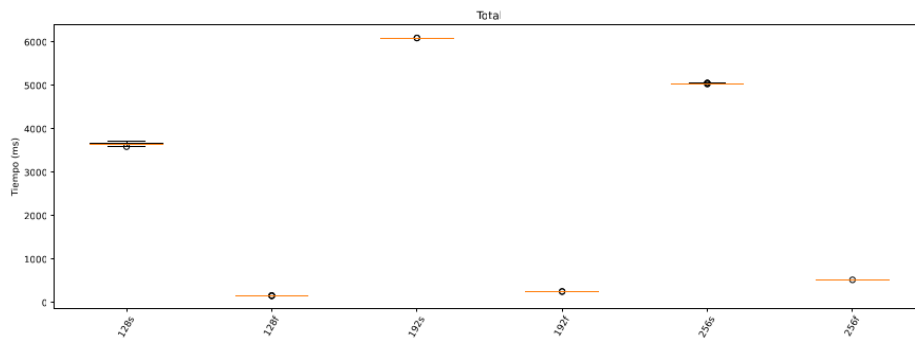
SPHINCS+ (SHAKE — C# Raspberry Pi) — Rendimiento



SPHINCS+ (SHA2 — C# Raspberry Pi) — Rendimiento



SPHINCS+ (SHAKE — C# Raspberry Pi) — Tiempo Total



SPHINCS+ (SHA2 — C# Raspberry Pi) — Tiempo Total

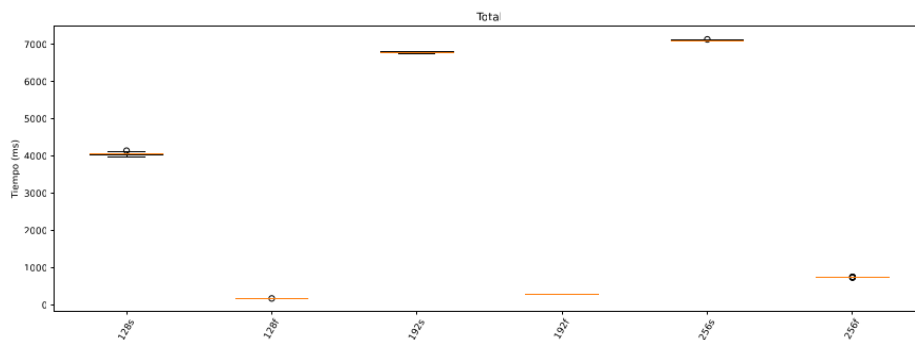


Tabla C.9. Resultados del algoritmo SPHINCS+ (implementación en C#) — Parte 2, en Raspberry Pi

Dilithium					
Lenguaje	Versión	T Generación de Claves	T Firma	T Verificación	T Total
C	Dilithium 2	0.1183 ms ($\pm$ 0.0158)	0.5900 ms ($\pm$ 0.3944)	0.1398 ms ($\pm$ 0.0185)	0.8481 ms ($\pm$ 0.4029)
C	Dilithium 3	0.2036 ms ($\pm$ 0.0014)	0.8584 ms ($\pm$ 0.5722)	0.2092 ms ($\pm$ 0.0012)	1.2712 ms ($\pm$ 0.5722)
C	Dilithium 5	0.3109 ms ($\pm$ 0.0025)	1.0467 ms ($\pm$ 0.5808)	0.3362 ms ($\pm$ 0.0020)	1.6937 ms ($\pm$ 0.5808)
C	Dilithium 2 native	0.1214 ms ($\pm$ 0.0188)	0.5579 ms ($\pm$ 0.3845)	0.1429 ms ($\pm$ 0.0217)	0.8222 ms ($\pm$ 0.3932)
C	Dilithium 3 native	0.2075 ms ($\pm$ 0.0013)	0.8517 ms ($\pm$ 0.5576)	0.2132 ms ($\pm$ 0.0013)	1.2724 ms ($\pm$ 0.5576)
C	Dilithium 5 native	0.3175 ms ($\pm$ 0.0026)	1.0586 ms ($\pm$ 0.6145)	0.3424 ms ($\pm$ 0.0023)	1.7185 ms ($\pm$ 0.6145)

Dilithium (C) — Rendimiento

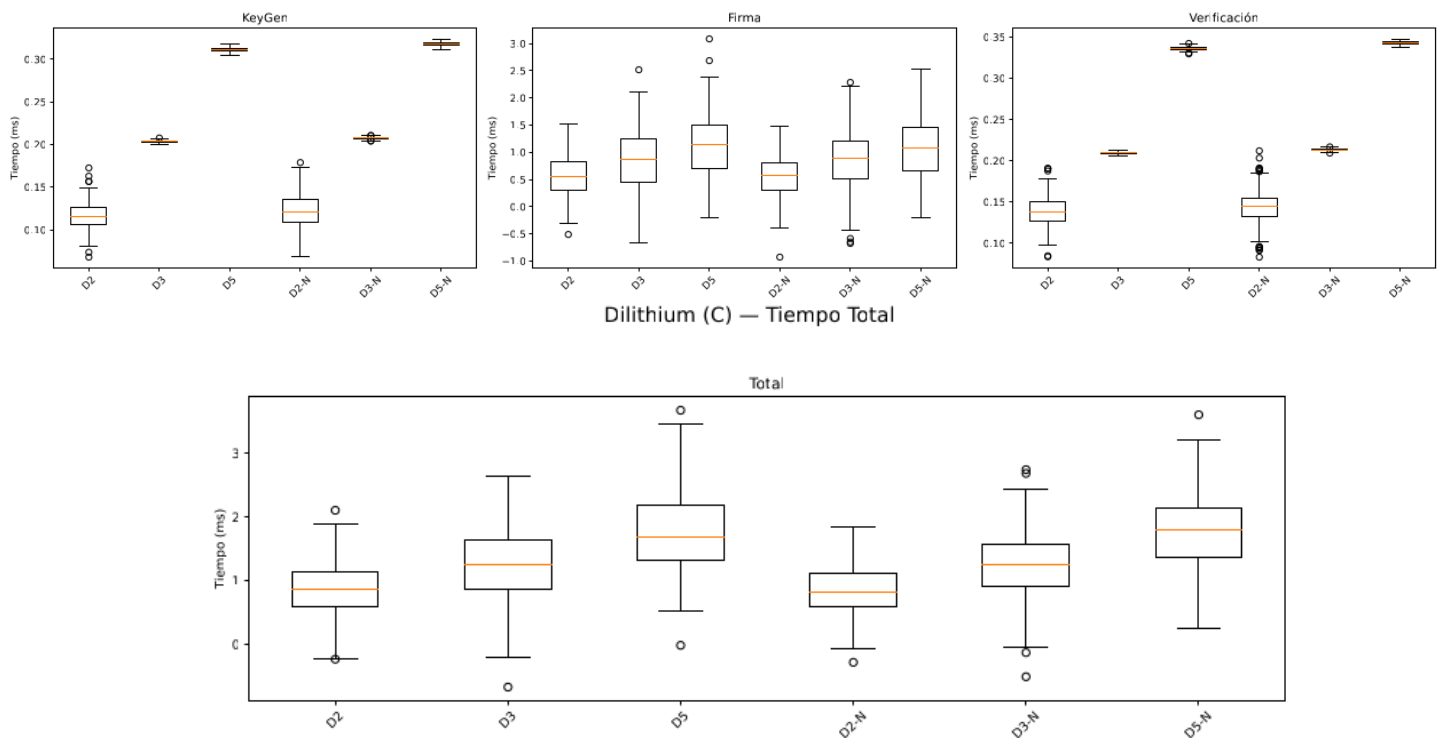


Tabla C.10. Resultados del algoritmo Dilithium (implementación en C) en Raspberry Pi 5.

Falcon					
Lenguaje	Versión	KeyGen (ms)	Sign (ms)	Verify (ms)	Total (ms)
C	Falcon-512 Reference	22.3999 ± 7.3489	6.5031 ± 0.0152	0.0543 ± 0.0010	28.9572
C	Falcon-1024 Reference	66.5583 ± 25.8308	14.2131 ± 0.0233	0.1149 ± 0.0009	80.8862
C	Falcon-512 NEON	22.1882 ± 7.3500	6.4347 ± 0.0147	0.0426 ± 0.0004	28.6655
C	Falcon-1024 NEON	65.8895 ± 25.7019	14.0623 ± 0.0308	0.0870 ± 0.0006	80.0388

Falcon (C) — Rendimiento

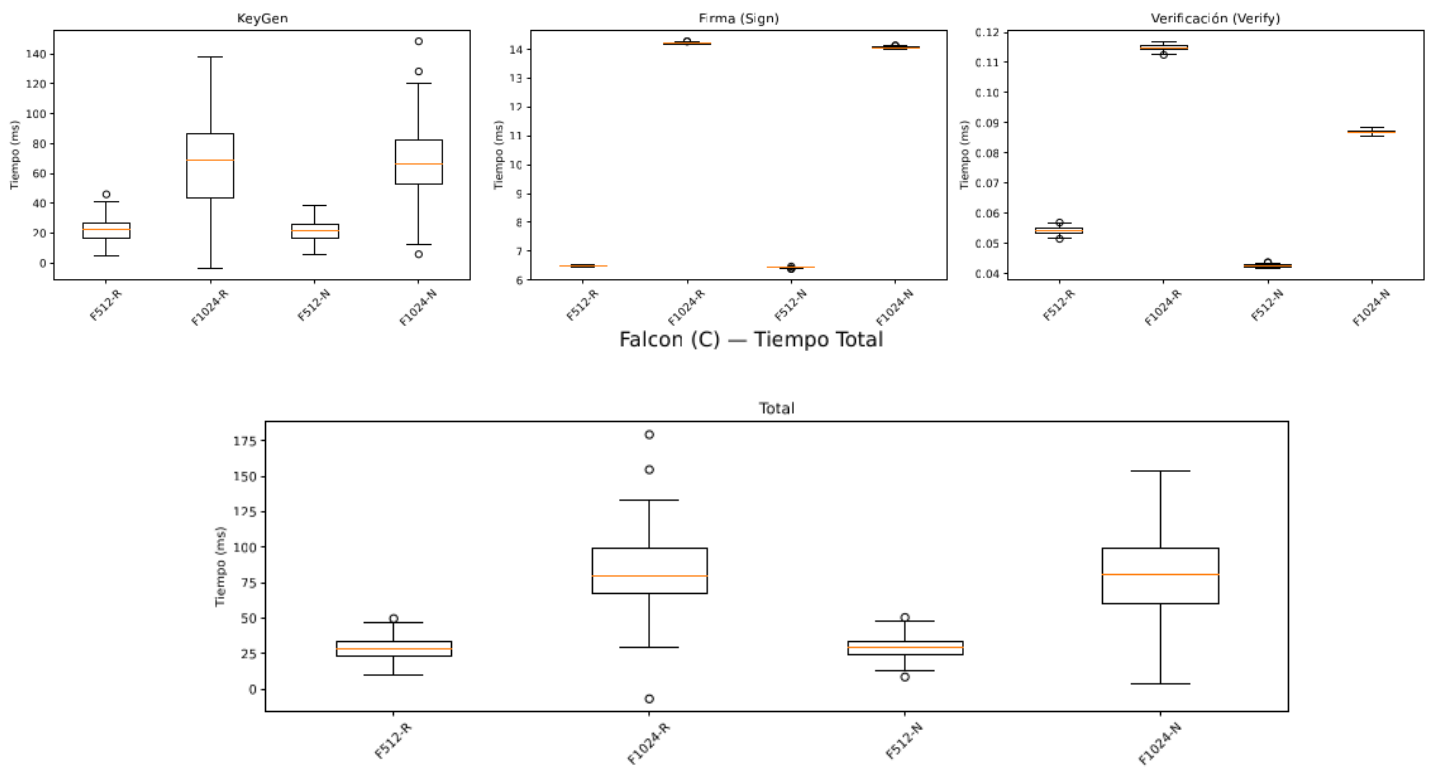


Tabla C.11. Resultados del algoritmo Falcon (implementación en C) en Raspberry Pi 5.

Kyber					
Lenguaje	Versión	KeyGen (ms)	Encaps (ms)	Decaps (ms)	Total (ms)
C	Kyber-512 Reference	0.0535 ± 0.0090	0.0711 ± 0.0120	0.0849 ± 0.0142	0.2095 ± 0.0352
C	Kyber-768 Reference	0.0868 ± 0.0147	0.1063 ± 0.0180	0.1243 ± 0.0209	0.3174 ± 0.0535
C	Kyber-1024 Reference	0.1361 ± 0.0249	0.1623 ± 0.0295	0.1859 ± 0.0338	0.4843 ± 0.0882
C	Kyber-512 NEON	0.0204 ± 0.0007	0.0267 ± 0.0006	0.0252 ± 0.0006	0.0724 ± 0.0013
C	Kyber-768 NEON	0.0435 ± 0.0099	0.0562 ± 0.0127	0.0534 ± 0.0121	0.1531 ± 0.0345
C	Kyber-1024 NEON	0.0502 ± 0.0008	0.0624 ± 0.0008	0.0601 ± 0.0007	0.1727 ± 0.0017

Kyber (C) — Rendimiento

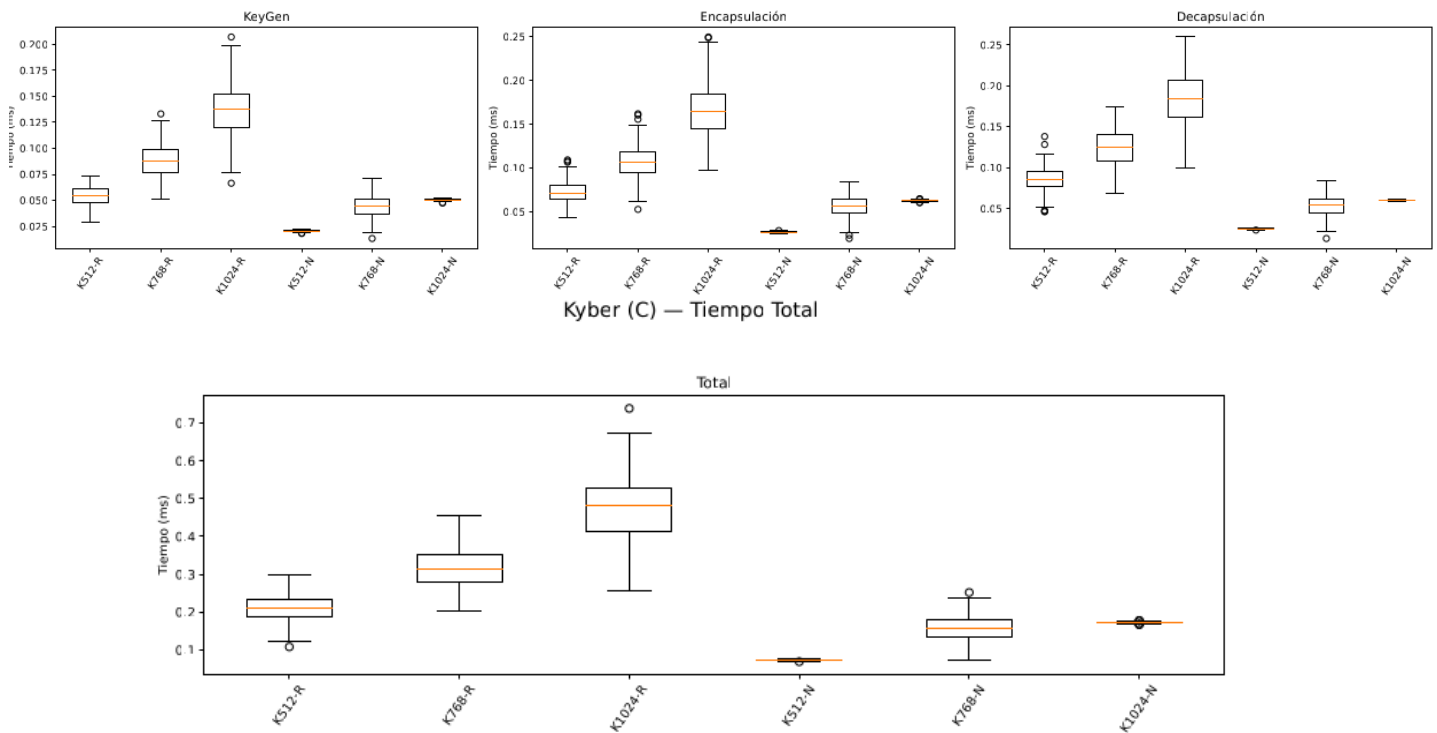
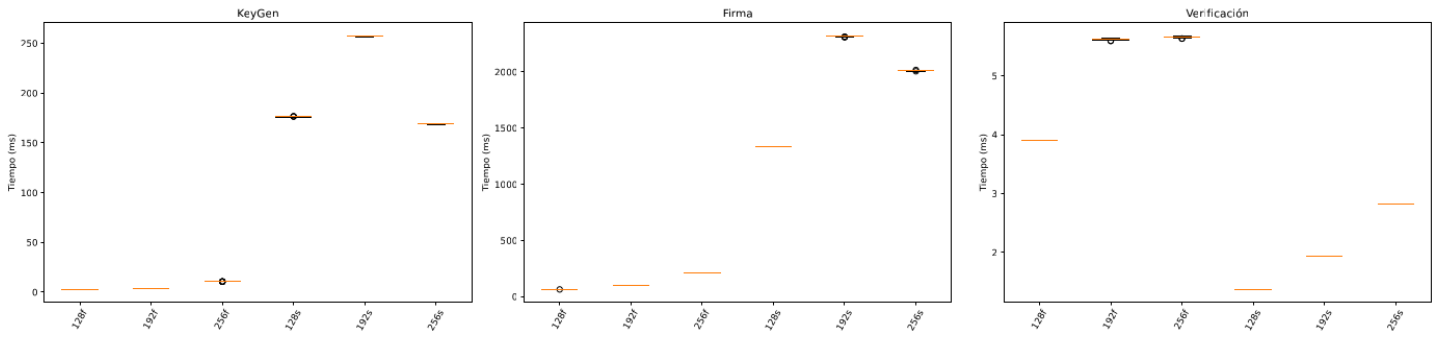


Tabla C.12. Resultados del algoritmo Kyber (implementación en C) en Raspberry Pi 5.

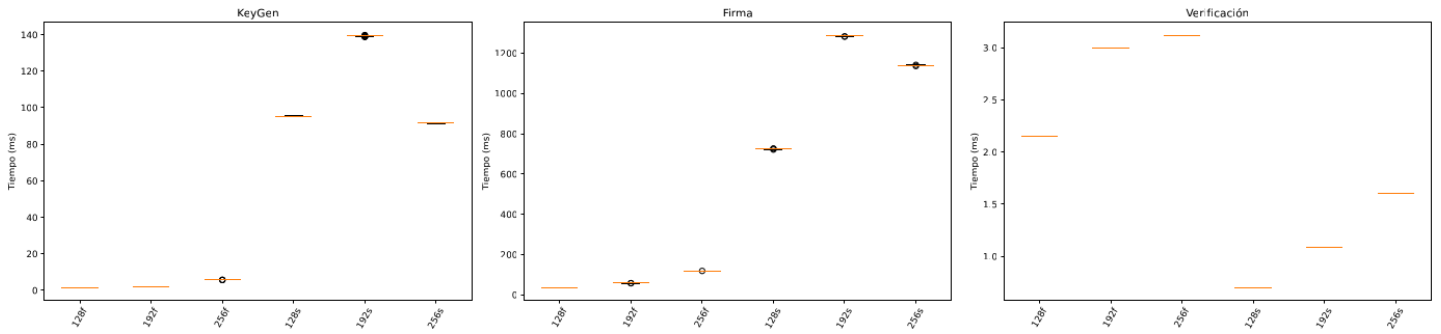
SPHINCS+					
Lenguaje	Versión	T Gen. Claves (ms)	T Firma (ms)	T Verificación (ms)	T Total (ms)
C	sphincs-shake-128f	2.75 (± 0.00)	64.36 (± 0.06)	3.90 (± 0.00)	71.01 (± 0.06)
C	sphincs-shake-192f	4.03 (± 0.00)	103.99 (± 0.10)	5.62 (± 0.01)	113.64 (± 0.10)
C	sphincs-shake-256f	10.57 (± 0.01)	214.46 (± 0.21)	5.66 (± 0.01)	230.69 (± 0.21)
C	sphincs-shake-128s	176.14 (± 0.18)	1334.92 (± 1.33)	1.37 (± 0.00)	1512.43 (± 1.35)
C	sphincs-shake-192s	257.14 (± 0.26)	2318.65 (± 2.31)	1.94 (± 0.00)	2577.73 (± 2.33)
C	sphincs-shake-256s	169.03 (± 0.17)	2012.16 (± 2.01)	2.83 (± 0.00)	2184.02 (± 2.02)
C	sphincs-sha2-128f	1.49 (± 0.00)	34.82 (± 0.03)	2.15 (± 0.00)	38.46 (± 0.03)
C	sphincs-sha2-192f	2.19 (± 0.00)	57.22 (± 0.06)	3.00 (± 0.00)	62.41 (± 0.06)
C	sphincs-sha2-256f	5.74 (± 0.01)	117.68 (± 0.12)	3.12 (± 0.00)	126.48 (± 0.12)
C	sphincs-sha2-128s	95.30 (± 0.09)	723.90 (± 0.72)	0.70 (± 0.00)	819.90 (± 0.73)
C	sphincs-sha2-192s	139.28 (± 0.14)	1287.16 (± 1.29)	1.09 (± 0.00)	1427.53 (± 1.30)
C	sphincs-sha2-256s	91.58 (± 0.09)	1139.45 (± 1.14)	1.60 (± 0.00)	1232.63 (± 1.14)

Tabla C.13. Resultados del algoritmo SPHINCS+ (implementación en C) — Parte 1, en Raspberry Pi

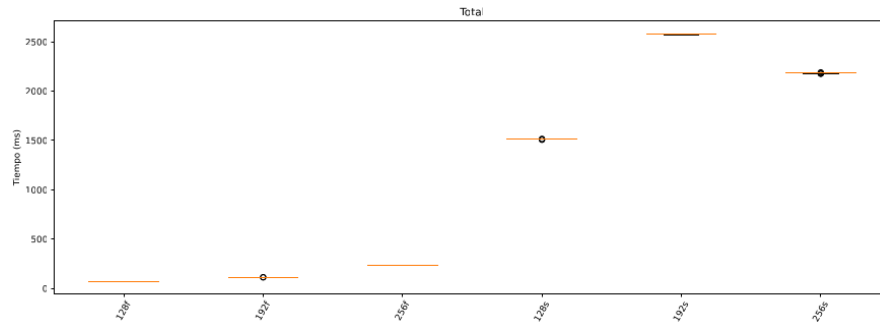
SPHINCS+ (SHAKE — C) — Rendimiento



SPHINCS+ (SHA2 — C) — Rendimiento



SPHINCS+ (SHAKE — C) — Tiempo Total



SPHINCS+ (SHA2 — C) — Tiempo Total

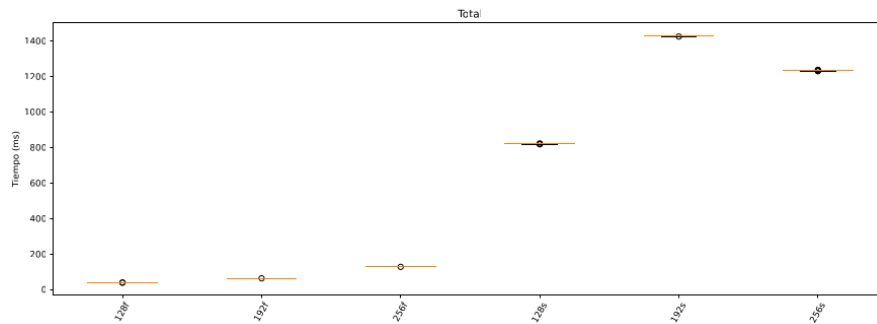


Tabla C.14. Resultados del algoritmo SPHINCS+ (implementación en C) — Parte 2, en Raspberry Pi

Lenguaje	Versión	T Gen. Claves (ms)	Bike		T Total (ms)
			T Encapsulación (ms)	T Desencapsulación (ms)	
C (NEON/F PU)	BIKE-12 8	2.8017 ($\pm$ 0.1556)	0.2336 ($\pm$ 0.0130)	34.7562 ($\pm$ 0.6939)	37.7915 ($\pm$ 0.7113)
C (NEON/F PU)	BIKE-19 2	11.3508 ($\pm$ 0.0933)	0.5592 ($\pm$ 0.0117)	55.5367 ($\pm$ 0.0231)	67.4468 ($\pm$ 0.0968)
C (NEON/F PU)	BIKE-25 6	24.3724 ($\pm$ 0.1569)	1.1863 ($\pm$ 0.0233)	121.4586 ($\pm$ 0.1625)	147.0173 ($\pm$ 0.2270)
C	BIKE-12 8	2.8025 ($\pm$ 0.0709)	0.2374 ($\pm$ 0.0065)	34.8111 ($\pm$ 0.1915)	37.8510 ($\pm$ 0.2043)
C	BIKE-19 2	11.3614 ($\pm$ 0.0938)	0.5671 ($\pm$ 0.0118)	55.6871 ($\pm$ 0.0224)	67.6156 ($\pm$ 0.0972)
C	BIKE-25 6	24.4089 ($\pm$ 0.1573)	1.1927 ($\pm$ 0.0221)	121.6545 ($\pm$ 0.0497)	147.2561 ($\pm$ 0.1664)

BIKE (C — Reference vs NEON/FPU) — Rendimiento

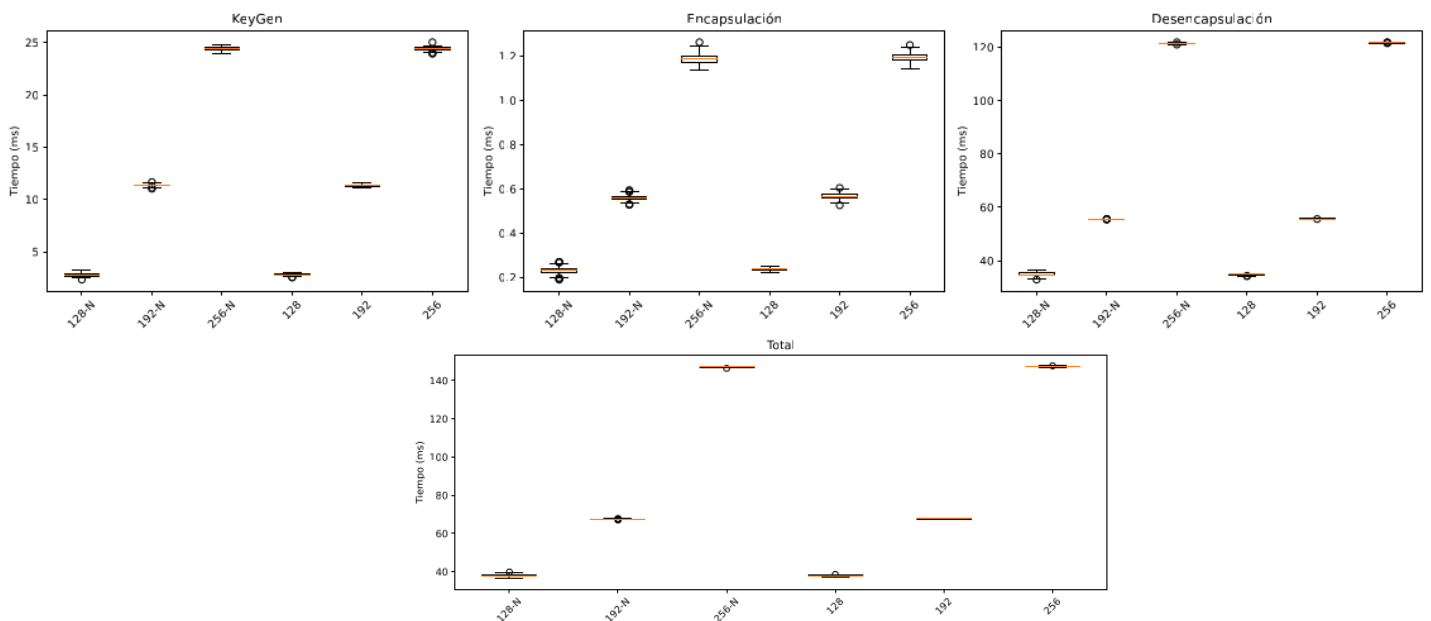


Tabla C.15. Resultados del algoritmo BIKE (implementación en C) en Raspberry Pi 5.

Lenguaje	Versión	HQC			
		T Gen. Claves (ms)	T Encapsulación (ms)	T Desencapsulación (ms)	T Total (ms)
C	HQC-12 8	2.2307 ($\pm$ 0.0852)	4.5384 ($\pm$ 0.1719)	7.4254 ($\pm$ 0.2595)	14.1944 ($\pm$ 0.5147)
C	HQC-19 2	6.8001 ($\pm$ 0.0062)	13.7103 ($\pm$ 0.0099)	21.3389 ($\pm$ 0.0154)	41.8493 ($\pm$ 0.0255)
C	HQC-25 6	12.4776 ($\pm$ 0.0128)	25.1947 ($\pm$ 0.0197)	39.7009 ($\pm$ 0.0286)	77.3733 ($\pm$ 0.0528)

HQC (C) — Rendimiento

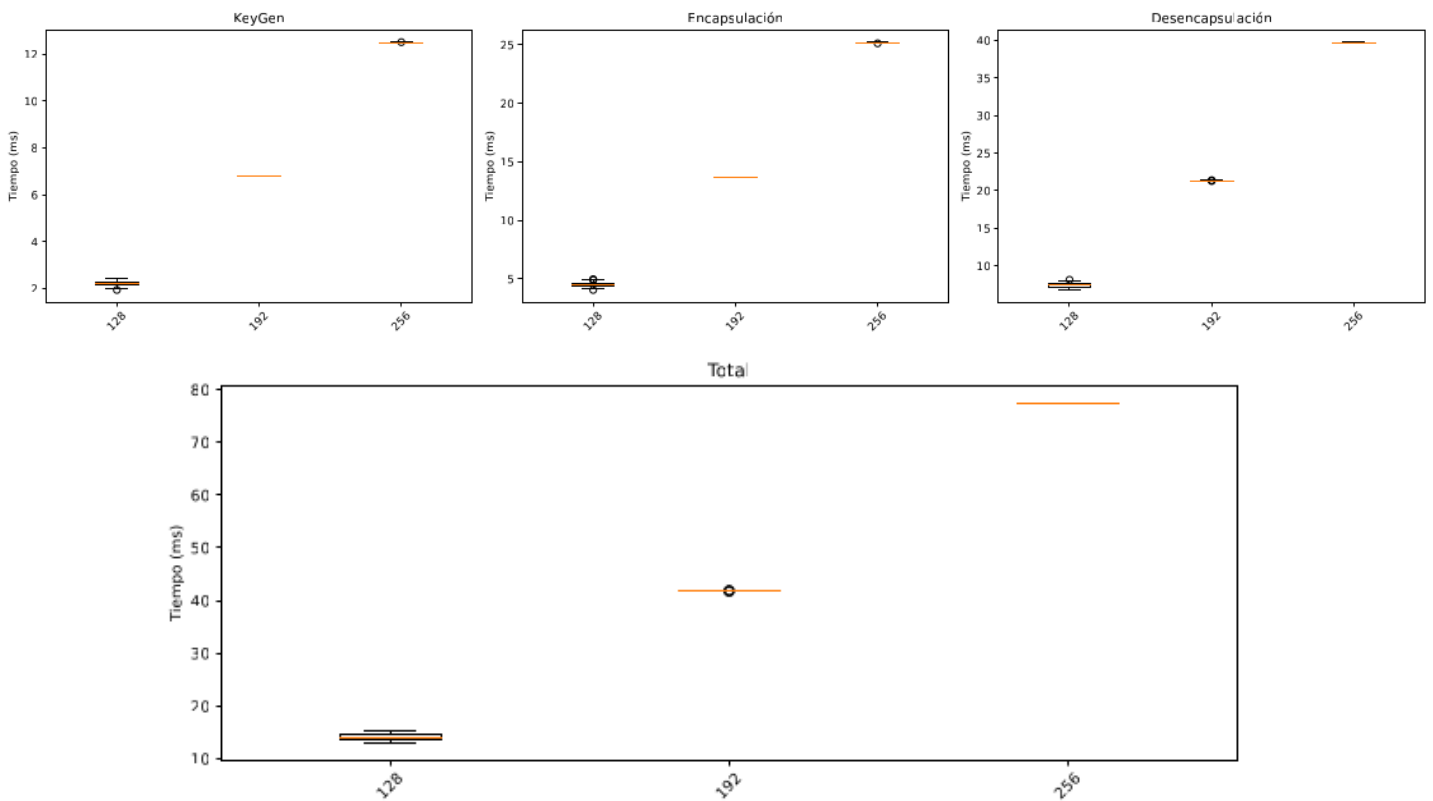
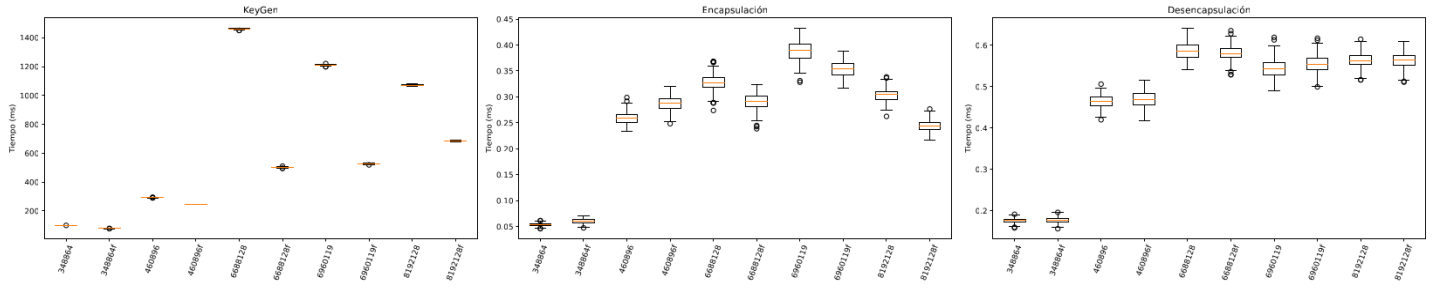


Tabla C.16. Resultados del algoritmo HQC (implementación en C) en Raspberry Pi 5.

Lenguaje	Versión	McEliece			
		T Gen. Claves (ms)	T Encapsulación (ms)	T Desencapsulación (ms)	T Total (ms)
C	mceliece348864 (ARM)	98.66 (± 0.42)	0.054 (± 0.003)	0.176 (± 0.006)	98.89 (± 0.43)
C	mceliece348864f (ARM)	78.01 (± 0.39)	0.060 (± 0.004)	0.178 (± 0.007)	78.25 (± 0.41)
C	mceliece460896 (ARM)	292.87 (± 0.91)	0.258 (± 0.012)	0.465 (± 0.014)	293.60 (± 0.93)
C	mceliece460896f (ARM)	246.01 (± 0.77)	0.286 (± 0.013)	0.471 (± 0.020)	246.77 (± 0.79)
C	mceliece6688128 (ARM)	1460.2 8 (± 3.54)	0.329 (± 0.015)	0.587 (± 0.021)	1461.2 0 (± 3.57)
C	mceliece6688128 f (ARM)	502.54 (± 2.62)	0.292 (± 0.017)	0.583 (± 0.018)	503.41 (± 2.64)
C	mceliece6960119 (ARM)	1210.4 8 (± 3.28)	0.388 (± 0.018)	0.544 (± 0.022)	1211.4 1 (± 3.31)
C	mceliece6960119 f (ARM)	526.41 (± 2.73)	0.354 (± 0.015)	0.555 (± 0.020)	527.32 (± 2.76)
C	mceliece8192128 (ARM)	1070.6 6 (± 3.10)	0.303 (± 0.012)	0.564 (± 0.018)	1071.5 3 (± 3.13)
C	mceliece8192128 f (ARM)	685.40 (± 3.03)	0.244 (± 0.010)	0.563 (± 0.019)	686.21 (± 3.06)

Tabla C.17. Resultados del algoritmo Classic McEliece (implementación en C) - Parte 1, en Raspberry Pi 5.

Classic McEliece (C — ARM) — Rendimiento



Classic McEliece (C — ARM) — Tiempo Total

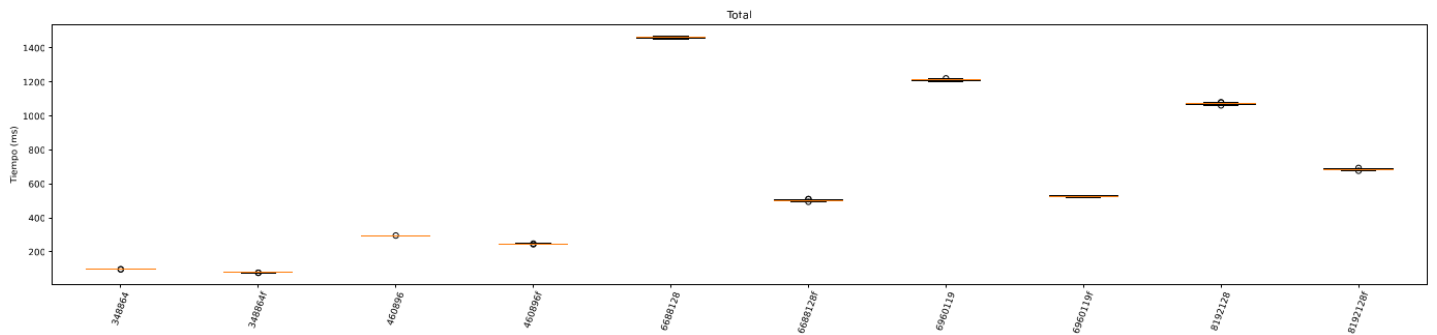


Tabla C.18. Resultados del algoritmo Classic McEliece (implementación en C) — Parte 2, en Raspberry Pi 5.

Bike					
Lenguaje	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
Java	BIKE-128	8.92 ms (± 1.93)	0.61 ms (± 0.67)	27.83 ms (± 2.16)	37.36 ms (± 4.72)
Java	BIKE-192	38.26 ms (± 0.06)	2.08 ms (± 0.04)	78.66 ms (± 0.60)	118.99 ms (± 0.61)
Java	BIKE-256	109.85 ms (± 0.10)	5.59 ms (± 0.02)	172.79 ms (± 1.55)	288.23 ms (± 1.54)

BIKE (Java) — Rendimiento

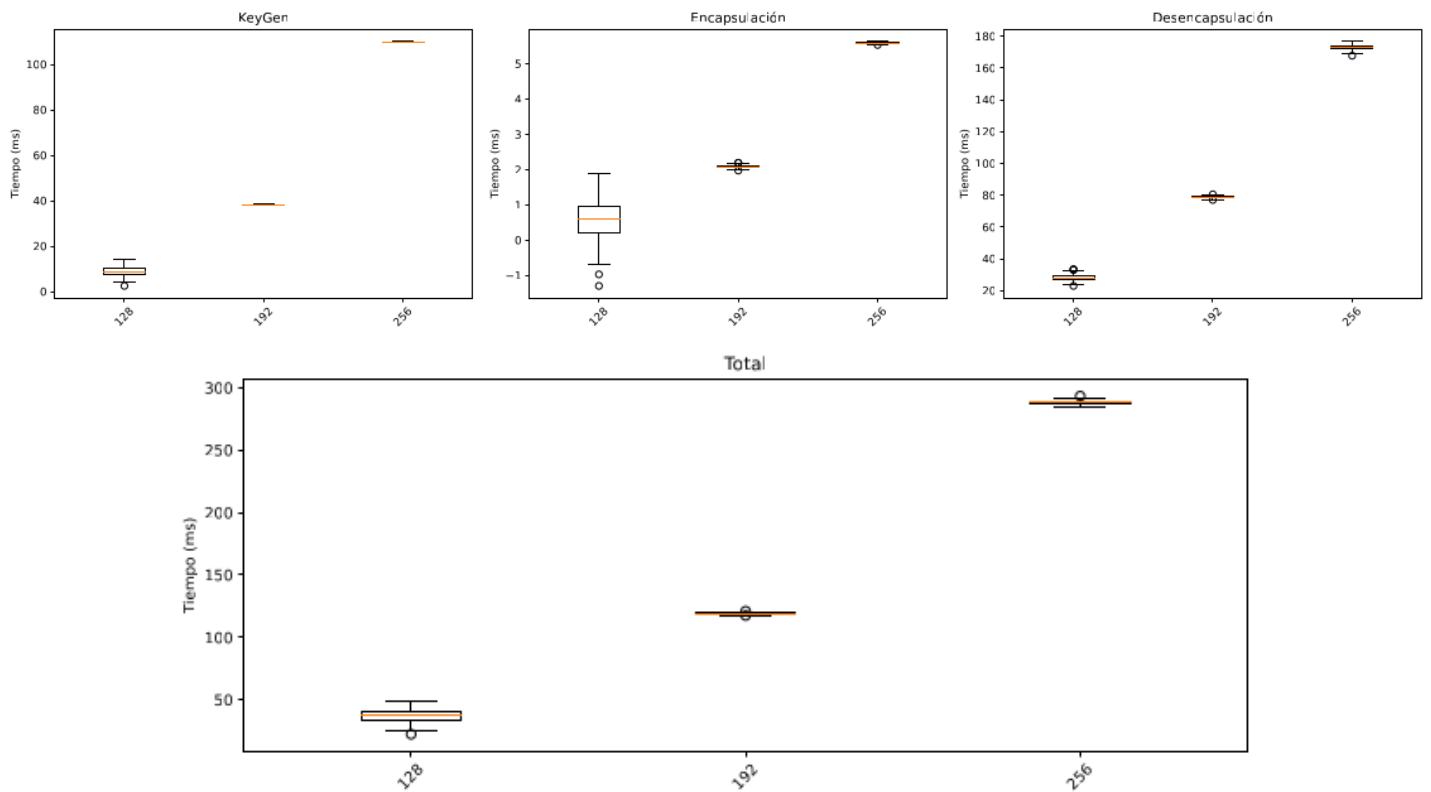


Tabla C.19. Resultados del algoritmo BIKE (implementación en Java) en Raspberry Pi 5.

Dilithium						
Lenguaje	Versión	T Gen. Claves (ms)	T Firma Pequeña (ms)	T Firma Grande (ms)	T Verif. Pequeña (ms)	T Verif. Grande (ms)
Java	Dilithium-2	0.25 (± 0.85)	0.82 (± 1.12)	0.83 (± 0.64)	0.21 (± 0.15)	0.25 (± 0.17)
Java	Dilithium-3	0.38 (± 1.06)	1.34 (± 1.29)	1.31 (± 0.91)	0.31 (± 0.13)	0.35 (± 0.15)
Java	Dilithium-5	0.53 (± 1.15)	1.57 (± 1.11)	1.58 (± 1.16)	0.49 (± 0.23)	0.53 (± 0.26)

Dilithium (Java) — Rendimiento

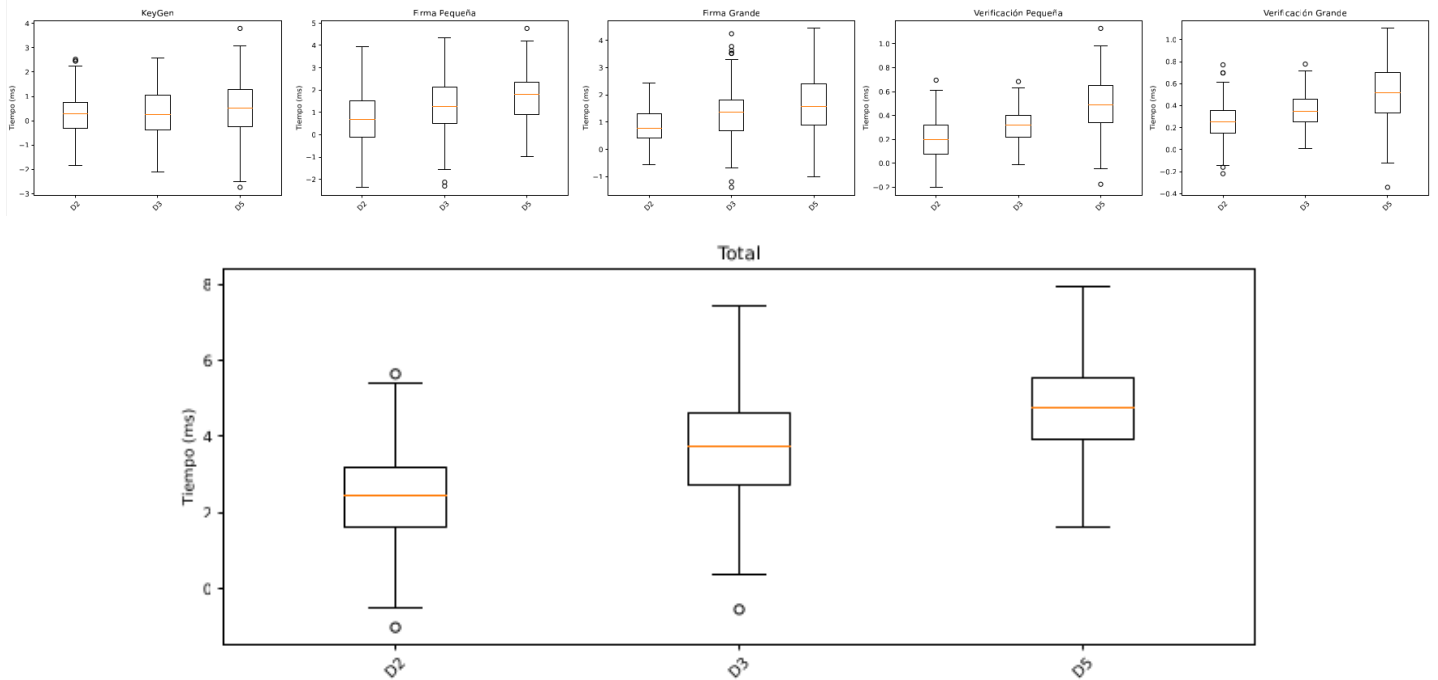


Tabla C.20. Resultados del algoritmo Dilithium (implementación en Java) en Raspberry Pi 5.

Falcon						
Lenguaje	Versión	Tipo	T Gen. Claves (ms)	T Firma (ms)	T Verificación (ms)	T Total (ms)
Java	Falcon-512	Corto	18.35 ($\pm$ 6.39)	1.89 ($\pm$ 1.08)	0.16 ($\pm$ 0.15)	20.40 ($\pm$ 7.01)
Java	Falcon-512	Largo	17.59 ($\pm$ 4.81)	1.66 ($\pm$ 0.16)	0.20 ($\pm$ 0.01)	19.46 ($\pm$ 4.81)
Java	Falcon-1024	Corto	51.56 ($\pm$ 16.26)	3.59 ($\pm$ 1.47)	0.25 ($\pm$ 0.21)	55.40 ($\pm$ 16.91)
Java	Falcon-1024	Largo	50.09 ($\pm$ 13.61)	3.42 ($\pm$ 0.31)	0.30 ($\pm$ 0.01)	53.82 ($\pm$ 13.63)

Falcon (Java) — Rendimiento

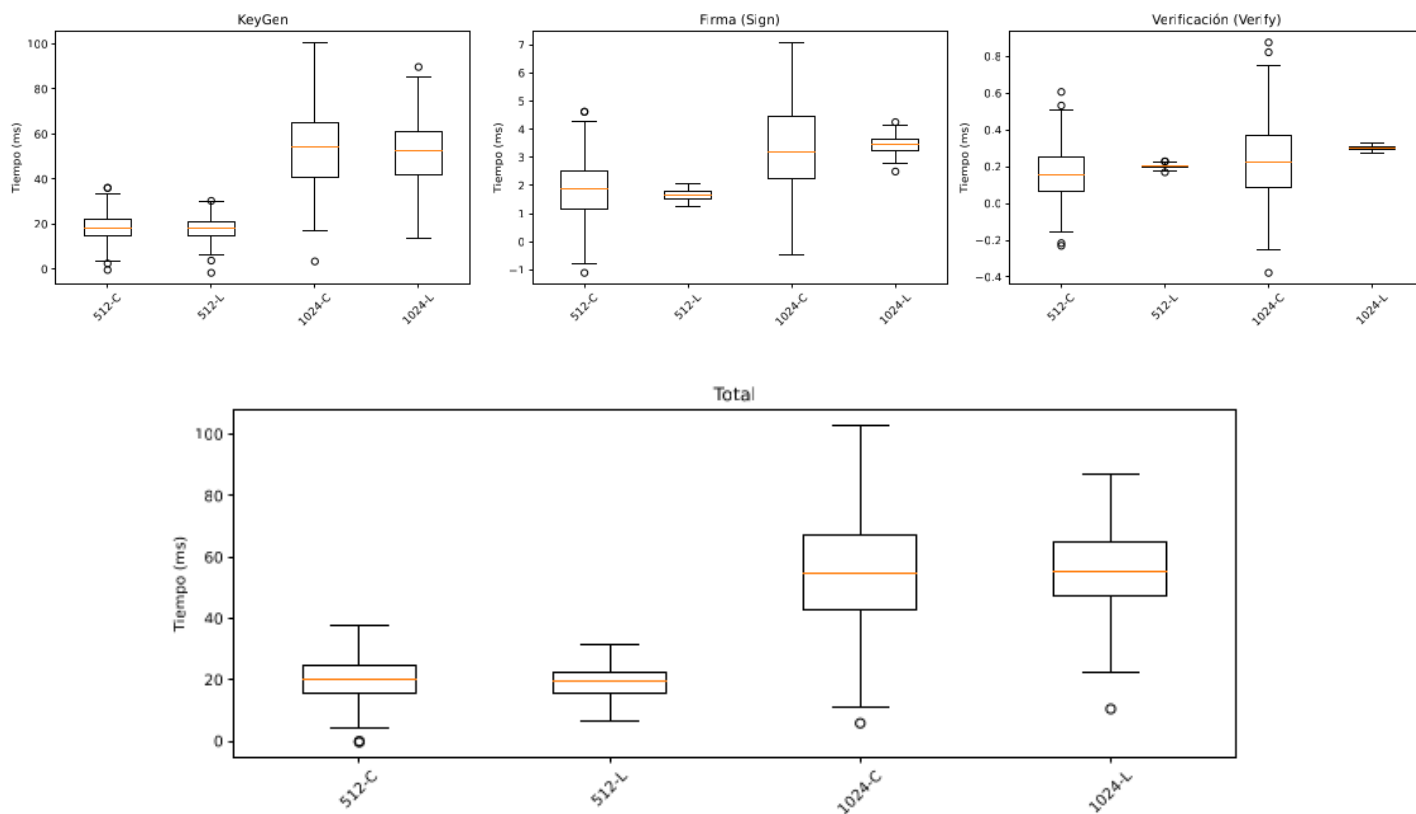


Tabla C.21. Resultados del algoritmo Falcon (implementación en Java) en Raspberry Pi 5.

Kyber					
Lenguaje	Versión	T Gen. Claves (ms)	T Encapsulación (ms)	T Decapsulación (ms)	T Total (ms)
Java	Kyber-512	0.18 ($\pm$ 0.76)	0.17 ($\pm$ 0.68)	0.18 ($\pm$ 0.34)	0.53 ($\pm$ 1.63)
Java	Kyber-768	0.13 ($\pm$ 0.02)	0.13 ($\pm$ 0.02)	0.15 ($\pm$ 0.18)	0.42 ($\pm$ 0.18)
Java	Kyber-1024	0.16 ($\pm$ 0.02)	0.18 ($\pm$ 0.03)	0.22 ($\pm$ 0.45)	0.57 ($\pm$ 0.45)

Kyber (Java) — Rendimiento

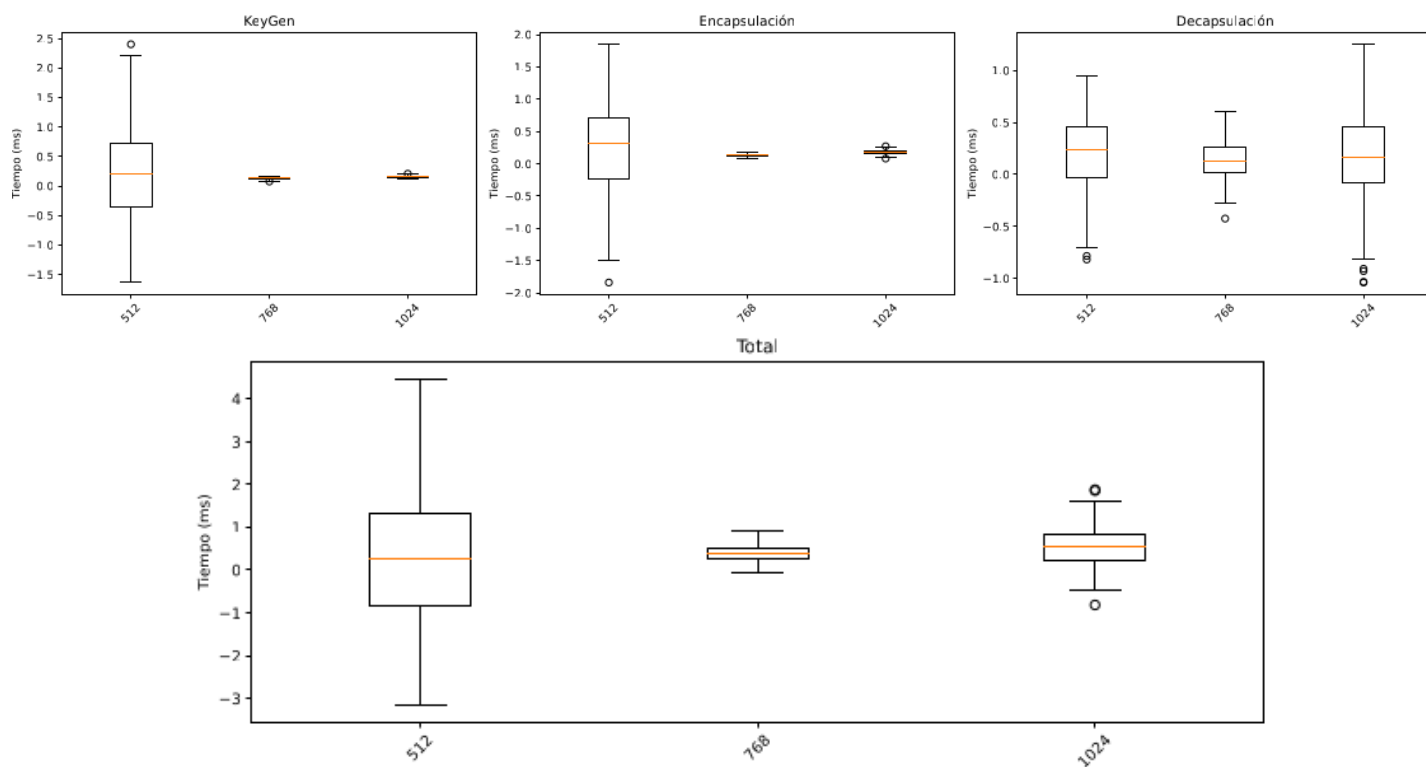
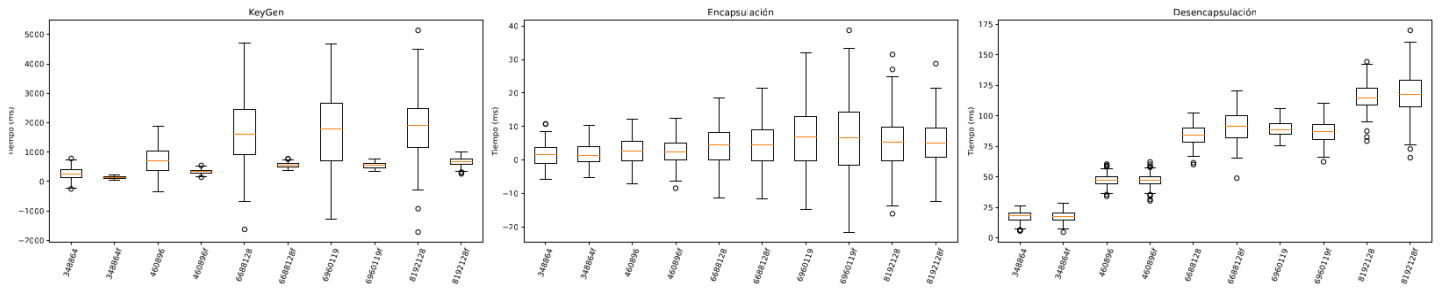


Tabla C.22. Resultados del algoritmo Kyber (implementación en Java) en Raspberry Pi 5.

Lenguaje	Versión	McEliece			
		T Gen. Claves (ms)	T Encapsulación (ms)	T Decapsulación (ms)	T Total (ms)
Java	McEliece-348864	253.98 (± 186.52)	1.61 (± 2.98)	17.30 (± 4.50)	273.02 (± 191.01)
Java	McEliece-348864f	122.65 (± 30.44)	1.85 (± 3.06)	17.30 (± 4.31)	141.91 (± 38.04)
Java	McEliece-460896	731.02 (± 453.21)	2.82 (± 4.26)	46.85 (± 4.75)	780.81 (± 459.31)
Java	McEliece-460896f	337.14 (± 68.61)	2.52 (± 3.85)	46.97 (± 4.79)	386.76 (± 77.63)
Java	McEliece-6688128	1596.76 (± 1180.10)	4.16 (± 5.82)	84.24 (± 7.84)	1685.28 (± 1188.22)
Java	McEliece-6688128 f	546.73 (± 71.07)	4.14 (± 5.92)	91.00 (± 12.29)	641.99 (± 89.71)
Java	McEliece-6960119	1708.14 (± 1243.34)	6.01 (± 7.80)	88.70 (± 6.36)	1802.99 (± 1244.21)
Java	McEliece-6960119 f	548.43 (± 88.46)	7.50 (± 11.04)	87.66 (± 9.14)	643.73 (± 107.55)
Java	McEliece-8192128	1882.28 (± 1074.37)	5.16 (± 7.58)	115.78 (± 10.87)	2003.36 (± 1075.24)
Java	McEliece-8192128 f	681.23 (± 132.35)	5.28 (± 7.38)	118.07 (± 17.24)	804.71 (± 150.95)

Tabla C.23. Resultados del algoritmo Classic McEliece (implementación en Java) — Parte 1, en Raspberry Pi 5.

Classic McEliece (Java) — Rendimiento



Classic McEliece (Java) — Tiempo Total

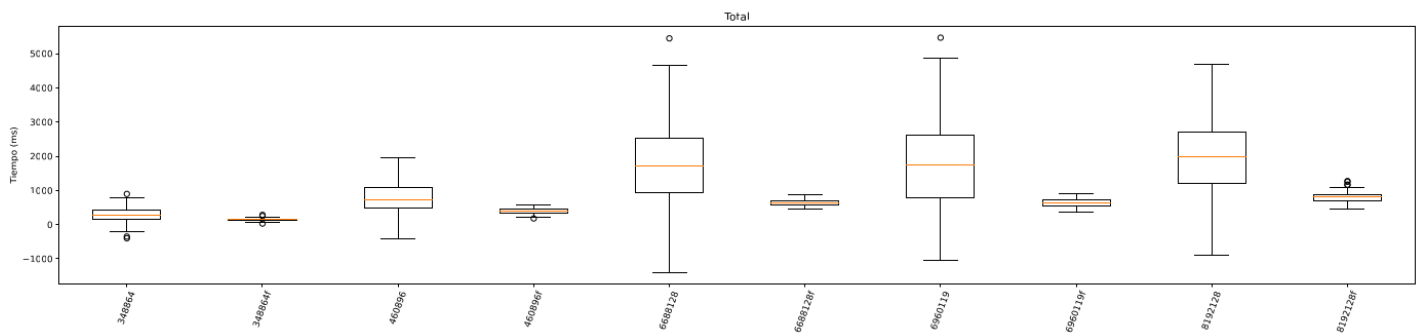


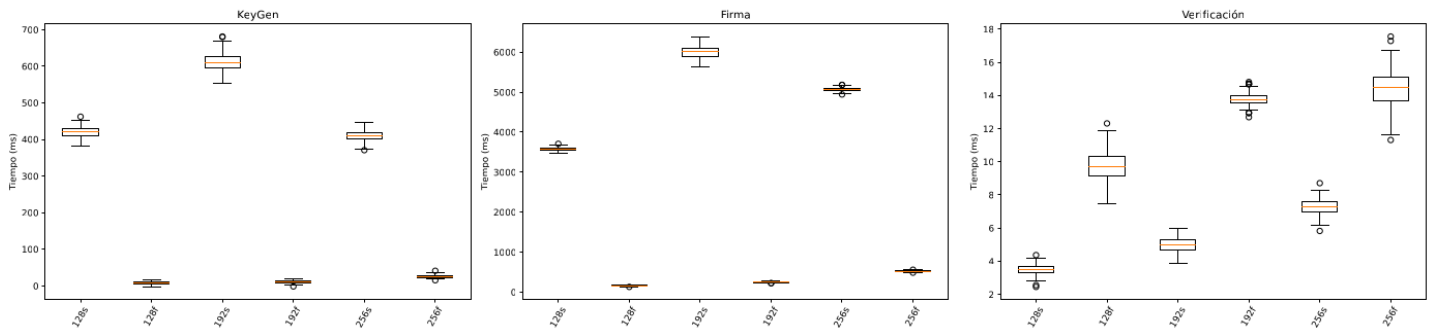
Tabla C.24. Resultados del algoritmo Classic McEliece (implementación en Java) — Parte 2, en Raspberry Pi 5.

SPHINCS+

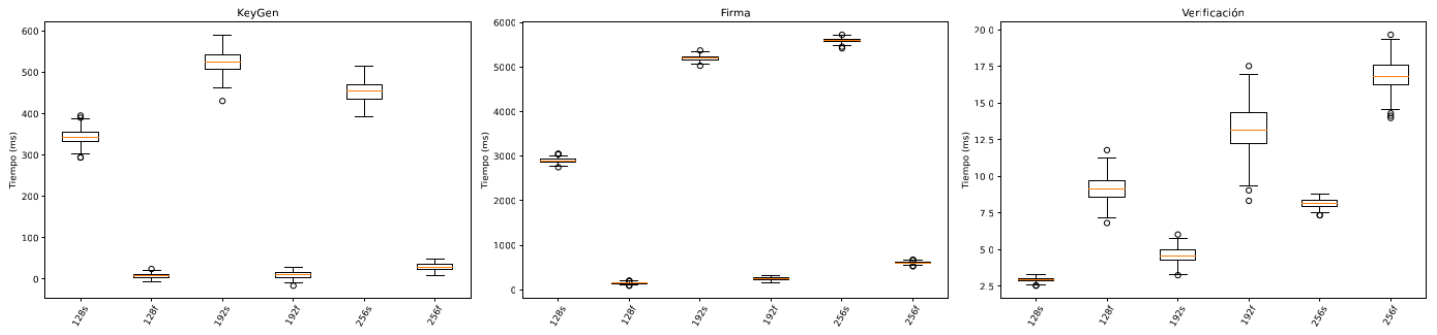
Lenguaje	Versión	T Gen. Claves (ms)	T Firma (ms)	T Verificación (ms)	T Total (ms)
Java	SPHINCS+-SHA 2-128s	343.41 ($\pm$ 18.87)	2904.76 ($\pm$ 51.12)	2.93 ($\pm$ 0.14)	3251.10 ($\pm$ 61.11)
Java	SPHINCS+-SHA 2-128f	6.85 ($\pm$ 6.17)	151.34 ($\pm$ 20.25)	9.19 ($\pm$ 0.97)	167.38 ($\pm$ 25.88)
Java	SPHINCS+-SHA 2-192s	525.94 ($\pm$ 27.06)	5196.93 ($\pm$ 61.89)	4.55 ($\pm$ 0.50)	5727.42 ($\pm$ 72.10)
Java	SPHINCS+-SHA 2-192f	9.86 ($\pm$ 7.89)	241.14 ($\pm$ 35.60)	13.21 ($\pm$ 1.63)	264.22 ($\pm$ 43.39)
Java	SPHINCS+-SHA 2-256s	456.59 ($\pm$ 26.99)	5586.03 ($\pm$ 54.02)	8.17 ($\pm$ 0.32)	6050.80 ($\pm$ 73.77)
Java	SPHINCS+-SHA 2-256f	30.30 ($\pm$ 8.25)	605.02 ($\pm$ 23.74)	16.77 ($\pm$ 1.03)	652.09 ($\pm$ 31.00)
Java	SPHINCS+-SHA KE-128s	420.59 ($\pm$ 15.56)	3573.24 ($\pm$ 48.46)	3.49 ($\pm$ 0.29)	3997.31 ($\pm$ 58.99)
Java	SPHINCS+-SHA KE-128f	7.16 ($\pm$ 4.25)	160.05 ($\pm$ 8.89)	9.79 ($\pm$ 0.88)	177.00 ($\pm$ 12.80)
Java	SPHINCS+-SHA KE-192s	612.82 ($\pm$ 23.67)	5971.50 ($\pm$ 158.28)	4.97 ($\pm$ 0.41)	6589.29 ($\pm$ 177.45)
Java	SPHINCS+-SHA KE-192f	9.84 ($\pm$ 3.47)	249.14 ($\pm$ 8.35)	13.77 ($\pm$ 0.40)	272.74 ($\pm$ 11.18)
Java	SPHINCS+-SHA KE-256s	409.35 ($\pm$ 13.58)	5061.84 ($\pm$ 41.42)	7.25 ($\pm$ 0.43)	5478.43 ($\pm$ 47.60)
Java	SPHINCS+-SHA KE-256f	25.83 ($\pm$ 4.07)	526.56 ($\pm$ 12.40)	14.43 ($\pm$ 1.10)	566.83 ($\pm$ 16.11)

Tabla C.25. Resultados del algoritmo SPHINCS+ (implementación en Java) — Parte 1, en Raspberry Pi 5.

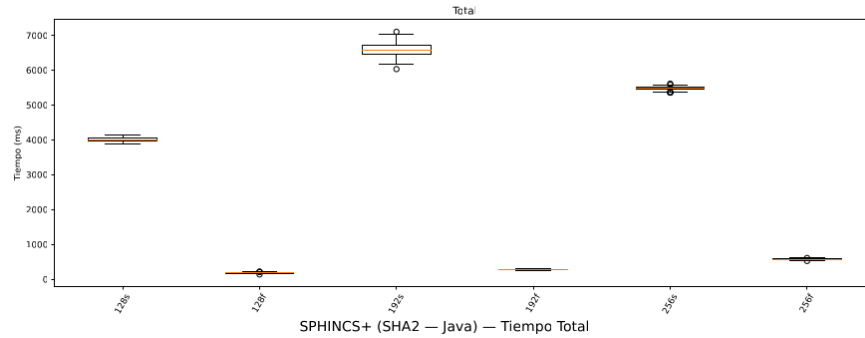
SPHINCS+ (SHAKE — Java) — Rendimiento



SPHINCS+ (SHA2 — Java) — Rendimiento



SPHINCS+ (SHAKE — Java) — Tiempo Total



SPHINCS+ (SHA2 — Java) — Tiempo Total

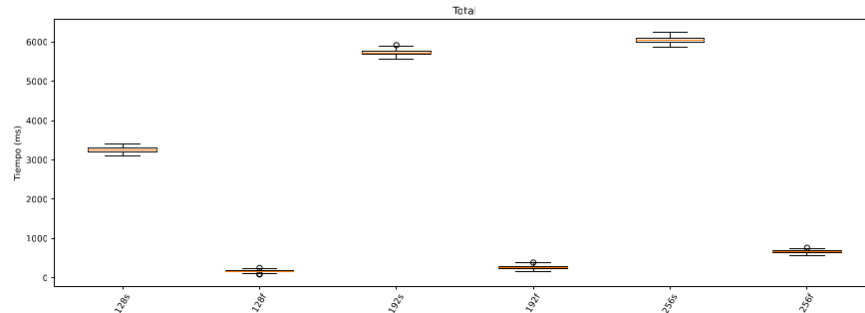


Tabla C.26. Resultados del algoritmo SPHINCS+ (implementación en Java) — Parte 2, en Raspberry Pi 5.

HQC

Lenguaje	Versión	T Gen. Claves (ms)	T Encapsulación (ms)	T Decapsulación (ms)	T Total (ms)
Java	hqc-128	4.398 ($\pm$ 2.469)	8.587 ($\pm$ 2.430)	12.944 ($\pm$ 1.325)	25.929 ($\pm$ 5.972)
Java	hqc-192	12.780 ($\pm$ 3.409)	25.078 ($\pm$ 1.670)	38.057 ($\pm$ 2.158)	75.914 ($\pm$ 6.786)
Java	hqc-256	23.084 ($\pm$ 3.958)	45.556 ($\pm$ 1.885)	68.708 ($\pm$ 2.348)	137.348 ($\pm$ 7.720)

HQC (Java) — Rendimiento

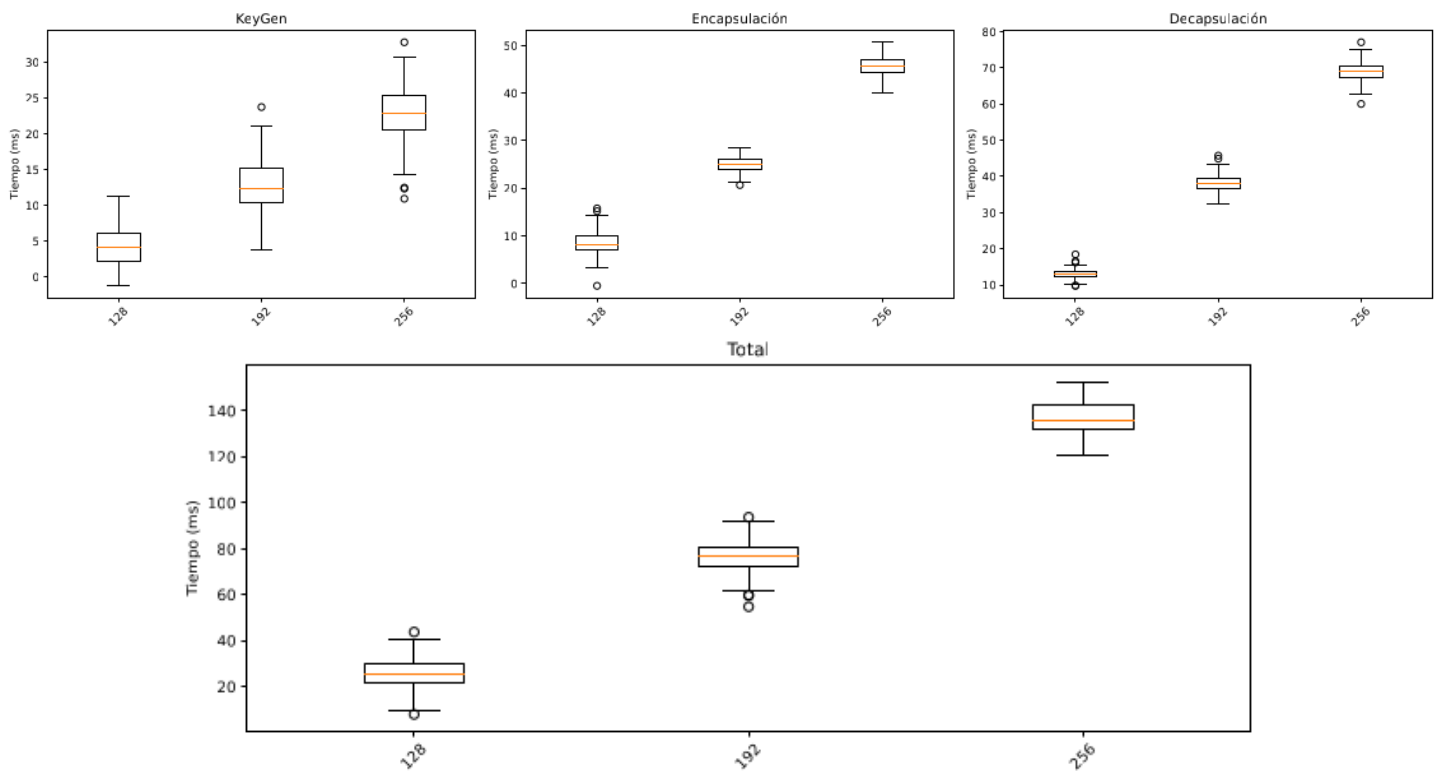


Tabla C.27. Resultados del algoritmo HQC (implementación en Java) en Raspberry Pi 5.

Lenguaje	Versión	Bike			
		T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
Python	BIKE-12 8	20.55 ms (± 0.59)	1.08 ms (± 0.03)	17.06 ms (± 0.49)	38.68 ms (± 0.77)
Python	BIKE-19 2	79.57 ms (± 11.05)	4.15 ms (± 0.78)	67.08 ms (± 10.07)	150.80 ms (± 14.97)
Python	BIKE-25 6	212.76 ms (± 18.14)	11.02 ms (± 1.95)	178.08 ms (± 16.35)	401.86 ms (± 24.49)

BIKE (Python) — Rendimiento

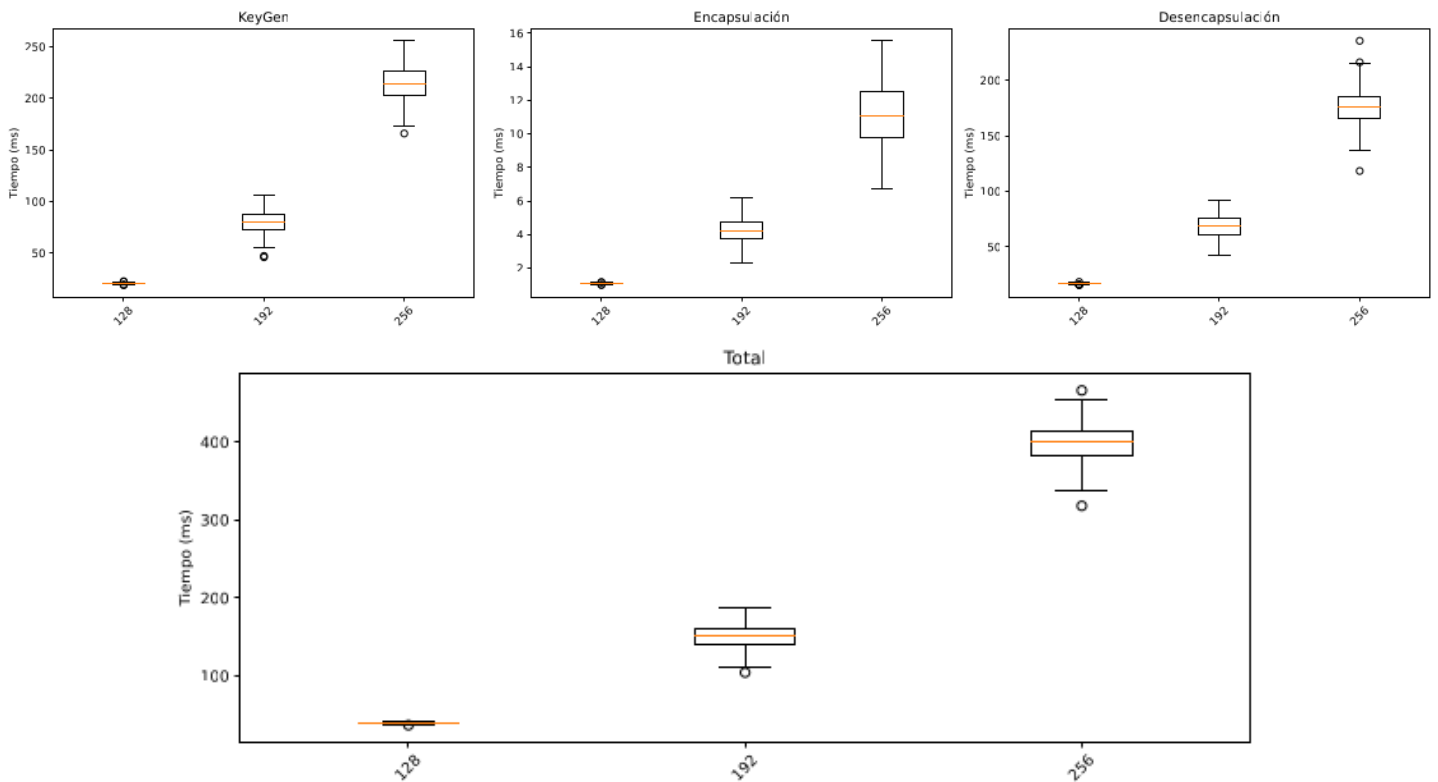


Tabla C.28. Resultados del algoritmo BIKE (implementación en Python) en Raspberry Pi 5.

Dilithium					
Lenguaje	Versión	T Gen. Claves (ms)	T Firma (ms)	T Verificación (ms)	T Total (ms)
Python	ML-DSA-2 Corto	9.21 ($\pm$ 1.32)	54.44 ($\pm$ 37.71)	11.17 ($\pm$ 1.60)	74.82
Python	ML-DSA-2 Largo	9.98 ($\pm$ 1.67)	59.59 ($\pm$ 40.20)	12.14 ($\pm$ 2.04)	81.71
Python	ML-DSA-3 Corto	17.06 ($\pm$ 2.96)	99.05 ($\pm$ 70.18)	19.52 ($\pm$ 3.35)	135.63
Python	ML-DSA-3 Largo	17.26 ($\pm$ 3.02)	99.06 ($\pm$ 71.58)	19.71 ($\pm$ 3.38)	136.03
Python	ML-DSA-5 Corto	26.96 ($\pm$ 4.58)	121.18 ($\pm$ 76.06)	31.17 ($\pm$ 5.18)	179.31
Python	ML-DSA-5 Largo	27.58 ($\pm$ 4.66)	117.30 ($\pm$ 74.49)	31.61 ($\pm$ 5.28)	176.49

Tabla C.29. Resultados del algoritmo Dilithium (implementación en Python) — Parte 1, en Raspberry Pi 5.

Dilithium / ML-DSA (Python) — Rendimiento

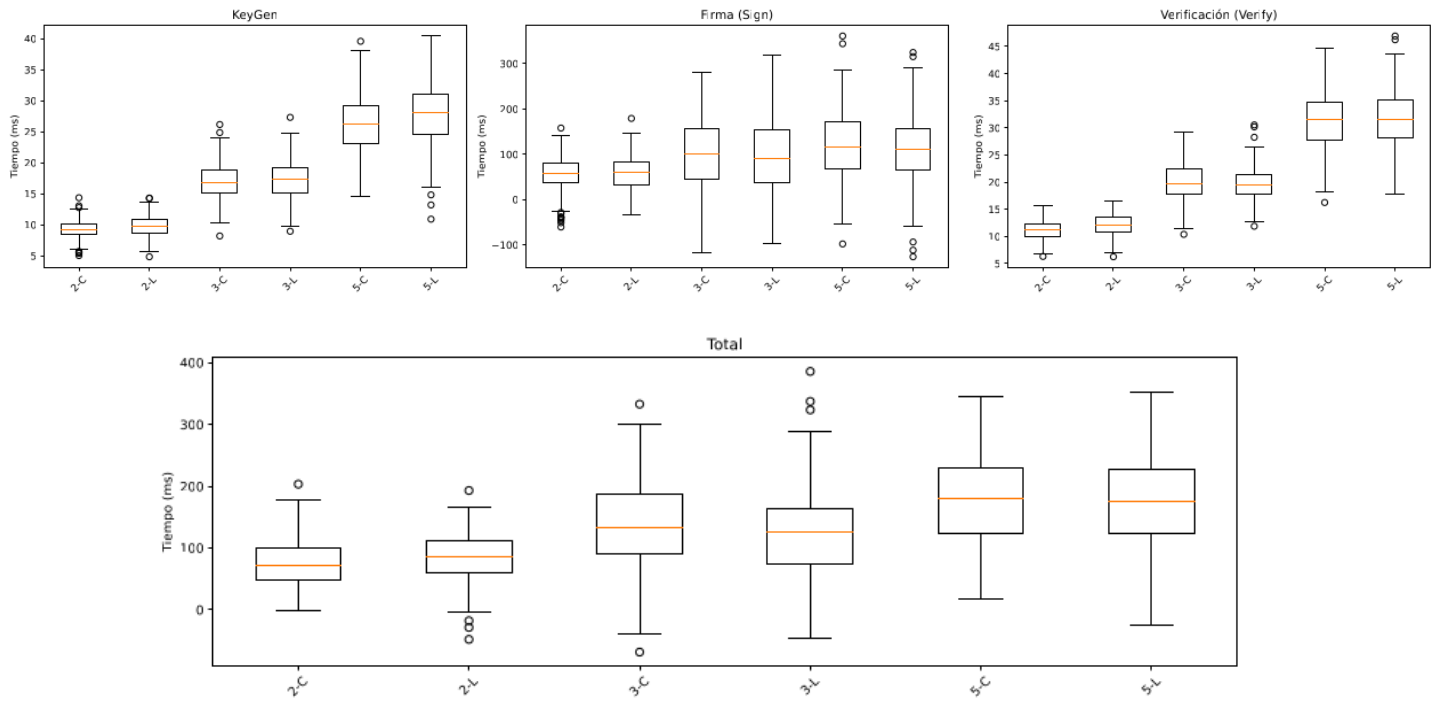


Tabla C.30. Resultados del algoritmo Dilithium (implementación en Python) — Parte 2, en Raspberry Pi 5.

Falcon

Lenguaje	Versión	T Gen. Claves (ms)	T Firma (ms)	T Verificación (ms)	T Total (ms)
Python	Falcon-512	13.4 ($\pm$ 4.5)	0.42 ($\pm$ 0.08)	0.09 ($\pm$ 0.02)	13.9 ($\pm$ 4.6)
Python	Falcon-1024 4	38.7 ($\pm$ 11.9)	0.82 ($\pm$ 0.15)	0.16 ($\pm$ 0.03)	39.7 ($\pm$ 12.0)

Falcon (Python) — Rendimiento

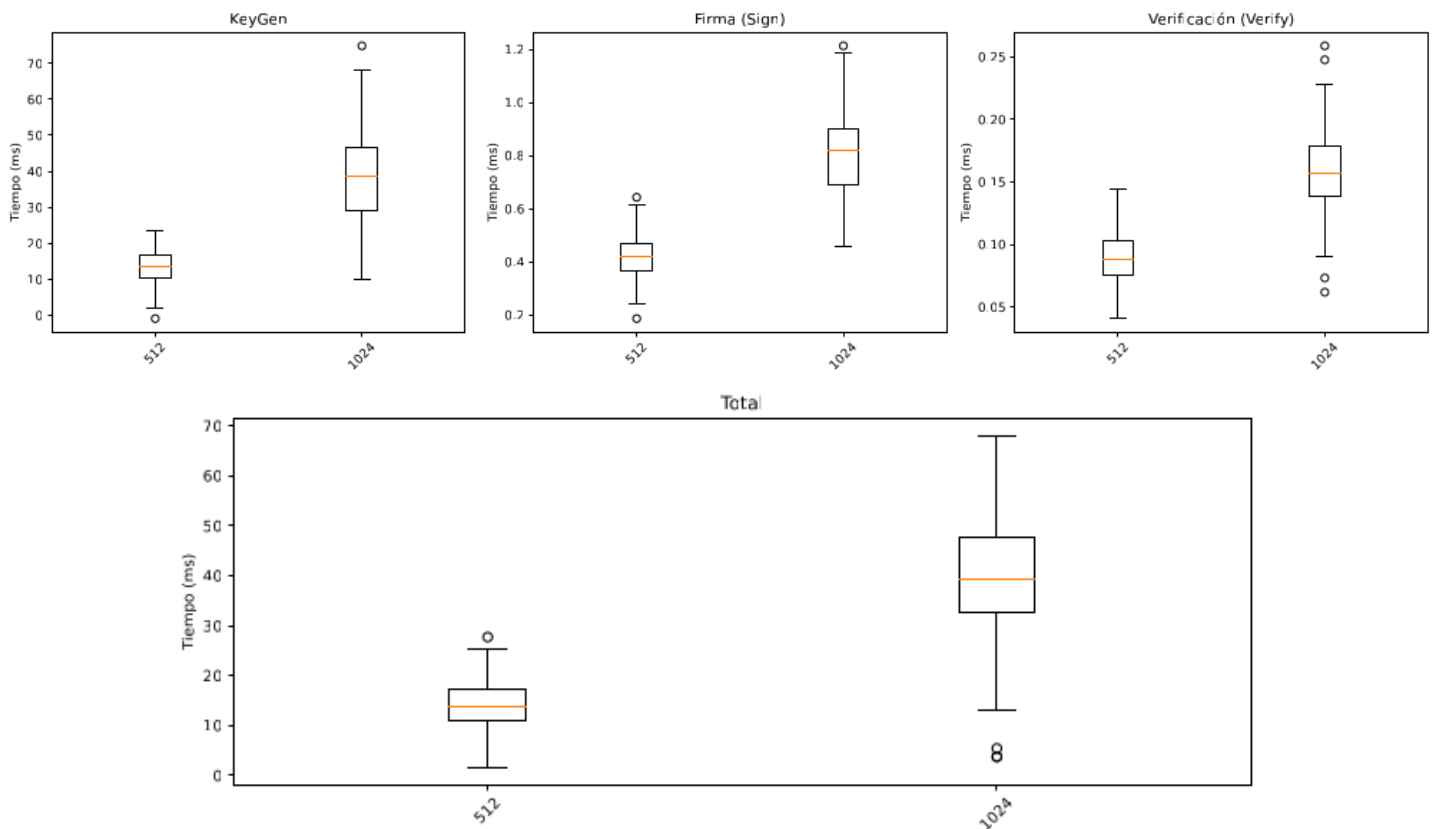


Tabla C.31. Resultados del algoritmo Falcon (implementación en Python) en Raspberry Pi 5.

HQC

Lenguaje	Versión	T Gen. Claves	T Encapsulación	T Decapsulación	T Total
Python	HQC-12 8	18.859 ms (± 1.152)	37.988 ms (± 2.255)	60.948 ms (± 3.588)	117.795 ms (± 6.323)
Python	HQC-19 2	57.094 ms (± 4.198)	114.507 ms (± 7.467)	177.092 ms (± 10.145)	348.693 ms (± 15.057)
Python	HQC-25 6	99.915 ms (± 11.084)	201.307 ms (± 16.765)	313.596 ms (± 19.356)	614.819 ms (± 25.076)

HQC (Python) — Rendimiento

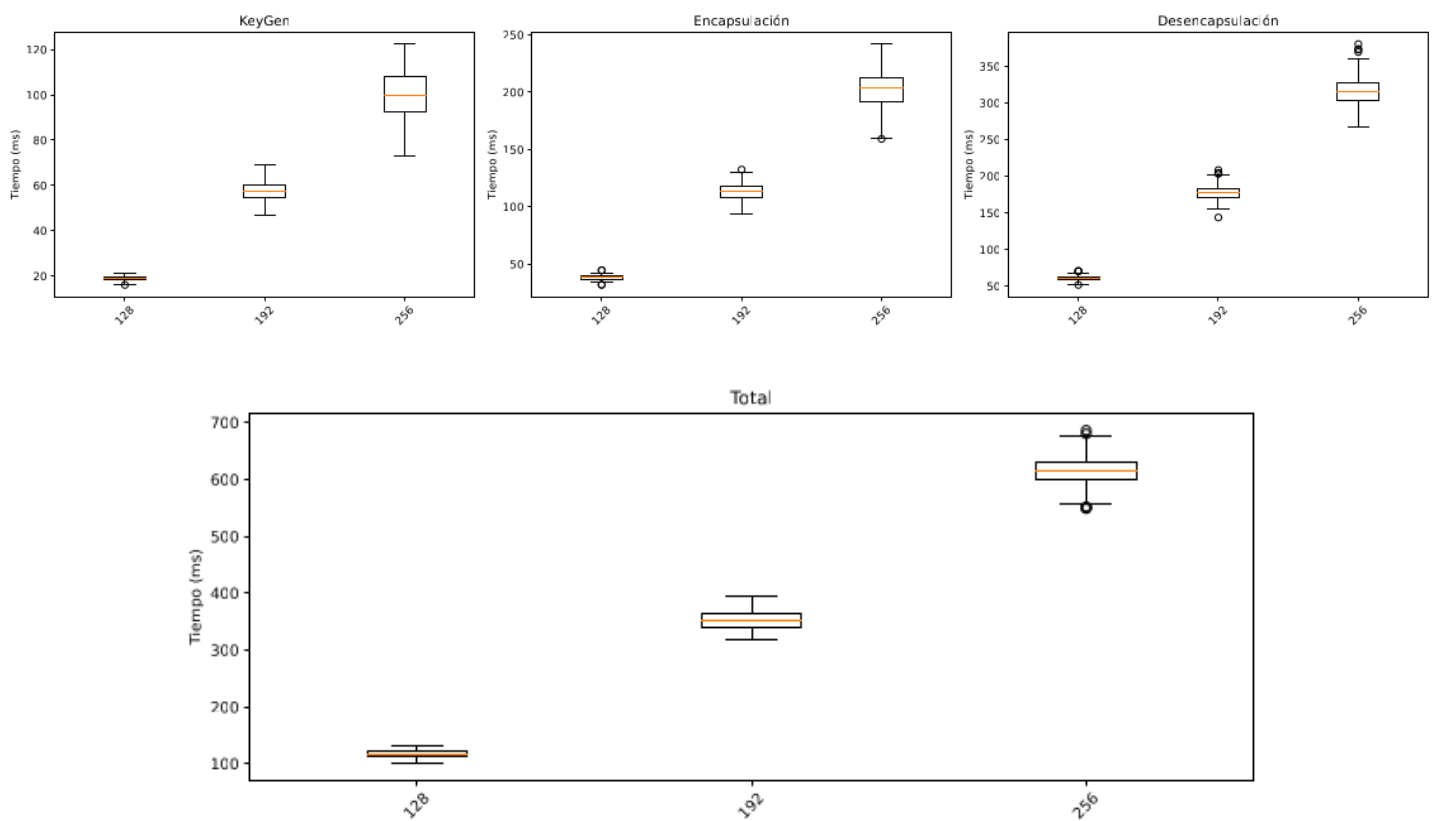


Tabla C.32. Resultados del algoritmo HQC (implementación en Python) en Raspberry Pi 5.

Kyber					
Lenguaje	Versión	T Gen. Claves (ms)	T Encapsulación (ms)	T Decapsulación (ms)	T Total (ms)
Python	Kyber-512	0.036 (± 0.092)	0.039 (± 0.009)	0.034 (± 0.007)	0.110 (± 0.099)
Python	Kyber-768	0.061 (± 0.096)	0.069 (± 0.006)	0.061 (± 0.002)	0.191 (± 0.101)
Python	Kyber-1024	0.087 (± 0.092)	0.098 (± 0.007)	0.090 (± 0.003)	0.275 (± 0.097)

Kyber (Python) — Rendimiento

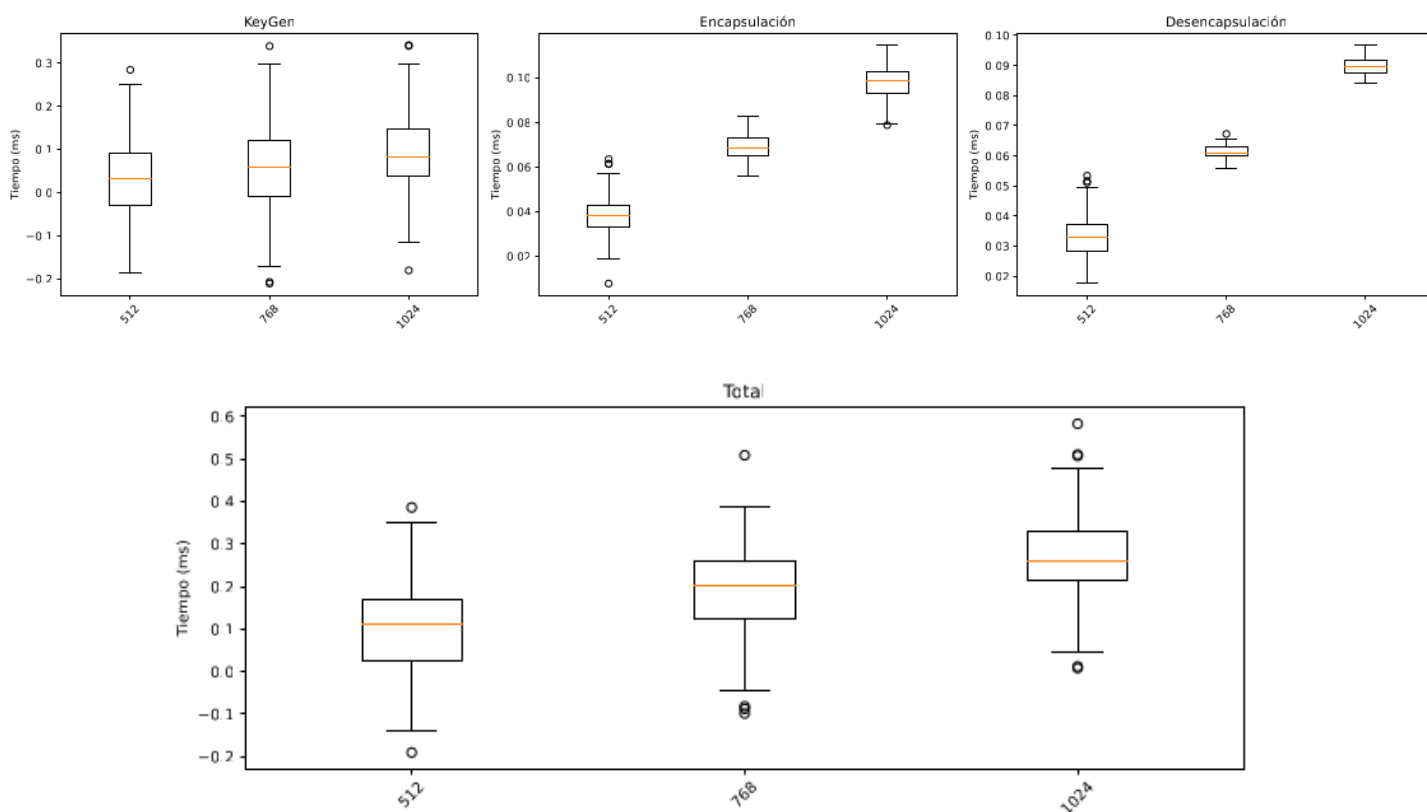


Tabla C.33. Resultados del algoritmo Kyber (implementación en Python) en Raspberry Pi 5.

McEliece					
LENGUAJE	Versión	T Gen. Claves	T Encapsulación	T Desencapsulación	T Total
Python	McEliece-348864	221.7 ms ($\pm$ 11.1)	0.2 ms ($\pm$ 0.1)	40.4 ms ($\pm$ 2.3)	262.3 ms ($\pm$ 13.5)
Python	McEliece-348864f	103.3 ms ($\pm$ 5.2)	0.1 ms ($\pm$ 0.1)	38.3 ms ($\pm$ 2.0)	141.8 ms ($\pm$ 7.1)
Python	McEliece-460896	591.7 ms ($\pm$ 27.4)	0.4 ms ($\pm$ 0.1)	80.0 ms ($\pm$ 3.9)	672.1 ms ($\pm$ 32.0)
Python	McEliece-460896f	309.6 ms ($\pm$ 15.2)	0.3 ms ($\pm$ 0.1)	75.6 ms ($\pm$ 3.6)	385.5 ms ($\pm$ 18.8)
Python	McEliece-6688128	1576.3 ms ($\pm$ 78.8)	0.8 ms ($\pm$ 0.1)	153.2 ms ($\pm$ 7.6)	1730.4 ms ($\pm$ 86.4)
Python	McEliece-6688128f	561.9 ms ($\pm$ 28.1)	0.8 ms ($\pm$ 0.1)	147.4 ms ($\pm$ 7.1)	710.0 ms ($\pm$ 35.0)
Python	McEliece-6960119	1454.1 ms ($\pm$ 72.7)	1.0 ms ($\pm$ 0.1)	146.1 ms ($\pm$ 7.3)	1601.3 ms ($\pm$ 80.1)
Python	McEliece-6960119f	591.8 ms ($\pm$ 29.6)	1.0 ms ($\pm$ 0.1)	144.8 ms ($\pm$ 7.2)	737.6 ms ($\pm$ 36.9)
Python	McEliece-8192128	1736.8 ms ($\pm$ 86.8)	0.9 ms ($\pm$ 0.1)	188.7 ms ($\pm$ 9.4)	1926.4 ms ($\pm$ 96.3)

Tabla C.34. Resultados del algoritmo Classic McEliece (implementación en Python) — Parte 1, en Raspberry Pi 5.

Python **McEliece-8192128f** 629.4 ms ($\pm$ 31.5) 0.9 ms ($\pm$ 0.1) 189.0 ms ($\pm$ 9.4) 819.2 ms ($\pm$ 41.0)

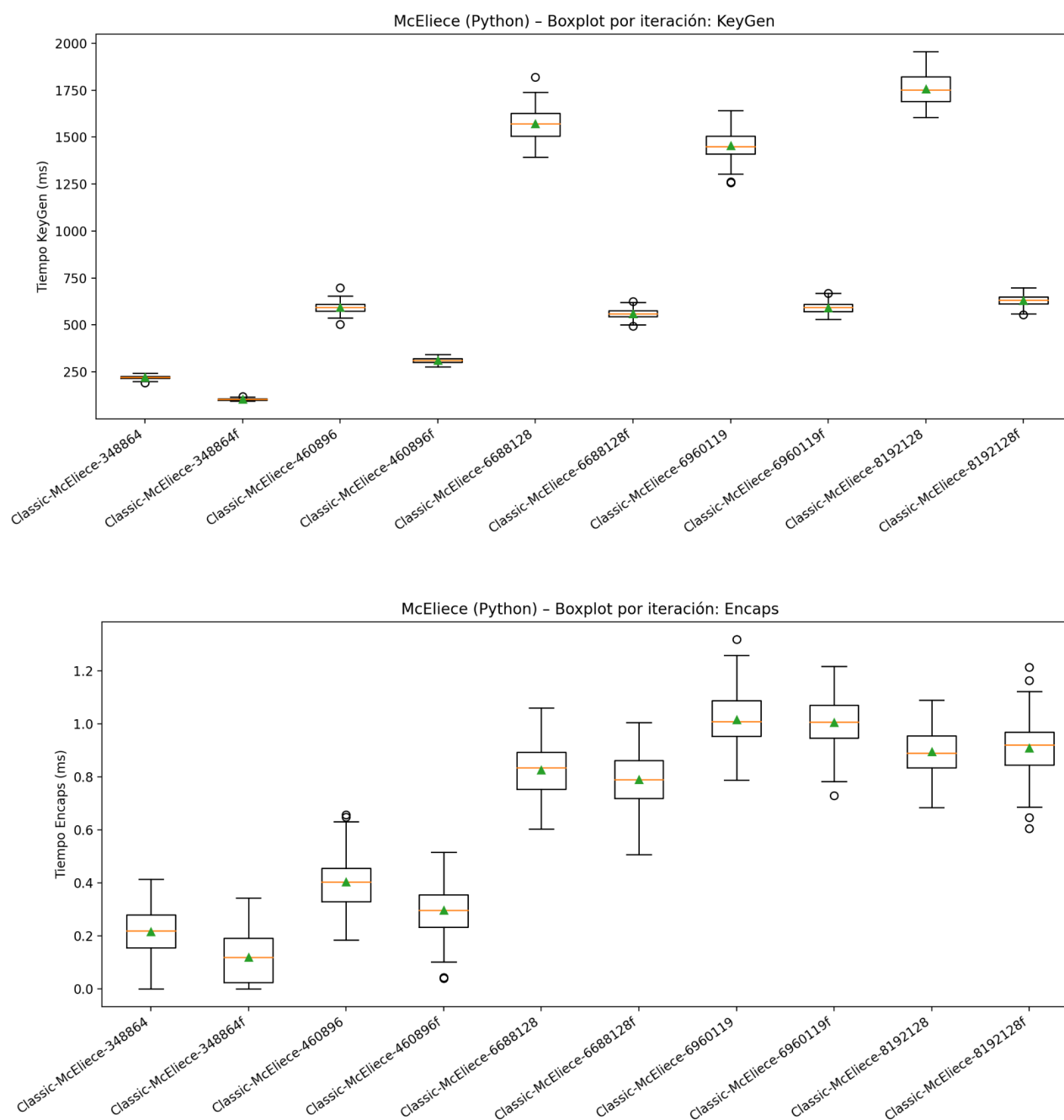


Tabla C.35. Resultados del algoritmo Classic McEliece (implementación en Python) — Parte 2, en Raspberry Pi 5.

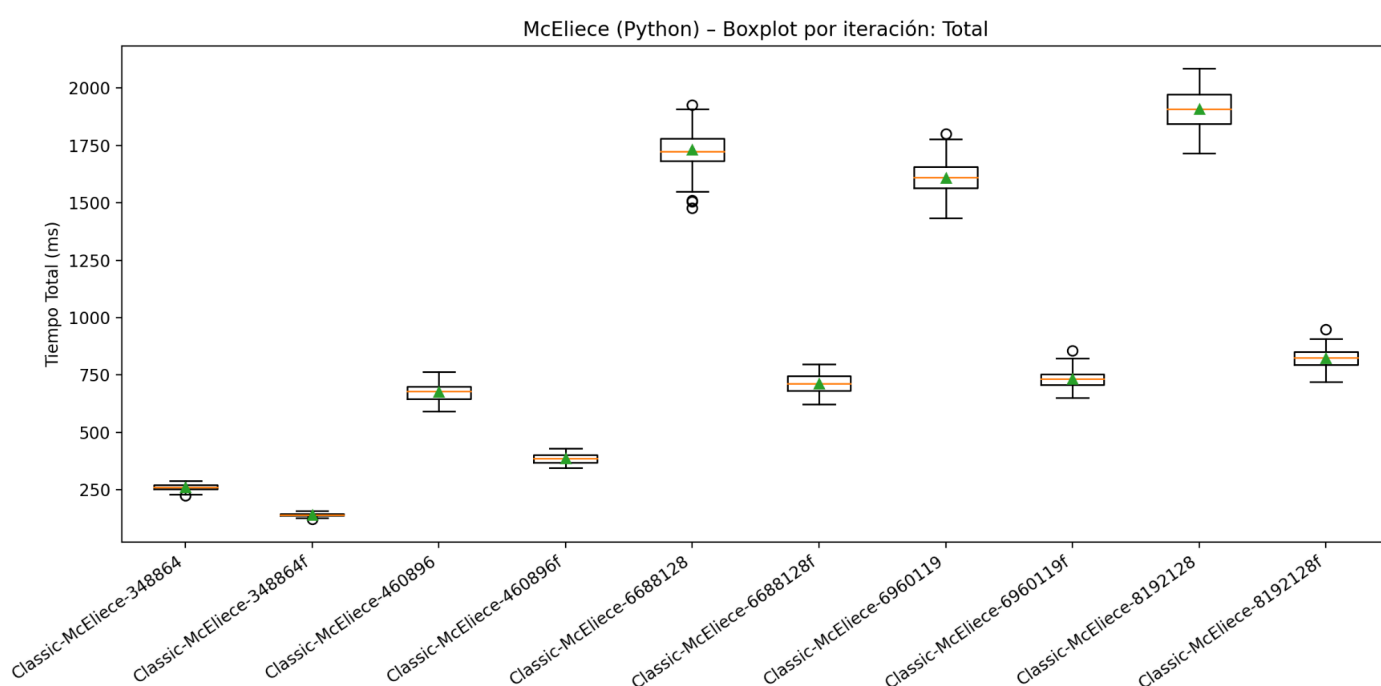
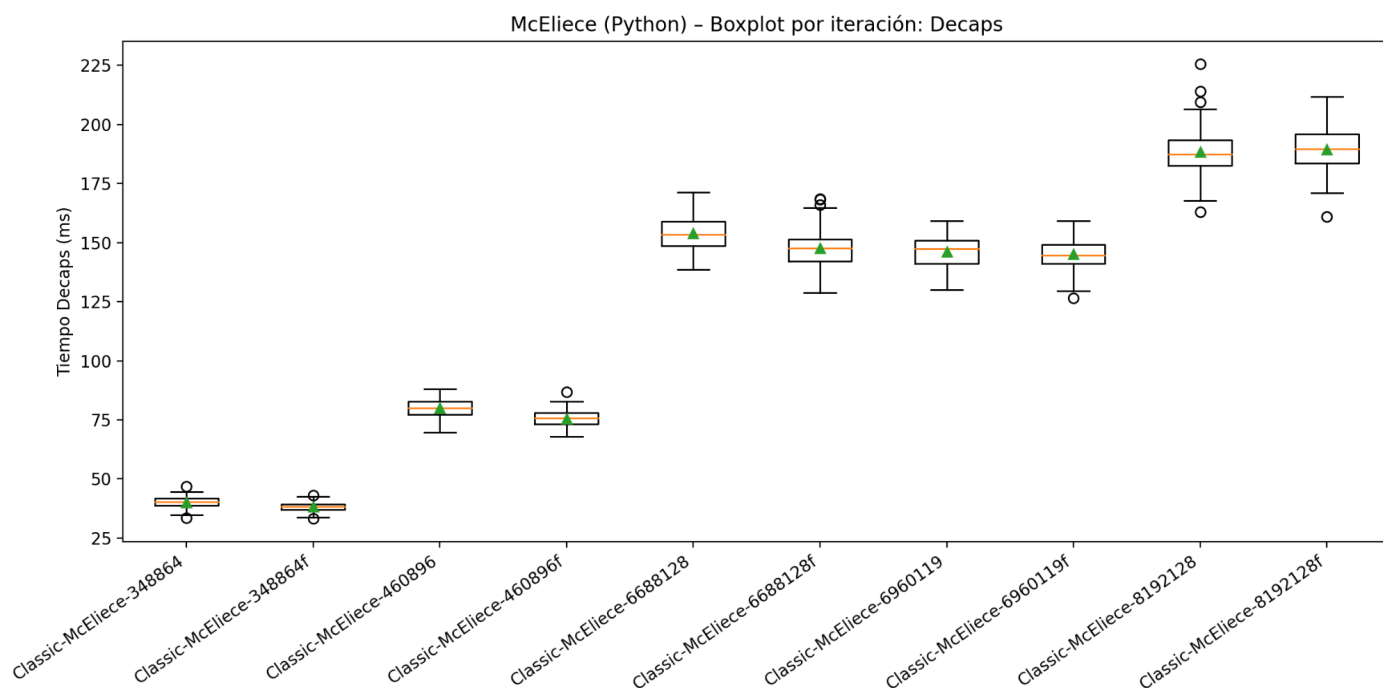


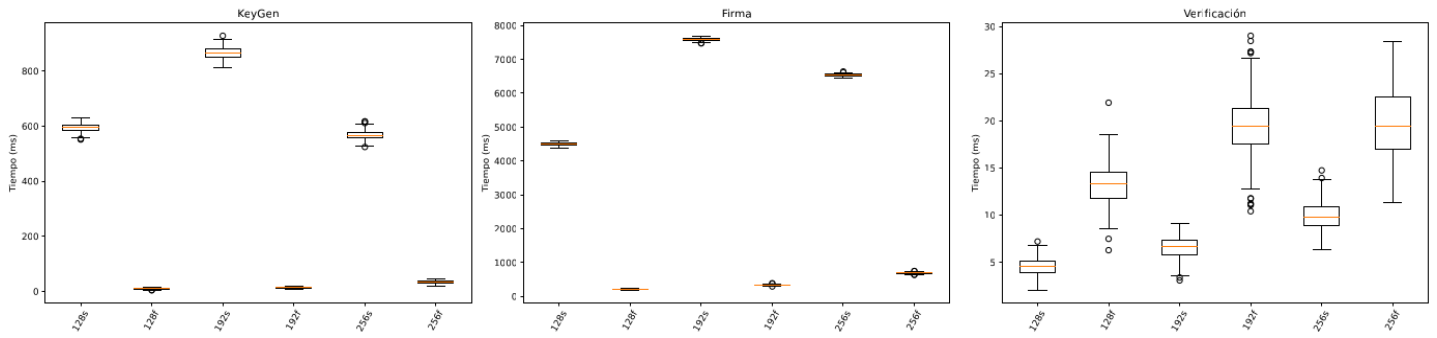
Tabla C.36. Resultados del algoritmo Classic McEliece (implementación en Python) — Parte 3, en Raspberry Pi 5.

SPHINCS+

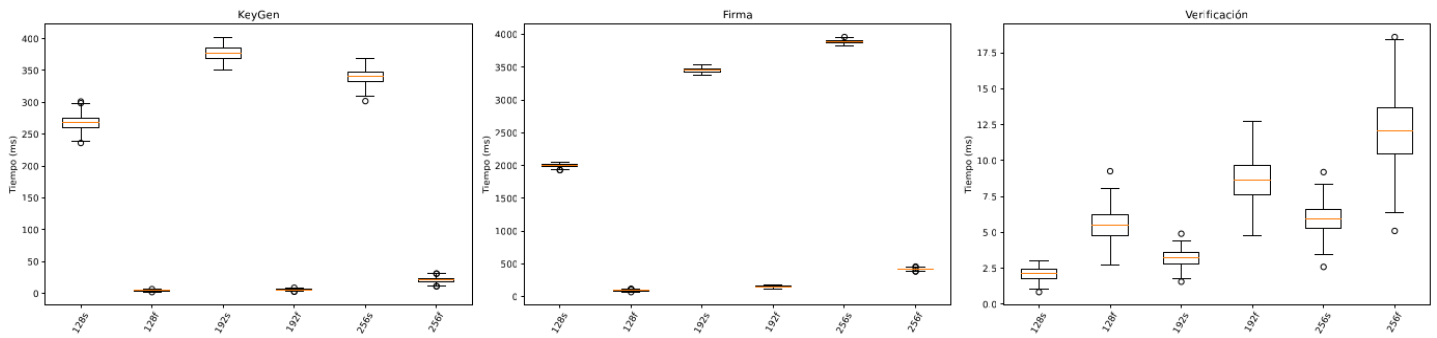
LENGUAJE	Versión	T Gen. Claves (ms)	T Firma (ms)	T Verificación (ms)	T Total (ms)
Python	shake_128s	593.8 (± 16.9)	4500.0 (± 40.7)	4.7 (± 0.9)	5098.5 (± 40.3)
Python	shake_128f	9.2 (± 1.7)	209.9 (± 10.2)	13.4 (± 2.4)	232.6 (± 10.5)
Python	shake_192s	864.8 (± 20.6)	7589.9 (± 45.2)	6.5 (± 1.2)	8461.2 (± 45.9)
Python	shake_192f	13.9 (± 2.3)	335.8 (± 14.3)	19.2 (± 3.5)	368.9 (± 14.1)
Python	shake_256s	566.0 (± 16.6)	6538.7 (± 36.2)	9.9 (± 1.7)	7114.6 (± 38.8)
Python	shake_256f	34.5 (± 5.7)	696.9 (± 18.2)	19.9 (± 3.5)	751.3 (± 18.6)
Python	sha2_128s	266.2 (± 12.1)	2001.2 (± 23.4)	2.1 (± 0.4)	2269.4 (± 23.5)
Python	sha2_128f	4.5 (± 0.8)	89.5 (± 8.3)	5.5 (± 1.0)	99.5 (± 8.7)
Python	sha2_192s	379.2 (± 11.3)	3452.4 (± 28.7)	3.2 (± 0.6)	3834.8 (± 30.6)
Python	sha2_192f	6.0 (± 1.1)	148.4 (± 10.7)	8.7 (± 1.6)	163.2 (± 10.1)
Python	sha2_256s	339.8 (± 12.2)	3888.1 (± 26.9)	5.9 (± 1.1)	4233.8 (± 26.6)
Python	sha2_256f	21.5 (± 3.9)	416.6 (± 11.7)	12.2 (± 2.3)	450.3 (± 12.8)

Tabla C.37. Resultados del algoritmo SPHINCS+ (implementación en Python) — Parte 1, en Raspberry Pi 5.

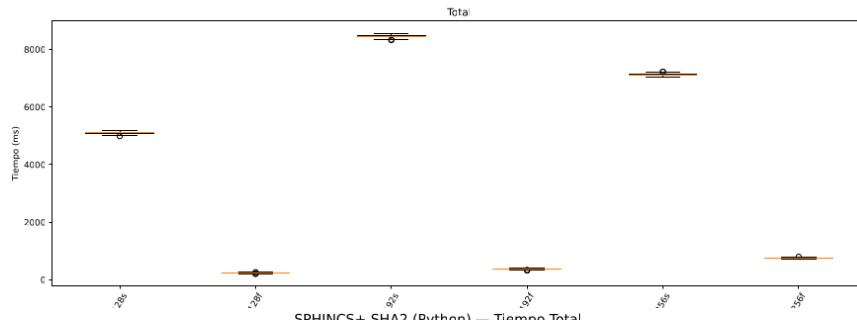
SPHINCS+ SHAKE (Python) — Rendimiento



SPHINCS+ SHA2 (Python) — Rendimiento



SPHINCS+ SHAKE (Python) — Tiempo Total



SPHINCS+ SHA2 (Python) — Tiempo Total

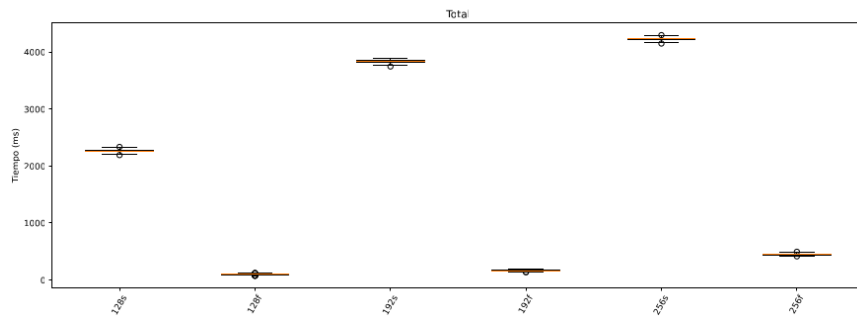


Tabla C.38. Resultados del algoritmo SPHINCS+ (implementación en Python) — Parte 2, en Raspberry Pi 5.

Consumo de Recursos

Versión	Tiempo total (s)	CPU (%)	Memoria máx. (kB)
Dilithium 2 Reference (C)	0.97	99	1 104
Dilithium 3 Reference (C)	1.52	99	1 088
Dilithium 5 Reference (C)	2.03	99	1 088
Dilithium 2 NEON (C)	0.92	100	1 088
Dilithium 3 NEON (C)	1.52	99	1 088
Dilithium 5 NEON (C)	2.06	100	1 088
Falcon-512 Reference (C)	68.65	99	1 072
Falcon-1024 Reference (C)	120.69	99	1 088
Falcon-512 NEON (C)	68.29	99	1 120
Falcon-1024 NEON (C)	119.82	99	1 072
BIKE-128 NEON (C)	37.78	99	9472
BIKE-192 NEON (C)	67.44	99	9408
BIKE-256 NEON (C)	147.00	99	9408
BIKE-128 Reference (C)	37.85	99	9408
BIKE-192 Reference (C)	67.61	99	9408
BIKE-256 Reference (C)	147.24	99	9904
HQC-128 Reference (C)	14.18	99	1120
HQC-192 Reference(C)	41.89	99	1072
HQC-256 Reference (C)	77.42	99	1072
mceliece348864 ARM (C)	0.100	99.68	6 285
mceliece348864f ARM (C)	0.079	99.32	6 291

Tabla C.39. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 — Parte 1.

mceliece460896 ARM (C)	0.294	99.62	7 320
mceliece460896f ARM (C)	0.249	99.68	7 167
mceliece6688128 ARM (C)	1.374	99.26	8 406
mceliece6688128f ARM (C)	0.546	99.48	8 344
mceliece6960119 ARM (C)	1.128	99.44	8 344
mceliece6960119f ARM (C)	0.503	99.54	8 344
mceliece8192128 ARM (C)	1.092	99.34	9 358
mceliece8192128f ARM (C)	0.645	99.38	8 824
sphincs-shake-128f (C)	0.0076	100	1648
sphincs-shake-192f (C)	0.0131	100	1616
sphincs-shake-256f (C)	0.0265	99	1616
sphincs-shake-128s (C)	0.0619	99	1616
sphincs-shake-192s (C)	0.0092	99	1616
sphincs-shake-256s (C)	0.0848	99	1616
sphincs-sha2-128f (C)	0.0041	100	1648
sphincs-sha2-192f (C)	0.0073	100	1616
sphincs-sha2-256f (C)	0.0148	99	1616
sphincs-sha2-128s (C)	0.0877	99	1616
sphincs-sha2-192s (C)	0.0740	99	1616
sphincs-sha2-256s (C)	0.0645	99	1648
BIKE-128 (Java)	37.89	105	149 920
BIKE-192 (Java)	121.60	101	149 840
BIKE-256 (Java)	291.23	100	149 920
Dilithium 2 (Java)	3.17	175	182 960

Tabla C.40. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 — Parte 2.

Dilithium 3 (Java)	4.40	155	184 384
Dilithium 5 (Java)	5.42	148	188 848
Falcon-512 (Java)	40.51	132	333 776
Falcon-1024 (Java)	109.87	111	284 832
Kyber-512 (Java)	2.33	286	210 256
Kyber-768 (Java)	2.38	277	218 288
Kyber-1024 (Java)	2.34	283	216 480
McEliece-348864 (Java)	8.70	138	126 576
McEliece-348864f (Java)	4.78	169	115 296
McEliece-460896 (Java)	23.93	116	161 200
McEliece-460896f (Java)	12.13	132	138 320
McEliece-6688128 (Java)	51.08	107	196 944
McEliece-6688128f (Java)	19.77	122	155 520
McEliece-6960119 (Java)	54.63	107	201 232
McEliece-6960119f (Java)	19.86	124	145 104
McEliece-8192128 (Java)	60.69	106	177 888
McEliece-8192128f (Java)	24.70	120	137 504
SPHINCS+-SHA2-128s (Java)	325.54	101	207 472
SPHINCS+-SHA2-128f (Java)	17.23	115	182 848
SPHINCS+-SHA2-192s (Java)	573.20	100	188 224
SPHINCS+-SHA2-192f (Java)	26.93	116	194 160
SPHINCS+-SHA2-256s (Java)	605.58	100	190 432

Tabla C.41. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 — Parte 3.

SPHINCS+-SHA2-256f (Java)	65.71	104	157 472
SPHINCS+-SHAKE-128s (Java)	400.26	100	149 712
SPHINCS+-SHAKE-128f (Java)	18.15	109	146 320
SPHINCS+-SHAKE-192s (Java)	659.46	100	146 816
SPHINCS+-SHAKE-192f (Java)	27.75	106	147 232
SPHINCS+-SHAKE-256s (Java)	548.31	100	148 496
SPHINCS+-SHAKE-256f (Java)	57.17	103	147 248
HQC-128 (Java)	3.00	145	148 784
HQC-192 (Java)	8.00	120	181 040
HQC-256 (Java)	14.14	111	148 112
SPHINCS+-SHAKE-192s (Python)	8.46	25	66 097
SPHINCS+-SHAKE-192f (Python)	0.37	25	66 208
SPHINCS+-SHAKE-256s (Python)	7.11	25	66 208
SPHINCS+-SHAKE-256f (Python)	0.75	25	66 272
SPHINCS+-SHA2-128s (Python)	2.27	25	66 288
SPHINCS+-SHA2-128f (Python)	0.10	25	66 289

Tabla C.42. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 — Parte 4.

SPHINCS+-SHA2-192s (Python)	3.83	25	66 304
SPHINCS+-SHA2-192f (Python)	0.16	25	66 304
SPHINCS+-SHA2-256s (Python)	4.23	25	66 304
SPHINCS+-SHA2-256f (Python)	0.45	25	66 304
BIKE-128 (C#)	0.27	91	44 240 kB
BIKE-192 (C#)	0.44	96	44 240 kB
BIKE-256 (C#)	0.63	97	44 800 kB
Dilithium 2 corto (C#)	0.17	87	41 870
Dilithium 2 largo (C#)	0.17	86	41 920
Dilithium 3 corto (C#)	0.18	86	42 000
Dilithium 3 largo (C#)	0.18	86	42 200
Dilithium 5 corto (C#)	0.18	87	42 400
Dilithium 5 largo (C#)	0.18	87	42 500
Falcon-512 pequeño (C#)	0.42	94	61 680
Falcon-512 grande (C#)	0.44	94	62 080
Falcon-1024 pequeño (C#)	0.71	99	62 000

Tabla C.43. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 — Parte 5.

Falcon-1024 grande (C#)	0.71	99	62 600
HQC-128 (C#)	473.6	94	48 080
HQC-192 (C#)	680.4	97	49 160
HQC-256 (C#)	636.8	97	50 280
Kyber-512 (C#)	328.0	93	47 800
Kyber-768 (C#)	398.4	94	48 200
Kyber-1024 (C#)	495.6	96	48 400
sha2-128s (C#)	7.31	99.5	50422
sha2-128f (C#)	1.01	97.0	48718
sha2-192s (C#)	11.92	99.3	50427
sha2-192f (C#)	1.37	99.0	49506
sha2-256s (C#)	12.91	94.0	50855
sha2-256f (C#)	1.99	100.0	50709
shake-128s (C#)	6.51	99.0	50433
shake-128f (C#)	1.07	98.0	46650
shake-192s (C#)	10.40	99.0	50252

Tabla C.44. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 — Parte 6.

shake-192f (C#)	1.49	99.0	49820
shake-256s (C#)	8.76	99.5	50277
shake-256f (C#)	1.70	99.0	50224
mceliece348864 (C#)	1.2094	98.92	50308.80
mceliece348864f (C#)	0.8698	97.52	49744.32
mceliece460896 (C#)	3.4470	99.72	52534.72
mceliece460896f (C#)	1.8740	98.98	52784.64
mceliece6688128 (C#)	8.5738	99.48	54196.48
mceliece6688128f (C#)	2.9144	99.24	53486.72
mceliece6960119 (C#)	6.7864	99.24	53786.24
mceliece6960119f (C#)	3.1452	95.14	53887.36
mceliece8192128 (C#)	9.6154	99.28	53948.80
mceliece8192128f (C#)	3.2290	99.42	53850.88

Tabla C.45. Consumo de recursos (memoria y CPU) en Raspberry Pi 5 — Parte 7.



ESTUDIO: RENDIMIENTO DE ALGORITMOS POST-CUÁNTICOS EN DISPOSITIVO ANDROID

A continuación se presentan los resultados completos obtenidos en las pruebas de rendimiento de los algoritmos post-cuánticos BIKE, HQC, Classic McEliece, Kyber, Dilithium, Falcon y SPHINCS+, evaluados en las implementaciones desarrolladas en lenguaje **Kotlin** para el dispositivo Android.

Las tablas incluidas en este anexo recogen los tiempos medios y sus desviaciones estándar, permitiendo comparar el rendimiento entre algoritmos y operaciones fundamentales —generación de claves, encapsulación, desencapsulación, firma y verificación— en un entorno móvil moderno.

BIKE

Lenguaje	Versión	T Gen. Claves	T Encap	T Decap	T Total
Kotlin	BIKE-128	18.02 ms ($\pm$ 6.84)	1.45 ms ($\pm$ 0.89)	32.51 ms ($\pm$ 10.09)	51.98 ms ($\pm$ 16.68)
Kotlin	BIKE-192	53.44 ms ($\pm$ 8.15)	3.34 ms ($\pm$ 1.30)	89.27 ms ($\pm$ 12.11)	146.06 ms ($\pm$ 20.49)
Kotlin	BIKE-256	143.13 ms ($\pm$ 8.26)	8.00 ms ($\pm$ 1.78)	192.81 ms ($\pm$ 13.74)	343.94 ms ($\pm$ 22.36)

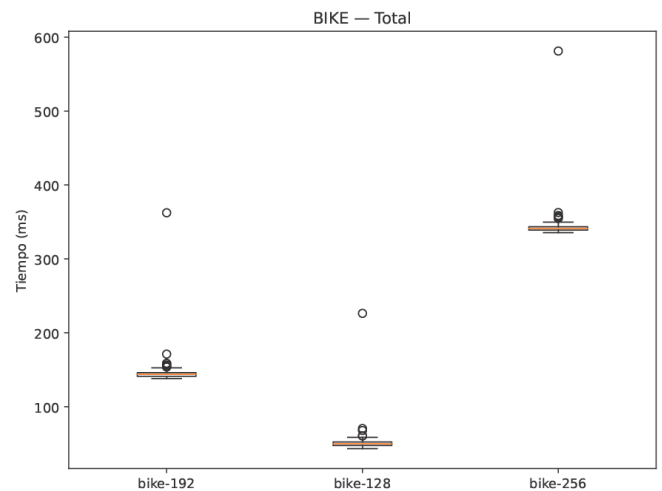
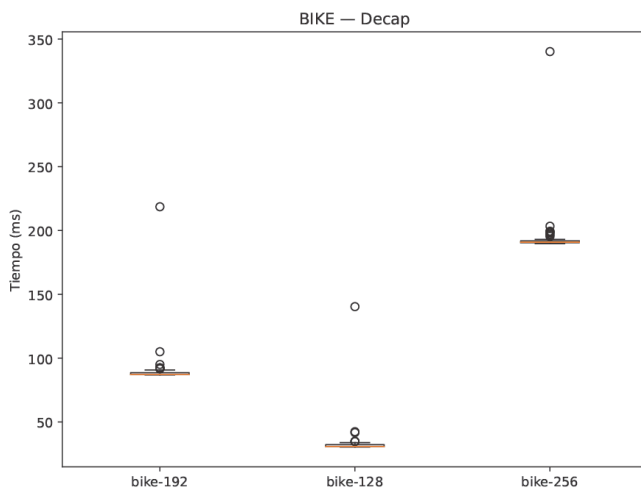
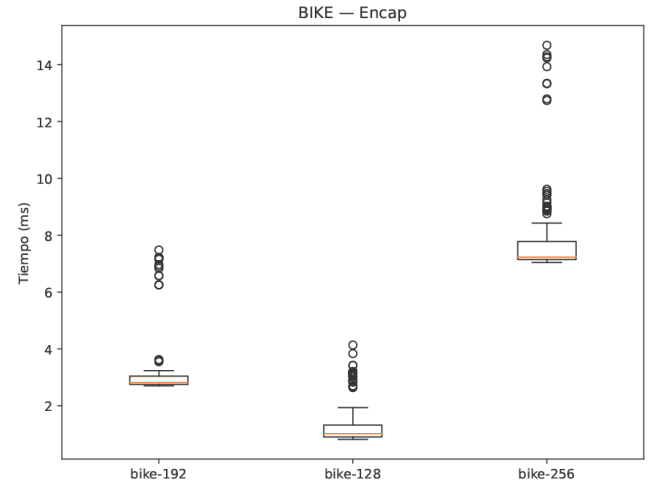
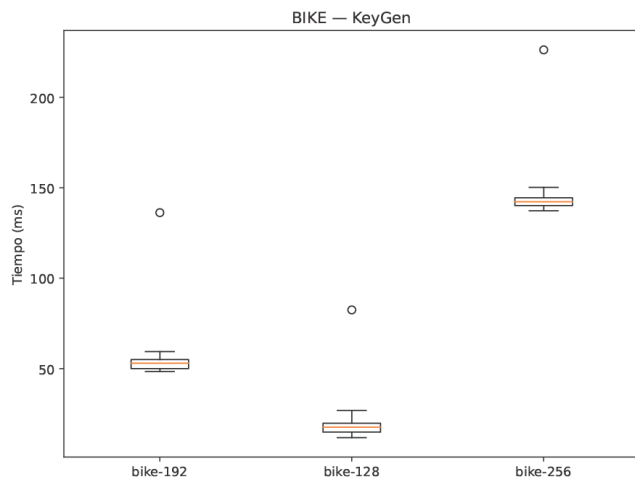


Tabla D.1. Resultados del algoritmo **BIKE** (implementación en **Kotlin**) en dispositivo Android.

Dilithium

Lenguaje	Versión	T Gen. Claves	T Firma	T Verificación	T Total
Kotlin	Dilithium2	5.01 ms ($\pm$ 4.49)	8.89 ms ($\pm$ 7.44)	2.18 ms ($\pm$ 1.82)	16.08 ms ($\pm$ 12.08)
Kotlin	Dilithium3	5.21 ms ($\pm$ 5.28)	10.64 ms ($\pm$ 11.11)	2.29 ms ($\pm$ 1.80)	18.14 ms ($\pm$ 17.05)
Kotlin	Dilithium5	6.23 ms ($\pm$ 6.31)	10.89 ms ($\pm$ 9.03)	2.97 ms ($\pm$ 2.72)	20.09 ms ($\pm$ 16.29)

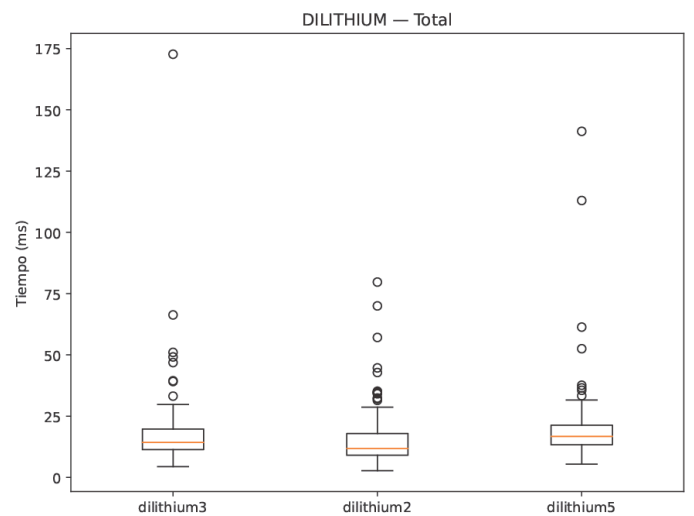
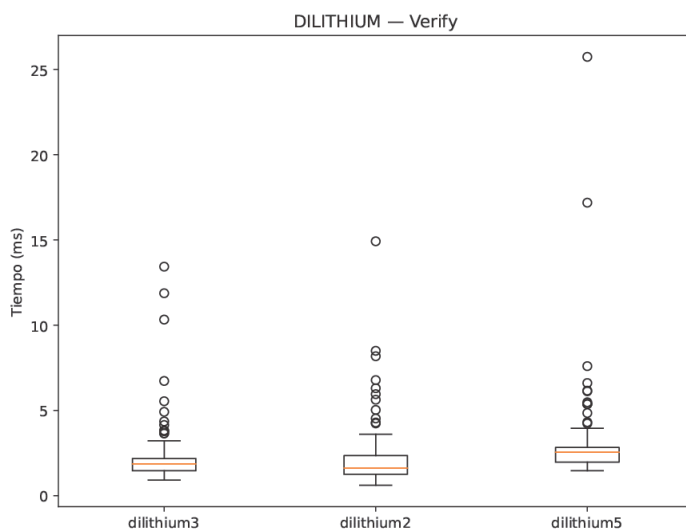
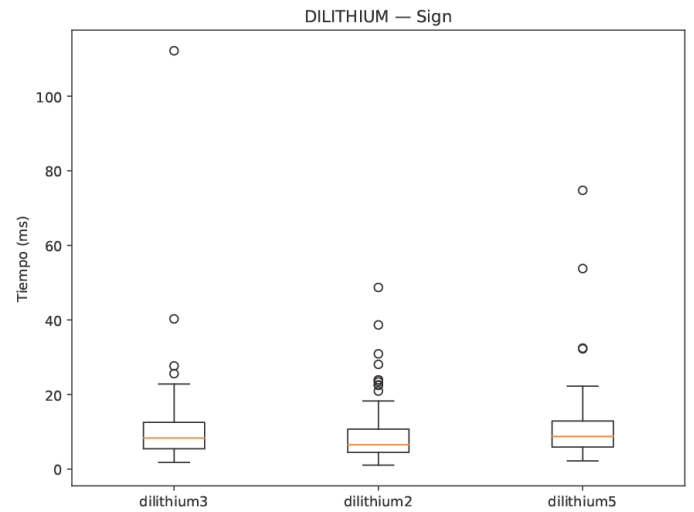
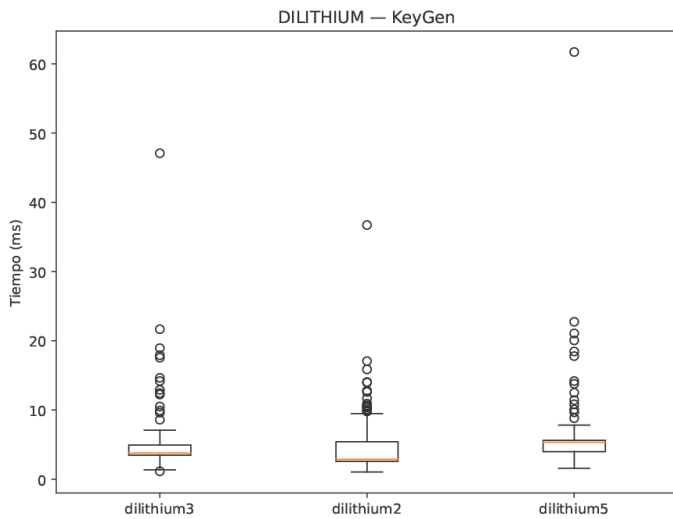


Tabla D.2. Resultados del algoritmo **Dilithium** (implementación en **Kotlin**) en dispositivo Android.

Falcon

Lenguaje	Versión	T Gen. Claves	T Firma	T Verificación	T Total
Kotlin	Falcon-512	63.93 ms ($\pm$ 39.98)	13.90 ms ($\pm$ 6.61)	0.93 ms ($\pm$ 0.73)	78.76 ms ($\pm$ 46.13)
Kotlin	Falcon-1024	174.27 ms ($\pm$ 69.64)	27.73 ms ($\pm$ 8.95)	1.14 ms ($\pm$ 1.14)	203.14 ms ($\pm$ 75.20)

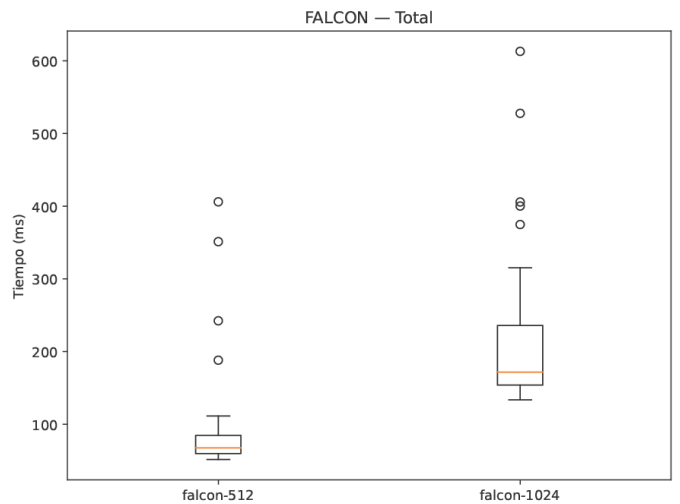
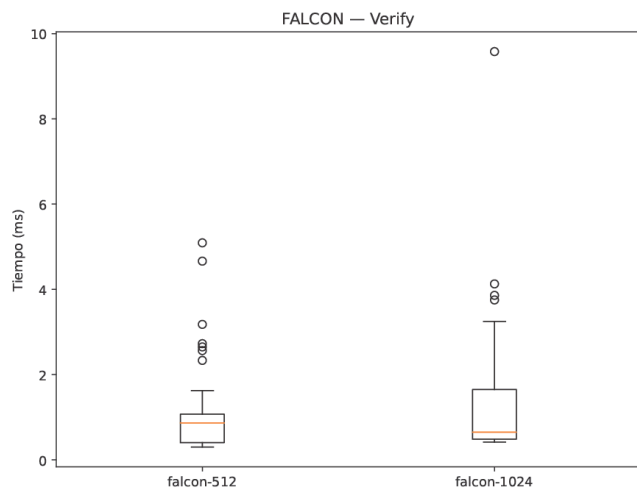
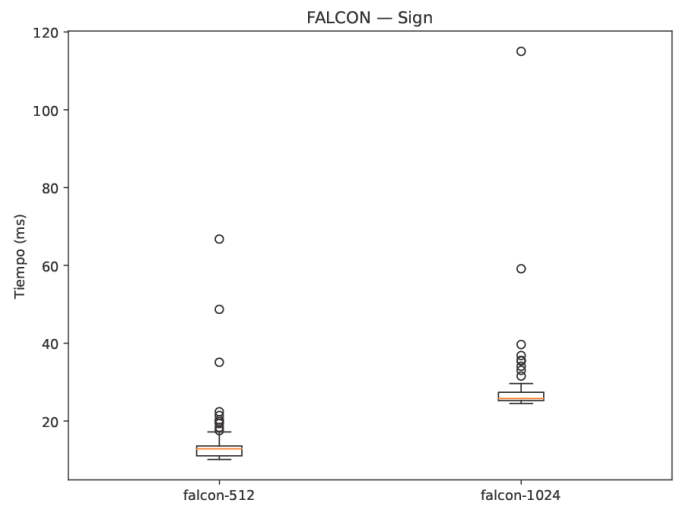
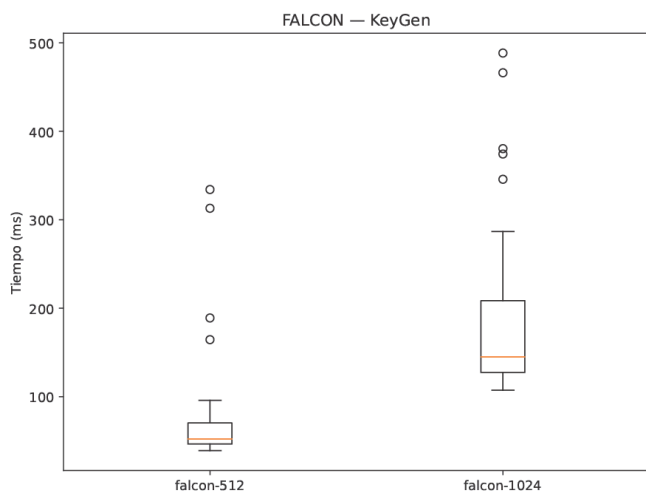


Tabla D.3. Resultados del algoritmo **Falcon** (implementación en **Kotlin**) en dispositivo Android.

HQC

Lenguaje	Versión	T Gen. Claves	T Encap	T Decap	T Total
Kotlin	HQC-128	7.50 ms ($\pm$ 4.78)	12.30 ms ($\pm$ 2.36)	17.21 ms ($\pm$ 5.64)	37.01 ms ($\pm$ 10.26)
Kotlin	HQC-192	19.21 ms ($\pm$ 5.67)	31.18 ms ($\pm$ 5.77)	47.15 ms ($\pm$ 3.15)	97.54 ms ($\pm$ 10.82)
Kotlin	HQC-256	30.64 ms ($\pm$ 6.14)	56.35 ms ($\pm$ 1.47)	83.98 ms ($\pm$ 6.88)	170.98 ms ($\pm$ 12.22)

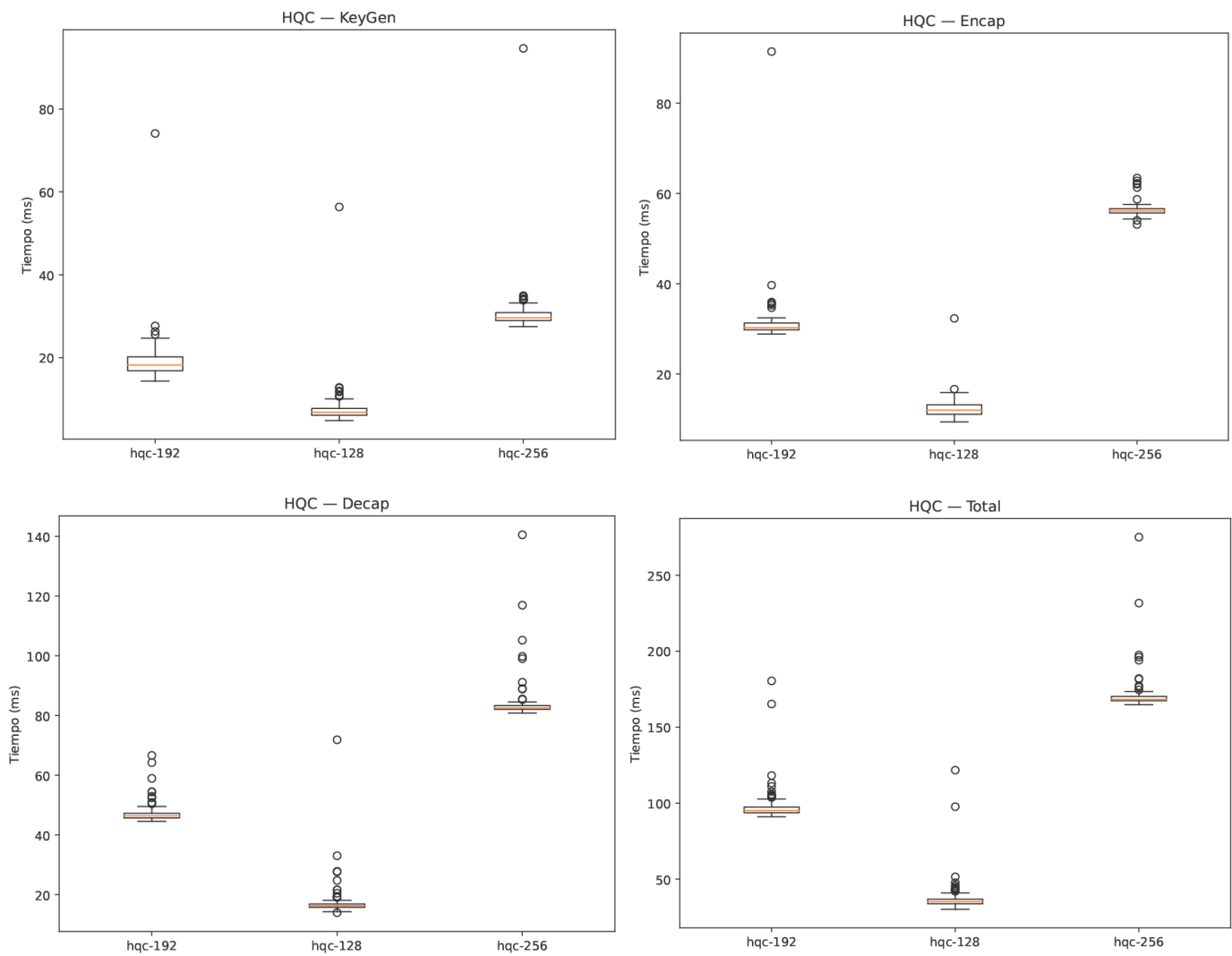


Tabla D.4. Resultados del algoritmo **HQC** (implementación en **Kotlin**) en dispositivo Android.

Kyber

Lenguaje	Versión	T Gen. Claves	T Encap	T Decap	T Total
Kotlin	Kyber-512	2.93 ms ($\pm$ 2.29)	2.85 ms ($\pm$ 2.01)	3.27 ms ($\pm$ 1.70)	9.05 ms ($\pm$ 5.71)
Kotlin	Kyber-768	3.34 ms ($\pm$ 3.11)	3.14 ms ($\pm$ 2.27)	3.52 ms ($\pm$ 2.06)	10.00 ms ($\pm$ 7.02)
Kotlin	Kyber-1024	3.94 ms ($\pm$ 3.30)	3.64 ms ($\pm$ 2.63)	4.04 ms ($\pm$ 2.71)	11.62 ms ($\pm$ 8.42)

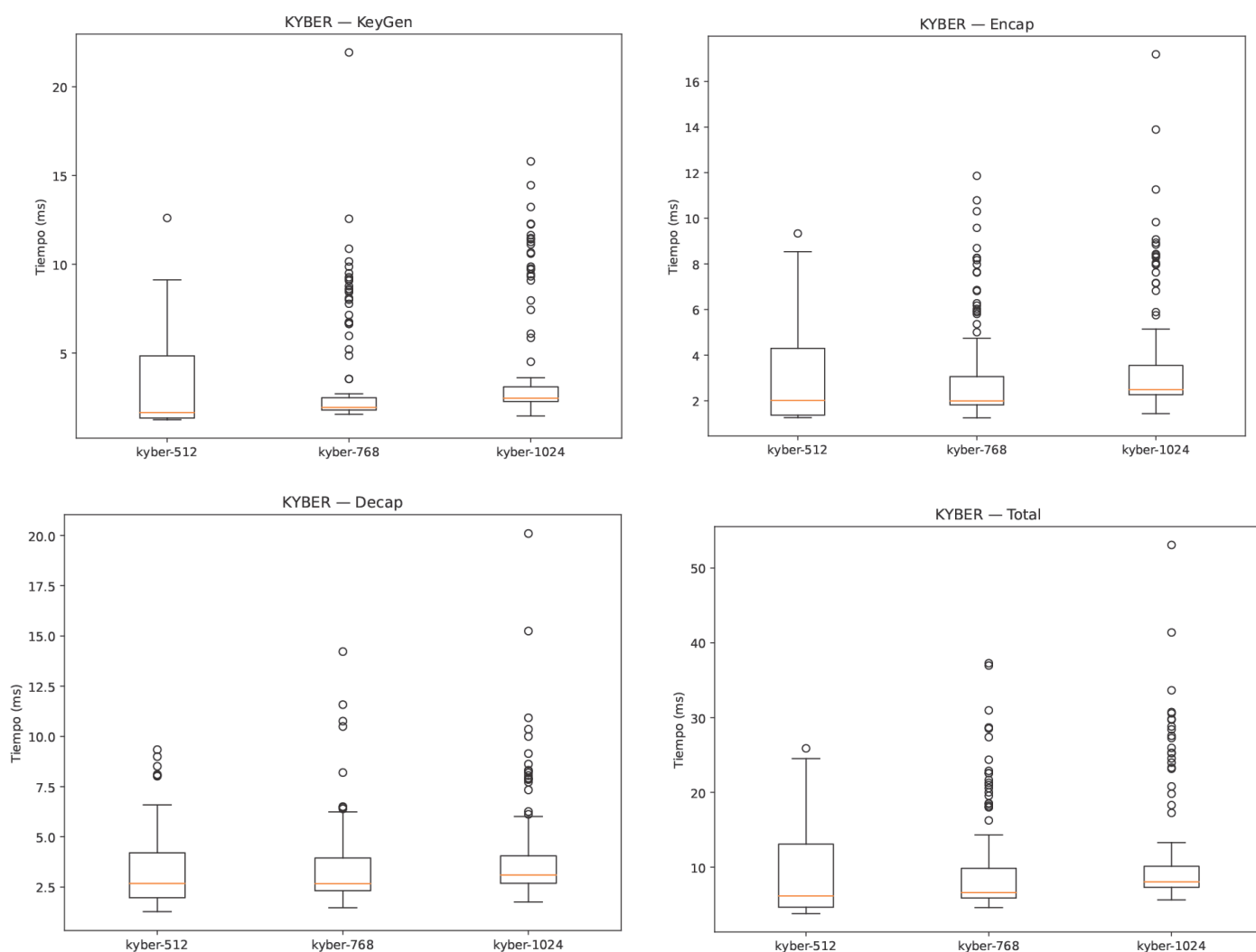


Tabla D.5. Resultados del algoritmo **Kyber** (implementación en **Kotlin**) en dispositivo Android.

Classic McEliece

Lenguaje	Versión	T Gen. Claves	T Encap	T Decap	T Total
Kotlin	CMCE-34886 4f	740.42 ms ($\pm$ 109.38)	2.45 ms ($\pm$ 2.47)	38.86 ms ($\pm$ 5.53)	781.72 ms ($\pm$ 116.83)
Kotlin	CMCE-46089 6f	2478.45 ms ($\pm$ 334.64)	4.56 ms ($\pm$ 3.25)	55.03 ms ($\pm$ 9.20)	2538.03 ms ($\pm$ 346.52)
Kotlin	CMCE-66881 28f	6182.06 ms ($\pm$ 834.83)	9.14 ms ($\pm$ 4.25)	105.25 ms ($\pm$ 10.19)	6296.45 ms ($\pm$ 848.48)

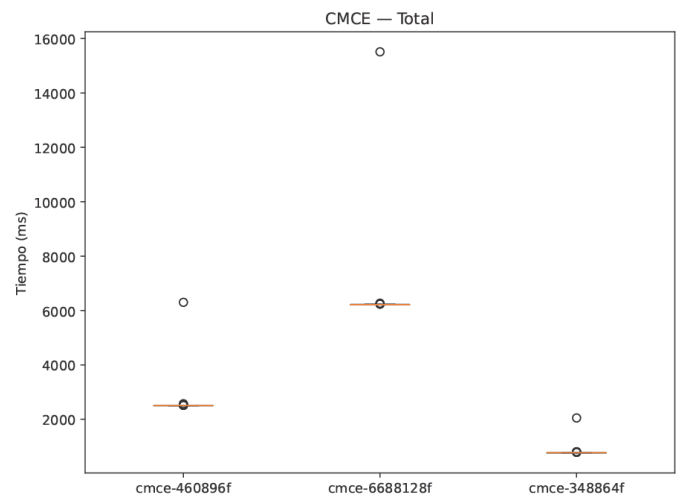
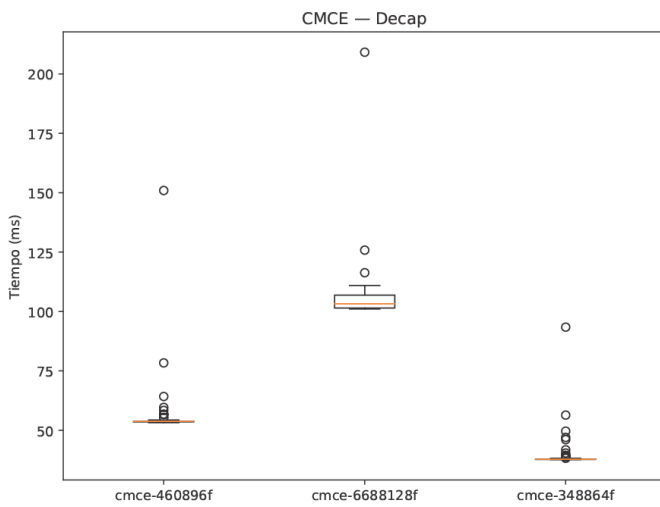
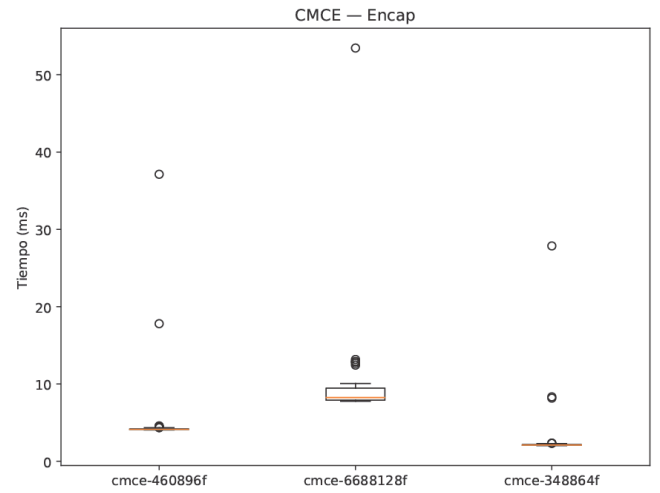
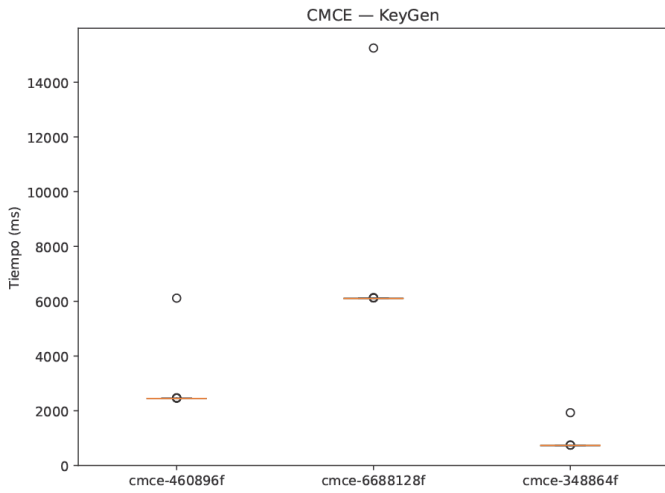


Tabla D.6. Resultados del algoritmo **Classic McEliece** (implementación en **Kotlin**) en dispositivo Android.

SPHINCS+

Lenguaje	Versión	T Gen. Claves	T Firma	T Verificación	T Total
Kotlin	SPHINCS+ SHA2-128f	13.55 ms ($\pm$ 8.90)	205.52 ms ($\pm$ 6.04)	12.23 ms ($\pm$ 0.64)	231.31 ms ($\pm$ 14.36)
Kotlin	SPHINCS+ SHA2-192f	17.35 ms ($\pm$ 9.84)	342.73 ms ($\pm$ 13.28)	18.14 ms ($\pm$ 0.72)	378.22 ms ($\pm$ 22.89)
Kotlin	SPHINCS+ SHA2-256f	35.87 ms ($\pm$ 13.21)	698.44 ms ($\pm$ 13.60)	19.33 ms ($\pm$ 0.60)	753.65 ms ($\pm$ 26.55)

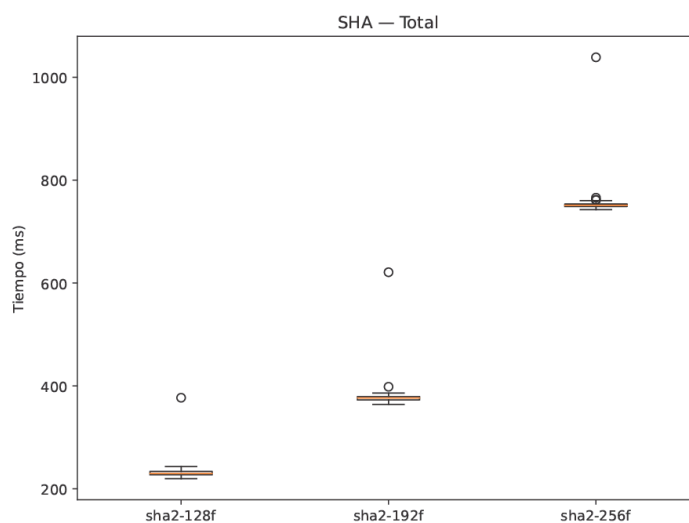
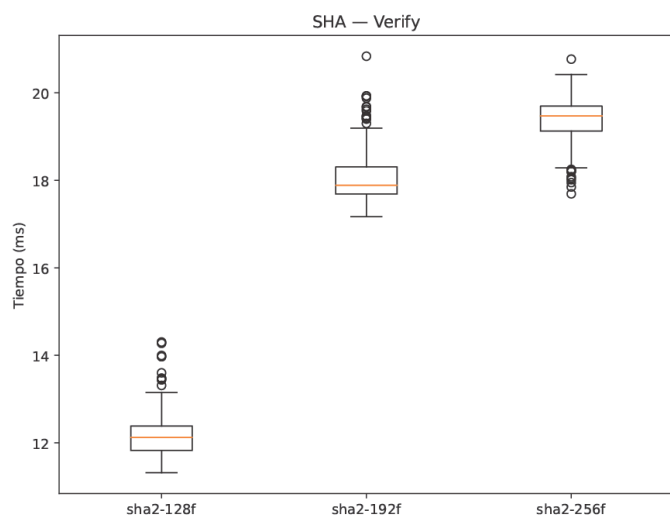
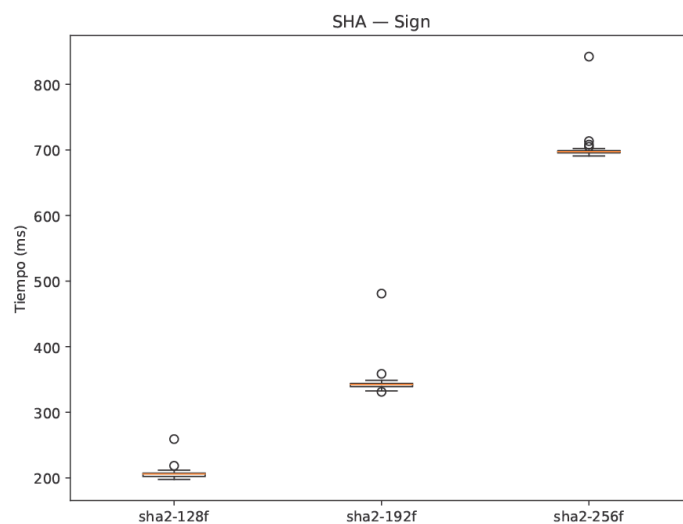
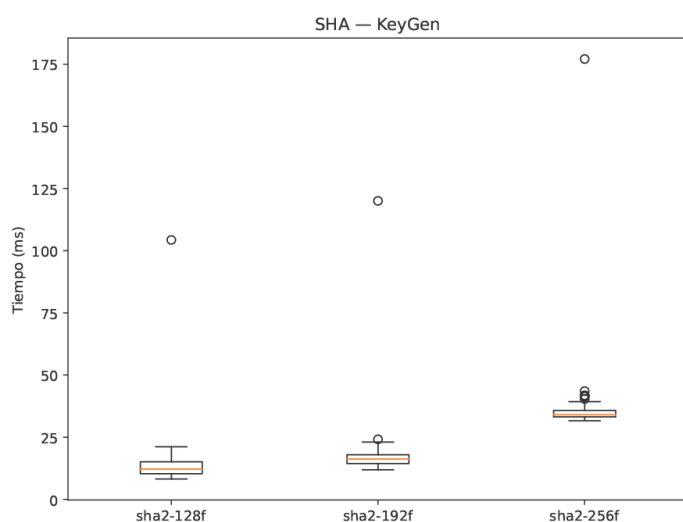


Tabla D.7. Resultados del algoritmo **SPHINCS+** (implementación en **Kotlin**) en dispositivo Android.

Consumo de recursos

Lenguaje	Algoritmo / Versión	Memoria (kB)	CPU Máx (%)
Kotlin	BIKE-128	175 912	89.86 %
Kotlin	BIKE-192	173 296	96.83 %
Kotlin	BIKE-256	173 948	98.64 %
Kotlin	CMCE-348864f	142 684	99.19 %
Kotlin	CMCE-460896f	132 064	99.59 %
Kotlin	CMCE-6688128f	130 648	99.81 %
Kotlin	Dilithium2	176 944	65.07 %
Kotlin	Dilithium3	178 924	70.99 %
Kotlin	Dilithium5	180 248	74.49 %
Kotlin	Falcon-512	290 804	120.69 %
Kotlin	Falcon-1024	305 472	126.94 %
Kotlin	HQC-128	376 032	105.84 %
Kotlin	HQC-192	376 972	121.02 %
Kotlin	HQC-256	377 020	128.14 %
Kotlin	Kyber-512	140 752	51.10 %
Kotlin	Kyber-768	145 268	52.82 %
Kotlin	Kyber-1024	149 724	54.58 %
Kotlin	SPHINCS+ SHA2-128f	181 756	100.97 %
Kotlin	SPHINCS+ SHA2-192f	182 548	102.58 %
Kotlin	SPHINCS+ SHA2-256f	184 736	104.45 %

Tabla D.8. Consumo de recursos (CPU y memoria) de las implementaciones en **Kotlin** en dispositivo Android.



Universidad Autónoma
de Madrid